Cognitive Modeling

Complete Lecture Notes

16 weeks · 6 chapters

Contents

Syllabus 1

0.0.1 Course description . . . 1
0.0.2 Learning goals . . . 2
0.0.3 Topics . . . 2
0.0.4 Weekly schedule . . . 2
0.0.5 Assessment (0–20 scale) . . . 4
0.0.6 Policies (summary) . . . 4
0.0.7 Software . . . 4
0.0.8 Readings . . . 5

Course blueprint 6

0.0.9 Design principles . . . 7
0.0.10 Weekly plan . . . 7
0.0.11 Grading (0–20 scale) . . . 16
0.0.12 Final project (overview; the full brief and rubric come in Stage 3) . . . 16
0.0.13 Computing on students' own laptops . . . 17

1 Fundamentals of Cognitive Modeling 18

1.1 Week 1: What Is a Cognitive Model? . . . 18
1.1.1 Key terms . . . 18
1.1.2 Learning outcomes . . . 19
1.1.3 Motivation & big picture . . . 20
1.1.4 Lecture notes . . . 21
1.1.5 Worked example . . . 30
1.1.6 Lab (2 hours) . . . 31
1.1.7 Connection to AI & LLMs . . . 36
1.1.8 Common mistakes & misconceptions . . . 37
1.1.9 Check your understanding . . . 38

1.1.10 Assignment . . . 38
1.1.11 Further reading . . . 39
1.2 Week 2: Anatomy of a Model: Signal Detection and Forgetting Curves . . . 40
1.2.1 Key terms . . . 40
1.2.2 Learning outcomes . . . 41
1.2.3 Motivation & big picture . . . 42
1.2.4 Lecture notes . . . 43
1.2.5 Worked example . . . 49
1.2.6 Lab (2 hours) . . . 53
1.2.7 Connection to AI & LLMs . . . 58
1.2.8 Common mistakes & misconceptions . . . 59
1.2.9 Check your understanding . . . 59
1.2.10 Assignment . . . 60
1.2.11 Further reading . . . 61

2 Parameter Estimation Methods 63
2.1 Week 3: Fitting Models by Least Squares: Error Surfaces and Optimization . . . 63
2.1.1 Key terms . . . 63
2.1.2 Learning outcomes . . . 64
2.1.3 Motivation & big picture . . . 65
2.1.4 Lecture notes . . . 65
2.1.5 Worked example . . . 72
2.1.6 Lab (2 hours) . . . 73
2.1.7 Connection to AI & LLMs . . . 78
2.1.8 Common mistakes & misconceptions . . . 78
2.1.9 Check your understanding . . . 79
2.1.10 Assignment . . . 80
2.1.11 Further reading . . . 80
2.2 Week 4: Maximum Likelihood Estimation . . . 80
2.2.1 Key terms . . . 80
2.2.2 Learning outcomes . . . 82
2.2.3 Motivation & big picture . . . 82
2.2.4 Lecture notes . . . 83
2.2.5 Worked example . . . 88
2.2.6 Lab (2 hours) . . . 88
2.2.7 Connection to AI & LLMs . . . 92

2.2.8 Common mistakes & misconceptions . . . 93
2.2.9 Check your understanding . . . 93
2.2.10 Assignment . . . 94
2.2.11 Further reading . . . 95
2.3 Week 5: Uncertainty in Estimates and Fitting Multiple Participants . . . 95
2.3.1 Key terms . . . 95
2.3.2 Learning outcomes . . . 96
2.3.3 Motivation & big picture . . . 97
2.3.4 Lecture notes . . . 97
2.3.5 Worked example . . . 102
2.3.6 Lab (2 hours) . . . 103
2.3.7 Connection to AI & LLMs . . . 107
2.3.8 Common mistakes & misconceptions . . . 107
2.3.9 Check your understanding . . . 108
2.3.10 Assignment . . . 109
2.3.11 Further reading . . . 109

3 Evaluation of Cognitive Models 110
3.1 Week 6: Evaluating Models I: Goodness of Fit, Overfitting and Cross-Validation . . . 110
3.1.1 Key terms . . . 110
3.1.2 Learning outcomes . . . 111
3.1.3 Motivation & big picture . . . 112
3.1.4 Lecture notes . . . 112
3.1.5 Worked example . . . 117
3.1.6 Lab (2 hours) . . . 118
3.1.7 Connection to AI & LLMs . . . 122
3.1.8 Common mistakes & misconceptions . . . 123
3.1.9 Check your understanding . . . 123
3.1.10 Assignment . . . 124
3.1.11 Further reading . . . 125
3.2 Week 7: Evaluating Models II: Information Criteria, Parameter Recovery and Model Recovery . . . 126
3.2.1 Key terms . . . 126
3.2.2 Learning outcomes . . . 127
3.2.3 Motivation & big picture . . . 127
3.2.4 Lecture notes . . . 128

3.2.5 Worked example . . . 132
3.2.6 Lab (2 hours) . . . 133
3.2.7 Connection to AI & LLMs . . . 137
3.2.8 Common mistakes & misconceptions . . . 138
3.2.9 Check your understanding . . . 138
3.2.10 Assignment . . . 139
3.2.11 Further reading . . . 139
3.3 Week 8: Midterm Review: The Whole Pipeline in One Case Study . . . 140
3.3.1 Key terms . . . 140
3.3.2 Learning outcomes . . . 141
3.3.3 Motivation & big picture . . . 142
3.3.4 Lecture notes . . . 142
3.3.5 Worked example . . . 145
3.3.6 Lab (2 hours): midterm exam + project consultation . . . 147
3.3.7 Connection to AI & LLMs . . . 149
3.3.8 Common mistakes & misconceptions (top 10 from Weeks 1–7) . . . 150
3.3.9 Check your understanding (practice questions in exam style) . . . 150
3.3.10 Assignment . . . 151
3.3.11 Further reading . . . 151

4 Bayesian Modeling 152
4.1 Week 9: Bayesian Modeling I: Priors, Posteriors and Grid Approximation . . . 152
4.1.1 Key terms . . . 152
4.1.2 Learning outcomes . . . 153
4.1.3 Motivation & big picture . . . 154
4.1.4 Lecture notes . . . 155
4.1.5 Worked example . . . 159
4.1.6 Lab (2 hours) . . . 160
4.1.7 Connection to AI & LLMs . . . 163
4.1.8 Common mistakes & misconceptions . . . 164
4.1.9 Check your understanding . . . 164
4.1.10 Assignment . . . 165
4.1.11 Further reading . . . 165
4.2 Week 10: Bayesian Modeling II: Posterior Predictive Checks and Bayes Factors . . . 166
4.2.1 Key terms . . . 166
4.2.2 Learning outcomes . . . 167

4.2.3 Motivation & big picture . . . 168
4.2.4 Lecture notes . . . 168
4.2.5 Worked example . . . 173
4.2.6 Lab (2 hours) . . . 174
4.2.7 Connection to AI & LLMs . . . 177
4.2.8 Common mistakes & misconceptions . . . 178
4.2.9 Check your understanding . . . 178
4.2.10 Assignment . . . 179
4.2.11 Further reading . . . 180

5 Parameter Estimation with MCMC 181
5.1 Week 11: Monte Carlo Methods and the Metropolis–Hastings Algorithm . . . 181
5.1.1 Key terms . . . 181
5.1.2 Learning outcomes . . . 182
5.1.3 Motivation & big picture . . . 183
5.1.4 Lecture notes . . . 183
5.1.5 Worked example . . . 187
5.1.6 Lab (2 hours) . . . 189
5.1.7 Connection to AI & LLMs . . . 192
5.1.8 Common mistakes & misconceptions . . . 192
5.1.9 Check your understanding . . . 193
5.1.10 Assignment . . . 194
5.1.11 Further reading . . . 194
5.2 Week 12: Gibbs Sampling and Convergence Diagnostics . . . 194
5.2.1 Key terms . . . 194
5.2.2 Learning outcomes . . . 195
5.2.3 Motivation & big picture . . . 196
5.2.4 Lecture notes . . . 197
5.2.5 Worked example . . . 201
5.2.6 Lab (2 hours) . . . 201
5.2.7 Connection to AI & LLMs . . . 204
5.2.8 Common mistakes & misconceptions . . . 205
5.2.9 Check your understanding . . . 205
5.2.10 Assignment . . . 206
5.2.11 Further reading . . . 207
5.3 Week 13: Probabilistic Programming and Hierarchical Models . . . 207

5.3.1 Key terms . . . 207
5.3.2 Learning outcomes . . . 209
5.3.3 Motivation & big picture . . . 209
5.3.4 Lecture notes . . . 210
5.3.5 Worked example . . . 214
5.3.6 Lab (2 hours) . . . 214
5.3.7 Connection to AI & LLMs . . . 218
5.3.8 Common mistakes & misconceptions . . . 218
5.3.9 Check your understanding . . . 219
5.3.10 Assignment . . . 220
5.3.11 Further reading . . . 220

6 Applications in Cognitive Science 221
6.1 Week 14: Modeling in Cognitive Science I: Categorization (GCM) and Decision Making (Drift-Diffusion) . . . 221
6.1.1 Key terms . . . 221
6.1.2 Learning outcomes . . . 222
6.1.3 Motivation & big picture . . . 223
6.1.4 Lecture notes . . . 224
6.1.5 Worked example . . . 228
6.1.6 Lab (2 hours) . . . 229
6.1.7 Connection to AI & LLMs . . . 232
6.1.8 Common mistakes & misconceptions . . . 233
6.1.9 Check your understanding . . . 233
6.1.10 Assignment . . . 234
6.1.11 Further reading . . . 234
6.2 Week 15: Modeling in Cognitive Science II: Reinforcement Learning, Bayesian Cognition and AI . . . 235
6.2.1 Key terms . . . 235
6.2.2 Learning outcomes . . . 236
6.2.3 Motivation & big picture . . . 236
6.2.4 Lecture notes . . . 237
6.2.5 Worked example . . . 242
6.2.6 Lab (2 hours) . . . 244
6.2.7 Connection to AI & LLMs . . . 247
6.2.8 Common mistakes & misconceptions . . . 248

6.2.9 Check your understanding . . . 248
6.2.10 Assignment . . . 249
6.2.11 Further reading . . . 249
6.3 Week 16: Final Project Presentations and Good Modeling Practice . . . 250
6.3.1 Key terms . . . 250
6.3.2 Learning outcomes . . . 251
6.3.3 Motivation & big picture . . . 251
6.3.4 Lecture notes . . . 252
6.3.5 Worked example: critiquing a modeling study . . . 255
6.3.6 Lab (2 hours): presentations and peer feedback . . . 256
6.3.7 Connection to AI & LLMs . . . 256
6.3.8 Common mistakes & misconceptions (in project presentations and reports) . 257
6.3.9 Check your understanding (integrative review questions for the final exam) . 257
6.3.10 Assignment . . . 258
6.3.11 Further reading . . . 259

Glossary 260
6.3.12 AI & LLM terms (from the “Connection to AI & LLMs” sections) . . . 265

Reading list 267
6.3.13 Course references (from the course specification) . . . 267
6.3.14 Chapter pointers by week . . . 268
6.3.15 Optional readings outside the course list . . . 269

Syllabus

Course	Cognitive Modeling (elective specialization course)
Credits	2 units: 1 theory unit (16 h) + 1 practical unit (32 h) = 48 h
Format	16 weeks × (1 h lecture + 2 h hands-on lab)
Audience	MSc students and final-year BSc students in cognitive science, psychology, computer science and AI
Prerequisites	Statistical methods in cognitive science; experimental methods in cognitive psychology. Basic probability, statistics and introductory programming are assumed; Bayesian statistics is *not* assumed.
Instructor	*[name, office, e-mail, office hours]*
Teaching assistant	*[name, e-mail, lab hours]*
Course page	*[LMS link]*

0.0.1 Course description

Cognitive models are explicit, mathematical or computational descriptions of the processes behind behavior. This course teaches how to build such models, fit them to experimental data, evaluate them, and interpret what they say about the mind. We move from intuition to formalism to code to interpretation: from the logic of a model, through least-squares and maximum-likelihood estimation, model evaluation and Bayesian inference, to Markov chain Monte Carlo and hierarchical models. We finish with models from memory, perception, categorization, decision making and learning, and their links to artificial intelligence. Every week includes a two-hour lab in Python in which students fit, evaluate and interpret a model on (simulated or real) experimental data.

0.0.2 Learning goals

General goals. By the end of the course students will be familiar with: 1. different approaches to and models of cognitive modeling; 2. how to design a model to address a cognitive problem; 3. applications of modeling in cognitive science and artificial intelligence.

Specific goals. Students will gain the skills to: 1. implement modeling methods on experimental data; 2. interpret research results obtained with cognitive models.

0.0.3 Topics

1. Fundamentals of cognitive modeling
2. Parameter estimation methods (least squares / mean squared error, maximum likelihood)
3. Evaluation of cognitive models
4. Bayesian modeling
5. Parameter estimation with data-driven methods (Monte Carlo, Metropolis–Hastings and Gibbs sampling, MCMC)
6. Examples of modeling in different areas of cognitive science

0.0.4 Weekly schedule

Week	Chapter	Lecture	Lab	Due
1	1	What is a cognitive model? Marr's levels; notation	Install the course environment; simulate a random walk	—
2	1	Anatomy of a model: SDT and forgetting curves	SDT, ROC/zROC; forgetting curves with binomial noise	A0
3	2	Least squares, error surfaces and optimization	Grid search, Nelder–Mead, L-BFGS-B; failure modes	—
4	2	Maximum likelihood	Binomial MLE; G^2; multi-condition SDT	A1
5	2	Uncertainty; fitting many participants	Hessian, profile and bootstrap CIs; aggregation artifacts	—

Week	Chapter	Lecture	Lab	Due
6	3	Fit vs. generalization; cross-validation	Overfitting; K-fold CV; generalization criterion	A2
7	3	LRT, AIC/BIC; parameter and model recovery	Recovery studies; confusion matrices	Project teams
8	1–3	Midterm review: the whole pipeline	Midterm exam (90 min) + project consultation	—
9	4	Bayesian modeling I: priors, posteriors, grids	Beta-binomial; 2-D grid posterior for SDT; prior predictive checks	A3, project proposal
10	4	Bayesian modeling II: PPCs and Bayes factors	PPC; Bayes factors by grid and Savage–Dickey; prior sensitivity	—
11	5	Monte Carlo and Metropolis–Hastings	Metropolis from scratch; tuning	—
12	5	Gibbs sampling; convergence diagnostics	Gibbs from scratch; R-hat and ESS; ArviZ	A4
13	5	Probabilistic programming; hierarchical models	Hierarchical SDT in PyMC	Progress check
14	6	Categorization (GCM) and decisions (drift-diffusion)	GCM vs. prototype; EZ-diffusion	A5
15	6	Reinforcement learning, Bayesian cognition and AI	Q-learning fits and recovery; ideal observer	—
16	—	Synthesis and good modeling practice	Project presentations	Presentations

Week	Chapter	Lecture	Lab	Due
Exam period	1–6	—	Final exam (120 min)	Report + code (before the exam)

Detailed materials for each week are in `weekNN_*.md`.

0.0.5 Assessment (0–20 scale)

Component	Points
Lab check-offs (weekly in-lab exercises)	2.0
Assignments A1–A5 (1.0 each; A0 required, ungraded)	5.0
In-lecture quizzes (about 10; lowest 2 dropped)	1.0
Midterm exam (Week 8)	3.0
Final project: proposal 1.0 · presentation 2.0 · report + code 3.0	6.0
Final exam (cumulative, emphasis on Chapters 4–6)	3.0
Total	20.0

Details, rubrics and policies are in `assessments/README.md`.

0.0.6 Policies (summary)

- Late work: −10% per day, maximum 3 days.
- Collaboration: discuss freely; write your own code and text (the project is a team product for pairs). Acknowledge discussions and reused code.
- AI tools: allowed for debugging, explanations and language editing. State how you used them; you must be able to explain everything you submit.
- Exams: closed book, formula sheet provided, non-programmable calculator allowed.
- Attendance: labs are where most learning happens; lab check-offs require attendance (or an agreed alternative).
- Accessibility: students needing accommodations should contact the instructor in the first two weeks.

0.0.7 Software

Python 3.11+ with numpy, scipy, pandas, matplotlib, JupyterLab, PyMC and ArviZ, installed on students' own laptops in Week 1 (Miniforge/conda-forge recommended; Google Colab as fallback).

See `environment.yml`, `requirements.txt` and the Week 1 lab.

0.0.8 Readings

Main references (see `reading_list.md` for the chapter mapping): - Farrell, S., & Lewandowsky, S. (2018). *Computational modeling of cognition and behavior.* Cambridge University Press. (primary text, Weeks 1–8) - Lee, M. D., & Wagenmakers, E.-J. (2014). *Bayesian cognitive modeling: A practical course.* Cambridge University Press. (primary text, Weeks 9–14) - Kruschke, J. K. (2014). *Doing Bayesian data analysis: A tutorial with R, JAGS, and Stan* (2nd ed.). Academic Press. - Martin, O. A., Kumar, R., & Lao, J. (2021). *Bayesian modeling and computation in Python.* Chapman and Hall/CRC. - Anderson, B. (2014). *Computational neuroscience and cognitive modelling: A student's introduction to methods and procedures.* SAGE. - Verguts, T. (2022). *Introduction to modeling cognitive processes.* MIT Press. - Ghodousian, A. (1401). *Mabani-ye ravesh-haye mostaghim dar behine-sazi gheyr-khatti* [Foundations of direct methods in nonlinear optimization] (in Persian). - Azimifar, Z., & Ahmadi, E. (1396). *Shenasayi-ye amari-ye olgu: yadgiri-ye nazari-ye moghaddamati* [Statistical pattern recognition: Introductory learning theory], vol. 1 (in Persian).

Course blueprint

Credits: 2 units (1 theory + 1 practical), 48 h in total · Format: 1 h lecture + 2 h lab per week Audience: MSc and final-year BSc students · Prerequisites: statistical methods in cognitive science; experimental methods in cognitive psychology Language / tools: English · Python 3 (numpy, scipy, pandas, matplotlib; PyMC + ArviZ from Week 13), run on students' own laptops

0.0.9 Design principles

- One modeling pipeline used all term. Every lab follows the same steps: *simulate* → *fit* → *evaluate* → *recover* → *interpret*.
- A few datasets come back again and again. Forgetting curves and signal detection theory (SDT) run through Weeks 2–13, so new methods are compared on familiar data. Chapter 6 adds categorization (GCM), response times (drift-diffusion) and reinforcement learning.
- "Write it yourself, then use the library." Students write least squares, maximum likelihood, grid posteriors, Metropolis–Hastings, Gibbs and R-hat by hand before using `scipy.optimize`, PyMC or ArviZ.
- Chapter weighting. Ch1: 2 weeks · Ch2: 3 · Ch3: 2 · Ch4: 2 · Ch5: 3 · Ch6: 2. One more week each goes to orientation, the midterm and project presentations.
- "Metropolis–Gibbs" means two separate algorithms: Metropolis–Hastings (Week 11) and Gibbs sampling (Week 12). MH-within-Gibbs is covered as an Advanced box.

0.0.10 Weekly plan

Wk	Ch	Lecture (1 h)	Lab (2 h)	Learning outcomes	Readings	Assignment / milestone
1	1	Orientation: what is a cognitive model? Why we model; verbal vs. formal theories; Marr's three levels; model families (symbolic, connectionist, stochastic/dynamical, Bayesian); course roadmap; notation table	Laptop setup (Miniforge/conda environment with the full course stack *including PyMC*, run an install-check script; Colab as fallback); simulate a random walk to a bound and plot simulated "data"	Distinguish descriptive, process and normative models; explain why verbal theories are underspecified; run and plot a model simulation	F&L ch. 1–2; Verguts, introductory chapter [verify]; Anderson, introductory chapter [verify]	A0 (ungraded): install check plus a one-page note: "Pick a verbal theory; list what it leaves unspecified"
2	1	Anatomy of a model: data, free vs. fixed parameters, model equations, noise (data) model; simulation as a thinking tool; first models: SDT (d', c) and exponential vs. power forgetting	Implement SDT (d', c, ROC curves); simulate exponential and power forgetting; explore how each parameter changes the predictions	Identify the components of a formal model; compute d' and c; predict how parameters shape model output	F&L ch. 2; L&W ch. 11 (SDT, a preview)	A1 released: "From verbal to formal" (due Wk 4)

Wk	Ch	Lecture (1 h)	Lab (2 h)	Learning outcomes	Readings	Assignment / milestone
3	2	Least squares & optimization: SSE, MSE, RMSD; error surfaces; grid search, Nelder–Mead, gradient methods; local minima, starting values, bounds and reparameterization	Fit forgetting curves by grid search, then with `scipy.optimize.minimize`; plot the error surface; multi-start fitting	Implement an LS fit from scratch; compare optimizers; diagnose failed fits	F&L ch. 3; Ghodousian (direct search methods)	—
4	2	Maximum likelihood: probability vs. likelihood; log-likelihood; Bernoulli, binomial and Gaussian likelihoods; LS as MLE under Gaussian noise	MLE for forgetting data (binomial recall counts) and for SDT; compare MLE and LS estimates	Derive a likelihood; implement NLL minimization; explain when LS and MLE agree	F&L ch. 4; Azimifar & Ahmadi (ML estimation)	A1 due · A2 released: LS vs. MLE (due Wk 6)

Wk	Ch	Lecture (1 h)	Lab (2 h)	Learning outcomes	Readings	Assignment / milestone
5	2	Uncertainty & multiple participants: Hessian standard errors, profile likelihood, parametric bootstrap; individual vs. aggregate fitting; aggregation artifacts	Bootstrap CIs; averaging exponential learners produces a spurious "power law of practice"; individual fits	Quantify estimation uncertainty; explain aggregation artifacts; choose a fitting level	F&L ch. 5	—
6	3	Evaluation I, fit vs. generalization: R^2, RMSD and deviance and their limits; flexibility and overfitting; hold-out, k-fold and LOO cross-validation	Polynomial overfitting demo; cross-validated comparison of three forgetting models	Explain why good fit $\neq$ good model; implement k-fold CV; interpret CV results	F&L ch. 10; Azimifar & Ahmadi (generalization)	A2 due · A3 released: comparison + recovery (due Wk 9)

Wk	Ch	Lecture (1 h)	Lab (2 h)	Learning outcomes	Readings	Assignment / milestone
7	3	Evaluation II, selection & recovery: likelihood-ratio test; AIC, AICc, BIC, Akaike weights; parameter and model recovery; identifiability and trade-offs	Recovery study: true vs. recovered parameter plots; model-recovery confusion matrix (AIC/BIC)	Interpret information criteria; design a recovery study; detect non-identifiability	F&L ch. 10	Project teams (1–2 students) formed; proposal brief released
8	1–3	Midterm review: integrated case study of the full pipeline	Midterm exam (lab slot, 90 min): conceptual questions + code reading/interpretation, Ch. 1–3	—	Review of Weeks 1–7	Midterm

Wk	Ch	Lecture (1 h)	Lab (2 h)	Learning outcomes	Readings	Assignment / milestone
9	4	Bayesian modeling I: prior, likelihood, posterior, evidence; beta-binomial conjugacy; grid approximation; posterior summaries and HDI; credible vs. confidence intervals	Grid posterior for a recall probability; 2-D grid for SDT; vary prior and sample size; prior predictive simulation	Compute a grid posterior; explain the role of the prior; interpret credible intervals	K ch. 5–6; L&W ch. 1, 3; MKL ch. 1; F&L ch. 6; Azimifar & Ahmadi (Bayesian estimation)	A3 due · Project proposal due
10	4 (+3)	Bayesian modeling II: posterior predictive checks; marginal likelihood and Bayes factors; Savage–Dickey; prior sensitivity; BIC as an approximation	PPCs for the forgetting model; Savage–Dickey BF for $d' > 0$; BF between retention models via grid integration	Conduct a PPC; compute and interpret a Bayes factor; evaluate prior sensitivity	L&W ch. 7; F&L ch. 11; K ch. 10	A4 released: Bayesian analysis by grid (due Wk 12)

Wk	Ch	Lecture (1 h)	Lab (2 h)	Learning outcomes	Readings	Assignment / milestone
11	5	Monte Carlo & Metropolis–Hastings: Monte Carlo integration; Markov chains; MH; proposal tuning, acceptance rate, burn-in	MH from scratch for the forgetting model; trace plots; compare with the grid posterior	Explain why MCMC works; implement MH; tune a proposal	K ch. 7; F&L ch. 7	—
12	5	Gibbs sampling & convergence diagnostics: full conditionals; Gibbs for a normal model; MH-within-Gibbs (Advanced); multiple chains, R-hat, ESS, autocorrelation	Gibbs from scratch; implement R-hat and ESS and check against ArviZ; break convergence and diagnose it	Implement Gibbs; interpret R-hat, ESS and trace plots; judge chain trustworthiness	K ch. 7; MKL ch. 2; F&L ch. 7	A4 due · A5 released: MCMC + diagnostics (due Wk 14)

Wk	Ch	Lecture (1 h)	Lab (2 h)	Learning outcomes	Readings	Assignment / milestone
13	5	Probabilistic programming & hierarchical models: PyMC; HMC/NUTS conceptually (Advanced); partial pooling and shrinkage (linked to Week 5)	Hierarchical SDT in PyMC: ArviZ diagnostics, PPCs, comparison with individual MLE fits; (Advanced) LOO/WAIC	Specify a PyMC model; explain partial pooling; report a full Bayesian workflow	L&W ch. 11; K ch. 9; F&L ch. 9; MKL ch. 2, 10	Project progress check (feedback only)
14	6	Modeling in cognitive science I: categorization (prototype vs. exemplar; GCM); choice/RT (drift-diffusion, EZ-diffusion)	Fit the GCM by MLE to 5–4 category-structure data; EZ-diffusion on simulated RTs; interpret the parameters	Fit an exemplar model; map DDM parameters to cognitive constructs	L&W ch. 17; F&L ch. 14 [verify]; Verguts (relevant chapters [verify])	A5 due

Wk	Ch	Lecture (1 h)	Lab (2 h)	Learning outcomes	Readings	Assignment / milestone
15	6	Modeling in cognitive science II: Rescorla–Wagner and Q-learning with softmax; Bayesian *models of cognition* vs. Bayesian *data analysis*; connectionist models and links to AI	Fit Q-learning (α, β) to simulated bandit data; parameter recovery; (Advanced) hierarchical version in PyMC	Fit an RL model; assess identifiability; contrast Bayesian cognition with Bayesian statistics	Verguts (learning/RL chapters [verify]); F&L ch. 13 [verify]; Anderson (relevant chapters [verify])	Final project work
16	—	Final project presentations; wrap-up on good modeling practice	Presentations continue; peer feedback	Communicate and critique a modeling study	—	Presentations · report + code due before the final exam
Exam period	1–6	Final exam (written, cumulative, emphasis on Ch. 4–6; formula sheet provided; code reading/interpretation, no laptops)	—	—	—	Final exam

Abbreviations: F&L = Farrell & Lewandowsky (2018) · L&W = Lee & Wagenmakers (2014) · K = Kruschke (2014) · MKL = Martin, Kumar & Lao (2021). Chapter numbers were written from memory. The ones marked [verify] are less certain; please check all of them against the books.

0.0.11 Grading (0–20 scale)

Component	Points
Lab check-offs (weekly in-lab exercises)	2.0
Assignments A1–A5 (1.0 each)	5.0
Short in-lecture quizzes (≈10, drop the lowest 2)	1.0
Midterm exam (Week 8)	3.0
Final project: proposal 1.0 · presentation 2.0 · report + reproducible code 3.0	6.0
Final exam (cumulative)	3.0
Total	20.0

0.0.12 Final project (overview; the full brief and rubric come in Stage 3)

- Teams of 1–2. The scope grows with team size. Everyone fits at least two candidate models, runs one model comparison and one parameter-recovery analysis. Pairs also do a Bayesian/MCMC analysis (or a hierarchical model) and submit an individual contribution statement.
- Data: simulated data, an open dataset, or the student's own experimental data.
- Milestones: team formation (Wk 7) → proposal (Wk 9) → progress check (Wk 13) → presentation (Wk 16) → report + code (before the final exam).

0.0.13 Computing on students' own laptops

- The full environment, PyMC included, is installed in Week 1, so Windows/macOS installation problems are solved long before Week 13.
- Recommended route: Miniforge (conda-forge) with a course `environment.yml`, plus a pip `requirements.txt` as an alternative (both delivered in Stage 4).
- Fallback: every notebook also runs on Google Colab.

Chapter 1

Fundamentals of Cognitive Modeling

1.1 Week 1: What Is a Cognitive Model?

> Chapter 1: Fundamentals of cognitive modeling · Lecture 1 h · Lab 2 h This week's running model: a simple random walk model of two-choice decisions.

1.1.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
cognitive model	مدل شناختی	توصیف دقیق و ریاضی یا محاسباتی از یک فرایند ذهنی
random walk	گام تصادفی	جمع شدن گام‌به‌گام شواهد تا رسیدن به یک مرز
verbal theory	نظریه‌ی کلامی	نظریه‌ای که فقط با کلمات بیان شده و پیش‌بینی دقیق نمی‌دهد
Marr's levels	سطوح تحلیل Marr	سطح محاسباتی، الگوریتمی و پیاده‌سازی
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
noise	نویز	تغییرات تصادفی در شواهد یا در داده
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
drift rate	نرخ رانش	سرعت میانگین جمع شدن شواهد؛ نشانه‌ی کیفیت شواهد

English	فارسی	توضیح کوتاه
boundary separation	فاصله‌ی مرزها	مقدار شواهد لازم برای پاسخ؛ نشانه‌ی احتیاط
evidence	شواهد	اطلاعاتی که برای تصمیم جمع می‌شود
accuracy	دقت	نسبت پاسخ‌های درست
response time (RT)	زمان واکنش	فاصله‌ی ارائه‌ی محرک تا پاسخ
formal model / formalize	فرمال کردن / مدل فرمال	نوشتن نظریه به شکل معادله یا الگوریتم
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
forward / inverse problem	معکوس / مسئله‌ی مستقیم	شبیه‌سازی (پارامتر ← داده) در برابر برآورد (داده ← پارامتر)
non-decision time	زمان غیرتصمیمی	زمان دیدن محرک و اجرای حرکت پاسخ
speed–accuracy trade-off	بده‌بستان سرعت و دقت	سریع‌تر پاسخ دادن به قیمت خطای بیشتر
trial	کوشش (trial)	یک بار انجام تکلیف توسط شرکت‌کننده
probabilistic programming	برنامه‌نویسی احتمالاتی	تعریف مدل با کد و نمونه‌گیری خودکار (PyMC)
temperature	دما (temperature)	پارامتر نویز در انتخاب token؛ $\beta = 1/T$
random seed	seed تصادفی	عدد شروع مولد اعداد تصادفی برای تکرارپذیری
quantile	چندک (quantile)	مقداری که درصد مشخصی از داده زیر آن است
large language model (LLM)	مدل بزرگ زبانی	شبکه‌ی عصبی‌ای که کلمه‌ی بعدی متن را پیش‌بینی می‌کند
token	token (واحد متن)	کلمه یا بخشی از کلمه که مدل زبانی پیش‌بینی می‌کند

1.1.2 Learning outcomes

By the end of this week, students will be able to:

1. Define a cognitive model and distinguish it from a verbal theory and from a purely statistical model of the data.
2. Classify models by purpose (descriptive, process, normative) and by family (symbolic, connectionist, stochastic/dynamical, Bayesian), and place them on Marr's three levels.
3. Explain how free parameters, a deterministic core and a noise model together turn a model into a *data-generating* machine.
4. Implement and simulate a random walk decision model in Python, and verify the simulation against analytic predictions.

5. Interpret how changes in the model's parameters (drift, boundary) map onto psychological constructs (evidence quality, caution).

1.1.3 Motivation & big picture

Consider a familiar verbal theory of simple decisions:

> *"When people decide, they gather evidence until they have enough, then they respond."*

It sounds complete, but try to get a prediction out of it:

- If we tell participants to *be accurate* rather than *be fast*, how much slower should they get? Twice as slow? 50 ms slower?
- Are errors faster or slower than correct responses, or equally fast?
- Should the response time (RT) distribution be symmetric or skewed?
- If a stimulus becomes harder to see, should RTs change in the same way as under an accuracy instruction?

The verbal theory cannot answer any of these. Each answer depends on decisions the theory never made: What counts as "evidence"? Does it arrive continuously or in chunks? What does "enough" mean? Is the threshold the same for both responses?

As soon as we write the theory down as a model, for example a random walk that steps up or down until it reaches a boundary, every one of those questions gets a definite answer. Some answers will turn out to be *wrong*, and that is useful: a model that can be wrong is a model we can learn from. This is the central message of the course:

> Formal models force us to make our assumptions explicit, produce precise predictions, and turn "Is this theory right?" into "How well, and why, does this model account for the data?"

Where is this used? Models like the random walk of this week are not only classroom examples. They are used every day in research and practice:

- Clinical psychology and psychiatry. Patients with depression, ADHD or anxiety are often "slower" in simple tasks. Decision models show *why*: lower evidence quality, more caution, or slower motor responses. These three explanations call for different interventions.
- Ageing research. Older adults respond more slowly. Fitting decision models showed that much of this slowing comes from more caution and slower motor execution, not from worse information processing.

- Neuroscience. Neurons in some brain areas increase their firing rate until a threshold before a decision, much like a random walk reaching a boundary. Model parameters are used to link behavior and brain activity.
- Applied settings. Eyewitness decisions, medical image reading and quality control all involve deciding under noisy evidence. The same model separates "how good is the evidence" from "how careful is the decision maker".
- Artificial intelligence. Ideas from cognitive models (evidence accumulation, learning from errors, probabilistic inference) are used in AI systems, and AI models are now tested with the methods of this course (see the AI & LLMs section).

1.1.4 Lecture notes

In plain words A *theory* says something in words ("people collect evidence until they have enough"). A *model* says the same thing so precisely that a computer can run it. Once it can run, the model makes exact predictions: how often people will be correct, how long they will take, and how both change when we change the task. This week's model is the simplest possible decision model. Imagine a token on a number line. Each moment it moves one step up (a bit of evidence for the correct answer) or one step down (a bit of evidence for the wrong answer). When it reaches the top line, the person answers correctly; when it reaches the bottom line, they make an error. Only three numbers control everything: how strongly the steps lean upward (drift, the quality of the evidence), how far apart the two lines are (boundary separation, how careful the person is), and where the token starts (starting point, bias). By running this little machine many times we can see a speed–accuracy trade-off appear by itself, without programming it in.

Psychology words in plain language

- Participant (subject): a person who takes part in an experiment.
- Stimulus (plural stimuli): whatever is shown or played to the participant, e.g. a picture, a word or a tone.
- Task: what the participant must do, e.g. "press left if the dots move left, right if they move right". A two-choice task has exactly two possible responses.
- Trial: one repetition of the task: one stimulus, one response. An experiment usually has hundreds of trials per participant.
- Response time (RT, reaction time): the time from stimulus onset to the key press, usually in milliseconds (ms).
- Accuracy: the proportion of correct responses.
- Condition / manipulation: a change the experimenter makes on purpose (e.g. "be fast!" vs. "be accurate!" instructions, or easy vs. hard stimuli), to see how behavior changes.
- Construct: a psychological quantity we cannot observe directly, e.g. *caution*, *attention* or *memory strength*. A good model parameter *measures* a construct.
- Behavioral data: what we can record: choices and RTs (not brain signals).

Suggested timing (60 min): motivation 10 · what and why 10 · kinds of models + Marr 15 · models as data-generating machines + pipeline 10 · first model (random walk) 12 · notation and course logistics 3.

1. What is a cognitive model?

A cognitive model is an explicit, usually mathematical or computational, description of the processes or representations that are assumed to produce behavior. It has three ingredients:

Ingredient	Question it answers	Random walk example
Assumptions / structure	What mechanism do we posit?	Evidence accumulates in discrete steps between two boundaries
Parameters	Which quantities are free to vary across people or conditions?	Drift v, boundary separation a, start point z
Mapping to data	How does the mechanism produce observable responses?	The boundary reached gives the choice; the number of steps gives the RT

Three related terms are easy to mix up:

- A theory is a set of verbal (often informal) claims about the mind.
- A model is one particular formal instantiation of a theory. One theory can be formalized in many different ways.
- A statistical model (a t-test, a regression) is also a model, but of the *data*. Its parameters (means, slopes) usually describe the data rather than a cognitive process. In a cognitive model the parameters are meant to measure latent psychological quantities, such as memory decay, caution or attention.

George Box's often-quoted remark that "all models are wrong, but some are useful" sums up the attitude of this course. We never ask whether a model is *true*. We ask whether it is *useful*: does it capture the important regularities, generalize to new data, and have interpretable parameters?

2. Why bother with formal models?

1. Precision. Writing the model down exposes hidden assumptions (as in the motivation example).
2. Quantitative predictions. Models predict *how much*, not just *whether*.
3. Deriving surprising consequences. Our intuitions about complex systems are poor. For example, averaging individual exponential learning curves produces a curve that looks like a *power* law (Week 5).

4. Measurement. Fitted parameters are measurements of latent constructs. d' in signal detection theory (Week 2) measures sensitivity separately from response bias, which raw accuracy cannot do.
5. Comparing theories. Two verbal theories that "both explain the data" can be pitted against each other once they are formalized (Weeks 6–7, 10).
6. Bridge to AI. The same formalisms (probabilistic models, learning rules, neural networks) are used to build artificial agents (Week 15).

3. Kinds of cognitive models

By purpose:

Type	Question	Example	Course weeks
Descriptive	*What* is the shape of the data?	Power law of practice; exponential forgetting curve	2–7
Process (mechanistic)	*How* does the mind produce the behavior?	Random walk / drift-diffusion; exemplar model (GCM)	1, 14
Normative (rational)	What *should* an ideal agent do?	Bayesian ideal observer for cue combination	15

The boundaries are fuzzy: an SDT model is partly descriptive and partly a process model.

By family:

Family	Core idea	Example
Symbolic / rule-based	Cognition as manipulation of symbols and production rules	ACT-R production systems
Connectionist	Distributed representations, learning by weight change	Delta-rule networks, deep networks
Stochastic / dynamical	Noisy processes unfolding in time	Random walk, drift-diffusion, Rescorla–Wagner
Bayesian (probabilistic)	Cognition as probabilistic inference	Bayesian observer models

> Watch out: "Bayesian modeling" in this course means two different things. (a) Bayesian models of cognition claim that the *mind* does Bayesian inference (Week 15). (b) Bayesian data analysis is how *we, the scientists,* estimate the parameters of any model, Bayesian or not (Weeks 9–13). Weeks 9–13 are mostly about (b).

4. Marr's three levels of analysis

David Marr proposed analyzing an information-processing system at three complementary levels:

Level	Question	Decision example
Computational	What problem is being solved, and why?	Choose the correct alternative while balancing speed and accuracy
Algorithmic / representational	What representations and procedures solve it?	Accumulate noisy evidence until a boundary is reached
Implementational	How is it realized physically?	Firing rates of neurons ramping up to a threshold

Normative models usually live at the computational level, process models at the algorithmic level, and computational neuroscience models at the implementational level. Most models in this course are algorithmic-level models.

5. A model is a data-generating machine

The single most important idea for the rest of the course: a model specifies a probability distribution over possible data.

$$\mathrm{y} \sim p(\mathrm{y} \mid \boldsymbol{\theta}, \mathcal{M})$$

Read this as: "under model $\mathcal{M}$ with parameters $\boldsymbol{\theta}$, the data y are generated from the distribution p." Many models split this into a deterministic core plus noise, for example $y_i = f(x_i; \boldsymbol{\theta}) + \varepsilon_i$ (Week 2). Others, like the random walk, are stochastic all the way through.

This gives two directions of reasoning:

Direction	Given	Find	Name	Course weeks
Forward	$\boldsymbol{\theta}$	Possible data y	Simulation / prediction	1–2
Inverse	Observed y	Plausible $\boldsymbol{\theta}$	Parameter estimation / fitting	3–5, 9–13

The inverse problem is harder. It is what we mean by "fitting a model to data". Because a model can always be run forwards, we can also test our fitting methods: simulate data from known parameters $\boldsymbol{\theta}^*$, fit, and check whether we get $\boldsymbol{\theta}^*$ back (*parameter recovery*, Week 7).

Reading the math $\mathrm{y} \sim p(\mathrm{y} \mid \boldsymbol{\theta}, \mathcal{M})$ is read: "the data y are drawn from the probability distribution p, given the parameter values $\boldsymbol{\theta}$ and the model $\mathcal{M}$."

- y (bold) is a *list* of observations, e.g. the 200 responses of one participant.
- The symbol $\sim$ means "is randomly generated from". It reminds us that the same model with the same parameters gives *different* data each time, just as a person does not respond identically on every trial.
- The vertical bar $\mid$ means "given" or "assuming". Everything to the right of it is fixed: the model's equations ($\mathcal{M}$) and the numbers we plug into them ($\boldsymbol{\theta}$).
- $\boldsymbol{\theta}$ (theta, bold) collects all parameters, e.g. $\boldsymbol{\theta} = (v, a, z)$ for the random walk.

Example. For a coin with $P(\text{heads}) = 0.6$, the model is "each flip is heads with probability θ", $\boldsymbol{\theta} = 0.6$, and y could be (H, T, H, H, T). Run it again and you get another sequence. *Forward* direction: choose θ, simulate y. *Inverse* direction (fitting, Weeks 3–5): see y, find a good θ.

6. The modeling pipeline (our roadmap)

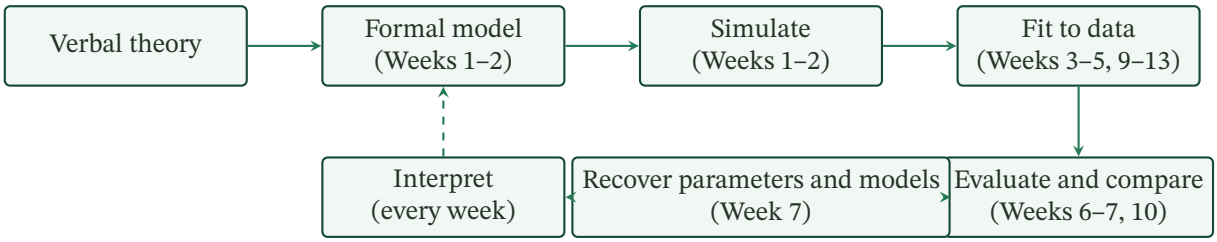


Figure 1.1: The cognitive modeling pipeline from Week 1

Every lab in this course runs some version of this loop. Today we focus on formalize $\rightarrow$ simulate $\rightarrow$ interpret, with a small preview of fit.

7. Our first model: a random walk to a boundary

Verbal $\rightarrow$ formal. Evidence is a single number X that starts at z. At each time step it moves up by 1 (evidence for the correct response) or down by 1 (evidence for the error response). The walk stops when X reaches the upper boundary a (correct response) or the lower boundary 0 (error).

$$X_0 = z, \qquad X_n = X_{n-1} + \delta_n, \qquad \delta_n = \begin{cases} +1 & \text{with probability } \frac{1+v}{2} \\ -1 & \text{with probability } \frac{1-v}{2} \end{cases}$$

- $v \in (-1, 1)$ is the drift, the average step $\mathbb{E}[\delta_n] = v$. It represents the quality of evidence: easy stimuli give a large v.
- a is the boundary separation. It represents caution: a wider gap needs more evidence.

- z is the start point ($0 < z < a$). With $z = a/2$ the decision maker is unbiased.
- The number of steps until absorption, T, maps onto RT through

$$\text{RT} = t_0 + T\,\Delta t,$$

  where t_0 is non-decision time (encoding plus motor execution) and Δt is the duration of one step.

Analytic predictions. Let $r = \frac{1-v}{1+v}$. For $v \neq 0$, the probability of a correct response and the mean number of steps are:

$$P_c = \frac{1 - r^z}{1 - r^a}, \qquad \mathbb{E}[T] = \frac{a\,P_c - z}{v}.$$

These two equations already settle the questions from the motivation:

- Increasing a raises P_c and $\mathbb{E}[T]$: a speed–accuracy trade-off.
- Increasing v raises P_c but lowers $\mathbb{E}[T]$: easier stimuli are both faster and more accurate.

So the two manipulations leave different signatures in the data. That is what lets us *measure* caution and evidence quality separately.

> Advanced: where the formulas come from. P_c is the classic *gambler's ruin* result. Let $h(x)$ be the probability of hitting a before 0 when starting from x. Conditioning on the first step gives $h(x) = \frac{1+v}{2}h(x+1) + \frac{1-v}{2}h(x-1)$ with $h(0) = 0$ and $h(a) = 1$. Solving this difference equation gives $h(x) = (1 - r^x)/(1 - r^a)$. For $\mathbb{E}[T]$, use *Wald's identity*: the final position satisfies $\mathbb{E}[X_T] = z + v\,\mathbb{E}[T]$. Since $X_T = a$ with probability P_c and $X_T = 0$ otherwise, $\mathbb{E}[X_T] = aP_c$, and rearranging gives $\mathbb{E}[T] = (aP_c - z)/v$. As the step size and Δt shrink together, the random walk becomes the drift-diffusion model (Week 14).

> Advanced: a property that turns out to be a weakness. With an unbiased start ($z = a/2$), the simple random walk predicts that correct and error responses have identical RT distributions. Empirically, errors are often *slower* than correct responses when accuracy is stressed, and sometimes *faster* under speed stress. The simple model cannot produce either pattern. Adding trial-to-trial variability in drift or start point can (lab Exercise 5). Finding where a model fails is exactly how models improve.

Reading the math The update rule. $X_n = X_{n-1} + \delta_n$ says: *the evidence now = the evidence one step ago + one new step*. The step δ_n is $+1$ or -1. The probability of $+1$ is $\frac{1+v}{2}$. With $v = 0$ this is 0.5 (a fair coin: no information). With $v = 0.2$ it is 0.6, so the walk drifts upward on average. The average step is $(+1)(0.6) + (-1)(0.4) = 0.2 = v$, which is why v is called the drift.

The accuracy formula. $P_c = \dfrac{1 - r^z}{1 - r^a}$ with $r = \dfrac{1 - v}{1 + v}$.

- r is the ratio "probability of a down step / probability of an up step". With $v = 0.2$: $r = 0.4/0.6 = 2/3$. A small r means the walk almost always goes up.
- r^z and r^a are small numbers when z and a are large, so P_c approaches 1. That is the math behind "more caution (larger a) gives higher accuracy".
- You do not need to derive the formula (see the Advanced box). It is enough to plug in numbers and check it against a simulation, as in the lab.

The time formula. $\mathbb{E}[T] = \dfrac{aP_c - z}{v}$. $\mathbb{E}[\cdot]$ means "average over many trials". The numerator is how far the walk travels on average (it ends at a with probability P_c, at 0 otherwise, and started at z). Dividing by the average speed v gives the average number of steps, just like *time = distance / speed*.

RT. RT $= t_0 + T\,\Delta t$: a fixed part t_0 (seeing the stimulus, pressing the key) plus the decision time, i.e. the number of steps times the duration of one step.

For an engineer You already know this model under another name.

- Sequential detection. The random walk is a discrete version of Wald's sequential probability ratio test (SPRT): accumulate the log-likelihood ratio of the observations, and decide as soon as it crosses an upper or a lower threshold. Under some assumptions the SPRT gives the smallest average number of observations for given error probabilities, which is one reason the brain might use this strategy.
- Drift ≈ signal-to-noise ratio. Each step is "signal + noise". The drift v is the mean of a step and the randomness of the step is the noise, so v plays the role of the SNR per sample. A cleaner stimulus is a higher SNR.
- Boundary ≈ decision threshold. Raising the threshold lowers the error probability but increases the detection delay: the classic trade-off between error rate and latency in sequential detection. In psychology this is called the speed–accuracy trade-off.
- Start point ≈ prior. Starting closer to one boundary is like using a non-zero initial log-likelihood ratio, i.e. a prior in favor of one hypothesis.
- Non-decision time ≈ fixed processing delay. Sensor/encoding latency plus actuator (motor) latency, added to every response.
- What is new for an engineer: in engineering we *design* the detector. Here we *observe* a human detector and try to infer its parameters (SNR, threshold) from its decisions and decision times. That inverse problem is the subject of the whole course.

8. Course-wide notation

This table is used in all weeks. When a later week introduces a symbol, it uses the meaning given here.

Data and design

Symbol	Meaning	First used
y_i, $\mathrm{y} = (y_1, \ldots, y_N)$	Observation i; vector of all observations	Wk 1
N	Number of observations (trials)	Wk 1
x_i	Experimental condition / independent variable of observation i	Wk 2
$j = 1, \ldots, J$	Participant index; number of participants	Wk 5

Models and parameters

Symbol	Meaning	First used
$\mathcal{M}$, $\mathcal{M}_1, \mathcal{M}_2$	A model; competing models	Wk 1
$\boldsymbol{\theta}$	Vector of free parameters	Wk 1
k	Number of free parameters	Wk 1
$\boldsymbol{\theta}^*$	True (data-generating) parameter values in a simulation	Wk 1
$\hat{\boldsymbol{\theta}}$	Estimate of $\boldsymbol{\theta}$	Wk 3
$f(x; \boldsymbol{\theta})$	Deterministic part of a model's prediction	Wk 2
$\hat{y}_i$	Model prediction for observation i	Wk 3
ε_i	Noise (error) term	Wk 2

Probability

Symbol	Meaning
$P(\cdot)$	Probability of an event
$p(\cdot)$	Probability mass or density function
$y \sim \mathcal{D}$	"y is distributed as $\mathcal{D}$"
$\mathcal{N}(\mu, \sigma)$	Normal distribution with mean μ and standard deviation σ (as in scipy and PyMC)
Bernoulli(θ), Binomial(n, θ)	Bernoulli and binomial distributions with success probability θ
Beta(a_0, b_0), Uniform(l, u)	Beta and uniform distributions
Φ, Φ^{-1}	Standard normal CDF and its inverse
$\mathbb{E}[\cdot]$, Var$[\cdot]$	Expectation and variance

Fitting, evaluation and Bayesian inference

Symbol	Meaning	First used
SSE, MSE, RMSD	Sum of squared errors, mean squared error, root mean squared deviation	Wk 3
$L(\boldsymbol{\theta}) = p(\mathrm{y} \mid \boldsymbol{\theta}, \mathcal{M})$	Likelihood	Wk 4
$\ell(\boldsymbol{\theta}) = \ln L(\boldsymbol{\theta})$; NLL $= -\ell$	Log-likelihood; negative log-likelihood	Wk 4
K	Number of cross-validation folds (not the same as k!)	Wk 6
AIC, BIC	Akaike and Bayesian information criteria	Wk 7
$p(\boldsymbol{\theta})$, $p(\boldsymbol{\theta} \mid \mathrm{y})$	Prior; posterior	Wk 9
$p(\mathrm{y} \mid \mathcal{M})$	Marginal likelihood (evidence)	Wk 10
BF_{12}	Bayes factor for $\mathcal{M}_1$ over $\mathcal{M}_2$	Wk 10
$\boldsymbol{\theta}^{(s)}$, $s = 1, \ldots, S$	s-th MCMC draw; number of draws	Wk 11
C	Number of MCMC chains	Wk 12
$\hat{R}$, ESS	Potential scale reduction factor (R-hat); effective sample size	Wk 12

Model-specific symbols

Model	Symbols	First used
Random walk / drift-diffusion	v drift; a boundary separation; z start point; t_0 non-decision time; Δt step duration; T number of steps	Wk 1, 14
Signal detection theory	H hit rate; F false-alarm rate; d' sensitivity; c criterion	Wk 2
Forgetting curves	t retention interval; A initial level; λ exponential decay rate; d power-law decay exponent	Wk 2
GCM (categorization)	w_m attention weights; c sensitivity	Wk 14
Reinforcement learning	α learning rate; β inverse temperature; $Q_t(o)$ value of option o on trial t	Wk 15

Overloaded symbols (kept because the literature uses them): c is the SDT criterion except in Week 14, where it is the GCM sensitivity. t is the retention interval in forgetting models and the trial index in RL. α is only used as the RL learning rate. We never use α for a significance level.

For the instructor

- Start with the questions, not the model. Read the verbal theory aloud and ask students to answer the four questions of the Motivation section. They will disagree. That disagreement is the reason formal models exist.
- Act out the random walk. Draw a vertical line on the board with a top line ("correct") and a bottom line ("error"). Let a student roll a die: 1–4 means up, 5–6 means down ($v \approx 0.33$). Count steps until a line is reached. Do this three times and you have three trials with different RTs. Then move the lines further apart and repeat.
- Keep the math light in this week. Students only need to *use* the formulas for P_c and $\mathbb{E}[T]$, not derive them. Show that the numbers match the simulation in the lab: that is the main message (simulation and analysis agree).
- Typical student questions.
  - *"Is evidence really discrete steps?"* No. It is a simplification; Week 14 shows the continuous version (drift-diffusion), which gives similar predictions.
  - *"Why not just compare mean RTs with a t-test?"* A t-test says *that* groups differ; the model says *how* (caution vs. evidence quality).
  - *"Which model is the true one?"* None. All models are simplifications; we ask which one is useful (Box's quote).
- Time management. Sections 3–4 (model types, Marr) can be shortened to 10 minutes if needed. Do not skip Section 7: everything in the lab depends on it.

1.1.5 Worked example

Setup. An unbiased decision maker with drift $v = 0.2$, boundary separation $a = 6$ and start point $z = 3$. Non-decision time $t_0 = 300$ ms; each step lasts $\Delta t = 20$ ms.

Step 1: step probabilities. $P(\delta = +1) = (1 + 0.2)/2 = 0.6$ and $P(\delta = -1) = 0.4$.

Step 2: simulate one trial by hand. Draw uniform random numbers u_n. Step up if $u_n < 0.6$, otherwise step down.

Step n	u_n	δ_n	X_n
0	—	—	3
1	0.31	+1	4
2	0.72	−1	3
3	0.55	+1	4

Step n	u_n	δ_n	X_n
4	0.12	+1	5
5	0.48	+1	6 = a $\rightarrow$ stop

The walk hits the upper boundary: a correct response after $T = 5$ steps, so RT $= 300 + 5 \times 20 = 400$ ms.

Step 3: analytic accuracy. $r = (1 - 0.2)/(1 + 0.2) = 2/3$, so $r^3 = 8/27 \approx 0.296$ and $r^6 = 64/729 \approx 0.088$.

$$P_c = \frac{1 - 8/27}{1 - 64/729} = \frac{1}{1 + 8/27} = \frac{27}{35} \approx 0.771.$$

(When $z = a/2$, the formula simplifies to $P_c = 1/(1 + r^{a/2})$.)

Step 4: analytic mean RT.

$$\mathbb{E}[T] = \frac{6 \times 0.771 - 3}{0.2} = \frac{57/35}{0.2} \approx 8.14 \text{ steps}, \qquad \mathbb{E}[\text{RT}] = 300 + 8.14 \times 20 \approx 462.9 \text{ ms}.$$

Step 5: two manipulations, two signatures.

Condition	v	a ($z = a/2$)	P_c	$\mathbb{E}[T]$	$\mathbb{E}[\text{RT}]$ (ms)
Baseline	0.2	6	0.771	8.14	462.9
“Be accurate!” (more caution)	0.2	10	0.884	19.18	683.6
Easier stimulus (better evidence)	0.3	6	0.865	7.30	446.0

Interpretation. Accuracy instructions buy about 11 percentage points of accuracy at a cost of roughly 220 ms. Easier stimuli give a *similar* accuracy gain while making responses slightly *faster*. A researcher who looked only at accuracy could not tell the two situations apart. The model, by using accuracy *and* RT together, attributes them to different psychological causes (caution vs. evidence quality).

1.1.6 Lab (2 hours)

Goals

1. Install a working course environment on your own laptop, *including* the Bayesian tools needed in Week 13.
2. Practice the basic numpy/matplotlib workflow: random numbers, vectorization, plotting.
3. Implement the random walk model, simulate an experiment, and verify the simulation against the analytic formulas.
4. Use the model to interpret a speed–accuracy trade-off.
5. Get a first taste of fitting: recover the hidden parameters of a "mystery participant", evaluate the fit, and interpret the result.

Timing: Part A 35 min · Part B 15 min · Part C 30 min · Part D 15 min · Part E 25 min. Exercises not finished in the lab are finished at home.

Part A: Environment setup (35 min)

We use Miniforge, a free installer for `conda` that uses the community `conda-forge` channel. It is the most reliable way to install PyMC on Windows, macOS and Linux.

1. Download and install Miniforge from the conda-forge website (choose your operating system; keep the default options).
2. Save the following file as `environment.yml` in your course folder:

```
# Cognitive Modeling course environment (same as environment.yml in the
↪  course repository)
name: cogmod
channels:
  - conda-forge
dependencies:
  - python=3.12
  - numpy>=1.26
  - scipy>=1.11
  - pandas>=2.1
  - matplotlib>=3.8
  - jupyterlab
  - pymc>=6.3    # Bayesian modeling (needed from Week 13)
  - arviz>=1.3   # MCMC diagnostics
```

3. Open the Miniforge Prompt (Windows) or a terminal (macOS/Linux), go to your course folder, and create the environment. This takes 5–15 minutes the first time.

```
conda env create -f environment.yml
```

```
conda activate cogmod
```

4. Save this script as `check_install.py` and run it with `pythoncheck_install.py`. Add `--sample` to also run a tiny PyMC model; the first run takes about a minute while PyMC compiles.

> Code · کد: `code/check_install.py` (save and run it as a separate script · به‌صورت یک اسکریپت جدا اجرا شود)

5. Start JupyterLab with `jupyterlab` and create a notebook called `week01_lab.ipynb`. Run all remaining code in that notebook, one block per cell.

> If the check prints a warning like "g++ not available": PyMC still works but runs much more slowly. This usually means PyMC was installed with `pip` instead of from the conda-forge environment above. The conda-forge installation normally brings the compiler that PyMC's backend uses [verify on a Windows lab machine before Week 1]. Recreate the environment with `condaenvcreate-fenvironment.yml`.

> If installation fails: do today's lab in Google Colab, where numpy, scipy, pandas and matplotlib are preinstalled, and fix the local install with the instructor before Week 3. The PyMC install must work by Week 12.

Part B: Python warm-up (15 min)

> Code · کد: `code/week01.py`, cell 1 · سلول 1

Expected output: a proportion close to 0.6 (0.574 with this seed) and a running-proportion curve that wobbles at first and then settles near the dashed line.

Part C: Implement, simulate and verify the random walk (30 min)

> Code · کد: `code/week01.py`, cell 2 · سلول 2

First, look at what the model does: plot a few evidence trajectories.

Code · کد: `code/week01.py`, cell 3 · سلول 3

Now simulate a whole experiment and compare the simulation with the analytic predictions. Always check a simulation against something you know; this habit will save you many times this term.

Code · کد: `code/week01.py`, cell 4 · سلول 4

Code · کد: `code/week01.py`, cell 5 · سلول 5

Expected output (seed 42):

```
Accuracy   simulated 0.764 | analytic 0.771
Mean steps simulated 8.27  | analytic 8.14
Mean RT    simulated 465.3 ms | analytic 462.9 ms

Mean RT by response (1 = correct, 0 = error):
        count   mean
choice
0        1180  466.1
1        3820  465.1
```

What to notice: - The simulation matches the analytic values to within sampling noise. - The RT distributions are right-skewed (a long tail of slow responses), which is the typical shape of empirical RT distributions. - Correct and error RTs are almost identical (see the Advanced box in Lecture §7).

Part D: Interpret, the speed–accuracy trade-off (15 min)

Code · کد: `code/week01.py`, cell 6 · سلول 6

Expected output: three curves rising from the bottom left toward 1.0. Moving along one curve (increasing a) trades speed for accuracy. Moving *between* curves (increasing v) improves accuracy at the same or lower RT. The table reproduces the worked example: e.g., $v = 0.2, a = 10$ gives accuracy 0.884 and mean RT 683.6 ms.

Discuss with your neighbor: an experimenter finds that a drug makes participants slower *and* more accurate. Which parameter would you suspect changed? What if the drug made them slower and *less* accurate?

Part E: A first fit, evaluation and interpretation (25 min)

Your instructor will give you `week01_mystery.csv`, the data of a "mystery participant" (1000 trials, columns `choice` and `rt`). If you do not have the file, the code below generates an equivalent dataset with parameters hidden behind a random seed. Don't peek: try to recover them.

We assume $t_0 = 300$ ms and $\Delta t = 20$ ms are known and $z = a/2$. Only v and a are free ($k = 2$).

Code · کد: `code/week01.py`, cell 7 · سلول 7

Fitting by grid search. For every combination of v and a on a grid, compute the predicted accuracy and mean RT and measure how far they are from the observed values. Each difference is expressed in standard-error units so that accuracy and RT are on a comparable scale. Week 3 develops this idea properly.

Code · کد: `code/week01.py`, cell 8 · سلول 8

Evaluating the fit. We fitted only two summary statistics (accuracy and mean RT). A stronger test is whether the model also reproduces something we did *not* fit: the shape of the RT distribution. We compare RT quantiles of correct responses in the data with those of a large simulation at the best-fitting parameters.

Code · کد: `code/week01.py`, cell 9 · سلول 9

Expected output (fallback dataset). Open only after you have run the fit.

```
Observed: accuracy = 0.836, mean RT = 746.4 ms (N = 1000)
Best fit: v = 0.15, a = 10 (pred. accuracy 0.819, pred. RT 725.7 ms,
↪  discrepancy 5.78)

   quantile  observed   model
0       0.1     440.0   440.0
1       0.3     520.0   520.0
2       0.5     640.0   640.0
3       0.7     800.0   800.0
4       0.9    1200.0  1160.0
```

The hidden values were $v^* = 0.16$ and $a^* = 10$. The grid search recovered a exactly and v to within one grid step (0.15). The small miss is not a bug: with 1000 trials, the observed accuracy and mean RT differ from their true values by sampling noise, so the best-fitting parameters do too. (Week 5 shows how to put an uncertainty interval around estimates like these.) RTs are multiples of 20 ms, which is why many quantiles match exactly. The model reproduces the shape of the distribution well, with only a slight underestimate of the slow tail (0.9 quantile).

Interpretation questions (write 2–3 sentences each in your notebook): 1. In psychological terms, what do the recovered v and a say about this participant? 2. The fit used only two numbers from the data. Why is the quantile comparison a more convincing check than the discrepancy value itself? 3. If this participant were re-tested under "be as fast as possible" instructions, which parameter should change and which should stay the same, according to the model?

Exercises (increasing difficulty)

Exercise 1 ⋆ (sampling noise). Simulate the baseline model ($v = 0.2$, $a = 6$) with $N = 20$, 100 and 1000 trials, five different seeds each. Make a table of the observed accuracies. Compare their spread with the standard error $\sqrt{P_c(1 - P_c)/N}$. *Expected:* with $N = 20$ accuracies can easily range from about 0.6 to 0.95. The spread shrinks roughly with $1/\sqrt{N}$.

Exercise 2 ⋆ (two conditions). Simulate a "speed" condition ($a = 4$) and an "accuracy" condition ($a = 10$) with the same $v = 0.2$, 1000 trials each. Report accuracy and mean RT per condition and plot both RT distributions. Which aspect of the RT distribution changes most: its minimum, its mode, or its tail?

Exercise 3 ⋆⋆ (response bias). Set $a = 8$ and $z = 5$ (start closer to the correct boundary). Compute P_c analytically and by simulation. Then compare the mean RT of correct and error responses. Explain the result in terms of the distance each boundary is from the start point. What would $z = 3$ do?

Exercise 4 ⋆⋆ (checking a model property). For $z = a/2$, verify by simulation (20 000 trials) that correct and error RT *quantiles* are equal to within sampling noise. Empirical data often show slower errors in difficult, accuracy-stressed conditions. Is the simple random walk falsified by such data? What would you change?

Exercise 5 ⋆⋆⋆ (Advanced: across-trial drift variability). Modify `simulate_trial` so that on every trial the drift is drawn as $v_i \sim \mathcal{N}(v, \eta)$, clipped to $[-0.95, 0.95]$. Use $v = 0.2$, $\eta = 0.2$, $a = 8$. Show that errors are now slower than correct responses (expected with 20 000 trials: accuracy ≈ 0.75, about 12 steps for correct vs. 14 steps for error responses), and explain why. *Hint:* which trials are most likely to end in an error, and how long do those trials take?

Lab check-off (counts toward the lab component)

Show the instructor or TA: (1) the output of `check_install.py` with all packages `[OK]`, (2) your speed–accuracy plot from Part D, and (3) your best-fit parameters and quantile plot from Part E.

1.1.7 Connection to AI & LLMs

Marr's levels for a chatbot. A large language model (LLM) can be described at all three of Marr's levels, and the levels can come apart just as they do for people. *Computational:* the model is trained to predict the next word (token) in a text. *Algorithmic:* a transformer network computes a score (a "logit") for every possible next token and turns the scores into probabilities. *Implementational:* matrix multiplications on GPUs. A claim like "the LLM understands the question" is usually made at the computational level, while the evidence (its outputs) is produced by the algorithm. Keep the levels apart when you read such claims.

A familiar decision rule. The final step of an LLM is a choice rule with a noise parameter, just like the cognitive models in this course. The probability of token i is a softmax with temperature T: $P(i) = e^{z_i/T} / \sum_j e^{z_j/T}$. A low T gives near-deterministic choices and a high T gives more random ones. This is the same rule as the softmax in Week 15, with $\beta = 1/T$.

Code · کد: `code/week01.py`, cell 10 · سلول 10

Expected output:

```
T = 0.2: P(A, B, C) = [0.993 0.007 0.   ], sampled shares = [0.994 0.006
↪  0.   ]
T = 1.0: P(A, B, C) = [0.652 0.24  0.108], sampled shares = [0.662 0.235
↪  0.103]
T = 2.0: P(A, B, C) = [0.497 0.301 0.202], sampled shares = [0.52  0.281
↪  0.199]
```

Try it (no API needed). - ⋆ Find the largest temperature at which "A" is still chosen at least 90% of the time. - ⋆ Is the LLM's temperature closer to the random walk's *noise* or to its *boundary* (Week 1, Part C)? Explain. - ⋆⋆ Discussion: someone says "the chatbot gave a different answer when I asked again, so it does not know the answer." Use this box to explain what else could be going on.

1.1.8 Common mistakes & misconceptions

1. "A model that fits well is true." Good fit is necessary but not sufficient. A flexible model can fit almost anything, including noise (Week 6). Fit says little until you know what the model *could not* have fitted.
2. "More parameters make a better model." More parameters make a model *more flexible*, which is not the same as better. We will penalize flexibility explicitly (AIC/BIC, cross-validation, Bayes factors).
3. Confusing simulation with fitting. Simulation goes from parameters to data; fitting goes from data to parameters. Students often "fit" by tweaking parameters until a plot looks nice. That is a legitimate *exploration* but not an estimation method.
4. Treating parameters as directly observable. v and a are *inferred* quantities whose values depend on the model's assumptions (e.g., that t_0 is known). Change the model and the "same" parameter may change meaning.
5. Forgetting random seeds. Unseeded simulations cannot be reproduced, and your results will change every time you run the notebook. Use `np.random.default_rng(seed)`.
6. Not verifying simulations. A bug in a simulator produces wrong conclusions that *look* plausible. Compare against an analytic result, a limiting case (e.g., $v \to 0$ should give $P_c \to z/a$), or a published value.
7. Mixing up the two meanings of "Bayesian". Bayesian models of cognition (a claim about the mind) vs. Bayesian data analysis (a method for scientists).
8. Treating Marr's levels as competing. They are complementary descriptions of the same system, not rival theories.

1.1.9 Check your understanding

1. In one sentence each, define a *theory*, a *cognitive model* and a *statistical model*. Answer A theory is a set of (usually verbal) claims about mental processes. A cognitive model is a formal, explicit instantiation of such claims that can generate predictions, with parameters that represent psychological quantities. A statistical model describes the structure of the data (e.g., group means, slopes) without committing to a cognitive mechanism.

2. Classify each as descriptive, process or normative: (a) RT $= A\,N^{-b}$ as a function of practice trials N; (b) the random walk model; (c) an ideal observer that combines visual and haptic cues weighted by their reliabilities. Answer (a) Descriptive: it summarizes the shape of the data without saying why. (b) Process: it specifies a mechanism that produces choices and RTs. (c) Normative: it specifies optimal behavior given the available information.

3. At which of Marr's levels does the random walk model mainly live? Give an example of a claim about the same decision at the implementational level. Answer The algorithmic/representational level: it describes a procedure (accumulate evidence to a boundary). An implementational claim would be, e.g., "the accumulated evidence is carried by the firing rate of neurons in a specific brain area, and the response is triggered when that rate reaches a threshold."

4. What is the difference between the forward and the inverse problem? Which one did we solve in Part E of the lab? Answer Forward: given parameters, generate (predict) data, i.e., simulation. Inverse: given data, infer the parameters, i.e., fitting. Part E solved an inverse problem (by grid search), but it relied on solving the forward problem many times, once per grid point.

5. Using the formulas, what happens to P_c when $v = 0$? (Hint: think about the limit, or about the symmetry of the walk.) Answer With no drift the walk is symmetric, and the probability of hitting the upper boundary equals z/a (the limit of the formula as $v \to 0$). With an unbiased start, $z = a/2$, accuracy is 0.5, i.e., chance. That is sensible: with no evidence the decision maker is guessing.

6. Two groups differ in accuracy (0.80 vs. 0.90). Why can't we tell from accuracy alone whether they differ in caution or in evidence quality? What extra data resolve this? Answer Both a larger boundary a and a larger drift v raise accuracy, so accuracy alone confounds them. RT resolves it: higher caution makes responses slower, while better evidence makes them faster (worked example, Step 5). Fitting the model to accuracy and RT jointly separates the two.

7. Why did we simulate data from *known* parameters before trusting the fitting procedure? Answer Only with known true values can we check whether the method returns the right answer (parameter recovery). If a method cannot recover parameters from data generated by the model itself, its estimates from real data are not interpretable. This is developed fully in Week 7.

1.1.10 Assignment

A0: Setup and "verbal theory audit" (ungraded, required; due at the start of Week 2)

A0 does not carry points, but it is required: an incomplete A0 must be revised before the Week 2 lab check-off can be awarded.

Part 1: Environment (required). Submit the text output of `pythoncheck_install.py--sample` showing all packages `[OK]` and the PyMC posterior mean. If PyMC fails, submit the error message and the steps you tried; the TA will help.

Part 2: Verbal theory audit (max. 1 page). Choose one verbal theory from your earlier courses, for example "levels of processing", "attention as a spotlight", "practice makes perfect", "dual-process theories of reasoning" or "interference causes forgetting".

1. State the theory in 1–2 sentences.
2. List at least five decisions the theory leaves unspecified that a formal model would have to make (e.g., functional form, what is noisy, what the free parameters are, how internal states map onto the measured behavior).
3. For two of these decisions, propose a concrete formal choice (an equation or an algorithmic step).
4. Describe one prediction that would differ depending on which choice is made, and an experiment that could tell them apart.

Feedback criteria (pass / revise):

Criterion	Pass	Revise
Environment	All packages `[OK]`, or a clear error report with attempted fixes	No output submitted
Underspecification	≥ 5 distinct, genuinely unspecified decisions	Fewer than 5, or restatements of the theory
Formalization	Two concrete, unambiguous formal choices	Vague ("use a function")
Diverging prediction	A specific prediction plus an experiment that distinguishes the choices	No prediction, or a prediction that does not depend on the choice

The best audits will be discussed briefly at the start of the Week 2 lecture.

1.1.11 Further reading

- Farrell & Lewandowsky (2018), chapters 1–2. Introduction to computational modeling and the move from verbal theories to formal models. *(Main reading for this week.)*

- Verguts (2022), introductory chapter [verify]. A short, accessible motivation for modeling cognitive processes.
- Anderson (2014), introductory chapter [verify]. Places cognitive modeling alongside computational neuroscience, useful context for Marr's implementational level.
- *Optional classic (outside the course list):* Marr, D. (1982). *Vision.* W. H. Freeman. The original source of the three levels of analysis.

Preview for Week 2: we turn signal detection theory and forgetting curves into Python models and study how each parameter shapes the predictions. Assignment A1 will be released.

1.2 Week 2: Anatomy of a Model: Signal Detection and Forgetting Curves

> Chapter 1: Fundamentals of cognitive modeling · Lecture 1 h · Lab 2 h This week's running models: equal-variance Gaussian signal detection theory (SDT); exponential and power forgetting curves. Both return in Weeks 3–13.

1.2.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان
signal detection theory (SDT)	نظریه‌ی تشخیص علامت	مدلی که حساسیت را از سوگیری پاسخ جدا می‌کند
exponential / power function	توانی / تابع نمایی	دو شکل رایج برای منحنی فراموشی و یادگیری
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
deterministic core	هسته‌ی قطعی	بخش بدون تصادف مدل که پیش‌بینی میانگین را می‌دهد
noise	نویز	تغییرات تصادفی در شواهد یا در داده
linking assumption	فرض پیوند	اینکه کمیت‌های مدل چطور به رفتار قابل مشاهده وصل می‌شوند
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز

English	فارسی	توضیح کوتاه
hit	اصابت (hit)	پاسخ «بله» وقتی علامت وجود دارد
response bias	سوگیری پاسخ	تمایل به یک پاسخ، مستقل از محرک
binomial distribution	توزیع دوجمله‌ای	توزیع تعداد موفقیت در n کوشش
noise model	مدل نویز	توزیع داده حول پیش‌بینی مدل
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
ROC / zROC curve	منحنی ROC / zROC	نرخ hit در برابر نرخ false alarm با تغییر ملاک
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
criterion (c)	ملاک	آستانه‌ی تصمیم؛ نشان‌دهنده‌ی سوگیری پاسخ
formal model / formalize	فرمال کردن / مدل فرمال	نوشتن نظریه به شکل معادله یا الگوریتم
trial	کوشش (trial)	یک بار انجام تکلیف توسط شرکت‌کننده
false alarm	هشدار کاذب (false alarm)	پاسخ «بله» وقتی علامت وجود ندارد
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
miss / correct rejection	رد درست / از دست دادن	پاسخ «نه» / پاسخ «نه» با وجود علامت بدون علامت
liberal / conservative	محافظه‌کار / آسان‌گیر	تمایل به گفتن «بله» / «نه»
free parameter	پارامتر آزاد	پارامتری که از روی داده برآورد می‌شود
verbal theory	نظریه‌ی کلامی	نظریه‌ای که فقط با کلمات بیان شده و پیش‌بینی دقیق نمی‌دهد

1.2.2 Learning outcomes

By the end of this week, students will be able to:

1. Decompose a cognitive model into its components: design, free and fixed parameters, deterministic core, noise (data) model, and linking assumptions.
2. Compute d' and c from a 2×2 table of counts, including a correction for hit or false-alarm rates of 0 or 1, and explain why d' separates sensitivity from bias.
3. Implement exponential and power forgetting models with a binomial noise model, and predict qualitatively how each parameter changes the curves.
4. Use simulation and diagnostic plots (ROC/zROC, semi-log and log-log plots) to evaluate whether a model's qualitative signature appears in data.
5. Interpret fitted parameters in psychological terms and identify the limits of the "fit a straight line to transformed data" approach.

1.2.3 Motivation & big picture

Two radiologists. Radiologist R1 flags 93% of scans that really contain a tumor, but also flags 31% of healthy scans. Radiologist R2 flags only 69% of tumors, but just 7% of healthy scans. Their percent correct is identical (81%). Who is the better diagnostician?

Percent correct cannot answer this, because it mixes two different things: how well a person can tell tumors from healthy tissue (*sensitivity*), and how willing they are to say "tumor" (*bias*). Signal detection theory is a small model with two parameters that pulls these apart. As the worked example will show, R1 and R2 have essentially the same sensitivity; they differ only in where they set their criterion. This is the first payoff of modeling: model parameters can measure latent constructs that raw scores conflate.

Ebbinghaus's curve. Since Ebbinghaus's experiments on himself in the 1880s, we know that forgetting is fast at first and then slows down. "Fast, then slow" is a verbal description that both an exponential and a power function satisfy. Yet they imply different things: an exponential means a memory loses a *constant proportion* of its strength per day, at any age; a power function means older memories become *more resistant*. Choosing between them is a question about the nature of memory, and it requires a formal model.

Where is this used?

- Medical diagnosis. Radiologists, pathologists and screening programs are evaluated with SDT and ROC curves, so that a doctor who "sees better" can be told apart from a doctor who simply says "abnormal" more often.
- Memory research. In a recognition test, studied words are "signal" and new words are "noise". d' measures memory strength separately from the tendency to say "old".
- Eyewitness identification. Police line-up procedures are compared with ROC analysis: some procedures do not improve accuracy, they only make witnesses more cautious.
- Perception and hearing tests. Thresholds for seeing or hearing faint stimuli are measured with SDT, so that a cautious listener is not mistaken for a hard-of-hearing one.
- Machine learning. Every binary classifier (spam filter, fraud detector, AI-text detector) has a hit rate and a false-alarm rate, and is evaluated with ROC curves and the area under the curve (AUC). This is the same theory.
- Forgetting curves are used to schedule learning: spaced-repetition apps (e.g. for vocabulary) decide when to show an item again from a model of how fast it is forgotten.

1.2.4 Lecture notes

In plain words This week has one big idea: a model has parts, and each part is a separate assumption. We look at two small models. Signal detection theory explains yes/no decisions under uncertainty. Every trial gives the observer a noisy "feeling of signal". If the feeling is stronger than a personal threshold, the observer says "yes". Two numbers describe the observer: how far apart the "signal" and "no signal" feelings are on average (sensitivity, d'), and where the threshold is (criterion, c). Two people can have the same percent correct for completely different reasons, and SDT shows which. The forgetting curve describes how recall drops with time. Two formulas, exponential and power, both look like "fast drop, then slow", but they say different things about memory. Choosing between them is the kind of question the rest of the course teaches us to answer with data.

Statistics words in plain language

- Distribution: a description of which values a random quantity takes and how often. A histogram of many measurements approximates it.
- Normal (Gaussian) distribution $\mathcal{N}(\mu, \sigma)$: the bell curve, with mean μ (center) and standard deviation σ (width). About 68% of values fall within $\mu \pm \sigma$ and 95% within $\mu \pm 1.96\sigma$.
- Standard normal: the normal distribution with $\mu = 0$, $\sigma = 1$. A value measured on this scale is called a z-score: "how many SDs above the mean".
- $\Phi(x)$ (CDF): the probability that a standard normal value is *below* x. $\Phi(0) = 0.5$, $\Phi(1) \approx 0.84$, $\Phi(1.96) \approx 0.975$. In Python: `norm.cdf(x)`.
- $\Phi^{-1}(p)$ (inverse CDF, "z-transform"): the opposite question: below which value lies a proportion p? $\Phi^{-1}(0.84) \approx 1$. In Python: `norm.ppf(p)`.
- Binomial distribution Binomial(n, p): the number of successes in n independent tries, each with success probability p (e.g. words recalled out of 50). Its mean is np and its variance $np(1-p)$.
- Proportion vs. probability: a proportion is what we *observe* (30 of 50 recalled = 0.60); a probability is the model's *underlying* value that the proportion estimates.

Suggested timing (60 min): anatomy of a model 15 · SDT 20 · forgetting curves 15 · simulation as a thinking tool + preview 10.

1. The anatomy of a model

In Week 1 we wrote a model as a data-generating machine, $y \sim p(y \mid \boldsymbol{\theta}, \mathcal{M})$. In practice, almost every model you will meet in this course can be taken apart into five pieces:

Component	Question	SDT	Forgetting (exponential)
1. Design x_i	What is manipulated and measured?	Trial type (signal/noise); response yes/no	Retention interval t_i; number of items recalled out of n

Component	Question	SDT	Forgetting (exponential)
2. Parameters $\boldsymbol{\theta}$	What is free to vary? What is fixed?	Free: d', c. Fixed: both SDs = 1, noise mean = 0	Free: A, λ. Fixed: n
3. Deterministic core $f(x;\boldsymbol{\theta})$	What does the model predict on average?	$H = \Phi(d'/2 - c)$, $F = \Phi(-d'/2 - c)$	$f(t) = Ae^{-\lambda t}$
4. Noise model	How do observations scatter around the prediction?	Hits $\sim$ Binomial(n_s, H); false alarms $\sim$ Binomial(n_n, F)	Recalled $\sim$ Binomial$(n, f(t))$
5. Linking assumptions	How do model quantities map onto behavior?	"Yes" if internal evidence exceeds the criterion	Recall probability equals memory strength $f(t)$

Some points about each component:

- Free vs. fixed parameters. Fixed parameters are not "unimportant". They are *assumptions*. In Week 1 we fixed $t_0 = 300$ ms. In SDT, fixing the noise mean at 0 and both standard deviations at 1 is what makes d' and c estimable at all: shifting or stretching the whole axis would not change any prediction. Such fixes are called scaling constraints.
- Parameter ranges. Every parameter has a meaningful range: $0 \leq A \leq 1$, $\lambda > 0$, $d' \geq 0$ for a non-perverse observer. Respecting these ranges matters when fitting (Week 3).
- The noise model is part of the model. Two researchers with the same $f(t)$ but different noise models (binomial vs. Gaussian) have *different* models and will get different parameter estimates (Week 4). The binomial is natural for counts: its variance $f(1-f)/n$ is largest at $f = 0.5$ and shrinks near 0 and 1.

2. Signal detection theory (SDT)

The task. On each trial a stimulus is either *signal* (e.g., a tumor, a studied word, a faint tone) or *noise*. The observer answers "yes" (signal) or "no". The outcomes form a 2×2 table:

	Respond "yes"	Respond "no"
Signal trial	Hit	Miss
Noise trial	False alarm (FA)	Correct rejection (CR)

with hit rate $H = \frac{\text{hits}}{\text{hits+misses}}$ and false-alarm rate $F = \frac{\text{FAs}}{\text{FAs+CRs}}$.

The model (intuition). Each trial produces an internal evidence value. It is noisy: even pure noise trials sometimes feel signal-like. Signal trials produce, on average, more evidence. The observer says "yes" whenever the evidence exceeds a criterion.

The model (formal), equal-variance Gaussian SDT:

$$\text{noise trials: } X \sim \mathcal{N}(0,1), \qquad \text{signal trials: } X \sim \mathcal{N}(d',1), \qquad \text{respond "yes" iff } X > k.$$

- d' is sensitivity: the distance between the two distributions in SD units.
- k is the absolute criterion location. It is more convenient to measure the criterion relative to the midpoint between the distributions: $c = k - d'/2$. With $c = 0$ the observer is unbiased, $c < 0$ is liberal (says "yes" readily), and $c > 0$ is conservative.

Predictions. Using the standard normal CDF Φ:

$$H = P(X > k \mid \text{signal}) = \Phi\left(\tfrac{d'}{2} - c\right), \qquad F = P(X > k \mid \text{noise}) = \Phi\left(-\tfrac{d'}{2} - c\right).$$

Inverting the model. Applying Φ^{-1} (often written z) to both equations and solving gives the familiar formulas:

$$d' = \Phi^{-1}(H) - \Phi^{-1}(F), \qquad c = -\tfrac{1}{2}\left[\Phi^{-1}(H) + \Phi^{-1}(F)\right].$$

Notice what just happened: this is our first parameter estimation. There are two free parameters and two data points (H, F), so the model can be solved exactly. With more data points than parameters, an exact solution usually does not exist and we need the fitting methods of Weeks 3–4.

The extreme-rate problem. If $H = 1$ or $F = 0$, then Φ^{-1} is $\pm\infty$ and d' is infinite. This happens often with few trials. A standard fix is the log-linear correction: add 0.5 to every cell count before computing the rates (i.e., add 0.5 to hits and FAs and 1 to the number of trials of each type). We use it by default in the lab.

The ROC curve: SDT's signature. Hold d' fixed and let c vary. The pair (F, H) traces a curve, the receiver operating characteristic (ROC). Every point on one ROC has the same sensitivity but a different bias. Plotting $\Phi^{-1}(H)$ against $\Phi^{-1}(F)$ gives the zROC, which under equal-variance SDT is a straight line:

$$\Phi^{-1}(H) = d' + 1 \cdot \Phi^{-1}(F).$$

So the model makes a *testable* prediction: the zROC is linear with slope 1. ROCs can be measured with confidence ratings (each confidence level acts as a separate criterion). In recognition memory, zROC slopes are typically *below* 1, which motivates the unequal-variance model below.

> Advanced: unequal-variance SDT. If signal trials have $X \sim \mathcal{N}(\mu_s, \sigma_s)$, the zROC is still linear but its slope is $1/\sigma_s$. In that case, d' computed from a single (H, F) pair *changes with the criterion*, so it is no longer a pure sensitivity measure. A common alternative index is $d_a = \mu_s\sqrt{2/(1+\sigma_s^2)}$. Estimating σ_s requires more than one criterion, i.e., rating data. You will see this effect directly in lab Part B.

Reading the math The two bell curves. Draw two normal curves on one axis: the "noise" curve centered at 0 and the "signal" curve centered at d'. Both have width 1. Draw a vertical line at the criterion k. Everything to the right of the line is a "yes".

- H = area of the signal curve to the right of the line. F = area of the noise curve to the right of the line.
- Why $H = \Phi(d'/2 - c)$? The criterion sits at $k = d'/2 + c$ (the midpoint plus c). The signal curve is centered at d', so the line is $k - d' = c - d'/2$ SDs from its center. The area to the right of that point is $1 - \Phi(c - d'/2) = \Phi(d'/2 - c)$, because the normal curve is symmetric.
- Numerical example. $d' = 2$, $c = 0$: $H = \Phi(1) = 0.84$ and $F = \Phi(-1) = 0.16$. Now make the observer liberal, $c = -0.5$: $H = \Phi(1.5) = 0.93$, $F = \Phi(-0.5) = 0.31$. Both rates went up, sensitivity is unchanged. This is radiologist R1 of the worked example.

Inverting. $d' = \Phi^{-1}(H) - \Phi^{-1}(F)$: convert each rate into a z-score and subtract. For R1: $1.476 - (-0.496) = 1.97$. c is minus the average of the two z-scores: if both rates are high (both z-scores positive), c is negative, i.e. liberal.

The zROC line. $\Phi^{-1}(H) = d' + \Phi^{-1}(F)$ is just the previous equation rearranged. In z-units, moving the criterion moves both z-scores by the same amount, so the points lie on a line with slope 1 and intercept d'.

For an engineer SDT *is* the detection theory of radar and communications, imported into psychology in the 1950s.

Psychology	Telecom / radar
noise trial / signal trial	hypothesis H_0 (noise only) / H_1 (signal + noise)
internal evidence X	test statistic, e.g. matched-filter output
criterion k (or c)	detection threshold
hit rate H	probability of detection P_D
false-alarm rate F	probability of false alarm P_{FA}
sensitivity d'	deflection $(\mu_1 - \mu_0)/\sigma$; d'^2 plays the role of an SNR
ROC curve	receiver operating characteristic (same name, same plot)
liberal / conservative observer	low / high threshold (high / low P_{FA})

Differences in emphasis: an engineer usually *chooses* the threshold (e.g. Neyman–Pearson: fix P_{FA}, maximize P_D). A psychologist *observes* a person's P_D and P_{FA} and *infers* both the SNR (d') and the threshold (c) the person used. The criterion c is measured from the midpoint between the two means, so $c = 0$ means "no bias". The optimal threshold depends on prior probabilities and costs exactly as in Bayesian detection, and people often move their criterion in the right direction when payoffs change (Week 4).

3. Forgetting curves

Data. At each retention interval t_i (e.g., in days), the participant is tested on n items and recalls y_i of them.

Two candidate deterministic cores:

	Exponential	Power
Equation	$f(t) = A\,e^{-\lambda t}$	$f(t) = A\,(1+t)^{-d}$
Parameters	A: initial level; λ: decay rate	A: initial level; d: decay exponent
Linearizing plot	$\ln f$ vs. t is a straight line with slope $-\lambda$	$\ln f$ vs. $\ln(1+t)$ is a straight line with slope $-d$
Relative loss rate $-f'(t)/f(t)$	λ (constant)	$d/(1+t)$ (decreasing with age)
Psychological reading	A memory loses the same *proportion* of its strength per unit time, whatever its age	Older memories lose strength more slowly; they become more resistant

The relative loss rate (in survival analysis, the *hazard*) is the key interpretive difference. It is why the choice between the two functions is a theoretical question, not a curve-fitting detail.

Noise model. Recall counts are binomial:

$$y_i \sim \text{Binomial}\big(n, f(t_i;\boldsymbol{\theta})\big).$$

With $n = 50$ items and $f = 0.5$, the SD of the observed proportion is $\sqrt{0.5 \cdot 0.5/50} \approx 0.07$. That is large compared with the differences between models over short intervals, which is a first hint that design (which intervals, how many items) determines whether data can discriminate between models.

> Advanced: the "+1" in the power model is not innocent. $A\,t^{-d}$ is infinite at $t = 0$, so a shift is added. But $(1 + t)$ depends on the unit of t: measuring in hours instead of days changes the shape of the curve, not just the scale of the axis. Some authors therefore write $A\,(1 + bt)^{-d}$ with a free scale parameter b, which adds a third parameter and raises the model comparison questions of Weeks 6–7. Keep the unit fixed (days) throughout this course.

Reading the math

- $f(t) = Ae^{-\lambda t}$: A is the recall level right after study ($t = 0$), and λ says how fast it falls. With $\lambda = 0.1$ per day, recall is multiplied by $e^{-0.1} \approx 0.905$ every day: about 9.5% of what is left is lost each day, on day 1 and on day 100 alike.
- $f(t) = A(1+t)^{-d}$: going from day 0 to day 1 multiplies recall by 2^{-d}; going from day 9 to day 10 multiplies it only by $(10/11)^d$, close to 1. Old memories hardly change any more.

- Relative loss rate $-f'(t)/f(t)$: "what fraction of the remaining memory is lost per day *right now*". Constant (λ) for the exponential, shrinking ($d/(1+t)$) for the power function. This single number is the psychological difference between the models.
- $y_i \sim \text{Binomial}(n, f(t_i))$: at delay t_i the participant recalls y_i of n items, and each item is recalled with probability $f(t_i)$. The observed proportion y_i/n scatters around $f(t_i)$ with SD $\sqrt{f(1-f)/n}$, about 0.07 for $n = 50$.

For an engineer

- The exponential curve is the step response of a first-order system (an RC circuit discharging): one time constant $\tau = 1/\lambda$. A constant relative loss rate is the "memoryless" property you know from exponential lifetimes in reliability theory.
- The power curve has no single time constant. A useful picture: a sum of many exponentials with *different* time constants (fast and slow components) looks like a power law. Week 5 shows this exactly, and it is one candidate explanation of power-law forgetting.
- On a log–log plot a power law is a straight line; on a semi-log plot an exponential is a straight line. Engineers use the same trick to identify system orders and $1/f$ noise.
- The binomial noise model is the counting statistics of a detector: like counting bit errors out of n bits, the uncertainty of a proportion shrinks with $1/\sqrt{n}$.

4. Simulation as a thinking tool

Before fitting anything, simulate. Simulation answers questions such as:

- What can the model produce, and what can it *not* produce? Equal-variance SDT with $d' > 0$ can never produce $H < F$, and it can never produce a zROC slope different from 1. Both exponential and power curves are always decreasing, so they cannot capture *reminiscence* (recall improving over time). The set of things a model cannot do is what makes it testable.
- Which parameters matter where? A moves the whole forgetting curve; λ or d controls the shape.
- Can the design discriminate between models? If two models make nearly identical predictions at the intervals we plan to test, no fitting method can tell them apart (lab Exercise 4; Assignment A1).

5. Preview: estimation by linearization, and why it is not enough

A traditional way to fit forgetting curves is to log-transform the observed proportions and fit a straight line by ordinary regression. It is fast and intuitive, and we try it in the lab. It has three problems that motivate Weeks 3–4:

1. $\ln(0)$ is undefined, so an interval with zero recalls breaks the method.
2. The log transform distorts the noise: the scatter of $\ln \hat{p}$ is much larger for small proportions, but regression treats all points as equally reliable.

3. The line is fit on the log scale, but we care about errors on the original (proportion) scale.

The principled alternative is to fit the model directly to the untransformed data, by minimizing squared error (Week 3) or maximizing the binomial likelihood (Week 4).

For the instructor

- Open with the radiologist puzzle (Motivation) and let students vote on who is better before showing any math. Most will pick by percent correct, then realize it is identical.
- Draw, don't derive. The two overlapping bell curves with a movable vertical line explain H, F, d' and c better than any formula. Move the line left and right on the board and ask what happens to H and F; then pull the curves apart and ask again.
- Use Φ tables or Python live. Compute R1's d' with `norm.ppf` in front of the class. Students who are new to statistics need to see that Φ^{-1} is "just a lookup".
- Connect to something they know: a COVID or pregnancy test has a hit rate (sensitivity) and a false-alarm rate (1 − specificity). Medical "sensitivity" is the hit rate, *not* d': point out this naming clash.
- Typical questions.
  - *"Why assume normal distributions?"* It is a convenient and often adequate assumption; the zROC test (slope 1) checks it, and the unequal-variance model relaxes it.
  - *"Is a liberal criterion bad?"* No: the best criterion depends on costs and base rates.
  - *"Exponential or power, which is right?"* For many datasets the power function fits better, but averaging over people can create a power shape (Week 5). Keep the question open.

1.2.5 Worked example

Part 1: Two radiologists (and a third)

Each radiologist reads 100 tumor scans and 100 healthy scans.

Observer	H	F	$\Phi^{-1}(H)$	$\Phi^{-1}(F)$	d'	c	Percent correct
R0 (neutral)	0.84	0.16	0.994	−0.994	1.99	0.00	84%
R1 (liberal)	0.93	0.31	1.476	−0.496	1.97	−0.49	81%
R2 (conservative)	0.69	0.07	0.496	−1.476	1.97	+0.49	81%

Step by step for R1: 1. $\Phi^{-1}(0.93) = 1.476$ and $\Phi^{-1}(0.31) = -0.496$ (`scipy.stats.norm.ppf`). 2. $d' = 1.476 - (-0.496) = 1.972$. 3. $c = -\frac{1}{2}(1.476 + (-0.496)) = -0.490$, which is negative, so R1 is liberal. 4. Check by plugging back in: $H = \Phi(1.972/2 + 0.490) = \Phi(1.476) = 0.93$. ✓

Interpretation. All three radiologists have essentially the same sensitivity ($d' \approx 2$). R1 and R2 have the *same* percent correct but *opposite* biases. Which bias is better is not a question of perception but of costs: missing a tumor is usually far worse than a false alarm, so a liberal criterion may be the *desirable* one. SDT lets us ask "can they see it?" and "how willing are they to say so?" separately.

The extreme-rate fix. A participant scores 50/50 hits and 3/50 false alarms. Then $H = 1$ and $d' = \infty$. With the log-linear correction: $H = 50.5/51 = 0.990$, $F = 3.5/51 = 0.069$, so $d' = 2.33 - (-1.49) = 3.82$.

Part 2: Exponential vs. power forgetting

Take $A = 0.95$ for both models, $\lambda = 0.25$/day for the exponential and $d = 0.6$ for the power model.

t (days)	0	1	2	7	14	30
Exponential $0.95\,e^{-0.25t}$	0.950	0.740	0.576	0.165	0.029	0.001
Power $0.95\,(1+t)^{-0.6}$	0.950	0.627	0.491	0.273	0.187	0.121

Half-life (time until recall halves from its initial level): - Exponential: $e^{-\lambda t} = 1/2 \Rightarrow t_{1/2} = \ln 2/\lambda = 0.693/0.25 = 2.77$ days. - Power: $(1+t)^{-d} = 1/2 \Rightarrow t_{1/2} = 2^{1/d} - 1 = 2^{1.667} - 1 = 2.17$ days.

Interpretation. Over the first two days the curves are fairly similar: they differ by about 0.1, which is only about 1.5 binomial SDs with $n = 50$ items. After a month they disagree completely: the exponential model says practically everything is forgotten (0.1%), while the power model says 12% is still retained. Long retention intervals are where these models can be told apart, a design lesson you will use in Assignment A1.

1.2.6 Lab (2 hours)

Goals

1. Implement SDT as a generative model, simulate yes/no experiments, and estimate d' and c.
2. Use ROC and zROC plots to evaluate the equal-variance assumption.
3. Implement exponential and power forgetting with binomial noise, and explore their parameters.
4. Fit both forgetting models by linearization, evaluate them with residuals, and interpret the result.

Timing: Part A 30 min · Part B 25 min · Part C 30 min · Part D 25 min · remaining time: exercises.

Setup

Code · کد: `code/week02.py`, cell 1 · سلول 1

Part A: SDT as a generative model (30 min)

Code · کد: `code/week02.py`, cell 2 · سلول 2

Simulate one participant and recover the parameters:

Code · کد: `code/week02.py`, cell 3 · سلول 3

Expected output:

```
Counts: {'hits': 68, 'misses': 32, 'fas': 11, 'crs': 89}
Predicted H = 0.674, F = 0.147
Estimated d' = 1.67 (true 1.5),  c = 0.37 (true 0.3)
```

One simulated participant tells us little about how good the *estimator* is. Let's repeat the experiment many times: this is a preview of the parameter recovery logic of Week 7.

Code · کد: `code/week02.py`, cell 4 · سلول 4

Expected output:

```
            d_hat         c_hat
             mean    std   mean    std
n_per_type
20          1.500  0.449  0.296  0.224
50          1.489  0.290  0.295  0.143
200         1.502  0.142  0.299  0.071
```

What to notice: the estimates scatter around the true values, and the scatter shrinks as the number of trials grows. With 20 trials per type, the SD of $\hat{d}'$ is about 0.45, so individual estimates are often off by ± 0.5 or more, and roughly $4\times$ more trials are needed to halve the scatter. On average the estimates are on target here. With higher d' and few trials, however, extreme rates become common, and the correction then pulls estimates toward 0 (try `true_d=3` with 20 trials). Keep this in mind whenever you compare d' across groups tested with different numbers of trials.

Part B: ROC curves and the equal-variance assumption (25 min)

First, the model's predicted ROCs:

Code · کد: `code/week02.py`, cell 5 · سلول 5

Now simulate a confidence-rating experiment. Responses are given on a 6-point scale (1 = "sure noise" … 6 = "sure signal"), which amounts to five criteria at once. We generate data from an equal-variance observer and from an unequal-variance observer whose signal distribution has SD 1.5.

Code · کد: `code/week02.py`, cell 6 · سلول 6

Expected output:

```
sigma_signal = 1.0: zROC slope = 0.96 (predicted 1.00); d' at each
↪  criterion = [1.52 1.48 1.48 1.43 1.43]
sigma_signal = 1.5: zROC slope = 0.67 (predicted 0.67); d' at each
↪  criterion = [1.46 1.34 1.16 0.96 0.84]
```

Evaluate and interpret. For the equal-variance observer, the zROC slope is close to 1 and d' is about the same at every criterion: the model's assumption holds. For the unequal-variance observer, the slope is close to $1/1.5 = 0.67$ and d' changes systematically with the criterion. If we had collected only yes/no data from such an observer, a group difference in "d'" could be produced purely by a difference in bias. Checking a model's signature prediction (here: zROC slope $= 1$) is a basic form of model evaluation.

Part C: Forgetting curves with binomial noise (30 min)

Code · کد: `code/week02.py`, cell 7 · سلول 7

Explore the parameters: how do A and the decay parameter change each curve?

Code · کد: `code/week02.py`, cell 8 · سلول 8

Now generate a simulated experiment. The data come from the *power* model, with 50 items tested at each of 8 retention intervals:

Code · کد: `code/week02.py`, cell 9 · سلول 9

Expected output:

```
       t  recalled   n  p_obs
0   0.25        42  50   0.84
1   1.00        30  50   0.60
2   2.00        26  50   0.52
3   4.00        25  50   0.50
4   7.00        13  50   0.26
5  14.00        13  50   0.26
6  21.00        13  50   0.26
7  30.00        14  50   0.28
```

Part D: Fit by linearization, evaluate, interpret (25 min)

Fit. Fit each model by a straight-line regression on the log scale (Lecture §5).

Code · کد: `code/week02.py`, cell 10 · سلول 10

Evaluate. Look at the fitted curves *and* the residuals. A good model leaves residuals that scatter randomly around zero; a systematic pattern means the model's *shape* is wrong.

Code · کد: `code/week02.py`, cell 11 · سلول 11

Expected output:

```
exp: A = 0.550, rate = 0.032, RMSD = 0.1308
pow: A = 0.797, rate = 0.370, RMSD = 0.0628
true (power): A = 0.9, d = 0.5
```

Interpret (write 2–3 sentences for each in your notebook):

1. Which model describes the data better, and what does the residual pattern of the *other* model look like? (Look for an inverted-U or U shape: the exponential overshoots at intermediate delays and undershoots at the long ones, or the reverse.)
2. Using the fitted parameters, what is each model's relative loss rate after 1 day and after 20 days? What does this mean for memory?
3. The power model is the true model here, yet its estimates ($\hat{A} \approx 0.80$, $\hat{d} \approx 0.37$) are noticeably off the true values ($A^* = 0.9$, $d^* = 0.5$). Name two reasons. (Hints: look at the last three data points; think about what the log transform does to the weights of small proportions.) In Week 3 we refit these same data on the original scale and compare.

Exercises (increasing difficulty)

Exercise 1 ⋆ (extreme rates). A participant has 40/40 hits and 0/40 false alarms. Compute d' without and with the log-linear correction. What does the corrected d' represent, and why should you report the correction you used?

Exercise 2 ⋆ (half-lives). Using your fitted parameters from Part D, compute each model's half-life. Then compute how long it takes recall to drop to 10% of its initial level. Where do the models disagree most?

Exercise 3 ⋆⋆ (Ebbinghaus's data). The classic savings data from Ebbinghaus's 1885 self-experiments are often reported as follows [verify against the original source before using them in class]:

Interval	20 min	1 h	8.8 h	1 day	2 days	6 days	31 days
Savings (%)	58.2	44.2	35.8	33.7	27.8	25.4	21.1

Convert the intervals to days, fit both models by linearization, and compare their residuals. Which shape fits better? Then redo the analysis with time measured in hours. Does the power fit change? Explain, using the Advanced box in Lecture §3.

Exercise 4 ⋆⋆ (which design can tell the models apart?). Choose (A, λ) and (A, d) so that the two models predict the *same* recall at $t = 1$ and $t = 7$ days. (Hint: two equations in two unknowns per model; take logs.) Then plot both curves from $t = 0$ to 60 days and mark where their difference exceeds 2 binomial SDs with $n = 50$. Which retention intervals would you test?

Exercise 5 ⋆⋆⋆ (Advanced: zero counts). Generate data from the exponential model with $A = 0.9$, $\lambda = 0.3$ at the intervals in Part C, with $n = 20$ items. How often (out of 1000 simulated datasets) does at least one interval produce zero recalls, which breaks `fit_linearized`? Propose and implement a quick fix (e.g., adding 0.5 to counts) and check whether it biases the estimate of λ. Explain why this is a sign that we need a method that works on the original scale (Weeks 3–4).

Lab check-off

Show the instructor or TA: (1) your SDT recovery table from Part A, (2) the two empirical zROCs with their slopes from Part B, and (3) the residual plot from Part D with a one-sentence interpretation.

1.2.7 Connection to AI & LLMs

AI-text detectors are observers in an SDT task. Universities increasingly use tools that score a text for "how likely it was written by AI". Each text is either AI-written ("signal") or human-written ("noise"), and the tool says "AI" when its score exceeds a threshold. This is exactly the SDT setting of this week: the tool has a sensitivity d' (how well the score separates the two kinds of texts) and a criterion c (the threshold the user chooses). A false alarm means a student is wrongly accused, so the choice of criterion is an ethical decision, not only a technical one.

Code · كد: `code/week02.py`, cell 12 · سلول 12

Expected output:

```
threshold 0.0: H = 0.84, F = 0.49, d' = 1.03, c = -0.50
threshold 0.6: H = 0.70, F = 0.28, d' = 1.11, c = 0.03
threshold 1.2: H = 0.51, F = 0.09, d' = 1.35, c = 0.66
```

Moving the threshold changes c a lot (from -0.50 to 0.66), while d' stays near its true value of 1.2. The variation in d' is sampling noise from 200 texts per group.

Try it (no API needed). - ⋆ Suppose 10% of submitted essays are AI-written. At threshold 1.2, what is $P(\text{AI-written} \mid \text{flagged})$? (Use Bayes' rule with H and F.) Would you act on a flag? - ⋆⋆ Plot the ROC of the detector (all thresholds from -2 to 3) and its zROC. Is the slope close to 1? Repeat with

human-text scores of SD 1.5 (an unequal-variance detector). - ⋆⋆ Discussion: detectors have been reported to flag texts by non-native English writers more often [verify: Liang et al., 2023, *Patterns*]. In SDT terms, what would that mean, and what should a university do?

1.2.8 Common mistakes & misconceptions

1. "Percent correct is good enough." It confounds sensitivity and bias (see the radiologists). Use d' and c whenever responses can be biased.
2. Computing d' as $\Phi^{-1}(H - F)$ or from percentages. The formula is $\Phi^{-1}(H) - \Phi^{-1}(F)$, with rates as proportions between 0 and 1.
3. Forgetting the extreme-rate problem. $H = 1$ or $F = 0$ gives an infinite d'; Python returns `inf` without an error. Always use (and report) a correction.
4. Assuming d' is bias-free no matter what. It is criterion-independent only under the equal-variance model. If the zROC slope differs from 1, d' from one (H, F) pair depends on bias.
5. "Exponential and power are just two curve shapes." They make different claims about how forgetting works (constant vs. decreasing relative loss rate).
6. Judging a model only by its fit statistic. A small RMSD with a systematic residual pattern means the model's shape is wrong. Always plot the residuals.
7. Ignoring the noise model. A model is not just $f(x; \boldsymbol{\theta})$. The binomial noise of recall proportions is part of the model, and it tells you which differences are meaningful.
8. Changing the units of t in the shifted power model and expecting the same shape.

1.2.9 Check your understanding

1. Name the five components of a model and give the SDT version of each. Answer Design (signal vs. noise trials; yes/no responses); parameters (free: d', c; fixed: noise mean 0, both SDs 1); deterministic core ($H = \Phi(d'/2 - c)$, $F = \Phi(-d'/2 - c)$); noise model (hit and FA counts are binomial); linking assumption (respond "yes" when the evidence exceeds the criterion).

2. Why are the noise mean and the SDs in SDT fixed rather than free? Answer Because they are not identifiable: shifting both distributions and the criterion by the same amount, or stretching the whole axis, leaves every prediction unchanged. Fixing them (scaling constraints) defines the units in which d' and c are measured.

3. An observer has $H = 0.80$ and $F = 0.20$. Compute d' and c. ($\Phi^{-1}(0.80) = 0.842$) Answer $d' = 0.842 - (-0.842) = 1.68$; $c = -\frac{1}{2}(0.842 - 0.842) = 0$, so the observer is unbiased.

4. What does equal-variance SDT predict about the zROC, and what does it mean if the empirical slope is 0.7? Answer A straight line with slope 1. A slope of 0.7 suggests the signal distribution is more variable than the noise distribution ($\sigma_s \approx 1/0.7 \approx 1.4$). Then single-point d' depends on the criterion, and an unequal-variance model (or d_a) should be used.

5. What is the relative loss rate of each forgetting model, and why does it matter psychologically? Answer Exponential: λ, constant. Power: $d/(1+t)$, decreasing over time. The exponential says memories lose the same proportion of strength per day at every age; the power function says older memories become more resistant to further forgetting.

6. Which plot would you make to check quickly whether forgetting data look exponential? And for power? Answer Exponential: $\ln(\text{recall})$ against t (semi-log), which should be a straight line. Power: $\ln(\text{recall})$ against $\ln(1+t)$ (log-log), which should be a straight line.

7. Give two reasons why fitting a straight line to log-transformed proportions is a poor estimation method. Answer (i) It breaks when an observed proportion is 0. (ii) It distorts the noise: small proportions become very variable on the log scale but are weighted equally, and the errors being minimized are on the wrong (log) scale. Fitting on the original scale with least squares or maximum likelihood avoids these problems.

1.2.10 Assignment

A1: From verbal to formal (1.0 point of 20; released Week 2, due at the start of the Week 4 lecture)

Scenario. Two research groups study forgetting of vocabulary learned in a foreign-language course. - Group C says: *"Memories decay at a constant rate."* - Group R says: *"Memories become more resistant to forgetting as they age."*

Both groups claim that the published data (recall drops quickly and then levels off) support their view.

Tasks. Submit a Jupyter notebook plus a PDF export.

1. Formalize (Group C and Group R). Write each verbal claim as a formal model with all five components from Lecture §1: design, free/fixed parameters with ranges, deterministic core, binomial noise model, and linking assumption. Justify in 2–3 sentences why each equation captures its verbal claim (use the relative loss rate).
2. Implement. Write Python functions that (a) return predicted recall probabilities and (b) simulate binomial recall counts for given intervals and number of items n.
3. Calibrate. Both groups agree that recall is 0.60 after 1 day and 0.35 after 7 days. Derive, by hand and in closed form, the parameters of each model that reproduce these two values exactly. Show the derivation (take logs) and verify it numerically.
4. Find where they diverge. With the calibrated parameters, plot both predicted curves from 0 to 90 days. For $n = 40$ items, compute at each interval a *discriminability index* $D(t) = \big(f_C(t) - f_R(t)\big)^2 / \text{Var}(t)$, where $\text{Var}(t) = \bar{f}(1-\bar{f})/n$ and $\bar{f}$ is the average of the two predictions. Plot $D(t)$.
5. Design an experiment. You can test each participant at only 4 retention intervals, chosen from {0.5, 1, 2, 4, 7, 14, 30, 60, 90} days. Choose the set that maximizes the summed $D(t)$, explain why it makes sense, and simulate one dataset from *each* model at your design. Show the two datasets on one plot.

6. Interpret (max. 300 words). Which data would favor Group C and which Group R? What does your analysis say about why both groups can claim support from "the published data"? Name one assumption of your models that could be wrong, and how it would change your conclusions.

Grading rubric (10 points, scaled to 1.0 of the final grade):

Criterion	Points	Full marks require
Formalization (Task 1)	2	All five components for both models; correct parameter ranges; the link between equation and verbal claim is explained via the relative loss rate
Code (Task 2)	2	Correct, commented, reproducible (fixed seeds); functions are vectorized or clearly looped
Calibration (Task 3)	2	Correct closed-form derivation and numerical check for both models
Divergence and design (Tasks 4–5)	2.5	Correct $D(t)$; the chosen design is justified and actually maximizes the criterion; clear plots
Interpretation (Task 6)	1.5	Clear, specific, refers to your own results; names a real limitation

Late submissions lose 10% per day (max. 3 days).

1.2.11 Further reading

- Farrell & Lewandowsky (2018), chapter 2. Building a model from a verbal theory; the components of a model and the role of parameters. *(Main reading.)*
- Lee & Wagenmakers (2014), chapter 11. Signal detection theory treated as a (Bayesian) cognitive model; we return to it in Weeks 9 and 13. Read the introductory part now, skip the code.
- Verguts (2022) and Anderson (2014): the chapters on model building and parameters [verify chapter numbers].

- *Background on forgetting curves:* Lee & Wagenmakers (2014), chapter 10 (memory retention) uses a closely related exponential retention model; we will fit it in a Bayesian way in Weeks 9–11.

Preview for Week 3: we drop linearization and fit models directly by minimizing squared error, which means learning how optimization algorithms search a parameter space.

Chapter 2

Parameter Estimation Methods

2.1 Week 3: Fitting Models by Least Squares: Error Surfaces and Optimization

Chapter 2: Parameter estimation methods · Lecture 1 h · Lab 2 h This week's running data: the simulated forgetting experiment from Week 2 (power model, $A^* = 0.9$, $d^* = 0.5$, $n = 50$ items at 8 intervals). We refit it properly and compare with last week's linearized fits.

2.1.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
least squares	کمترین مربعات	روش برازش با کمینه کردن مجموع مربع خطاها
error surface	سطح خطا	خطا به‌عنوان تابعی از پارامترها
optimizer	بهینه‌ساز	الگوریتمی که کمینه‌ی تابع را پیدا می‌کند
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان

English	فارسی	توضیح کوتاه
exponential / power function	توانی / تابع نمایی	دو شکل رایج برای منحنی فراموشی و یادگیری
SSE / RMSD	ریشه‌ی میانگین / مجموع مربعات خطا مربع انحراف	معیارهای فاصله‌ی داده و پیش‌بینی
noise	نویز	تغییرات تصادفی در شواهد یا در داده
parameter trade-off	بده‌بستان پارامترها	تغییر یک پارامتر که با پارامتر دیگر جبران می‌شود
grid approximation	تقریب شبکه‌ای (grid)	محاسبه‌ی پسین روی شبکه‌ای از مقادیر
Nelder–Mead	روش Nelder–Mead	بهینه‌ساز بدون نیاز به مشتق
Bayes factor (BF)	عامل بیز (Bayes factor)	نسبت درست‌نمایی‌های حاشیه‌ای دو مدل
local minimum	کمینه‌ی محلی	نقطه‌ای پایین‌تر از اطراف ولی نه بهترین نقطه
bounds / constraints	قیدها / کران‌ها	محدودیت‌های مجاز برای مقدار پارامترها
reparameterization	بازپارامتری‌سازی	نوشتن پارامتر در مقیاس دیگر، مثلاً $\lambda = e\hat{}\eta$
multi-start	شروع چندگانه (multi-start)	اجرای بهینه‌ساز از چند نقطه‌ی شروع
maximum likelihood estimate (MLE)	برآورد بیشینه‌ی درست‌نمایی	پارامترهایی که داده را محتمل‌ترین می‌کنند
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
gradient	گرادیان	جهت و شیب تغییر تابع
convergence	همگرایی	رسیدن زنجیره‌ها به توزیع هدف
large language model (LLM)	مدل بزرگ زبانی	شبکه‌ی عصبی‌ای که کلمه‌ی بعدی متن را پیش‌بینی می‌کند
scaling law	قانون مقیاس	رابطه‌ی توانی بین اندازه‌ی مدل و خطا

2.1.2 Learning outcomes

By the end of this week, students will be able to:

1. Define SSE, MSE and RMSD as discrepancy functions and explain what minimizing them assumes about the noise.
2. Visualize an error surface for a two-parameter model and interpret its shape (minimum, valleys, plateaus) in terms of parameter trade-offs.
3. Implement least-squares fitting by grid search and with `scipy.optimize.minimize` (Nelder–Mead and L-BFGS-B), and compare the algorithms' strengths and failure modes.
4. Diagnose and remedy common fitting failures: bad starting values, plateaus, local minima, and out-of-range parameters (using bounds, reparameterization and multi-start).

5. Evaluate a least-squares fit with RMSD and residuals, and interpret the fitted parameters.

2.1.3 Motivation & big picture

Last week we fitted forgetting curves by drawing a straight line through log-transformed proportions. The power model was the true model, yet its estimates were noticeably off ($\hat{A} = 0.80$, $\hat{d} = 0.37$ instead of 0.9 and 0.5). The method also breaks whenever a count is zero, and it minimizes errors on the wrong (log) scale.

The fix is conceptually simple: choose the parameters that make the model's predictions as close as possible to the data, on the data's own scale. "As close as possible" needs a number, a *discrepancy function*. "Choose the parameters" needs a search procedure, an *optimization algorithm*. This week covers both.

There is a deeper reason this matters for cognitive science. Model parameters are our *measurements* of latent constructs (decay rate, caution, sensitivity). A poor fitting procedure gives poor measurements, and every conclusion drawn from them (group differences, correlations with brain activity, model comparisons) inherits that error. Worse, as we will see, an optimizer can report "success" while returning nonsense.

Where is this used?

- Every model-based study in psychology has a fitting step. Learning curves, forgetting curves, psychometric functions (how detection improves with stimulus strength) and dose–response curves in psychopharmacology are all fitted by minimizing a discrepancy.
- Clinical and developmental research compares fitted parameters between groups (e.g. the forgetting rate of patients vs. controls). If the fit is poor or stuck at a bound, the group comparison is wrong.
- Neuroscience fits tuning curves of neurons and hemodynamic response functions in fMRI with exactly these optimizers.
- Machine learning trains models by minimizing a loss with gradient methods; the vocabulary (loss surface, learning rate, local minima) is the same as this week's.

2.1.4 Lecture notes

In plain words A model has knobs (parameters). Fitting means turning the knobs until the model's curve passes as close as possible to the data points. To do this with a computer we need two things. First, a single number that says how bad the current fit is: the sum of squared errors (SSE), the squared vertical distances between data points and the curve, added up. Second, a strategy to turn the knobs so that this number goes down: an optimizer. Think of the SSE as a landscape over all possible

knob settings, with hills (bad fits) and valleys (good fits); the optimizer is a hiker looking for the lowest point. Problems arise when the landscape has a long flat valley (two knobs can compensate each other), a flat plateau (the hiker cannot tell which way is down), or several separate valleys (the hiker may stop in the wrong one). Most of this week is about recognizing those situations and protecting yourself against them.

Suggested timing (60 min): discrepancy functions 10 · error surfaces 10 · optimization algorithms 20 · practical problems and a fitting checklist 15 · preview 5.

1. Discrepancy functions

For observations y_i and model predictions $\hat{y}_i = f(x_i; \boldsymbol{\theta})$, $i = 1, \ldots, N$:

$$\text{SSE}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \big(y_i - f(x_i; \boldsymbol{\theta})\big)^2, \qquad \text{MSE} = \frac{\text{SSE}}{N}, \qquad \text{RMSD} = \sqrt{\text{MSE}}.$$

- All three have the same minimizer: they are monotone transformations of each other. We *minimize* SSE and *report* RMSD, because RMSD is in the units of the data (here: proportion recalled), so "RMSD = 0.057" means predictions are off by about 0.06 on average.
- The least-squares estimate is
$$\hat{\boldsymbol{\theta}}_{\text{LS}} = \arg\min_{\boldsymbol{\theta}} \text{SSE}(\boldsymbol{\theta}).$$

Why squares? Squaring makes all errors positive, penalizes large errors heavily, and gives a smooth (differentiable) function that optimizers handle well. More deeply, minimizing SSE is exactly maximum likelihood estimation if the noise is Gaussian with the same variance for every observation (Week 4). That "if" is the hidden assumption of least squares.

When the assumption fails. Recall proportions are binomial: their variance $f(1-f)/n$ depends on f. Points near 0.5 are noisier than points near 0 or 1, but plain SSE treats them as equally reliable. Weighted least squares fixes this by weighting each squared error by the inverse of its variance:

$$\text{WSSE}(\boldsymbol{\theta}) = \sum_{i} \frac{\big(y_i - \hat{y}_i\big)^2}{\text{Var}(y_i)}.$$

You will try this in Exercise 3. Next week, maximum likelihood handles this issue more naturally.

> Note: some textbooks call RMSD "RMSE". They are the same quantity.

Reading the math

- $\sum_{i=1}^{N}$ means "add up over all N data points". $y_i - f(x_i; \boldsymbol{\theta})$ is the residual of point i: observed minus predicted. Squaring and adding gives one number per parameter setting $\boldsymbol{\theta}$.
- $\arg\min_{\boldsymbol{\theta}} \text{SSE}(\boldsymbol{\theta})$ reads "the value of $\boldsymbol{\theta}$ at which SSE is smallest". Note the difference: min is the smallest *SSE value*, arg min is the *parameter value* that achieves it. We want the arg min.

- Small example. Data $y = (0.62, 0.41)$, predictions $(0.66, 0.44)$: residuals $(-0.04, -0.03)$, SSE $= 0.0016 + 0.0009 = 0.0025$, MSE $= 0.00125$, RMSD $= 0.035$: the curve misses the points by about 3.5 percentage points on average.
- In WSSE each squared residual is divided by the variance of that point, so a noisy point counts less. With binomial data the variance is $f(1-f)/n$.

2. The error surface

For a model with k parameters, SSE($\boldsymbol{\theta}$) is a surface over a k-dimensional parameter space. With $k = 2$ we can draw it as a contour map: the fitting problem becomes "find the lowest point of the landscape."

Features to recognize:

Feature	What it looks like	What it means
Single, round bowl	Concentric, nearly circular contours	Parameters well determined and roughly independent; any decent algorithm finds the minimum
Long, diagonal valley	Elongated contours tilted at an angle	Parameter trade-off: increasing one parameter can be compensated by changing the other. Estimates will be correlated and uncertain along the valley (Week 5, Week 7)
Plateau	Very widely spaced contours	The model's predictions barely change with the parameters (e.g., a decay rate so large that the predicted curve is 0 everywhere). Gradient-based methods stall
Several separate basins	More than one closed set of contours	Local minima: the algorithm's answer depends on where it starts

For exponential forgetting, A and λ trade off: a higher starting level with faster decay can predict nearly the same mid-range values as a lower level with slower decay. The valley is therefore tilted (you will see it in the lab).

3. Optimization algorithms

All algorithms below start from initial values $\boldsymbol{\theta}^{(0)}$ and generate a sequence $\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}, \ldots$ that (hopefully) descends to the minimum.

3.1 Grid search Evaluate the SSE at every point of a regular grid and keep the best.

- Pros: trivially simple; cannot be fooled by local minima *on the grid*; gives a picture of the whole surface.
- Cons: precision limited by grid spacing; cost grows exponentially with the number of parameters. With 100 values per parameter: $k = 2$ needs 10^4 evaluations, $k = 5$ needs 10^{10}. This is the curse of dimensionality.
- Use it for: visualizing 1–2-parameter surfaces, and for generating good starting values.

3.2 Nelder–Mead simplex (derivative-free) A simplex is a set of $k + 1$ points in a k-dimensional space (a triangle when $k = 2$). Each iteration tries to replace the worst vertex $\boldsymbol{\theta}_w$ by moving it through the centroid $\bar{\boldsymbol{\theta}}$ of the other vertices:

Move	Formula (standard coefficients)	When
Reflect	$\boldsymbol{\theta}_r = \bar{\boldsymbol{\theta}} + (\bar{\boldsymbol{\theta}} - \boldsymbol{\theta}_w)$	Always tried first
Expand	$\boldsymbol{\theta}_e = \bar{\boldsymbol{\theta}} + 2(\bar{\boldsymbol{\theta}} - \boldsymbol{\theta}_w)$	The reflected point is better than the current best; try going further
Contract	$\boldsymbol{\theta}_c = \bar{\boldsymbol{\theta}} + \frac{1}{2}(\boldsymbol{\theta}_w - \bar{\boldsymbol{\theta}})$ (inside), or the same step on the reflected side (outside)	The reflected point is not better than the second-worst vertex
Shrink	Move all vertices halfway toward the best one	Contraction also failed

The reflected point is accepted when it is better than the second-worst vertex but not better than the best. The simplex "tumbles" downhill, stretches along valleys, and shrinks around the minimum.

- Pros: needs only SSE values (no derivatives); robust to noisy or non-smooth objective functions; the default workhorse in cognitive modeling.
- Cons: can be slow in high dimensions; can stall in long narrow valleys; no guarantee of finding the global minimum.

> Direct-search methods like Nelder–Mead are treated in depth in Ghodousian's book on direct methods in nonlinear optimization (see Further reading).

3.3 Gradient-based methods The gradient $\nabla\mathrm{SSE}(\boldsymbol{\theta})$ points uphill, so we step the other way:

$$\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \eta\,\nabla\mathrm{SSE}\big(\boldsymbol{\theta}^{(m)}\big), \qquad \frac{\partial\,\mathrm{SSE}}{\partial\theta} = -2\sum_i \big(y_i - \hat{y}_i\big)\,\frac{\partial \hat{y}_i}{\partial\theta},$$

with step size (learning rate) η. For the exponential model, $\partial\hat{y}/\partial A = e^{-\lambda t}$ and $\partial\hat{y}/\partial\lambda = -A\,t\,e^{-\lambda t}$.

In practice we use quasi-Newton methods such as BFGS. They also estimate the *curvature* of the surface and choose the step size automatically, so they converge much faster near a minimum. L-BFGS-B is the variant that accepts bounds on parameters. If you do not supply the gradient, scipy approximates it numerically by finite differences.

- Pros: fast and precise on smooth surfaces; scale well to many parameters (this is how neural networks are trained).
- Cons: need a smooth objective; get stuck where the gradient is (nearly) zero, at plateaus as well as at minima; sensitive to starting values.

> Advanced: why plateaus trap gradient methods. If λ is very large, $e^{-\lambda t} \approx 0$ at every observed t, so both partial derivatives above are ≈ 0. A gradient method sees a "flat floor" and stops, sometimes reporting successful convergence. Nelder–Mead compares function values at distant points and can step off the plateau. You will see both behaviors in lab Part D.

Summary:

Method	Needs derivatives	Bounds	Typical use
Grid search	No	Built in	Visualization, starting values, $k \leq 2$
Nelder–Mead	No	Yes (scipy ≥ 1.7)	Default for small cognitive models; noisy objectives
BFGS	Numerical or analytic	No	Smooth objectives, unconstrained parameters
L-BFGS-B	Numerical or analytic	Yes	Smooth objectives with bounded parameters

Reading the math The gradient step $\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \eta \, \nabla\mathrm{SSE}(\boldsymbol{\theta}^{(m)})$ reads: *new guess = old guess − step size × slope*. The superscript (m) is the iteration number, not a power. ∇SSE is the vector of partial derivatives (one slope per parameter). The minus sign makes the step go downhill.

The derivative formula $\partial\,\mathrm{SSE}/\partial\theta = -2\sum_i (y_i - \hat{y}_i)\,\partial\hat{y}_i/\partial\theta$ follows from the chain rule. It says: points where the residual is large *and* where the prediction is sensitive to θ pull hardest on θ. At the minimum these pulls cancel and the sum is zero.

For an engineer

- Least squares is the same criterion as in channel estimation and curve fitting; the difference here is that the models are nonlinear in the parameters ($e^{-\lambda t}$), so there is no closed-form

normal-equation solution and we need iterative search.

- Weighted least squares is noise whitening: dividing by the variance is what you do before LS when the noise is not white (heteroscedastic noise).
- Gradient descent with step size η is the steepest-descent idea behind the LMS adaptive filter. BFGS is a quasi-Newton method (it builds an approximation of the Hessian), close to Gauss–Newton / Levenberg–Marquardt used in nonlinear LS.
- A long diagonal valley in the error surface is an ill-conditioned problem: the Hessian has a large eigenvalue ratio, as with strongly correlated regressors. Parameters are well determined along one direction and poorly along the other.
- Reparameterization ($\lambda = e^{\eta}$) is a change of variables that turns a constrained problem into an unconstrained one; it also often improves conditioning.
- What is different from engineering practice: data sets are small (8 points, 50 items each), the noise is large, and the parameters must be *interpreted* psychologically. So it is not enough that the predicted curve looks good: the individual parameter values must be trustworthy.

4. Practical problems and how to handle them

1. Starting values. Use informed starts: read them off a plot, use a quick linearized fit (Week 2), or take the best point of a coarse grid.

2. Local minima $\rightarrow$ multi-start. Run the optimizer from many random starting points and keep the best solution. If many starts agree on the same minimum, you can be reasonably confident it is the global one. Multi-start is cheap insurance; use it routinely for models with more than two parameters.

3. Parameter bounds. Many parameters have natural ranges ($0 \leq A \leq 1$, $\lambda > 0$). Without constraints, optimizers happily return $A = 29.6$. Three approaches:

Approach	How	Comment
Bounded optimizer	`method="L-BFGS-B"` or `"Nelder-Mead"` with `bounds=`	Easiest; recommended default
Reparameterization	Optimize over unconstrained η and transform: $\lambda = e^{\eta}$ (positive), $A = 1/(1 + e^{-\alpha_A})$ (between 0 and 1)	Works with any optimizer; also used in Bayesian modeling (Week 13)
Penalty	Return a huge SSE when parameters are out of range	Creates discontinuities; avoid

4. Non-identifiability. Sometimes *several* parameter combinations fit equally well. An example is a two-component model whose two components can swap labels. This is a property of the model, not a failure of the optimizer, and multi-start reveals it (lab Part D).

5. Trust, but verify. `result.success==True` only means that the algorithm's stopping rule was met. Always (i) plot the fitted curve over the data, (ii) look at the residuals, and (iii) check that the

estimates are within plausible ranges.

A fitting checklist (use it in every lab from now on):

1. Plot the data.
2. Write the model and the discrepancy function; test them on a few hand-picked parameter values.
3. Choose sensible bounds and starting values.
4. Optimize, preferably with multi-start.
5. Check the convergence message, the plot, the residuals and the parameter ranges.
6. Run a recovery check: simulate data from the fitted parameters, refit, and compare (Week 7 makes this systematic).

> Advanced: conditionally linear parameters. In $f(t) = A\,e^{-\lambda t}$, for any fixed λ the best A has a closed form, because the model is linear in A: $\hat{A}(\lambda) = \sum_i y_i g_i / \sum_i g_i^2$ with $g_i = e^{-\lambda t_i}$. Substituting it back gives a one-dimensional problem in λ alone. This trick ("profiling out" linear parameters, or *variable projection*) makes fitting faster and more stable, and the resulting 1-D function is related to the *profile likelihood* of Week 5.

For the instructor

- Show the landscape first. A contour plot of SSE over (A, λ) (lab Part B) makes every later idea concrete: the minimum, the diagonal valley, the plateau. Put it on the projector early and keep coming back to it.
- Act out Nelder–Mead. Draw a triangle on the contour plot and do one reflection by hand (worked example Step 2). Students remember the "tumbling triangle" much better than the table of moves.
- Your strength: you know gradient descent and least squares. Spend less time on the algorithms and more on the *failure modes* (Part D of the lab): an optimizer reporting success at $A = 29.6$ is the most memorable lesson of the week.
- Typical questions.
  - *"Which optimizer should I use?"* For small cognitive models: Nelder–Mead or L-BFGS-B with bounds, always with multi-start.
  - *"Why not always use a grid?"* The cost grows exponentially with the number of parameters.
  - *"Is SSE the right measure?"* Only if noise is Gaussian with equal variance; next week replaces it with the likelihood.

2.1.5 Worked example

Data. A participant recalls proportions $y = (0.62, 0.41, 0.19)$ at $t = (1, 3, 7)$ days. Model: $f(t) = A\,e^{-\lambda t}$.

Step 1: a small grid search by hand

For $(A, \lambda) = (0.8, 0.2)$: predictions $0.8e^{-0.2} = 0.655$, $0.8e^{-0.6} = 0.439$, $0.8e^{-1.4} = 0.197$, so

$$\text{SSE} = (0.62-0.655)^2+(0.41-0.439)^2+(0.19-0.197)^2 = 0.00123+0.00084+0.00005 \approx 0.00212.$$

Repeating this for a 2×3 grid:

	$\lambda = 0.15$	$\lambda = 0.20$	$\lambda = 0.25$
$A = 0.8$	0.02281	0.00212	0.00364
$A = 0.9$	0.06638	0.02172	0.00791

The grid's best point is $(0.8, 0.20)$. Note the tilt: $(0.9, 0.25)$ beats $(0.9, 0.15)$ by far. A larger A "wants" a larger λ, which is the trade-off valley.

Step 2: one Nelder–Mead iteration by hand

Start with the simplex (triangle) of three grid points:

Vertex	(A, λ)	SSE	Rank
$\boldsymbol{\theta}_b$	(0.8, 0.25)	0.00364	best
$\boldsymbol{\theta}_g$	(0.8, 0.15)	0.02281	second-worst
$\boldsymbol{\theta}_w$	(0.9, 0.15)	0.06638	worst

1. Centroid of all vertices except the worst: $\bar{\boldsymbol{\theta}} = \big((0.8 + 0.8)/2,\ (0.25 + 0.15)/2\big) = (0.8, 0.20)$.
2. Reflect the worst vertex through the centroid: $\boldsymbol{\theta}_r = (0.8, 0.20) + \big((0.8, 0.20) - (0.9, 0.15)\big) = (0.7, 0.25)$.
3. Evaluate: predictions $(0.545, 0.331, 0.122)$, so $\text{SSE}(\boldsymbol{\theta}_r) = 0.01657$.
4. Decide: $0.00364 \le 0.01657 < 0.02281$. The reflected point is better than the second-worst but not better than the best, so we accept the reflection (no expansion).
5. New simplex: $\{(0.8, 0.25),\ (0.7, 0.25),\ (0.8, 0.15)\}$, and the worst SSE has dropped from 0.066 to 0.023.

Step 3: let the computer finish

Running `scipy.optimize.minimize(...,method="Nelder-Mead")` from $(0.8, 0.2)$ converges to $\hat{A} = 0.754$, $\hat{\lambda} = 0.200$, with SSE $= 3.7 \times 10^{-5}$ (RMSD $= 0.0035$).

Interpretation. The fitted model says this participant would recall about 75% immediately after learning and loses about 18% of the remaining material per day ($1 - e^{-0.2} = 0.181$), with a half-life of $\ln 2/0.2 \approx 3.5$ days. With only three data points and two parameters, the near-perfect fit says very little about whether the exponential model is *right*. Any reasonable two-parameter decreasing curve would fit three points well (Week 6).

2.1.6 Lab (2 hours)

Goals

1. Write an SSE function and explore the error surface of two forgetting models.
2. Fit by grid search, then with Nelder–Mead and L-BFGS-B; compare with last week's linearized fits.
3. Provoke and fix fitting failures: plateaus, local minima and out-of-range estimates.
4. Evaluate the fits (RMSD, residuals, comparison with the true values) and interpret the parameters.

Timing: Part A 15 min · Part B 25 min · Part C 25 min · Part D 35 min · Part E 20 min.

Setup and data

Code · کد `code/week03.py`, cell 1 · سلول 1

Part A: Get a feel for the discrepancy function (15 min)

Code · کد `code/week03.py`, cell 2 · سلول 2

Expected output:

```
theta = (0.9, 0.5):  power SSE = 0.0349   exponential SSE = 0.4511
theta = (0.8, 0.4):  power SSE = 0.0328   exponential SSE = 0.4145
theta = (0.9, 0.1):  power SSE = 0.9047   exponential SSE = 0.2192
theta = (0.6, 0.05):  power SSE = 0.3303   exponential SSE = 0.1145
```

Before running the next parts, predict: which parameter values will give the smallest SSE for each model? Write your guess down.

Part B: Grid search and the error surface (25 min)

Code · کد: `code/week03.py`, cell 3 · سلول 3

Expected output:

```
exp: grid minimum A = 0.655, rate = 0.0560, SSE = 0.10607  (42300
↪  evaluations)
pow: grid minimum A = 0.870, rate = 0.4239, SSE = 0.02592  (42300
↪  evaluations)
```

What to notice: both surfaces have a single basin, but it is an elongated, tilted valley: higher A goes with higher decay. That is the parameter trade-off. For the power model the true value (white ×) lies inside the valley but not at its bottom: with finite data, the best fit is never exactly the truth.

Part C: Fitting with an optimizer (25 min)

Code · کد: `code/week03.py`, cell 4 · سلول 4

Expected output:

```
model            method      A   rate    SSE   RMSD  f_evals success
  exp          L-BFGS-B 0.6568 0.0566 0.1061 0.1151     39.0    True
  exp       Nelder-Mead 0.6568 0.0566 0.1061 0.1151     69.0    True
  exp              grid 0.6550 0.0560 0.1061 0.1151  42300.0    True
  exp linearized (Wk 2) 0.5500 0.0320 0.1373 0.1310      NaN     NaN
  pow          L-BFGS-B 0.8712 0.4251 0.0259 0.0569     36.0    True
  pow       Nelder-Mead 0.8711 0.4251 0.0259 0.0569     71.0    True
  pow              grid 0.8700 0.4239 0.0259 0.0569  42300.0    True
  pow linearized (Wk 2) 0.7970 0.3700 0.0315 0.0628      NaN     NaN
```

Code · کد: `code/week03.py`, cell 5 · سلول 5

What to notice: - Nelder–Mead and L-BFGS-B agree to three or four decimals; the grid is within its spacing (0.005 for A) of them. The optimizers needed only 36–71 function evaluations instead of 42 300. - Least squares gives a lower SSE than the linearized fits of Week 2 (it is minimizing exactly that quantity), and the power estimates are much closer to the truth. - The path first heads quickly into the valley and then crawls along it, a typical pattern.

Part D: When fitting goes wrong (35 min)

D1. A plateau and a false "success". Start the exponential fit with an absurd decay rate, $\lambda = 20$ per day:

Code · کد: `code/week03.py`, cell 6 · سلول 6

Expected output:

```
BFGS         A =   29.571  lambda =  14.244  SSE = 1.1616  success =
 ↪  True  (Optimization terminated successfully.)
L-BFGS-B     A =   29.905  lambda =  14.289  SSE = 1.1616  success =
 ↪  True  (CONVERGENCE: NORM_OF_PROJECTED_GRADIENT_<=_PGTOL)
Nelder-Mead  A =    0.657  lambda =   0.057  SSE = 0.1061  success =
 ↪  True  (Optimization terminated successfully.)
```

(The wording of the L-BFGS-B message varies slightly between SciPy versions.) The two gradient-based methods report success but return a nonsensical fit: A is around 30 (a recall probability of 3000%!) and the predicted curve is essentially zero at every interval. They started on a plateau where the gradient is almost zero (Lecture §3.3, Advanced box). Nelder–Mead escaped. Lessons: (1) always check estimates and plots, not just `success`; (2) use bounds; (3) use sensible or multiple starting values.

D2. Reparameterization. Make the parameters unconstrained so that *any* optimizer respects the ranges $0 < A < 1$ and $\lambda > 0$:

Code · کد: `code/week03.py`, cell 7 · سلول 7

Expected output:

```
BFGS on reparameterized power model: A = 0.8712, d = 0.4251, SSE =
 ↪  0.02592
```

Same answer as in Part C, and the estimates are guaranteed to lie in range.

D3. Local minima and multi-start. A popular idea in memory research is that forgetting reflects two memory traces: a fast component that decays quickly and a slow one that lasts:

$$f(t) = A_1 e^{-\lambda_1 t} + A_2 e^{-\lambda_2 t}.$$

This model has $k = 4$ parameters, and its error surface has several basins. Fit it from 50 random starting points:

Code · کد: `code/week03.py`, cell 8 · سلول 8

Expected output:

```
Distinct final SSE values (rounded) and how often each was reached:
  SSE = 0.0201: 42 starts
  SSE = 0.0203: 1 starts
  SSE = 0.0337: 1 starts
  SSE = 0.0340: 1 starts
  SSE = 0.0359: 1 starts
  SSE = 0.1064: 1 starts
  SSE = 0.2010: 1 starts
  SSE = 0.2456: 1 starts
  SSE = 0.9092: 1 starts
Best 3 solutions (A1, lam1, A2, lam2):
   [0.58  0.36  0.264 0.   ] SSE = 0.02013
   [0.264 0.    0.58  0.36 ] SSE = 0.02013
   [0.264 0.    0.58  0.36 ] SSE = 0.02013
```

What to notice: - About 40 of 50 starts end in the same best basin, but the rest (about 1 in 6) get stuck in worse local minima, some with an SSE many times larger. (The exact counts differ slightly between SciPy versions; with SciPy 1.18 it is 41 of 50.) A single run from an unlucky start would have given a wrong answer, and no warning. - Among the best solutions, the fast and slow components may appear with labels swapped ($A_1, \lambda_1 \leftrightarrow A_2, \lambda_2$). Both describe the same curve. This is a non-identifiability built into the model, and a common fix is to impose an order constraint such as $\lambda_1 > \lambda_2$. - In the best solution one component has a rate of exactly 0 (a flat, non-decaying component of 0.264). The fitted two-trace model has therefore collapsed into the exponential-with-asymptote model of Exercise 4, with the same SSE (0.0201). Is that a meaningful "permanent memory" or just the extra parameters absorbing noise? Remember that the data came from a *power* model. Fit alone cannot tell (Weeks 6–7).

Part E: Evaluate and interpret (20 min)

Code · کد: `code/week03.py`, cell 9 · سلول 9

Expected output:

```
Power model: relative loss rate at t =  1 days: 0.213 per day
Power model: relative loss rate at t = 20 days: 0.020 per day
Exponential model: constant relative loss rate 0.057 per day
```

Interpret (write 2–3 sentences for each in your notebook):

1. Compare the residual patterns. Which model shows a systematic pattern, and what does that pattern say about the shape of forgetting in these data?
2. The power model's relative loss rate falls from about 0.21 per day after 1 day to about 0.02 per day after 20 days. Explain in plain language what this claims about memory.
3. The least-squares estimates ($\hat{A} \approx 0.87$, $\hat{d} \approx 0.43$) are closer to the truth than last week's linearized estimates but still not equal to it. Is this a failure of the method? (Hint: what would happen if you reran the experiment with a new random sample?)

Exercises (increasing difficulty)

Exercise 1 $\star$ (same minimizer). Fit the power model by minimizing SSE, MSE and RMSD in turn. Show that the estimates agree (up to optimizer tolerance). Why do we still prefer to *report* RMSD?

Exercise 2 $\star$ (Ebbinghaus by least squares). Refit the Ebbinghaus savings data from Week 2, Exercise 3 (values marked [verify]) with least squares, for both models. Compare the parameters and RMSD with your linearized fits from Week 2.

Exercise 3 $\star\star$ (weighted least squares). Binomial proportions have variance $f(1-f)/n$. Implement WSSE (Lecture §1) using the predicted f in the variance, refit the power model and compare the estimates with plain least squares. Which data points gain influence and which lose it? (Next week you will see that maximum likelihood does this weighting automatically.)

Exercise 4 $\star\star$ (more parameters, better fit?). Fit the three-parameter exponential-with-asymptote model $f(t) = B + (A - B)e^{-\lambda t}$, where B is the long-term asymptote, using bounds $0 \le B \le A \le 1$ (hint: reparameterize, or check the constraint after fitting). *Expected:* SSE ≈ 0.020, lower than the power model's ≈ 0.026, even though the data were generated by the power model! What does this tell you about using SSE to choose between models with different numbers of parameters? (We answer this properly in Weeks 6–7.)

Exercise 5 $\star\star\star$ (Advanced: Nelder–Mead from scratch). Implement Nelder–Mead for $k = 2$ using the moves in Lecture §3.2 (reflect, expand, inside/outside contract, shrink) and a stopping rule based on the spread of SSE values in the simplex. Run it on the power model from $(0.8, 0.2)$, plot the simplex every few iterations on the error surface, and compare the result and number of function evaluations with `scipy`.

Lab check-off

Show the instructor or TA: (1) the error-surface plots with grid minima (Part B), (2) the comparison table from Part C, (3) your multi-start histogram (Part D3) with one sentence on what it shows, and (4) your answer to interpretation question 1 (Part E).

2.1.7 Connection to AI & LLMs

Scaling laws are curve fitting. One of the most influential results in modern AI is that the test loss of language models falls smoothly with model size N, data size and compute, following a power law with an asymptote: $L(N) = E + B\,N^{-\alpha}$ (Kaplan et al., 2020; Hoffmann et al., 2022). Companies use such fits to decide how big the next model should be. Mathematically, this is the same problem as fitting a forgetting curve: a nonlinear model with three parameters, fitted by least squares with a numerical optimizer and multiple starting points. It has the same pitfalls too: extrapolation far beyond the data, and trade-offs between parameters (here E and α).

Code · کد: `code/week03.py`, cell 10 · سلول 10

Expected output:

```
E = 1.70, B = 3.22, alpha = 0.358, SSE = 0.00424
Extrapolated loss at N = 1e11: 1.754
```

Try it (no API needed). - ⋆ Fit a pure power law without the asymptote ($E = 0$). Compare the SSE and the extrapolation to $N = 10^{11}$. Which conclusion about "bigger is better" changes? - ⋆⋆ Fit only the four smallest models and extrapolate to the four largest. How far off is the prediction? Repeat with different seeds to see how stable the extrapolation is. - ⋆⋆ Plot the SSE surface over E and α (with B at its best value for each pair). Is there a ridge, as in the Week 3 lab?

Optional reading: Kaplan, J., et al. (2020). Scaling laws for neural language models. *arXiv:2001.08361*. Hoffmann, J., et al. (2022). Training compute-optimal large language models. *arXiv:2203.15556*.

2.1.8 Common mistakes & misconceptions

1. "**`success=True`** means the fit is good." It only means a stopping rule was met (Part D1). Always inspect estimates, plots and residuals.
2. One start is enough. For anything beyond simple two-parameter models, run multi-start. The cost is small; the risk of reporting a local minimum is not.
3. Ignoring parameter ranges. Unbounded optimizers can return impossible values (negative rates, probabilities above 1). Use bounds or reparameterization.
4. Comparing SSE across models with different numbers of parameters. The more flexible model almost always wins on SSE (Exercise 4). That is a question of model *evaluation*, not estimation (Weeks 6–7).
5. Comparing SSE across datasets of different sizes. SSE grows with N; use RMSD for comparisons.

6. Treating the minimizer as the true value. $\hat{\boldsymbol{\theta}}$ is an estimate with sampling error. A different sample would give a different estimate (Week 5 quantifies this).
7. Forgetting what least squares assumes. Plain SSE assumes equal, Gaussian noise for all observations. Proportions, counts and RTs usually violate this.
8. Using grid search for many parameters. The cost explodes (100^k evaluations). Use grids only for visualization and starting values.

2.1.9 Check your understanding

1. Why do SSE, MSE and RMSD have the same minimizer? Which one would you report, and why? Answer MSE = SSE/N and RMSD = $\sqrt{}$MSE are strictly increasing transformations of SSE (N is fixed), so the same parameters minimize all three. Report RMSD, because it is in the units of the data and comparable across datasets of different sizes.

2. What is a parameter trade-off, and how does it appear on an error surface? Answer Changes in one parameter can be compensated by changes in another, leaving the predictions (and the SSE) almost unchanged. On the surface it appears as an elongated, tilted valley of low SSE. Estimates are then correlated and poorly determined along the valley direction.

3. With 50 grid values per parameter, how many SSE evaluations does a grid search need for a model with 6 parameters? What does this imply? Answer $50^6 \approx 1.56 \times 10^{10}$ evaluations. Grid search is infeasible beyond 2–3 parameters (the curse of dimensionality); we need iterative optimizers.

4. In one Nelder–Mead iteration, the reflected point is better than the current best vertex. What does the algorithm do next? Answer It tries an expansion: a point twice as far from the centroid in the same direction. It keeps the expanded point if it is better than the reflected point, otherwise the reflected point.

5. Why can a gradient-based optimizer stop on a plateau and report success? Answer Its stopping rule is based on the gradient (or step size) being nearly zero. On a plateau the gradient is close to zero even though the point is not a minimum, so the convergence criterion is met.

6. Give two ways to keep a decay-rate parameter positive during fitting. Answer (i) Use a bounded optimizer (e.g., L-BFGS-B or Nelder–Mead with `bounds=[(1e-6,None)]`). (ii) Reparameterize: optimize η on the whole real line and set $\lambda = e^{\eta}$.

7. Multi-start fits of the two-component model reach the same minimum SSE with $(A_1, \lambda_1, A_2, \lambda_2) = (0.58, 0.36, 0.26, 0.00)$ and $(0.26, 0.00, 0.58, 0.36)$. Is this a problem with the optimizer? Answer No. It is label switching: swapping the two components gives the identical curve, so the model is not identifiable without an ordering constraint (e.g., $\lambda_1 > \lambda_2$). The optimizer correctly found two equivalent global minima.

2.1.10 Assignment

No new graded assignment is released this week. A1 (From verbal to formal) is due at the start of the Week 4 lecture. Today's lab gives you everything needed for A1's calibration step if you want to check your closed-form derivation numerically.

Optional practice (ungraded, recommended before Week 4): complete lab Exercises 3 and 4. Exercise 3 (weighted least squares) is the bridge to maximum likelihood, and Exercise 4 previews the central problem of model evaluation. Solutions will be discussed briefly at the start of the Week 4 lab.

2.1.11 Further reading

- Farrell & Lewandowsky (2018), chapter 3. Basic parameter estimation: discrepancy functions, error surfaces, the simplex algorithm, and practical issues such as local minima and starting values. *(Main reading.)*
- Ghodousian (1401), *Mabani-ye ravesh-haye mostaghim dar behine-sazi gheyr-khatti* (in Persian): direct search methods for nonlinear optimization, including simplex-type methods. Recommended for students who want the theory behind Nelder–Mead [verify chapter].
- Verguts (2022): the chapter on optimization and gradient descent, useful for the gradient-based view that returns in Week 15 (learning models) [verify chapter].

Preview for Week 4: instead of asking "which parameters minimize squared error?", we ask "which parameters make the observed data most probable?" This is maximum likelihood. It handles binomial data correctly, and it is the foundation for AIC/BIC (Week 7) and Bayesian inference (Weeks 9–13).

2.2 Week 4: Maximum Likelihood Estimation

> Chapter 2: Parameter estimation methods · Lecture 1 h · Lab 2 h Running data: the Week 2–3 forgetting experiment (binomial recall counts), plus a new SDT experiment with three payoff conditions.

2.2.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
estimate / estimator	برآورد / برآوردگر	مقدار به‌دست‌آمده برای پارامتر / روش به دست آوردن آن
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان
binomial distribution	توزیع دوجمله‌ای	توزیع تعداد موفقیت در n کوشش
signal detection theory (SDT)	نظریه‌ی تشخیص علامت	مدلی که حساسیت را از سوگیری پاسخ جدا می‌کند
log-likelihood / NLL	لگاریتم درست‌نمایی / منفی آن	مقداری که بهینه‌سازها کمینه یا بیشینه می‌کنند
least squares	کمترین مربعات	روش برازش با کمینه کردن مجموع مربع خطاها
maximum likelihood estimate (MLE)	برآورد بیشینه‌ی درست‌نمایی	پارامترهایی که داده را محتمل‌ترین می‌کنند
noise	نویز	تغییرات تصادفی در شواهد یا در داده
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
deviance (G^2)	انحراف (deviance)	معیار عدم برازش نسبت به مدل اشباع‌شده
bounds / constraints	قیدها / کران‌ها	محدودیت‌های مجاز برای مقدار پارامترها
response bias	سوگیری پاسخ	تمایل به یک پاسخ، مستقل از محرک
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز
trial	کوشش (trial)	یک بار انجام تکلیف توسط شرکت‌کننده
optimizer	بهینه‌ساز	الگوریتمی که کمینه‌ی تابع را پیدا می‌کند
SSE / RMSD	مجموع مربعات خطا / ریشه‌ی میانگین مربع انحراف	معیارهای فاصله‌ی داده و پیش‌بینی
saturated model	مدل اشباع‌شده	مدلی که داده را دقیقاً بازتولید می‌کند
likelihood-ratio test (LRT)	آزمون نسبت درست‌نمایی	مقایسه‌ی دو مدل تودرتو
exponential / power function	توانی / تابع نمایی	دو شکل رایج برای منحنی فراموشی و یادگیری
liberal / conservative	آسان‌گیر / محافظه‌کار	تمایل به گفتن «بله» / «نه»
token	token (واحد متن)	کلمه یا بخشی از کلمه که مدل زبانی پیش‌بینی می‌کند
perplexity	perplexity (سردرگمی)	e به توان NLL هر token

2.2.2 Learning outcomes

By the end of this week, students will be able to:

1. Distinguish probability (data varying, parameters fixed) from likelihood (data fixed, parameters varying).
2. Derive the likelihood and log-likelihood for binomial (Bernoulli) and Gaussian data models, and show that least squares is maximum likelihood under equal-variance Gaussian noise.
3. Implement maximum likelihood estimation (MLE) by minimizing the negative log-likelihood (NLL) in Python, for forgetting curves and for a multi-condition SDT model.
4. Evaluate absolute goodness of fit with the deviance G^2 and compare MLE with least-squares estimates.
5. Interpret constrained models (e.g., "payoffs change bias, not sensitivity") as substantive psychological hypotheses.

2.2.3 Motivation & big picture

In Week 3, least squares treated every data point as equally reliable. But a recall proportion of 0.50 measured on 50 items is *noisier* than a proportion of 0.95: binomial variance $f(1-f)/n$ depends on f. Least squares also has nothing to say about data that are not continuous numbers, such as a single yes/no response, a choice among options, or a confidence rating.

Maximum likelihood replaces "make predictions close to data" with a sharper question:

> Which parameter values make the data we actually observed most probable?

To answer it, the model must say how probable *any* possible dataset is, which is exactly the noise model (component 4 of Week 2's anatomy). Once we have it, MLE handles counts, choices, RTs and mixtures of these in the same way. It also gives us, for free, an absolute measure of misfit (the deviance), the basis for standard errors (Week 5), information criteria (Week 7), and the likelihood term in Bayes' rule (Weeks 9–13). The likelihood is the single most important object in this course.

Where is this used?

- Almost every published cognitive model is fitted by maximum likelihood or by its Bayesian extension (Weeks 9–13). When a paper reports "model fits", "AIC" or "log-likelihood", MLE is underneath.
- Choice data: which option a person picks (in a memory test, a gamble, a learning task) is not a continuous number, so least squares does not apply. The likelihood handles choices naturally.
- Psychophysics: the psychometric function (probability of detecting a stimulus as a function of its strength) is fitted by binomial MLE in every vision and hearing lab.

- Testing hypotheses inside a model: "does the payoff change sensitivity or only bias?" becomes a comparison of two likelihoods (Section 6).
- AI: large language models are trained by maximizing the likelihood of text (see the AI & LLMs section).

2.2.4 Lecture notes

In plain words Imagine trying different parameter values one by one. For each value, ask the model: "if this were the true value, how probable would the data I actually collected be?" That number is the likelihood of that parameter value. The maximum likelihood estimate (MLE) is the parameter value under which the observed data would have been most probable. Example: someone recalls 7 of 10 words. If their true recall probability were 0.5, getting exactly 7 would be fairly unlikely (0.117); if it were 0.7, it would be more likely (0.267). So 0.7 is the better explanation, and indeed the MLE is 7/10. In practice we work with the logarithm of the likelihood (so products become sums) and we let the computer *minimize* its negative (NLL), using the optimizers of Week 3. Two bonuses: least squares turns out to be a special case of MLE, and the likelihood gives an absolute test of misfit (G^2).

Suggested timing (60 min): probability vs. likelihood 10 · binomial MLE 15 · log-likelihood and NLL 10 · Gaussian case (LS = MLE) 10 · goodness of fit G^2 10 · constrained models 5.

1. Probability vs. likelihood

The same function $p(\mathrm{y} \mid \boldsymbol{\theta})$ can be read in two ways:

Reading	Fixed	Varies	Name	Sums/integrates to 1?
Before the experiment	$\boldsymbol{\theta}$	y	Probability (mass/density) of data	Yes, over y
After the experiment	y (observed)	$\boldsymbol{\theta}$	Likelihood $L(\boldsymbol{\theta}) = p(\mathrm{y} \mid \boldsymbol{\theta}, \mathcal{M})$	No, not over $\boldsymbol{\theta}$

The likelihood is not the probability that the parameters are correct. It is the probability of the data *under* each candidate parameter value. (Turning it into a probability distribution over $\boldsymbol{\theta}$ requires a prior: Bayes, Week 9.)

Example. A participant recalls $y = 7$ of $n = 10$ items. Under a binomial model with recall probability θ:

$$L(\theta) = \binom{10}{7} \theta^7 (1-\theta)^3.$$

$L(0.5) = 0.117$, $L(0.7) = 0.267$: the observed data are about 2.3 times more probable if $\theta = 0.7$ than if $\theta = 0.5$.

Reading the math $L(\theta) = \binom{10}{7}\theta^7(1-\theta)^3$:

- θ^7: the probability that 7 specific items are recalled (each with probability θ).
- $(1-\theta)^3$: the probability that the other 3 are forgotten.
- $\binom{10}{7} = 120$ ("10 choose 7"): the number of different ways to pick which 7 of the 10 items are the recalled ones. It does not depend on θ, so it never changes *where* the maximum is.
- Plug in $\theta = 0.7$: $120 \times 0.7^7 \times 0.3^3 = 120 \times 0.0824 \times 0.027 = 0.267$.

The key mental switch: in probability, θ is fixed and we ask about different data; in likelihood, the data are fixed (we already have them) and we compare different θ.

2. The maximum likelihood estimate

$$\hat{\boldsymbol{\theta}}_{\text{ML}} = \arg\max_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \arg\max_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}), \qquad \ell(\boldsymbol{\theta}) = \ln L(\boldsymbol{\theta}).$$

Why the log? 1. With independent observations the likelihood is a product, $L = \prod_i p(y_i \mid \boldsymbol{\theta})$, which underflows to 0 on a computer for even a few hundred trials. The log turns it into a sum, $\ell = \sum_i \ln p(y_i \mid \boldsymbol{\theta})$. 2. The log is strictly increasing, so it has the same maximizer. 3. Sums are easier to differentiate.

Optimizers *minimize*, so in code we minimize the negative log-likelihood, $\text{NLL}(\boldsymbol{\theta}) = -\ell(\boldsymbol{\theta})$.

Analytic example (binomial). $\ell(\theta) = \text{const} + y \ln\theta + (n-y)\ln(1-\theta)$. Setting the derivative to zero:

$$\frac{d\ell}{d\theta} = \frac{y}{\theta} - \frac{n-y}{1-\theta} = 0 \quad \Rightarrow \quad \hat{\theta} = \frac{y}{n}.$$

The MLE of a probability is the observed proportion, which is reassuring. For cognitive models the predicted probability is a *function* of the parameters, $\theta_i = f(x_i; \boldsymbol{\theta})$, and we usually need a numerical optimizer.

3. The binomial likelihood for forgetting curves

With y_i items recalled out of n_i at interval t_i, and independent intervals:

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^{N} \left[\ln \binom{n_i}{y_i} + y_i \ln f(t_i;\boldsymbol{\theta}) + (n_i - y_i)\ln\big(1 - f(t_i;\boldsymbol{\theta})\big) \right].$$

- The binomial coefficient does not depend on $\boldsymbol{\theta}$, so it does not affect the MLE (but keep it if you want to compare log-likelihoods across different data codings; `scipy.stats.binom.logpmf` includes it).
- Observations predicted to be near 0 or 1 have small variance, so a prediction that misses them is punished more heavily. MLE performs the variance weighting of Week 3's weighted least squares *automatically*.

- Numerical safety: if the model predicts $f = 0$ where $y > 0$, then $\ln 0 = -\infty$. Clip predictions to $[\epsilon, 1-\epsilon]$ with, e.g., $\epsilon = 10^{-10}$.

Bernoulli vs. binomial. If we keep the individual 0/1 responses, $\ell = \sum_{\text{trials}}[y \ln f + (1-y)\ln(1-f)]$. This differs from the binomial version only by the constant binomial coefficients, so it gives the same MLE.

Reading the math The forgetting log-likelihood is the single-proportion log-likelihood from Section 2, written once per retention interval and added up:

- for interval i: $y_i \ln f(t_i)$ rewards predicting a high recall probability where many items were recalled; $(n_i - y_i)\ln(1 - f(t_i))$ rewards predicting a low recall probability where many were forgotten.
- Adding over intervals is allowed because the intervals are assumed independent: the log of a product of probabilities is the sum of the logs.
- Why it weights points automatically: if the model predicts $f = 0.95$ but only 40 of 50 are recalled, the term $10\ln(0.05) \approx -30$ is very costly. Near $f = 0.5$, the same absolute miss costs much less. Points that *should* be precise are trusted more.

4. The Gaussian case: least squares is maximum likelihood

Suppose $y_i = f(x_i; \boldsymbol{\theta}) + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma)$, independent and with the same σ. Then

$$\ell(\boldsymbol{\theta}, \sigma) = \sum_{i=1}^{N} \ln\left[\frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{(y_i - f(x_i;\boldsymbol{\theta}))^2}{2\sigma^2}\right)\right] = -\frac{N}{2}\ln(2\pi\sigma^2) - \frac{\text{SSE}(\boldsymbol{\theta})}{2\sigma^2}.$$

For any fixed σ, maximizing ℓ over $\boldsymbol{\theta}$ means minimizing SSE. So least squares = MLE under i.i.d. equal-variance Gaussian noise. Maximizing over σ as well gives $\hat{\sigma}^2 = \text{SSE}/N$.

This explains *when* least squares is appropriate (continuous, roughly symmetric noise with constant variance) and when it is not (counts, proportions, choices, skewed RTs).

> Advanced: properties of the MLE. Under regularity conditions (identifiable model, true parameters not on the boundary, N large), the MLE is *consistent* (converges to the true value), *asymptotically normal*, and *asymptotically efficient* (no consistent estimator has smaller variance in the limit). Its sampling variance is approximated by the inverse of the Fisher information, the curvature of $-\ell$ at its minimum. Week 5 uses exactly this to compute standard errors. In small samples the MLE can be biased (e.g., $\hat{\sigma}^2 = \text{SSE}/N$ underestimates σ^2).

5. Absolute goodness of fit: the deviance G^2

The log-likelihood value alone has no absolute meaning. To judge it, compare the model with the saturated model, which has one free probability per data cell and therefore reproduces every observed proportion exactly ($\hat{f}_i = y_i/n_i$). The deviance is

$$G^2 = 2\big[\ell_{\text{saturated}} - \ell(\hat{\boldsymbol{\theta}})\big] = 2\sum_i \left[y_i \ln \frac{y_i}{n_i \hat{f}_i} + (n_i - y_i) \ln \frac{n_i - y_i}{n_i(1-\hat{f}_i)} \right],$$

with $0 \ln 0 = 0$. If the model is correct and counts are not too small, G^2 approximately follows a χ^2 distribution with df $=$ (number of cells) $- k$. A large G^2 (small p) means the model misfits *more than sampling noise can explain*.

Caveats: with very large N, any small misfit becomes "significant"; with small N, real misfit can go undetected. A non-significant G^2 does not prove the model is right (Week 6).

Reading the math

- The saturated model is the "cheating" model that predicts every observed proportion exactly. Its log-likelihood is the best any model could possibly achieve on these data.
- $G^2 = 2(\ell_{\text{saturated}} - \ell(\hat{\boldsymbol{\theta}}))$ measures how far our model falls short of that ceiling. $G^2 = 0$ would be a perfect fit.
- Degrees of freedom = number of data cells − number of fitted parameters. With 8 intervals and 2 parameters, df = 6.
- If the model is true, G^2 behaves like a χ^2 variable with that df, whose average is df. So $G^2 \approx 6$ with df = 6 is a normal amount of misfit (just noise); $G^2 = 26$ with df = 6 ($p = .0002$) means the model systematically misses the data.

6. Constrained models as hypotheses

MLE makes it easy to fit several conditions at once with some parameters shared. Example: an SDT experiment with three payoff conditions (rewarding "yes" responses, neutral, rewarding "no").

Model	Free parameters	k	Hypothesis
Shared sensitivity	d'; c_1, c_2, c_3	4	Payoffs change *bias* only
Separate	d'_1, d'_2, d'_3; c_1, c_2, c_3	6	Payoffs may also change *sensitivity* (this is the saturated model for 6 rates)

The joint log-likelihood is the sum over conditions of the binomial log-likelihoods of hits and false alarms. Comparing the two models' maximized log-likelihoods tests a substantive question. With nested models like these, $2(\ell_6 - \ell_4)$ is approximately χ^2 with 2 df: the likelihood-ratio test, developed in Week 7.

> Advanced: SDT MLE and the formulas. For a single condition, the MLE of (d', c) from counts is exactly what the Week 2 formulas give *without* correction, because the model is saturated (2 parameters, 2 rates) and the MLE reproduces the observed rates. The log-linear correction is *not* MLE; it is closer to a Bayesian estimate with a weak prior (Week 9).

For an engineer

- This is maximum-likelihood estimation exactly as in estimation theory (ML channel or frequency estimation, ML sequence detection). The only change is the noise model: binomial counts instead of AWGN.
- LS = ML under AWGN (Section 4) is a result you already use: with white Gaussian noise of known, equal variance, the ML estimator minimizes the squared error.
- The Fisher information and the asymptotic variance of the MLE (Advanced box) are the same quantities as in the Cramér–Rao lower bound. Week 5 uses them to compute standard errors.
- Comparing a constrained and a free model with $2(\ell_{\text{free}} - \ell_{\text{constrained}})$ is the generalized likelihood-ratio test (GLRT): both hypotheses have unknown parameters that are replaced by their ML estimates.
- Payoffs and criterion shifts: in Bayesian detection the optimal threshold moves with the costs and priors. Section 6 tests whether *people* move only their threshold (c) when the payoffs change, while their "SNR" (d') stays the same.

For the instructor

- The one sentence to repeat: "the likelihood is the probability of the data we observed, as a function of the parameters". Write it on the board and leave it there all session.
- Plot it. Draw $L(\theta)$ for 7 of 10 (worked example, Part 1) as a curve over θ from 0 to 1. Ask: where is the peak? Does the area under the curve equal 1? (No.) Then draw it for 70 of 100: same peak, much narrower curve. That narrowness is precision, the topic of Week 5.
- Use your background. Many students find MLE abstract; you can present it as "the estimation method engineers use when the noise is not Gaussian".
- Typical questions.
  - *"Is the likelihood the probability that θ is correct?"* No. That needs a prior (Week 9).
  - *"Why minimize NLL instead of maximizing L?"* Same answer, but sums of logs avoid numerical underflow, and optimizers are written to minimize.
  - *"Is a non-significant G^2 proof the model is right?"* No: it only means the misfit is not larger than the noise, and with few data, real misfit can hide.
- Time management. Section 4 (Gaussian case) can be shown quickly as "LS is a special case". Spend the saved time on Section 5 (G^2).

2.2.5 Worked example

Part 1: likelihood of a single proportion

$y = 7$ recalled out of $n = 10$:

θ	0.5	0.6	0.7	0.8
$L(\theta) = \binom{10}{7}\theta^7(1-\theta)^3$	0.117	0.215	0.267	0.201
$\ell(\theta)$	−2.144	−1.537	−1.321	−1.603

The maximum is at $\hat{\theta} = 7/10 = 0.7$. Note that the likelihood values do not sum to 1 across θ: likelihood is not a probability distribution over parameters.

Part 2: a two-interval forgetting experiment

$y = (30, 13)$ items recalled out of $n = 50$ at $t = (1, 7)$ days. Model: $f(t) = Ae^{-\lambda t}$.

Step 1: NLL for a candidate. For $(A, \lambda) = (0.70, 0.15)$: $f(1) = 0.7e^{-0.15} = 0.6025$, $f(7) = 0.7e^{-1.05} = 0.2450$.

$$\ell = \ln \text{Binom}(30; 50, 0.6025) + \ln \text{Binom}(13; 50, 0.2450) = -2.167 + (-2.088) = -4.255,$$

so NLL $= 4.255$.

Step 2: compare. For $(0.70, 0.12)$: $f = (0.621, 0.302)$ and NLL $= 4.488$, which is worse. The first candidate makes the data more probable.

Step 3: the MLE. With 2 parameters and 2 data points the model can match both observed proportions exactly: $Ae^{-\lambda} = 0.60$ and $Ae^{-7\lambda} = 0.26$. Dividing gives $e^{6\lambda} = 0.60/0.26$, so $\hat{\lambda} = \ln(2.308)/6 = 0.139$ and $\hat{A} = 0.60\, e^{0.139} = 0.690$, with NLL $= 4.225$ (the minimum). A numerical optimizer gives the same values.

Interpretation. The MLE says this participant recalls about 69% at the start and loses about 13% of what remains per day ($1 - e^{-0.139}$). Because $k = N$ here, $G^2 = 0$: the model *cannot* misfit, so the perfect fit carries no evidence for the exponential form. Only data with more cells than parameters can test a model.

2.2.6 Lab (2 hours)

Goals

1. Plot a likelihood function and see why we work with log-likelihoods.
2. Fit exponential and power forgetting models by MLE; compare with least squares; evaluate with G^2.
3. Verify numerically that Gaussian MLE equals least squares.
4. Fit a constrained multi-condition SDT model and interpret what it says about payoffs.

Timing: Part A 15 min · Part B 35 min · Part C 15 min · Part D 35 min · Part E 20 min (exercises).

Setup

Code · کد: `code/week04.py`, cell 1 · سلول 1

Part A: the likelihood function (15 min)

Code · کد: `code/week04.py`, cell 2 · سلول 2

Expected output:

```
Area under L(theta): 0.091 (not 1: L is not a distribution over theta)
Product of 2000 trial probabilities: 0.0
Sum of their logs: -1239.52
```

Part B: MLE for forgetting curves (35 min)

Code · کد: `code/week04.py`, cell 3 · سلول 3

Expected output:

```
model method      A   rate     NLL    SSE
  exp    MLE 0.6377 0.0458 29.8123 0.1100
  exp     LS 0.6568 0.0566 30.6983 0.1061
  pow    MLE 0.8773 0.4214 19.8303 0.0262
  pow     LS 0.8712 0.4251 19.8585 0.0259
True (power): A = 0.9, d = 0.5
```

What to notice: - Each method wins on its *own* criterion: MLE has the lowest NLL, LS the lowest SSE. - For the (true) power model the two methods agree closely. - For the exponential model the

two methods differ more (decay rate 0.046 vs. 0.057). The binomial likelihood gives more weight to the precisely measured points near the floor (long delays), so the ML curve decays more slowly in order to stay closer to them.

> Warning: a real optimizer trap. Replace `"L-BFGS-B"` with `"Nelder-Mead"` in the ML fit above (keeping `bounds=bounds`) and rerun. For the exponential model, scipy's bounded Nelder–Mead collapses against the bound `A=1` and reports `success` with NLL = 79.5, far worse than the true minimum of 29.8 (it is even worse than the LS estimate's NLL). Bounded simplex methods can degenerate when vertices are pushed onto a bound. Two quick checks would catch this: an estimate sitting exactly on a bound, and a "MLE" whose NLL is *higher* than that of some other parameter values you have tried. Multi-start (Week 3) is the general cure.

Now evaluate absolute fit with the deviance:

> Code · کد: `code/week04.py`, cell 4 · سلول 4

Expected output:

```
exp: G2 =  26.27, df = 6, p = 0.0002
pow: G2 =   6.31, df = 6, p = 0.3895
```

Interpret: the exponential model misfits far more than binomial noise allows ($p = .0002$): we can reject it as a description of these data. The power model's misfit is consistent with sampling noise. That does not prove it is the right model; it means these data give no reason to reject it.

> Code · کد: `code/week04.py`, cell 5 · سلول 5

Part C: Gaussian MLE equals least squares (15 min)

Treat the proportions as continuous and assume Gaussian noise with a free σ (a third parameter). The estimates of A and d should match least squares, and $\hat{\sigma}$ should equal $\sqrt{\mathrm{SSE}/N}$.

> Code · کد: `code/week04.py`, cell 6 · سلول 6

Expected output:

```
Gaussian MLE: A = 0.8712, d = 0.4251, sigma = 0.0569
Least squares: A = 0.8711, d = 0.4251, sqrt(SSE/N) = 0.0569
```

Part D: a constrained multi-condition SDT model (35 min)

The experiment. Participants detect a faint tone under three payoff schemes: liberal (hits rewarded), neutral, and conservative (false alarms penalized). 80 signal and 80 noise trials per condition. We simulate one participant whose sensitivity is the same in all conditions and whose criterion shifts with the payoffs.

Code · کد: `code/week04.py`, cell 7 · سلول 7

Expected output:

```
      condition  hits  n_signal  fas  n_noise
0       liberal    73        80   33       80
1       neutral    58        80   16       80
2  conservative    44        80    7       80

Shared d':   d' = 1.496, c = [-0.555  0.124  0.618], NLL = 13.134
Separate d': d' = [1.577 1.439 1.482], c = [-0.568  0.122  0.615], NLL =
↪  13.043

Likelihood-ratio statistic = 0.18, df = 2, p = 0.913
```

Note that the estimated `d'` (about 1.5) is higher than the generating value of 1.2. With 80 trials per cell that is ordinary sampling error. Week 5 shows how to put a confidence interval around it.

Interpret (write in your notebook): 1. Does constraining d' to be equal across payoff conditions cost much likelihood? What do you conclude about the effect of payoffs? 2. The separate-d' estimates differ from each other by a few tenths. Why is that not evidence that payoffs change sensitivity? 3. Check that the separate-d' model reproduces the observed hit and false-alarm rates exactly. Why must it?

Exercises (increasing difficulty)

Exercise 1 $\star$. Derive by hand the MLE of θ for a Bernoulli sequence with y successes in n trials, and check it numerically with `minimize_scalar` for $y = 13$, $n = 40$.

Exercise 2 $\star$. In Part B, refit both models with the Bernoulli likelihood on item-level data (expand each count into 0/1 responses). Show that the estimates are identical to the binomial fits and that the NLLs differ by a constant. What is the constant?

Exercise 3 $\star\star$ (zero counts). Set the last two counts to 0 (`recalled[-2:]=0`). Which fitting methods from Weeks 2–4 still work? Fit both models by MLE and comment on what happens to the estimates and to G^2.

Exercise 4 ⋆⋆ (a third SDT model). Fit a model in which the criterion is the *same* in all conditions but d' differs ($k = 4$). Compare its NLL with the shared-d' model. Both have $k = 4$; why can you compare them directly by NLL here? Which one is psychologically more plausible for a payoff manipulation?

Exercise 5 ⋆⋆⋆ (Advanced: LS vs. MLE by simulation). Simulate 500 datasets from the power model ($A = 0.9$, $d = 0.5$, the 8 intervals, $n = 50$), fit each by LS and by MLE, and compare the two estimators of d: mean (bias), SD, and root-mean-squared error. Which is better here? Repeat with $n = 10$. (This is good preparation for A2.)

Lab check-off

Show: (1) the LS-vs-MLE table and G^2 results from Part B, (2) the Gaussian-vs-LS comparison from Part C, and (3) your answers to the interpretation questions of Part D.

2.2.7 Connection to AI & LLMs

LLMs are trained by maximum likelihood. The "cross-entropy loss" used to train every language model is the negative log-likelihood of the next token, averaged over a huge text corpus. Training an LLM is therefore an MLE problem with billions of parameters, solved by gradient descent instead of Nelder–Mead. Two quantities you meet in AI papers are this week's quantities in disguise: loss = NLL per token, and perplexity = $e^{\text{NLL per token}}$, the effective number of equally likely choices the model is "confused" between.

The simplest language model, a bigram model, has a closed-form MLE: $P(w_2 \mid w_1) = \text{count}(w_1 w_2)/\text{count}(w_1)$. It shows a familiar problem. Any word pair never seen in training gets probability 0, so a single new pair makes the likelihood of a whole sentence 0 (NLL = ∞). Smoothing solves this, in the same way that the log-linear correction of Week 2 solved hit rates of 1.

> Code · کد: `code/week04.py`, cell 8 · سلول 8

Expected output:

```
P(cat | the) = 0.333
'the dog sat on the mat .'   MLE      NLL =  3.584  perplexity =   1.82
'the dog sat on the mat .'   add-one  NLL =  9.537  perplexity =   4.90
'the dog saw the cat .'      MLE      NLL =    inf  perplexity =    inf
'the dog saw the cat .'      add-one  NLL =  9.624  perplexity =   6.85
```

Try it (no API needed). - ⋆ Which word pair in "the dog saw the cat ." was never seen in the corpus? Check by hand that the MLE of $P(\text{cat} \mid \text{the})$ is 1/3. - ⋆⋆ Smoothing makes the *training* sentence less likely (NLL 3.6 → 9.5) but makes the new sentence possible. Replace add-one with add-k and choose k by leave-one-sentence-out cross-validation (a preview of Week 6). - ⋆⋆ Discussion: the MLE of a

bigram model reproduces the training corpus as closely as possible. Why is "maximizing likelihood on internet text" not the same as "learning to tell the truth"?

2.2.8 Common mistakes & misconceptions

1. "The likelihood is the probability that the parameters are true." It is the probability (density) of the *data* given the parameters. It does not integrate to 1 over $\boldsymbol{\theta}$.
2. Multiplying probabilities instead of summing logs. The product underflows to 0 for realistic numbers of trials (Part A).
3. Maximizing when the optimizer minimizes. `minimize` needs the negative log-likelihood. Forgetting the minus sign sends the optimizer to the *worst* parameters.
4. Not clipping probabilities. Predicted probabilities of exactly 0 or 1 produce $-\infty$ log-likelihoods and crash the optimizer.
5. Comparing NLLs across different datasets or data codings. NLL depends on the number of observations and on constants (e.g., binomial coefficients). Compare only models fitted to the same data, coded the same way.
6. Using least squares for choices or counts by default. It ignores the data's noise structure. Use the likelihood that matches how the data were generated.
7. Reading a non-significant G^2 as proof the model is correct. It only says the misfit is not detectably larger than sampling noise at this sample size.
8. Forgetting that a saturated or exactly-determined model always fits perfectly (worked example, Part D separate-d' model). A perfect fit is then uninformative.

2.2.9 Check your understanding

1. In one sentence each: what varies and what is fixed in a probability distribution vs. a likelihood function? Answer Probability: parameters fixed, data vary, and it sums/integrates to 1 over data. Likelihood: data fixed at the observed values, parameters vary, and it need not integrate to 1 over parameters.

2. Give two reasons for working with log-likelihoods. Answer (i) Numerical: products of many probabilities underflow, while sums of logs do not. (ii) The log is monotone, so the maximizer is unchanged, and sums are easier to differentiate and optimize.

3. Under what assumptions is least squares equivalent to maximum likelihood? Show the key step. Answer Independent Gaussian errors with equal variance σ^2. Then $\ell = -\frac{N}{2}\ln(2\pi\sigma^2) - \text{SSE}/(2\sigma^2)$, so for any σ, maximizing ℓ over $\boldsymbol{\theta}$ is minimizing SSE.

4. A model with 2 parameters is fitted to 8 binomial cells and gives $G^2 = 4.1$. What are the df, and what do you conclude? Answer df $= 8 - 2 = 6$. A χ^2_6 has mean 6, so 4.1 is well within the range expected from sampling noise ($p \approx 0.66$). There is no evidence of misfit, but that does not prove the model correct.

5. Why does MLE with a binomial likelihood weight data points differently than plain least squares does? Answer The binomial log-likelihood reflects the variance $f(1-f)/n$: points predicted near 0 or 1 (small variance) are fitted more tightly, and points with larger n count more. Plain least squares weights every squared error equally.

6. In the payoff experiment, what psychological hypothesis does the shared-d' model express, and how would you test it? Answer That payoffs change only response bias (criterion), not perceptual sensitivity. Test it by comparing the maximized log-likelihood with the separate-d' model: $2(\ell_6 - \ell_4) \sim \chi^2_2$ approximately (likelihood-ratio test), or with AIC/BIC (Week 7).

7. You fit a model and the optimizer returns NLL $= -35.2$ (negative). Is something wrong? Answer Not necessarily. For continuous data, densities can exceed 1 (e.g., a narrow Gaussian), so log-densities can be positive and the NLL negative. For discrete data (binomial), probabilities are ≤ 1, so the NLL must be ≥ 0, and a negative value would indicate a bug.

2.2.10 Assignment

A2: Least squares vs. maximum likelihood (1.0 point of 20; released Week 4, due at the start of the Week 6 lecture)

Data. Generate the dataset with exactly this code (everyone gets the same data):

Code · كد: `code/week04.py`, cell 9 · سلول 9

Tasks.

1. Fit (2 pts). Fit the exponential and power models to these data by (a) least squares on proportions and (b) binomial MLE. Report all estimates in one table with SSE and NLL.
2. Evaluate (2 pts). Compute G^2, df and p for each model's MLE fit. Plot the data with 95% binomial intervals, both fitted curves, and residuals. Which model, if any, can be rejected?
3. Simulation study (3 pts). Treat the power model with $A = 0.95$, $d = 0.7$ as the truth. Simulate 500 datasets with the same design, fit each with LS and MLE, and report the bias, SD and RMSE of $\hat{d}$ for each method. Show histograms. Repeat with $n = 100$ items and describe what changes.
4. Interpret (2 pts, max. 300 words). Which estimator would you recommend for recall data, and why? Relate your answer to the noise structure of binomial data and to the near-floor counts (1–2 of 20) at the longest delays. What would happen to the Week 2 linearization method if one of those counts were 0?
5. Code quality (1 pt). Reproducible, commented notebook; seeds fixed; functions reused rather than copy-pasted.

Late submissions lose 10% per day (max. 3 days).

2.2.11 Further reading

- Farrell & Lewandowsky (2018), chapter 4. Maximum likelihood parameter estimation: likelihood functions for common data types, log-likelihood, and fitting in practice. *(Main reading.)*
- Azimifar & Ahmadi (1396), *Shenasayi-ye amari-ye olgu*, vol. 1 (in Persian): the section on maximum likelihood parameter estimation, with the statistical-learning view [verify chapter].
- Lee & Wagenmakers (2014), chapter 3. Inferences with binomials. Read it now for the binomial set-up; we revisit it from the Bayesian side in Week 9.

Preview for Week 5: an estimate without an uncertainty is half a result. We compute standard errors and confidence intervals from the curvature of the likelihood and by bootstrapping, and we discover how averaging participants can create a "law" that no individual follows.

2.3 Week 5: Uncertainty in Estimates and Fitting Multiple Participants

> Chapter 2: Parameter estimation methods · Lecture 1 h · Lab 2 h Running data: the Week 2–4 forgetting data (how certain are our estimates?) and a new simulated practice experiment with 20 participants (the power law of practice).

2.3.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
power law of practice	قانون توانی تمرین	کاهش توانی زمان واکنش با تمرین
exponential / power function	توانی / تابع نمایی	دو شکل رایج برای منحنی فراموشی و یادگیری
sampling distribution	توزیع نمونه‌گیری	توزیع برآورد در تکرارهای فرضی آزمایش

English	فارسی	توضیح کوتاه
standard error (SE)	خطای استاندارد	پراکندگی برآورد در تکرارهای فرضی آزمایش
confidence interval (CI)	بازه‌ی اطمینان	بازه‌ای که با روش مشخص پارامتر واقعی را در بر می‌گیرد
Hessian	ماتریس Hessian	ماتریس مشتق‌های دوم؛ انحنای درست‌نمایی
log-likelihood / NLL	منفی آن / لگاریتم درست‌نمایی	مقداری که بهینه‌سازها کمینه یا بیشینه می‌کنند
Wald interval	بازه‌ی Wald	برآورد $\pm$ 1.96 SE
profile likelihood	درست‌نمایی نیم‌رخ (profile)	درست‌نمایی یک پارامتر با بهینه کردن بقیه
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
bootstrap	bootstrap (بازنمونه‌گیری)	تخمین عدم قطعیت با شبیه‌سازی یا بازنمونه‌گیری
parameter trade-off	بده‌بستان پارامترها	تغییر یک پارامتر که با پارامتر دیگر جبران می‌شود
aggregation artifact	خطای ناشی از میانگین‌گیری	الگویی در میانگین که در هیچ فردی نیست
trial	کوشش (trial)	یک بار انجام تکلیف توسط شرکت‌کننده
bounds / constraints	قیدها / کران‌ها	محدودیت‌های مجاز برای مقدار پارامترها
accuracy	دقت	نسبت پاسخ‌های درست
complete / no / partial pooling	ادغام جزئی / بدون ادغام / ادغام کامل (pooling)	/ جدا برای هر فرد / یک پارامتر برای همه سلسله‌مراتبی
individual differences	تفاوت‌های فردی	فرق پارامترها بین افراد
noise	نویز	تغییرات تصادفی در شواهد یا در داده
benchmark	benchmark (مجموعه‌آزمون استاندارد)	مجموعه سؤال برای مقایسه‌ی مدل‌های هوش مصنوعی

2.3.2 Learning outcomes

By the end of this week, students will be able to:

1. Explain the sampling distribution of an estimator and distinguish a standard error from a standard deviation of the data.
2. Compute standard errors and confidence intervals for model parameters in three ways: from the Hessian of the NLL (Wald), from the profile likelihood, and by parametric bootstrap. Compare these three methods.
3. Diagnose parameter trade-offs from the estimated correlation between parameters.

4. Demonstrate by simulation that averaging individual curves can produce a functional form (a "power law") that no individual follows.
5. Choose between aggregate, individual and (preview) hierarchical fitting, and justify the choice.

2.3.3 Motivation & big picture

"$\hat{d} = 0.42$" is half a result. Last week the power model's MLE for the Week 2 data was $\hat{d} = 0.42$ (true value 0.5). Is 0.42 compatible with 0.5? With 0.3? With 0.6? Without a measure of uncertainty we cannot say whether two participants, two groups or two conditions really differ. Every parameter estimate we report from now on comes with a confidence interval.

The law that nobody obeys. The *power law of practice* is one of the most cited regularities in cognitive psychology: response times fall with practice approximately as a power function of the number of trials. Some researchers have argued that the power shape can arise from averaging over people (or items) who each improve *exponentially* but at different rates (see Further reading). If so, the "law" describes the average but no single learner, and a theory built to explain a power function in each individual mind would be explaining an artifact. This week we demonstrate the effect with simulated data and learn when to fit individuals rather than group averages.

Where is this used?

- Any claim of a difference ("patients forget faster than controls", "training improved sensitivity") needs uncertainty estimates. Without them a difference of 0.05 in a parameter may be pure noise.
- Clinical assessment of individuals: to say that one patient's memory decay is abnormal, you need the standard error of *that patient's* estimate, not only the group average.
- Learning and skill research: the "power law of practice" was long treated as a universal law. This week's aggregation artifact shows why researchers now fit individual learners (and why the same caution applies to group-averaged curves in education, sports science and human factors).
- Replication and meta-analysis combine estimates from many studies by weighting each one by its precision ($1/\mathrm{SE}^2$).

2.3.4 Lecture notes

In plain words If you ran the same experiment again on the same person, you would get slightly different data and therefore a slightly different parameter estimate. The standard error (SE) says how much the estimate would jump around between such repetitions; a confidence interval is a range built from it (roughly estimate $\pm$ 2 SE). We cannot repeat the experiment many times, so we estimate

this jumping in three ways: (1) from how sharply peaked the likelihood is (a sharp peak means the data pin the parameter down; this is the Wald method using the Hessian), (2) by tracing how fast the fit gets worse as we move one parameter (profile likelihood), and (3) by simulating many fake experiments from the fitted model and refitting each (bootstrap). The second half of the week warns about a trap: averaging data across people can create a curve shape that no single person has. If every learner improves exponentially but at a different speed, the group average looks like a power law.

Suggested timing (60 min): sampling distributions 10 · Hessian standard errors 15 · profile likelihood 8 · bootstrap 10 · multiple participants and aggregation 17.

1. Estimates have sampling distributions

If we reran the same experiment on the same participant (same true $\boldsymbol{\theta}^*$), we would get different data and therefore a different $\hat{\boldsymbol{\theta}}$. The distribution of $\hat{\boldsymbol{\theta}}$ over hypothetical replications is its sampling distribution.

- The standard error (SE) of a parameter is the SD of its sampling distribution: how much the *estimate* would vary across replications.
- A 95% confidence interval is constructed by a procedure that, across replications, contains θ^* in 95% of cases. (It is *not* "a 95% probability that θ^* is in this particular interval"; that is a Bayesian credible interval, Week 9.)

We cannot rerun the experiment many times, so we approximate the sampling distribution in three ways.

2. Standard errors from the curvature of the likelihood (Wald)

Intuition. If the NLL has a sharp, narrow minimum, small changes in $\boldsymbol{\theta}$ make the data much less likely, so the data pin $\boldsymbol{\theta}$ down precisely. A flat, wide minimum means many values fit almost equally well, so the uncertainty is large. Curvature is measured by second derivatives.

Formalism. The Hessian of the NLL at the MLE is the matrix of second derivatives,

$$\mathrm{H}_{mn} = \left.\frac{\partial^2 \,\mathrm{NLL}(\boldsymbol{\theta})}{\partial\theta_m \,\partial\theta_n}\right|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}},$$

also called the observed Fisher information. For large samples,

$$\widehat{\mathrm{Cov}}(\hat{\boldsymbol{\theta}}) \approx \mathrm{H}^{-1}, \qquad \mathrm{SE}(\hat{\theta}_m) = \sqrt{[\mathrm{H}^{-1}]_{mm}}, \qquad 95\%\ \text{Wald CI: } \hat{\theta}_m \pm 1.96\,\mathrm{SE}(\hat{\theta}_m).$$

The off-diagonal elements of H^{-1}, rescaled to correlations, show parameter trade-offs: a correlation of -0.9 between two estimates means their errors largely compensate each other (the tilted valley of Week 3).

Sanity check (binomial). For one proportion, $\mathrm{NLL}''(\theta) = y/\theta^2 + (n-y)/(1-\theta)^2$. At $\hat{\theta} = y/n$ this equals $n/[\hat{\theta}(1-\hat{\theta})]$, so $\mathrm{SE} = \sqrt{\hat{\theta}(1-\hat{\theta})/n}$, the familiar textbook formula.

In practice: compute H numerically by finite differences (lab Part A). The `hess_inv` returned by BFGS is only a rough approximation, so do not use it for SEs.

When Wald fails: small samples, estimates near a bound (e.g., a recall level $\hat{A}$ close to 1), strongly non-quadratic likelihoods, or parameters with skewed sampling distributions (rates, variances). Then the symmetric ± 1.96 SE interval can extend into impossible values.

> Advanced: why this works. Near its minimum, any smooth NLL is approximately quadratic (second-order Taylor expansion). A quadratic NLL is exactly the NLL of a multivariate normal distribution whose inverse covariance is the Hessian. Combined with the asymptotic normality of the MLE (Week 4), this gives the result. The same quadratic approximation, called the *Laplace approximation*, reappears in Bayesian model comparison (Week 10).

Reading the math

- $\mathrm{H}_{mn} = \partial^2 \mathrm{NLL} / \partial\theta_m \partial\theta_n$ at $\hat{\boldsymbol{\theta}}$: the matrix of second derivatives, i.e. the curvature of the NLL bowl at its bottom. Diagonal entries: how steep the bowl is along each parameter. Off-diagonal entries: whether the bowl is tilted (parameters trade off).
- $\widehat{\mathrm{Cov}}(\hat{\boldsymbol{\theta}}) \approx \mathrm{H}^{-1}$: a sharply curved bowl (large H) gives a small covariance, i.e. precise estimates. Taking the inverse is what turns "steepness" into "uncertainty".
- $\mathrm{SE}(\hat{\theta}_m) = \sqrt{[\mathrm{H}^{-1}]_{mm}}$: take the m-th diagonal entry of the inverse and its square root.
- One-parameter check: for a proportion, $\mathrm{NLL}'' = n/[\hat{\theta}(1-\hat{\theta})]$, so $\mathrm{SE} = \sqrt{\hat{\theta}(1-\hat{\theta})/n}$. With 30 of 50 recalled: $\sqrt{0.6 \times 0.4/50} = 0.069$, and the 95% interval is $0.60 \pm 1.96 \times 0.069 = [0.46, 0.74]$.
- Correlation from the covariance: $\mathrm{corr} = \mathrm{Cov}_{12}/(\mathrm{SE}_1 \mathrm{SE}_2)$. A value like −0.9 means the two estimates are strongly traded off.

3. Profile likelihood intervals

For a parameter of interest ψ (say d), the profile NLL is obtained by fixing ψ on a grid and minimizing the NLL over all *other* parameters at each grid value:

$$\mathrm{NLL}_{\mathrm{prof}}(\psi) = \min_{\text{other params}} \mathrm{NLL}(\psi, \text{others}).$$

The 95% likelihood-ratio (profile) interval is the set of ψ values that are not much worse than the best:

$$\left\{\psi : 2\left[\mathrm{NLL}_{\mathrm{prof}}(\psi) - \mathrm{NLL}_{\mathrm{min}}\right] \le 3.84\right\}, \qquad 3.84 = \chi^2_{1,\,0.95}.$$

Profile intervals follow the actual shape of the likelihood, so they can be asymmetric and respect natural bounds. They are usually more accurate than Wald intervals in small samples.

Reading the math

- $\text{NLL}_{\text{prof}}(\psi)$: fix the parameter of interest ψ at one value, let the optimizer choose the best values of all *other* parameters, and record the resulting NLL. Repeat for many values of ψ. The result is a 1-D curve.
- The interval keeps every ψ whose profile NLL is within $3.84/2 = 1.92$ units of the minimum. The number 3.84 is the 95% point of a χ^2 distribution with 1 df; it comes from the likelihood-ratio test (Week 7).
- Unlike Wald, the curve is not forced to be a symmetric parabola, so the interval can be lopsided and never extends to impossible values.

4. The parametric bootstrap

A simulation answer to "what if we reran the experiment?":

1. Fit the model to the data: $\hat{\boldsymbol{\theta}}$.
2. Simulate B new datasets from the model at $\hat{\boldsymbol{\theta}}$, with the same design as the real experiment.
3. Refit each dataset: $\hat{\boldsymbol{\theta}}^{*(1)}, \ldots, \hat{\boldsymbol{\theta}}^{*(B)}$.
4. The SD of the bootstrap estimates approximates the SE. The 2.5th and 97.5th percentiles give a percentile CI.

The bootstrap needs no derivatives, works for any statistic (e.g., a half-life, a difference between conditions), and shows the full shape and correlations of the sampling distribution. Its cost is B refits ($B \approx 500$–2000). The non-parametric bootstrap resamples the observed trials with replacement instead of simulating from the model.

Comparison:

Method	Cost	Symmetric?	Handles bounds?	Needs
Wald (Hessian)	One Hessian	Always	No	Smooth NLL, large N
Profile likelihood	One fit per grid value	Can be asymmetric	Yes	A likelihood
Parametric bootstrap	B fits	Can be asymmetric	Yes	A model you can simulate

5. Fitting multiple participants

Real experiments have J participants (index j). There are three broad strategies:

Strategy	What is fitted	Pros	Cons
Aggregate (complete pooling)	One model to the averaged data	Stable; works with few trials per person	Assumes everyone is the same; can distort the functional form
Individual (no pooling)	One model per participant; then analyze the J sets of estimates	Respects individual differences; standard practice	Noisy estimates with few trials; second-stage analysis ignores estimation uncertainty
Hierarchical (partial pooling)	Individual parameters drawn from a group distribution, estimated jointly	Best of both; principled shrinkage	More complex; usually Bayesian (Week 13)

Aggregation artifacts. The average of nonlinear curves is generally not a curve of the same family. For example, averaging exponential curves with different rates does not give an exponential: fast learners finish early, so the average's relative rate of change *slows down* over time, which is the defining property of a power function (Week 2, relative loss rate).

> Advanced: an exact result. Suppose each participant follows $e^{-\lambda_j t}$ and the rates are Gamma-distributed across people, $\lambda_j \sim \text{Gamma}(\text{shape } s, \text{rate } b)$. Then the group average curve is
>
> $$\mathbb{E}\left[e^{-\lambda t}\right] = \left(1 + \frac{t}{b}\right)^{-s},$$
>
> which is *exactly* a shifted power function (this is the moment-generating function of the Gamma distribution, evaluated at $-t$). Everyone forgets exponentially, yet the average forgets by a power law with exponent equal to the Gamma shape s. You verify this numerically in Exercise 4.

Practical guideline. Fit individuals whenever each participant has enough data to constrain the model; use the aggregate only as a summary, never as evidence about the *form* of an individual-level process. When individual data are sparse, use hierarchical models (Week 13).

For an engineer

- Wald SE from the Hessian = the inverse Fisher information, i.e. the CRLB evaluated at the estimate. MLE reaches the bound asymptotically, so for large N the Wald interval is the CRLB-based interval.
- The parametric bootstrap is the Monte Carlo performance evaluation you do for an estimator or a receiver (e.g. simulating many noise realizations to get an estimator's variance or a BER curve), done at the fitted parameter values.
- Profile likelihood is what you do when you have *nuisance parameters*: optimize them out and look at the likelihood of the parameter you care about.

- Aggregation artifact = the difference between an ensemble average and a single realization. Averaging exponential decays with different time constants gives a slower, heavy-tailed decay, just as the average of many channel realizations does not look like any one realization. The process is not "ergodic across people": the group mean does not describe the individual.
- Partial pooling (preview of Week 13) is like estimating each user's SNR by combining their own few measurements with the distribution of SNRs in the whole cell: an empirical-Bayes estimate.

For the instructor

- Start with the question "is 0.42 different from 0.5?" (Motivation). Students will want a yes/no; show that the answer depends entirely on the SE.
- Show the three methods side by side on the same data (lab Part C table). The message is not "which is correct" but "they agree when the likelihood is well-behaved, and differ near bounds or with little data".
- The aggregation demo is the highlight. Plot 5 individual exponential curves and their average on a log–log plot: the average is almost a straight line (power law), while each individual is curved. Ask the class what a researcher who only saw the average would conclude.
- Typical questions.
  - *"What is the difference between SD and SE?"* SD describes the spread of the *data*; SE describes the spread of an *estimate* across repeated experiments. SE shrinks with more data; SD does not.
  - *"Does a 95% CI contain the true value with probability 0.95?"* Not this particular interval; 95% of intervals built this way would. (The Bayesian credible interval of Week 9 does have the direct reading.)
  - *"Should I always fit individuals?"* Yes, if each person has enough data; otherwise use hierarchical models (Week 13).

2.3.5 Worked example

Part 1: three intervals for a recall probability

A participant recalls $y = 30$ of $n = 50$ items; $\hat{\theta} = 0.60$.

Wald. SE $= \sqrt{0.6 \times 0.4/50} = 0.0693$, so the 95% CI is $0.60 \pm 1.96 \times 0.0693 = [0.464, 0.736]$.

Profile (likelihood ratio). Solve $2[\ell(0.6) - \ell(\theta)] = 3.84$ numerically on both sides of 0.6. The 95% CI is $[0.462, 0.728]$, slightly asymmetric: it extends 0.138 below the estimate and 0.128 above, because the likelihood is skewed toward 0.5, where binomial variance is largest.

Interpretation. With 50 items, a recall probability is only known to about ± 0.13. A difference of 0.10 between two conditions of this size would not be convincing on its own.

Part 2: averaging two exponential learners

Participant 1 forgets at $\lambda = 0.1$, participant 2 at $\lambda = 1.0$. The average curve $\bar{f}(t) = (e^{-0.1t} + e^{-t})/2$:

t	0	1	2	5	10
$\bar{f}(t)$	1.000	0.636	0.477	0.307	0.184
Relative loss rate $-\bar{f}'(t)/\bar{f}(t)$	0.55	0.36	0.23	0.11	0.10

Each participant has a *constant* relative loss rate (0.1 and 1.0), but the average's rate falls from 0.55 to 0.10, the signature of a power-like curve. The reason: at later times the average is dominated by the slow forgetter, whose rate is low. Interpretation: a researcher fitting only the average would conclude that "memories become more resistant with age", a claim true of neither participant.

2.3.6 Lab (2 hours)

Goals

1. Compute Wald, profile and bootstrap intervals for the power model fitted to the Week 2 data, and compare them.
2. Visualize parameter trade-offs through the estimated correlation and the bootstrap cloud.
3. Simulate 20 exponential learners, fit exponential and power models to the average and to each individual, and interpret the contradiction.

Timing: Part A 20 min · Part B 15 min · Part C 25 min · Part D 40 min · Part E 20 min.

Setup

Code · کد: `code/week05.py`, cell 1 · سلول 1

Part A: Wald standard errors from a numerical Hessian (20 min)

Code · کد: `code/week05.py`, cell 2 · سلول 2

Expected output:

```
   estimate      SE   lower   upper
A    0.8773  0.0626  0.7546  1.0000
d    0.4214  0.0532  0.3171  0.5257
Estimated correlation between A-hat and d-hat: 0.751
```

Interpret: both 95% intervals contain the true values ($A^* = 0.9$, $d^* = 0.5$), so the "miss" we worried about in Weeks 3–4 is well within sampling error. Note that the Wald interval for A happens to end at 1.000, right at the boundary of possible values. With slightly noisier data, the symmetric Wald interval would include impossible values (probabilities above 1), whereas the bootstrap interval cannot. The positive correlation is the trade-off valley from Week 3: when the estimate of A is too high, the estimate of d tends to be too high as well.

Part B: profile likelihood interval for d (15 min)

> Code · کد: `code/week05.py`, cell 3 · سلول 3

Expected output:

```
Profile 95% CI for d: [0.320, 0.525]  (grid step 0.005)
```

Part C: parametric bootstrap (25 min)

> Code · کد: `code/week05.py`, cell 4 · سلول 4

Expected output (takes about 30–60 s):

```
   boot SE    2.5%  97.5%
A   0.0563  0.7724  1.000
d   0.0501  0.3304  0.528
Bootstrap correlation: 0.703
```

Compare: put the Wald, profile and bootstrap intervals for d side by side. Are they similar? Which is asymmetric? With these data (8 cells × 50 items) all three methods should agree reasonably well. They diverge more with fewer items or with estimates near a bound (Exercise 5).

Part D: the power law of practice as an aggregation artifact (40 min)

Design. $J = 20$ participants each complete 60 practice blocks of a visual-search task. Each participant improves exponentially:

$$\mathrm{RT}_j(m) = E + B\, e^{-r_j (m-1)} + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, 40\ \mathrm{ms}),$$

with asymptote $E = 400$ ms, initial gain $B = 600$ ms, and individual learning rates $r_j \sim \mathrm{Gamma}(\mathrm{shape} = 0.7, \mathrm{scale} = 0.5)$ (mean 0.35 per block; a skewed distribution, with a few very fast learners and several very slow ones). Here m is the practice block, and ε is the noise of a block-mean RT.

Code · کد: `code/week05.py`, cell 5 · سلول 5

Expected output:

```
True learning rates: [1.000e-03 7.000e-03 7.000e-03 2.000e-02 3.000e-02
↪  4.500e-02 1.220e-01
 1.490e-01 1.720e-01 2.360e-01 2.820e-01 3.260e-01 3.870e-01 3.950e-01
 5.340e-01 7.100e-01 7.900e-01 1.002e+00 1.869e+00 2.534e+00]
Average curve, exp: E = 507.5, B = 423.8, r = 0.205, RMSD = 19.26 ms
Average curve, pow: E = 419.2, B = 579.4, r = 0.529, RMSD = 10.16 ms
```

Code · کد: `code/week05.py`, cell 6 · سلول 6

Expected output (summary line):

```
Which model fits each participant better? {'exp': 20}
(with SciPy 1.18: {'exp': 19, 'pow': 1}; one participant with a
↪  near-zero learning rate is fitted differently)
```

Code · کد: `code/week05.py`, cell 7 · سلول 7

Part E: interpret (20 min)

Code · کد: `code/week05.py`, cell 8 · سلول 8

Expected output:

```
Mean of true individual rates:      0.481
Mean of estimated individual rates: 0.449 (SD 0.626)
Exponential rate fitted to the average curve: 0.205
Correlation of true and estimated individual rates: 0.971
(with SciPy 1.18: estimated mean 0.457 (SD 0.621), correlation 0.968,
 ↪  because of the same participant)
```

Write 2–3 sentences for each: 1. The average curve is better described by a power function (RMSD about 10 vs. 19 ms), yet all (or, depending on the SciPy version, 19) of the 20 individuals are better described by an exponential. Which statement about *learning* is justified, and which is not? 2. Why is the exponential rate fitted to the average curve smaller than the mean of the individual rates? (Hint: which participants dominate the average late in practice?) 3. Individual fits recover the true rates well here, with 60 blocks per person. What would change with only 10 blocks per person, and what would you do then? (Preview of Week 13.)

Exercises (increasing difficulty)

Exercise 1 $\star$. Compute Wald 95% CIs for the *exponential* model's MLE on the Week 2 data. Do they contain sensible values? Then compute them at the bogus "estimate" `(A,lam)=(1.0,0.107)` from last week's bounded-Nelder–Mead warning. What happens to the Hessian at a point that is not a minimum?

Exercise 2 $\star$. Rerun the bootstrap with $B = 50, 200, 1000$ and different seeds. How stable are the percentile limits? How large should B be for a 95% CI?

Exercise 3 $\star\star$ (a derived quantity). Use the bootstrap samples from Part C to compute a 95% CI for the half-life of the power model, $t_{1/2} = 2^{1/d} - 1$. Why is this easy with the bootstrap and awkward with the Hessian?

Exercise 4 $\star\star$ (the Gamma–power result). Draw 100 000 rates $\lambda \sim$ Gamma(shape $s = 0.8$, rate $b = 2$), average $e^{-\lambda t}$ over them for $t \in [0, 30]$, and overlay the analytic curve $(1 + t/b)^{-s}$ (Lecture §5, Advanced box). Then fit exponential and power functions to the averaged curve.

Exercise 5 $\star\star\star$ (Advanced: coverage). How often do 95% intervals actually contain the true value? Simulate 300 datasets from the power model ($A = 0.9$, $d = 0.5$) with $n = 10$ items per interval, and compute the Wald and profile CIs for d for each. Report the coverage (proportion containing 0.5) of each method. Which is closer to 95%? Repeat with $n = 50$.

Lab check-off

Show: (1) a table comparing the Wald, profile and bootstrap 95% CIs for d; (2) the bootstrap scatter plot with one sentence on the trade-off; (3) the individual-vs-average comparison and your answer to interpretation question 1.

2.3.7 Connection to AI & LLMs

Benchmark scores have standard errors. AI models are compared on benchmarks, and leaderboards often rank models that differ by one or two percentage points. A benchmark is a *sample* of items, so its accuracy has sampling error just like a parameter estimate. Two lessons from this week apply directly: 1. Report uncertainty. A bootstrap over items gives a confidence interval for the accuracy and for the difference between two models. 2. Respect the design. Both models answered the *same* items, so the comparison is paired, like a within-participant design. Resampling the two models independently ignores the shared item difficulty and gives intervals that are too wide.

Item difficulty also varies a lot, which is the item-level version of the participant heterogeneity in Part D: a single average accuracy hides it.

Code · کد: `code/week05.py`, cell 9 · سلول 9

Expected output:

```
Accuracy A = 0.74, B = 0.67, difference = 0.07
95% CI, paired bootstrap:   [-0.03, 0.17]
95% CI, unpaired bootstrap: [-0.06, 0.20]
```

A 7-point lead on 100 items is not clearly different from zero. A leaderboard that ranks A above B without an interval is overconfident.

Try it (no API needed). - ⋆ How many items would the benchmark need for the paired interval to exclude 0, if the true difference stays the same? Find out by simulation. - ⋆⋆ Compute the Wald SE of each accuracy, $\sqrt{p(1-p)/n}$, and compare it with the bootstrap SE. - ⋆⋆ Discussion: a paper reports "Model A: 74%, Model B: 67%" with no interval and concludes that A "reasons better". List what you would ask the authors.

2.3.8 Common mistakes & misconceptions

1. Confusing SE with SD. The SD describes the spread of the *data*; the SE describes the uncertainty of an *estimate* and shrinks as the data grow.
2. "There is a 95% probability that the true value is in my CI." For a frequentist CI, 95% refers to the long-run behavior of the procedure. The probability statement belongs to Bayesian credible intervals (Week 9).
3. Using BFGS's **`hess_inv`** for standard errors. It is a crude by-product of the optimizer. Compute the Hessian properly.
4. Trusting Wald intervals near bounds. If $\hat{\theta}$ is on or near a bound, or the interval crosses impossible values, use profile or bootstrap intervals.
5. Reporting parameters without uncertainty. Comparing two point estimates ("participant 1 has $d = 0.42$, participant 2 has $d = 0.51$") is meaningless without intervals.

6. Inferring individual-level processes from averaged data. Averages of nonlinear curves change functional form (Part D). This applies to items, trials and participants alike.
7. Treating individual estimates as if they were data. Second-stage t-tests on fitted parameters ignore that some estimates are much noisier than others. Hierarchical models fix this (Week 13).

2.3.9 Check your understanding

1. What is the sampling distribution of an estimator, and why can't we observe it directly? Answer The distribution of the estimates we would get across many hypothetical replications of the experiment with the same true parameters. We have only one dataset, so we approximate it with the Hessian (asymptotic theory), the profile likelihood, or simulation (bootstrap).

2. The NLL has a very flat minimum along one direction of the parameter space. What does that imply for the standard errors and the correlation of the estimates? Answer Low curvature means a small Hessian entry in that direction, so a large variance in H^{-1}: large SEs. If the flat direction is diagonal, the estimates are strongly correlated (a trade-off).

3. Derive the Wald SE for a binomial proportion from the second derivative of the NLL. Answer $\mathrm{NLL}''(\theta) = y/\theta^2 + (n-y)/(1-\theta)^2$. At $\hat{\theta} = y/n$ this is $n/\hat{\theta} + n/(1-\hat{\theta}) = n/[\hat{\theta}(1-\hat{\theta})]$. Inverting and taking the square root gives $\mathrm{SE} = \sqrt{\hat{\theta}(1-\hat{\theta})/n}$.

4. List the steps of a parametric bootstrap for a CI on a model parameter. Answer Fit the model to get $\hat{\boldsymbol{\theta}}$. Simulate B datasets from the model at $\hat{\boldsymbol{\theta}}$ using the original design. Refit each one. Use the SD of the refitted estimates as the SE, and their 2.5th and 97.5th percentiles as the 95% CI.

5. Why can a profile-likelihood interval be asymmetric while a Wald interval cannot? Answer The Wald interval assumes a quadratic NLL (a normal sampling distribution), which is symmetric by construction. The profile interval uses the actual likelihood, which can be skewed, e.g., near a bound or for rate parameters.

6. Twenty participants each learn exponentially, with different rates. The averaged curve is fitted better by a power function. Should we conclude that learning follows a power law? Answer No. The power shape is an aggregation artifact: averaging exponentials with heterogeneous rates produces a curve whose relative rate of change decreases over time. Conclusions about the form of individual learning must come from individual (or hierarchical) fits.

7. Give one advantage and one disadvantage of fitting each participant separately. Answer Advantage: respects individual differences and avoids aggregation artifacts. Disadvantage: with few trials per person, estimates are noisy (and sometimes extreme or on bounds), and analyzing them in a second stage ignores their different uncertainties.

2.3.10 Assignment

No new graded assignment this week. A2 (LS vs. MLE) is due at the start of the Week 6 lecture. Part A of today's lab shows how to add standard errors to your A2 estimates. Doing so is not required, but it will strengthen your interpretation.

Optional practice (ungraded): Exercise 5 (coverage) is the best preparation for Week 7's recovery studies.

2.3.11 Further reading

- Farrell & Lewandowsky (2018), chapter 5. Combining information from multiple participants: aggregate vs. individual fitting and the dangers of averaging. *(Main reading.)*
- Farrell & Lewandowsky (2018), chapter 4 (the sections on parameter uncertainty and the Hessian), revisited from last week.
- *Optional, outside the course list:* Heathcote, A., Brown, S., & Mewhort, D. J. K. (2000). The power law repealed: The case for an exponential law of practice. *Psychonomic Bulletin & Review, 7*, 185–207. The classic analysis of averaging artifacts in practice data.

Preview for Week 6: Exercise 4 of Week 3 showed a three-parameter model beating the true model on SSE. Next week we confront this directly: why good fit is not enough, and how cross-validation measures what we really want, which is prediction of new data.

Chapter 3

Evaluation of Cognitive Models

3.1 Week 6: Evaluating Models I: Goodness of Fit, Overfitting and Cross-Validation

Chapter 3: Evaluation of cognitive models · Lecture 1 h · Lab 2 h Running data: the Week 2–5 forgetting data, now with four candidate models: exponential ($k = 2$), power ($k = 2$), exponential-with-asymptote ($k = 3$) and two-trace ($k = 4$).

3.1.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
goodness of fit	نیکویی برازش	اینکه مدل داده‌ی موجود را چقدر خوب توصیف می‌کند
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
overfitting	بیش‌برازش (overfitting)	برازش نویز نمونه به‌جای الگوی واقعی
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان
exponential / power function	توانی / تابع نمایی	دو شکل رایج برای منحنی فراموشی و یادگیری
trace plot	نمودار trace	مقدار نمونه‌ها در برابر شماره‌ی تکرار
model flexibility / complexity	پیچیدگی مدل / انعطاف‌پذیری	دامنه‌ی الگوهایی که مدل می‌تواند تولید کند

English	فارسی	توضیح کوتاه
training / test data	آزمون / داده‌ی آموزش	داده‌ی کنارگذاشته برای / داده‌ی برازش سنجش
cross-validation (CV)	اعتبارسنجی متقابل (cross-validation)	برازش روی بخشی از داده و آزمون روی بخش کنارگذاشته
log-likelihood / NLL	منفی آن / لگاریتم درست‌نمایی	مقداری که بهینه‌سازها کمینه یا بیشینه می‌کنند
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
generalization	تعمیم	پیش‌بینی خوب داده‌ی جدید
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
SSE / RMSD	ریشه‌ی میانگین / مجموع مربعات خطا مربع انحراف	معیارهای فاصله‌ی داده و پیش‌بینی
noise	نویز	تغییرات تصادفی در شواهد یا در داده
response bias	سوگیری پاسخ	تمایل به یک پاسخ، مستقل از محرک
criterion (c)	ملاک	آستانه‌ی تصمیم؛ نشان‌دهنده‌ی سوگیری پاسخ
bias–variance trade-off	بده‌بستان سوگیری و واریانس	مدل ساده سوگیر، مدل منعطف پرواریانس
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
data leakage	نشت داده	رسیدن اطلاعات داده‌ی آزمون به برازش
AIC / BIC	بیزی / معیار اطلاعاتی آکائیکه	برازش به‌علاوه‌ی جریمه برای تعداد پارامتر
benchmark	benchmark (مجموعه‌آزمون استاندارد)	مجموعه سؤال برای مقایسه‌ی مدل‌های هوش مصنوعی
benchmark contamination	benchmark آلودگی	بودن سؤال‌های آزمون در داده‌ی آموزش مدل
parameter recovery	بازیابی پارامتر	آیا برازش، پارامترهای معلوم شبیه‌سازی را برمی‌گرداند

3.1.2 Learning outcomes

By the end of this week, students will be able to:

1. Explain why a good fit is necessary but not sufficient evidence for a model, using the concepts of flexibility and overfitting.
2. Demonstrate overfitting by simulation: training error falls monotonically with model complexity while prediction error does not.

3. Implement hold-out, K-fold and leave-one-out cross-validation with an out-of-sample log-likelihood score, keeping the training and test data properly separated.
4. Choose what to hold out (trials, conditions, participants) according to the generalization claim being made.
5. Interpret cross-validation results, including their uncertainty, to compare cognitive models.

3.1.3 Motivation & big picture

In Week 3, Exercise 4, the three-parameter exponential-with-asymptote model fitted the Week 2 data *better* than the power model (SSE ≈ 0.020 vs. 0.026), although the data were generated by the power model. In Week 3, Part D3, a four-parameter two-trace model did just as well. If we chose models by fit, we would choose the wrong one, and we would keep adding parameters forever.

The problem is not specific to forgetting curves. A flexible enough model can fit any data, including the noise. Roberts and Pashler famously asked how persuasive a good fit really is (see Further reading). Their answer: not very, unless we also know what the model could *not* have fitted.

The way out is to remember why we fit models. We want to capture the *process* that generated the data, and a model that has captured the process should predict new data from that process. Noise, by definition, does not repeat. So the question changes from "how well does the model fit these data?" to "how well does it predict data it has not seen?" Cross-validation answers this question directly with data we already have.

Where is this used?

- Choosing between theories. Whenever two models of memory, decision or learning are compared, a good fit alone is not convincing; reviewers ask for cross-validation or a measure that penalizes flexibility.
- Prediction in clinical psychology and psychiatry: models that predict relapse, treatment response or diagnosis must be validated on *new patients*, not on the patients they were fitted to. Many published predictors fail exactly this test.
- Machine learning: train/validation/test splits and k-fold cross-validation are the standard way to choose models and hyperparameters; overfitting is the central danger.
- Evaluating AI systems: benchmark scores are only meaningful if the test items were not in the training data (see the AI & LLMs section).

3.1.4 Lecture notes

In plain words A model with enough knobs can pass through every data point, including the random noise. That does not make it a good model: the noise will be different next time, so a model

that copied the noise will predict *new* data badly. This is overfitting. The fix is to judge models by how well they predict data they have not seen. Cross-validation does this with the data we already have: hide part of the data, fit the model on the rest, predict the hidden part, and repeat until every part has been hidden once. The model with the best predictions on hidden data wins. The week's main message: a better fit to the *same* data is not evidence for a better model; better *prediction* of new data is.

Suggested timing (60 min): measures of fit and their limits 10 · overfitting and flexibility 15 · the logic of generalization 5 · cross-validation 20 · what to hold out 10.

1. Measures of goodness of fit

Measure	Definition	Scale	Remarks
RMSD	$\sqrt{\text{SSE}/N}$	Data units	Easy to read; assumes equal weighting
R^2	$1 - \text{SSE}/\sum_i (y_i - \bar{y})^2$	0–1 (for linear models)	Popular but treacherous for nonlinear models: can be negative, rewards flexibility, and is dominated by large between-condition variance (fitting a monotone decline earns a high R^2 almost automatically)
Log-likelihood $\ell(\hat{\boldsymbol{\theta}})$, NLL	Week 4	Nats	The natural measure for likelihood-based models
Deviance G^2	$2(\ell_{\text{sat}} - \ell)$	$\approx \chi^2$	Absolute test of misfit (Week 4)

All of these measure descriptive adequacy: how well the model reproduces the data it was fitted to. None of them accounts for how easily the model *could* have produced those data.

2. Overfitting and flexibility

Write each observation as signal + noise: $y_i = g(x_i) + \varepsilon_i$, where g is the true process. A fitted model tries to reproduce y_i, but only the g part will be there again in new data.

- A model that is too simple misses part of g: systematic misfit, or underfitting (high bias).
- A model that is too flexible bends to follow the particular ε_i of this sample: overfitting (high variance). Its fit to the sample improves; its predictions for new data get worse.

As complexity increases, training error always goes down, while expected prediction error first decreases and then increases: a U-shape (lab Part A).

Flexibility is more than the number of parameters. Two models with the same k can differ greatly in flexibility, i.e., in the range of data patterns they can produce. A parameter that enters as an exponent or a shift usually buys more flexibility than one that scales the curve. The two-trace model can mimic exponential, power-like and asymptotic curves, which is exactly why its good fit is unimpressive. Information criteria (Week 7) count parameters; cross-validation and Bayes factors (Week 10) respond to actual flexibility.

> Advanced: the bias–variance decomposition. For squared error, the expected prediction error at a point x (over new datasets and new noise) decomposes as
>
> $$\mathbb{E}\big[(y_{\text{new}} - \hat{f}(x))^2\big] = \underbrace{\sigma^2}_{\text{irreducible noise}} + \underbrace{\big(g(x) - \mathbb{E}[\hat{f}(x)]\big)^2}_{\text{bias}^2} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{variance}}.$$
>
> Adding parameters lowers bias but raises variance. The best model for prediction balances the two, and it is not necessarily the true model when data are scarce. This decomposition is treated in depth in Azimifar & Ahmadi (see Further reading).

Reading the math The bias–variance decomposition (Advanced box) splits the average squared prediction error into three parts:

- σ^2, irreducible noise: even the true process cannot predict the random part of new data. No model can go below this.
- bias$^2 = (g(x) - \mathbb{E}[\hat{f}(x)])^2$: how far the model's *average* prediction (averaged over many possible datasets) is from the truth. Too-simple models have large bias.
- variance $= \text{Var}[\hat{f}(x)]$: how much the fitted curve jumps around from one dataset to the next. Too-flexible models have large variance, because they chase each dataset's noise.

Adding parameters moves error from the bias term into the variance term. The best predictor sits where their sum is smallest, which is why a slightly wrong but simple model can predict better than the true but complex one when data are scarce.

3. Generalization as the criterion

We prefer the model that best predicts new data from the same process. Three kinds of "new" correspond to three scientific claims:

New data are…	Claim being tested	Hold out…
New trials of the same participant in the same conditions	The model captures this person's process	Random trials or items
New conditions (e.g., longer delays, new stimuli)	The model captures the mechanism, so it can extrapolate	Whole conditions (the *generalization criterion* approach)
New participants	The model captures what is common to people	Whole participants

Always state which kind of generalization your evaluation targets.

4. Cross-validation (CV)

Hold-out validation. Split the data into a training set (fit) and a test set (evaluate). Simple, but wasteful and noisy: the result depends on one particular split.

K-fold CV. Split the data into K roughly equal folds. For each fold $k = 1, \ldots, K$: 1. Fit the model to all data except fold k, obtaining $\hat{\boldsymbol{\theta}}_{(-k)}$. 2. Score its predictions for fold k.

Sum the scores over folds. Every observation is used once for testing, and the model is never tested on data it was trained on. Typical values are $K = 5$ or 10. Leave-one-out (LOO) is the special case where every observation is its own fold.

The score. For likelihood-based models, use the out-of-sample log-likelihood (log predictive score):

$$\text{CV-NLL} = -\sum_{k=1}^{K} \sum_{i \in \text{fold } k} \ln p\big(y_i \mid \hat{\boldsymbol{\theta}}_{(-k)}, \mathcal{M}\big).$$

Lower is better. For Gaussian models, out-of-sample squared error is an equivalent choice.

Rules for honest CV: 1. Everything that is learned from data, including parameter estimates, starting-value choices from grids and data-driven transformations, must be done *inside* each training fold. Otherwise information "leaks" from the test set. 2. Split along the right unit (§3). For trial-level CV with binomial counts, split the items within each condition, so that every training fold still contains every condition. 3. CV estimates are noisy. Repeat with several random fold assignments and report the mean and spread, or the SE of the *difference* between models across folds. 4. With the item-level split, CV compares models fitted to $(K-1)/K$ of the data, so it slightly penalizes models that need a lot of data.

> Advanced: LOO, AIC and Bayes. For regular models and large N, leave-one-out CV with the log score and AIC (Week 7) choose the same model asymptotically. In Bayesian modeling, an efficient approximation to LOO (PSIS-LOO) is computed from posterior samples. You will meet it with ArviZ in Week 13.

Reading the math $\text{CV-NLL} = -\sum_k \sum_{i \in \text{fold } k} \ln p(y_i \mid \hat{\boldsymbol{\theta}}_{(-k)}, \mathcal{M})$:

- $\hat{\boldsymbol{\theta}}_{(-k)}$: the parameters estimated without fold k (the minus sign in the subscript means "leaving out").
- $\ln p(y_i \mid \hat{\boldsymbol{\theta}}_{(-k)})$: how probable the model, trained without them, finds the held-out observations. A model that predicted them well gives them high probability.
- The double sum adds this over all folds and all held-out points; the minus sign makes it a quantity to *minimize*, like the NLL of Week 4. It is the same NLL, but each observation is scored by a model that never saw it.

5. Interpreting a model comparison

- Differences in CV-NLL are in nats. As a rough guide, differences smaller than about 1–2 nats, or smaller than twice their SE across folds or repetitions, should not be over-interpreted.
- A model that wins CV is the best predictor among those compared, for this kind of generalization and this amount of data. It is not "the true model". With more data, more complex models often start to win, because their variance shrinks.
- Combine CV with the qualitative checks you already know: residual patterns, parameter plausibility, and the model's interpretability.

For an engineer

- Overfitting is the familiar problem of fitting a high-order polynomial or a filter with too many taps to a short, noisy record: the training error goes to zero and the prediction error explodes.
- The bias–variance trade-off is the same one you meet when choosing a model order (AR order, number of filter taps) or a smoothing window: too few degrees of freedom gives bias, too many gives variance.
- Cross-validation is the data-driven version of model-order selection. AIC/MDL-type criteria (Week 7) are the analytic shortcuts you may know from AR model order selection.
- Training on the test set is like tuning a receiver on the same noise realization you later measure its BER on: the reported performance is optimistic.
- Nats are the natural-log version of bits: a difference of 1 nat is a likelihood ratio of $e \approx 2.7$.

For the instructor

- Start with the polynomial worked example on the board. Four points, four models. Let students pick the best model by training SSE (they pick the cubic), then ask what it predicts at 16 days (-8.5). The laugh is the lesson.
- Do leave-one-out by hand for one point (cover the 8-day point with your hand, refit the line by eye, predict). Students then understand what the code in the lab does.
- Insist on "what is held out". Write the three kinds of "new" data (Section 3) on the board; most mistakes in student projects come from holding out the wrong thing.
- Typical questions.

- *"If the true model overfits less, why can a wrong model win CV?"* With little data, a simpler wrong model can have lower variance. CV picks the best *predictor*, not the true model.
- *"How many folds?"* 5 or 10 is standard; LOO for very small datasets.
- *"Is $R^2 = 0.98$ good?"* For a decreasing curve, almost any monotone model gets a high R^2; it says little.

3.1.5 Worked example

Data. Four recall proportions $y = (0.62, 0.50, 0.47, 0.27)$ at $t = (1, 2, 4, 8)$ days. We compare polynomials of increasing degree as "models".

Step 1: fit to all the data

Model	k	Fitted equation	Training SSE
Constant	1	$\hat{y} = 0.465$	0.0633
Linear	2	$\hat{y} = 0.637 - 0.046\,t$	0.0032
Quadratic	3	$\hat{y} = 0.647 - 0.053\,t + 0.0008\,t^2$	0.0031
Cubic	4	passes exactly through all 4 points	0.0000

By training SSE, the cubic is "perfect". But ask it to predict recall at $t = 16$ days: it predicts -8.5, i.e., *minus 850% recall*. The linear model predicts -0.10 (also impossible, but less absurd), and the constant predicts 0.47.

Step 2: leave-one-out CV

Leave out each point in turn, refit on the remaining three, and predict the left-out point.

Left-out t	Observed	Constant predicts	Linear predicts	Quadratic predicts
1	0.62	0.413	0.560	0.498
2	0.50	0.453	0.570	0.570
4	0.47	0.463	0.448	0.330
8	0.27	0.530	0.275	1.250
LOO SSE		0.1125	0.0090	1.0000

(The cubic cannot be cross-validated here: three points do not determine four parameters.)

Interpretation. The quadratic's training SSE is essentially equal to the linear model's, but its LOO error is more than 100 times larger. When it does not see the 8-day point, it bends its curvature to

fit the three short delays and predicts recall *above 1* at 8 days. The extra parameter fitted noise, not forgetting. The linear model generalizes best of the three. (None of these polynomials is a sensible *cognitive* model of forgetting, because none respects the 0–1 range; that is exactly why we use exponential and power models instead.)

3.1.6 Lab (2 hours)

Goals

1. See overfitting happen: training vs. test error as polynomial degree increases.
2. Implement item-level K-fold CV with the out-of-sample binomial log-likelihood, and compare four forgetting models.
3. Run a generalization-criterion test: train on short delays, predict long delays.
4. Interpret which model generalizes, and why the best-fitting model is not the best predicting one.

Timing: Part A 25 min · Part B 45 min · Part C 25 min · Part D 25 min.

Setup: data and four models

Code · کد: `code/week06.py`, cell 1 · سلول 1

Expected output:

```
exp      k = 2  training NLL = 29.812  params = [0.638 0.046]
pow      k = 2  training NLL = 19.830  params = [0.877 0.421]
expasym  k = 3  training NLL = 18.919  params = [0.861 0.38  0.308]
two      k = 4  training NLL = 18.919  params = [0.265 0.    0.596 0.38 ]
```

By training NLL, the more complex models win. Notice that the two-trace model's best fit has a slow component with rate 0: it has collapsed into the exponential-with-asymptote model (same NLL), as in Week 3. Now let's see whether they also *predict* better.

Part A: overfitting with polynomials (25 min)

We use polynomials in $x = \ln(1 + t)$ as deliberately "dumb" flexible models. The test data are 500 fresh datasets simulated from the true power model, with the same design.

Code · کد: `code/week06.py`, cell 2 · سلول 2

Expected output:

```
 degree  k  train_MSE  test_MSE
      0  1    0.03980   0.05072
      1  2    0.00663   0.00787
      2  3    0.00204   0.00634
      3  4    0.00202   0.00640
      4  5    0.00173   0.00661
      5  6    0.00087   0.00750
      6  7    0.00087   0.00750
      7  8    0.00000   0.00833
```

What to notice: training error decreases with every added parameter and reaches 0 at $k = 8$ (interpolation). Test error is lowest at an intermediate complexity and then rises: the extra parameters chase noise. Even the best polynomial predicts worse than the *true* power model, whose test MSE you can compute as an exercise.

Part B: K-fold cross-validation over items (45 min)

Each interval tested 50 items. We assign each item to one of $K = 5$ folds *within* each interval, so that every training set contains all 8 intervals.

Code · کد: `code/week06.py`, cell 3 · سلول 3

Expected output (about 1–2 minutes):

```
         k  train_NLL  CV_NLL_mean  CV_NLL_sd
exp      2     29.812      256.458      1.058
pow      2     19.830      245.791      0.734
expasym  3     18.919      247.370      1.574
two      4     18.919      247.531      1.172

CV-NLL difference to the power model (mean, SD over repeats):
         exp  pow  expasym    two
mean  10.667  0.0    1.579  1.740
std    1.137  0.0    1.807  1.349
```

Note that training NLL (binomial, with the binomial coefficients) and CV-NLL (Bernoulli, item level) are on different scales. Compare models within a column, not across columns.

Interpret: 1. Which model predicts held-out items best? Is its advantage over the runner-up larger than the variability across CV repetitions? 2. The two-trace model has the best training fit. What happens to it under CV, and why? 3. Can CV tell the power and exponential-with-asymptote models apart with these data? What would help?

Part C: the generalization criterion: predicting longer delays (25 min)

A model that has captured *how* forgetting works should be able to extrapolate. Train on the five shortest delays ($t \leq 7$ days) and predict the three longest (14, 21, 30 days), which the models have never seen.

Code · کد: `code/week06.py`, cell 4 · سلول 4

Expected output:

```
         test_NLL_long_delays  pred_t14  pred_t21  pred_t30
model
exp                    72.550     0.086     0.028     0.007
pow                     9.627     0.228     0.186     0.156
expasym                 6.268     0.260     0.257     0.257
two                    46.072     0.131     0.057     0.020
observed: {14: 0.26, 21: 0.26, 30: 0.28}
```

What to notice: here the exponential-with-asymptote model extrapolates best, *not* the true power model. The three held-out points happen to be flat (0.26, 0.26, 0.28), and a curve that levels off predicts them well. The power model keeps declining, as the true process does, but in this sample the noise at the long delays made recall look flat. The pure exponential and the two-trace model (whose short-delay fit gives no constraint on the slow component) extrapolate badly. Lessons: (1) different generalization criteria can rank models differently; (2) a single test on three data points is itself noisy, so a model's win on one test set is not proof; (3) agreement across several criteria (Part B, Part C, Week 7's AIC/BIC) is more convincing than any one of them.

Part D: put it together (25 min)

Fill in this table in your notebook from Parts B and C, and write a short paragraph (5–8 sentences) answering: Which model would you recommend as the best account of these forgetting data, and how confident are you?

Model	k	Training NLL	5-fold CV-NLL (mean $\pm$ SD)	Long-delay test NLL	Residual pattern (Week 4)	Parameters plausible?
exp						
pow						
expasym						
two						

Exercises (increasing difficulty)

Exercise 1 $\star$. In Part A, add the true power model with the true parameters as a "model" (it has no fitted parameters). Compute its test MSE and compare it with the best polynomial's. What is the irreducible error?

Exercise 2 $\star$. Rerun Part B with $K = 2$, $K = 10$ and leave-one-item-out ($K = 50$). Does the ranking change? Which K gives the most stable results across repetitions?

Exercise 3 $\star\star$ (data leakage). Write a "wrong" CV in which the starting values for each fold are taken from the fit to the *full* dataset, and another in which the fold's test items are accidentally included in training. Compare the results with the correct CV. Which mistake matters more, and why?

Exercise 4 $\star\star$ (more data). Simulate a new dataset from the true power model with $n = 500$ items per interval and rerun Parts B and C. Does the ranking become clearer? Does the two-trace model ever win?

Exercise 5 $\star\star\star$ (Advanced: leave-one-participant-out). Use the practice data generator from Week 5, Part D, with 20 participants. Implement CV in which each fold leaves out *one participant*: fit an aggregate (group-average) model on 19 participants and score the prediction of the held-out participant's curve with the Gaussian log-likelihood. Compare the exponential and power models. Then do the same with individual-level fits. Explain why the two CV schemes answer different scientific questions.

Lab check-off

Show: (1) the training-vs-test plot from Part A, (2) the CV summary table from Part B, and (3) your Part D paragraph.

3.1.7 Connection to AI & LLMs

Benchmark contamination is data leakage. LLMs are trained on huge amounts of internet text. If benchmark questions (or their answers) were published online, they may be in the training data, so the model is tested on items it has already seen. This is exactly the leakage problem of cross-validation: the test set is no longer independent of the training set, so the score measures memorization as well as generalization. It is one of the most serious problems in evaluating AI systems, including claims that LLMs show human-like cognitive abilities on classic psychology tasks, whose materials are widely published.

Code · کد: `code/week06.py`, cell 5 · سلول 5

Expected output:

```
Reported accuracy (all items): 0.73
Accuracy on leaked items:      0.96  (n = 120)
Accuracy on clean items:       0.63  (n = 280)
```

The reported score (0.73) overstates the ability to solve *new* problems (0.63). The fix is the same as in this week's lab: evaluate on items that were guaranteed not to be used in fitting, for example newly written items, or systematic variations of classic tasks.

Try it (no API needed). - ⋆ Change the share of leaked items from 0 to 0.9 and plot the reported accuracy. Can you tell from the reported accuracy alone how much leaked? - ⋆⋆ In practice we do not know which items leaked. Suppose a "fresh" variant of every item (same logic, new wording) is available. Simulate fresh variants with $P(\text{correct}) = 0.60$ and estimate the leakage effect as the difference between original and fresh accuracy, with a paired bootstrap CI (Week 5). - ⋆⋆ Discussion: an LLM solves a famous reasoning puzzle from a 1970s psychology paper. Why is this weak evidence that it "reasons like a human"? Which test would be stronger?

3.1.8 Common mistakes & misconceptions

1. "The model with the best fit is the best model." Fit rewards flexibility. Always ask how well the model predicts new data.
2. "Flexibility = number of parameters." Functional form matters too. Models with equal k can differ greatly in what they can fit.
3. Leaking test information into training, e.g., choosing starting values, bounds or transformations from the full dataset, or normalizing with full-data statistics.
4. Splitting along the wrong unit. If the claim is about new participants, holding out random trials overestimates generalization.
5. Over-reading small CV differences. CV estimates are noisy. Repeat the splits and look at the variability of the differences.
6. "CV picked the true model." CV picks the best *predictor* for this amount of data. With little data, a simpler (even wrong) model can predict better than the true one.
7. Using R^2 to compare nonlinear cognitive models. It is not comparable across models and rewards flexibility.

3.1.9 Check your understanding

1. Why does training error always decrease (or stay equal) when we add a parameter to a nested model? Answer The larger model contains the smaller one as a special case (e.g., by setting the extra parameter to 0), so its best fit can never be worse. The optimizer can always reproduce the smaller model's fit and usually improves on it by fitting noise.

2. Define overfitting in terms of signal and noise. Answer Data = signal (the systematic process) + noise (random variation). An overfitting model adapts to the noise of the particular sample. That improves its fit to these data but harms prediction of new data, in which the noise is different.

3. Describe K-fold cross-validation in four steps. Answer (1) Split the data into K folds. (2) For each fold, fit the model on the other $K - 1$ folds. (3) Score the fitted model's predictions for the held-out fold (e.g., with the log-likelihood). (4) Sum or average the scores over folds; repeat with new random splits if possible.

4. In the forgetting experiment, why do we assign items to folds *within* each retention interval rather than leaving out whole intervals? Answer Within-interval splitting tests prediction of new trials under the same conditions, and every training fold still contains every condition. Leaving out whole intervals tests a different claim: extrapolation to new conditions (the generalization criterion, Part C).

5. Model A has CV-NLL 2.1 nats lower than model B on average, but the difference varies with an SD of 3.0 across CV repetitions. What do you conclude? Answer The evidence is weak. Strictly, the SD across repetitions reflects split-to-split variability rather than the sampling uncertainty of the data, but either way a 2-nat difference with this much instability should not be over-interpreted. Collect more data or report the models as roughly equally predictive.

6. Why can a simpler, "wrong" model sometimes cross-validate better than the true, more complex model? Answer With limited data, the complex model's parameters are estimated with high variance, and that variance hurts prediction more than the simpler model's bias does (the bias–variance trade-off). With more data, the true model's variance shrinks and it eventually wins.

7. A student reports $R^2 = 0.97$ for a nonlinear forgetting model and concludes that it is correct. Give two objections. Answer (i) R^2 is high for almost any monotone decreasing curve, because most variance in forgetting data is the overall decline. (ii) A good fit says nothing about generalization or flexibility; residual patterns, CV and a comparison with alternative models are needed.

3.1.10 Assignment

A3: Model comparison and recovery (1.0 point of 20; released Week 6, due at the start of the Week 9 lecture)

A3 spans this week and next: Tasks 1–3 use this week's material, Tasks 4–6 use Week 7's (AIC/BIC, parameter and model recovery).

Data. A forgetting experiment with 12 participants, generated by this code:

> Code · کد: `code/week06.py`, cell 6 · سلول 6

Tasks.

1. Individual fits (1.5 pts). Fit the exponential, power and exponential-with-asymptote models to each participant by multi-start MLE. Report a table of parameters and NLLs; flag estimates on bounds.
2. Absolute fit (1 pt). Compute G^2 per participant and model. How many participants does each model fail to fit ($p < .05$)?
3. Cross-validation (2 pts). For each participant, run 5-fold item-level CV (repeated 5 times) for the three models. Report which model wins for how many participants, and the summed CV-NLL across participants with its variability.
4. Information criteria (1.5 pts, Week 7). Compute AIC, AICc and BIC for every participant and model, and the Akaike weights. Do the criteria agree with CV?
5. Recovery (2.5 pts, Week 7). (a) Parameter recovery for the power model at this design: 200 simulations with parameters drawn from plausible ranges; plot true vs. recovered values, report correlations, and comment on any trade-off. (b) Model recovery for the three models with AIC and BIC: a 3×3 confusion matrix from 100 simulations per generating model.
6. Interpretation (1.5 pts, max. 400 words). Which model best describes forgetting in this study, how strong is the evidence, and how does the recovery analysis qualify your conclusion?

Rubric (10 points, scaled to 1.0): the points per task are given above. Within each task: correctness 60%, clarity of tables and figures 20%, interpretation 20%. Late submissions lose 10% per day (max. 3 days).

3.1.11 Further reading

- Farrell & Lewandowsky (2018), chapter 10. Model comparison: why fit is not enough, flexibility, and cross-validation in relation to other comparison methods. *(Main reading.)*
- Azimifar & Ahmadi (1396), vol. 1 (in Persian): the material on generalization, overfitting and the bias–variance trade-off from the pattern-recognition perspective [verify chapter].
- *Optional classics, outside the course list:* Roberts, S., & Pashler, H. (2000). How persuasive is a good fit? A comment on theory testing. *Psychological Review, 107*, 358–367. Busemeyer, J. R., & Wang, Y.-M. (2000). Model comparisons and model selections based on generalization criterion methodology. *Journal of Mathematical Psychology, 44*, 171–189.

Preview for Week 7: cross-validation is general but computationally expensive. Information criteria (AIC, BIC) estimate generalization from a single fit. And before trusting any comparison, we must check whether our method can recover the parameters and the models that actually generated the data.

3.2 Week 7: Evaluating Models II: Information Criteria, Parameter Recovery and Model Recovery

Chapter 3: Evaluation of cognitive models · Lecture 1 h · Lab 2 h Running data: the Week 2–6 forgetting data and models, plus large simulation studies at the same design.

3.2.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
criterion (c)	ملاک	آستانه‌ی تصمیم؛ نشان‌دهنده‌ی سوگیری پاسخ
parameter recovery	بازیابی پارامتر	آیا برازش، پارامترهای معلوم شبیه‌سازی را برمی‌گرداند
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
model recovery	بازیابی مدل	آیا مقایسه‌ی مدل، مدل مولد را درست انتخاب می‌کند
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
research design	طرح آزمایش	شرایط، تعداد کوشش‌ها و نمونه‌ها
likelihood-ratio test (LRT)	آزمون نسبت درست‌نمایی	مقایسه‌ی دو مدل تودرتو
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
nested models	مدل‌های تودرتو (nested)	یک مدل حالت خاص مدل دیگر است
AIC / BIC	بیزی / معیار اطلاعاتی آکائیکه	برازش به‌علاوه‌ی جریمه برای تعداد پارامتر
response bias	سوگیری پاسخ	تمایل به یک پاسخ، مستقل از محرک
confusion matrix	ماتریس درهم‌ریختگی (confusion matrix)	P(مدل انتخاب‌شده
inversion matrix	ماتریس معکوس (inversion matrix)	P(مدل مولد
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
exponential / power function	توانی / تابع نمایی	دو شکل رایج برای منحنی فراموشی و یادگیری

English	فارسی	توضیح کوتاه
identifiability	شناسایی‌پذیری	آیا داده پارامترها را به‌طور یکتا تعیین می‌کند
parameter trade-off	بده‌بستان پارامترها	تغییر یک پارامتر که با پارامتر دیگر جبران می‌شود
log-likelihood / NLL	منفی آن / لگاریتم درست‌نمایی	مقداری که بهینه‌سازها کمینه یا بیشینه می‌کنند
Akaike weights	وزن‌های آکائیکه	پشتیبانی نسبی از هر مدل
cross-validation (CV)	اعتبارسنجی متقابل (cross-validation)	برازش روی بخشی از داده و آزمون روی بخش کنارگذاشته
trial	کوشش (trial)	یک بار انجام تکلیف توسط شرکت‌کننده
model mimicry	تقلید مدل	تولید داده‌ای که شبیه داده‌ی مدل دیگر است
in-context learning	یادگیری درون‌متنی	یادگیری از مثال‌های داخل prompt

---

3.2.2 Learning outcomes

By the end of this week, students will be able to:

1. Conduct a likelihood-ratio test for nested models and explain why it cannot compare non-nested models.
2. Compute and interpret AIC, AICc, BIC and Akaike/BIC weights, and explain the different goals of AIC and BIC.
3. Design and run a parameter-recovery study, and diagnose biased, noisy or traded-off parameters.
4. Design and run a model-recovery study, and interpret the confusion and inversion matrices.
5. Evaluate whether a given experimental design and comparison method can support the conclusions a researcher wants to draw.

3.2.3 Motivation & big picture

Cross-validation (Week 6) is general but expensive: every fold means refitting, and refitting complex models with multi-start can take hours. Information criteria estimate out-of-sample performance from a *single* fit, by adding a complexity penalty to the training fit.

But there is a more fundamental question that no criterion answers on its own: can our method even work at this design? Suppose we conclude “participants forget by a power law, with decay exponent 0.42”. Two things must be true for this to mean anything:

1. If the data had come from a power model with $d = 0.42$, our fitting procedure would have returned something close to 0.42 (parameter recovery).
2. If the data had come from an exponential model, our comparison method would have said "exponential", not "power" (model recovery).

Both can be checked by simulation, because we know the truth for simulated data. Modern guidelines for computational modeling treat these checks as essential; skipping them is like running an experiment without checking that the equipment works. This week completes the "simulate $\rightarrow$ fit $\rightarrow$ evaluate $\rightarrow$ recover" part of our pipeline.

Where is this used?

- Model comparison tables with AIC/BIC appear in almost every modeling paper in psychology, neuroscience, ecology and economics.
- Computational psychiatry: before claiming that patients have a lower learning rate, researchers must show by parameter recovery that the learning rate can be estimated at their design. Reviewers increasingly require this.
- Designing experiments: recovery simulations are run *before* collecting data, to choose the number of trials and conditions (a modeling version of power analysis).
- Any study that says "model X explains the data best" should report model recovery; otherwise the winner may only reflect a bias of the criterion.

3.2.4 Lecture notes

In plain words Cross-validation is honest but slow. Information criteria (AIC, BIC) are quick formulas that take the fit of each model and subtract a penalty for every parameter, so that a model only wins if its extra parameters improve the fit by more than they "cost". AIC aims at the best predictor, BIC at the true model, and BIC's penalty is larger. The second, more important half of the week asks whether our methods work at all. We test them on fake data where we *know* the truth. Parameter recovery: simulate data with known parameter values, fit, and check that the fitted values come back close to the true ones. Model recovery: simulate data from each candidate model and check that the comparison method picks the model that actually generated them. If a method cannot find the truth in simulated data, its answer on real data means nothing.

Suggested timing (60 min): likelihood-ratio test 8 · AIC and BIC 20 · parameter recovery 14 · model recovery 14 · identifiability 4.

1. The likelihood-ratio test (nested models)

Model $\mathcal{M}_0$ is nested in $\mathcal{M}_1$ if $\mathcal{M}_0$ is a special case of $\mathcal{M}_1$ obtained by fixing some parameters. Example: the exponential model is the exponential-with-asymptote model with $s = 0$ (asymptote at 0). Under the null hypothesis that $\mathcal{M}_0$ is true,

$$\Lambda = 2\big[\ell_1(\hat{\boldsymbol{\theta}}_1) - \ell_0(\hat{\boldsymbol{\theta}}_0)\big] \mathrel{\dot\sim} \chi^2_{\Delta k},$$

where Δk is the difference in the number of free parameters. A large Λ means the extra parameters improve the fit more than chance would.

Limitations: it only works for nested models (power vs. exponential cannot be tested this way); it is a null-hypothesis test (it can reject the simpler model but never support it); and it is approximate when the fixed parameter value lies on the boundary of the parameter space (as $s = 0$ does).

> Advanced: boundary nulls. When the null value is on the boundary (a variance of 0, an asymptote of 0, a mixture weight of 0), the null distribution of Λ is a 50:50 mixture of χ^2_0 (a point mass at 0) and χ^2_1. Using χ^2_1 is then conservative: the correct p-value is half of the naive one.

2. Information criteria

Information criteria trade off fit against complexity. With maximized log-likelihood $\hat{\ell}$, k free parameters and N observations:

$$\text{AIC} = -2\hat{\ell} + 2k, \qquad \text{AICc} = \text{AIC} + \frac{2k(k+1)}{N-k-1}, \qquad \text{BIC} = -2\hat{\ell} + k \ln N.$$

Lower is better. Only differences between models fitted to the same data are meaningful.

	AIC (and AICc)	BIC
Goal	Best prediction of new data (minimize expected Kullback–Leibler divergence to the truth)	Pick the true model if it is among the candidates (consistency); approximates the Bayes factor
Penalty per parameter	2	$\ln N$ (bigger than 2 once $N \geq 8$)
As $N \to \infty$	May keep choosing over-complex models	Chooses the true model with probability $\to 1$
Small-sample correction	AICc (use when $N/k < 40$)	—

What is N? The number of *independent observations* that enter the likelihood. For binomial recall data that is the number of item responses (here $8 \times 50 = 400$), not the number of cells (8). This choice affects BIC (through $\ln N$) and AICc. Report it.

Weights. With $\Delta_i = \text{IC}_i - \min_j \text{IC}_j$,

$$w_i = \frac{\exp(-\Delta_i/2)}{\sum_j \exp(-\Delta_j/2)}.$$

Akaike weights estimate the probability that model i is the best predictor among the candidates. BIC weights approximate posterior model probabilities under equal prior odds (Week 10).

Rules of thumb for Δ: 0–2, essentially equivalent support; 4–7, considerably less support; more than 10, essentially none. These are conventions, not laws.

Limitation shared by AIC and BIC: both count parameters, k, so they are blind to differences in functional-form flexibility between models with the same k (Week 6). Cross-validation and Bayes factors (Week 10) are not.

> Advanced: where the AIC penalty comes from. The maximized log-likelihood is an optimistically biased estimate of the expected log-likelihood of new data, because the same data were used to choose $\hat{\boldsymbol{\theta}}$. Akaike showed that, for regular models and large N, the bias is approximately k. Correcting for it gives $-2(\hat{\ell} - k) =$ AIC. That is also why AIC and LOO-CV agree asymptotically.

Reading the math

- $-2\hat{\ell}$ is the misfit part (twice the minimized NLL); $2k$ or $k \ln N$ is the penalty part. Lower AIC/BIC is better.
- Each extra parameter must lower $-2\hat{\ell}$ by more than 2 (AIC) or more than $\ln N$ (BIC) to be worth it. With $N = 400$, $\ln N \approx 6$: a parameter must improve the NLL by about 3 units to pass BIC, but only by 1 unit to pass AIC.
- Weights: $\exp(-\Delta_i/2)$ converts a difference in AIC into a relative likelihood. $\Delta = 2$ gives $e^{-1} = 0.37$, so a model 2 AIC units worse is about a third as well supported. Dividing by the sum makes the weights add up to 1.
- Worked numbers: power model AIC $= 2(19.830) + 2(2) = 43.66$; exponential-with-asymptote $= 2(18.919) + 2(3) = 43.84$. The asymptote model fits better (lower NLL) but pays for one more parameter, so the two end up almost tied.

3. Parameter recovery

Question: at this design, does the fitting procedure return the parameters that generated the data?

Procedure: 1. Choose a set of true parameter values $\boldsymbol{\theta}^*$ covering the plausible range (a grid, or random draws from plausible ranges, e.g., around the estimates from real data). 2. For each $\boldsymbol{\theta}^*$, simulate a dataset with exactly the real design (same conditions, numbers of trials). 3. Fit the model with exactly the real fitting procedure (same optimizer, starts, bounds). 4. Compare $\hat{\boldsymbol{\theta}}$ with $\boldsymbol{\theta}^*$: scatter plots of true vs. recovered; correlation; bias (mean of $\hat{\theta} - \theta^*$); RMSE. 5. Also plot the recovered parameters against each other. A strong correlation between, say, $\hat{\lambda}$ and $\hat{s}$ that is absent in the true values reveals a trade-off.

Reading the results: points on the identity line mean good recovery; a cloud parallel to the x-axis means the parameter is not recoverable at this design; a curve bending away at the extremes means bias near the bounds. If recovery fails, do not interpret that parameter. Change the design (more trials, other conditions) or simplify the model.

4. Model recovery

Question: if model $\mathcal{M}_a$ generated the data, does our comparison method select $\mathcal{M}_a$?

Procedure: 1. For each candidate model $\mathcal{M}_a$: simulate many datasets at the real design, with parameters drawn from plausible ranges. 2. Fit every candidate model to every dataset. 3. Select a winner by the chosen criterion (AIC, BIC, CV, ...). 4. Tabulate the confusion matrix $P(\text{selected} = b \mid \text{generated} = a)$.

A good method gives a matrix with large diagonal entries. Off-diagonal mass shows model mimicry: one model can produce data that look like another's.

What we actually need when interpreting real data is the reverse: given that model b won, how likely is it that b generated the data? This is the inversion matrix $P(\text{generated} = a \mid \text{selected} = b)$, obtained by normalizing each *column* of the confusion counts (assuming equal prior frequency of the generating models).

> Note: model recovery also reveals biases of criteria: BIC tends to favor simpler models at small N, AIC more complex ones. The "best" criterion is the one that recovers your candidate models best *at your design*, which is exactly what this study measures.

Reading the math

- Confusion matrix $P(\text{selected} = b \mid \text{generated} = a)$: rows are the true (generating) models, columns the selected ones. Each row adds up to 1. Read a row as: "when the truth is a, how often do we pick each model?"
- Inversion matrix $P(\text{generated} = a \mid \text{selected} = b)$: the same counts, but each *column* is normalized to 1. Read a column as: "when we picked b, how often was b really the truth?" This is the question that matters for a real dataset, where we only see the winner.
- Small example. 100 datasets from each of two models. From A: 90 select A, 10 select B. From B: 40 select A, 60 select B. Confusion row A = (0.90, 0.10). But among the 130 datasets that selected A, only 90 came from A: $P(\text{A true} \mid \text{A selected}) = 90/130 = 0.69$.

5. Identifiability

- Structurally non-identifiable: different parameter values give *exactly* the same predictions, for any data. Examples: label switching in the two-trace model (Week 3), or the SDT location and scale (Week 2). The fix is a constraint or a reparameterization.
- Practically non-identifiable: parameters are identifiable in principle but not with the available data. Recovery is poor and estimates trade off. The fix is a better design or a simpler model.

Parameter recovery is the practical test of identifiability.

For an engineer

- AIC and BIC are the model-order selection criteria you may know from AR/ARMA modeling; BIC is essentially Rissanen's MDL criterion. The LRT for nested models is again the GLRT.
- Parameter recovery is testing an estimator on synthetic signals with known ground truth, at the real SNR and record length, before trusting it on measured data. Engineers do this routinely; in psychology it is often skipped, which is why the course insists on it.
- The confusion matrix is the channel transition matrix of a discrete channel: input = generating model, output = selected model. The inversion matrix is the matrix of *a posteriori* probabilities $P(\text{input} \mid \text{output})$, i.e. what an MAP decoder needs, assuming equal input priors.
- Structural non-identifiability is like an unobservable state or a rank-deficient system matrix: no amount of data helps. Practical non-identifiability is an ill-conditioned problem: identifiable in principle, but the noise is too large for the available record.

For the instructor

- Frame recovery as "checking the equipment". No one would trust a thermometer that was never tested in water of known temperature. Recovery is that test for a model.
- Show one scatter plot of true vs. recovered values (lab Part C) before any theory: good recovery (points on the diagonal) next to bad recovery (a flat cloud). The picture explains itself.
- Teach confusion vs. inversion with the small 2×2 example above; students confuse them in the exam more than anything else this week. The medical-test analogy helps: the sensitivity of a test is not the probability that a positive patient is ill.
- Typical questions.
  - *"AIC or BIC?"* Depends on the goal (prediction vs. finding the true model) and, practically, on which one recovers your models better at your design.
  - *"How many simulated datasets?"* 50–100 per condition is enough to see the pattern; more for publication.
  - *"Recovery failed, what now?"* Do not interpret that parameter. Change the design or simplify the model.

3.2.5 Worked example

The four forgetting models fitted by MLE to the Week 2 data (training NLLs from Week 6), with $N = 400$ item responses:

Model	k	NLL	AIC $= 2\,\text{NLL} + 2k$	BIC $= 2\,\text{NLL} + k \ln 400$
exp	2	29.812	63.62	71.61
pow	2	19.830	43.66	51.64
expasym	3	18.919	43.84	55.81
two	4	18.919	45.84	61.80

Step 1: AIC for the power model. $2 \times 19.830 + 2 \times 2 = 43.66$.

Step 2: BIC for the power model. $\ln 400 = 5.991$, so $2 \times 19.830 + 2 \times 5.991 = 51.64$.

Step 3: Akaike weights. Differences from the best AIC: $\Delta = (19.96, 0, 0.18, 2.18)$. Then $e^{-\Delta/2} = (0.00005, 1, 0.914, 0.336)$, which sum to 2.250, so

$$w^{\text{AIC}} = (0.000,\ 0.444,\ 0.406,\ 0.149).$$

BIC weights computed the same way: $(0.000,\ 0.884,\ 0.110,\ 0.005)$.

Step 4: likelihood-ratio test, exp nested in expasym: $\Lambda = 2(29.812 - 18.919) = 21.79$ on 1 df, $p \approx 3 \times 10^{-6}$. Adding an asymptote improves the exponential model significantly.

Interpretation. Both criteria reject the pure exponential decisively. AIC cannot separate the power and asymptote models (weights 0.44 vs. 0.41): for *prediction* they are about equally good, consistent with the Week 6 CV results. BIC's larger penalty favors the two-parameter power model (0.88). The two-trace model's extra parameter buys nothing: identical NLL to expasym, so it pays the penalty for nothing. Whether we should *trust* these preferences at this design is a question for model recovery (lab Part D).

3.2.6 Lab (2 hours)

Goals

1. Run a likelihood-ratio test and compute AIC, AICc, BIC and weights for the forgetting models.
2. Run a parameter-recovery study for the power and exponential-with-asymptote models, and check for trade-offs.
3. Run a model-recovery study for three models with AIC and BIC; build confusion and inversion matrices.
4. Interpret: what can and cannot be concluded from the Week 2 data at this design?

Timing: Part A 15 min · Part B 15 min · Part C 35 min · Part D 40 min · Part E 15 min.

Setup

Code · کد: `code/week07.py`, cell 1 · سلول 1

Part A: likelihood-ratio test (15 min)

Code · کد: `code/week07.py`, cell 2 · سلول 2

Expected output:

```
NLL exp = 29.812, NLL expasym = 18.919
LR statistic = 21.79, df = 1, naive p = 3.05e-06, boundary-corrected p =
↪  1.52e-06
```

Question: why can't you use the same test to compare the power and exponential models?

Part B: information criteria and weights (15 min)

> Code · کد: `code/week07.py`, cell 3 · سلول 3

Expected output:

```
         k     NLL     AIC    AICc     BIC  w_AIC  w_AICc  w_BIC
model
exp      2  29.812  63.625  63.655  71.608  0.000   0.000  0.000
pow      2  19.830  43.661  43.691  51.644  0.444   0.449  0.884
expasym  3  18.919  43.837  43.898  55.812  0.406   0.405  0.110
two      4  18.919  45.837  45.939  61.803  0.150   0.146  0.006
```

Sensitivity check: recompute BIC with $N = 8$ (the number of cells) instead of 400. Does the ranking change? Which choice of N is defensible here, and why?

Part C: parameter recovery (35 min)

> Code · کد: `code/week07.py`, cell 4 · سلول 4

Expected output (about 1 minute):

```
pow:
   r(true, rec)   bias   RMSE
A         0.888  0.005  0.055
d         0.965  0.003  0.070

expasym:
     r(true, rec)   bias   RMSE
A           0.802  0.008  0.077
lam         0.799  0.034  0.230
s           0.918 -0.008  0.060

expasym: corr(true lam, true s) = -0.08;  corr(recovered lam, recovered
↪  s) = 0.03
```

Interpret: 1. Which parameters are well recovered at this design? Which would you hesitate to interpret? 2. The expasym decay rate λ has by far the largest RMSE (0.23 on a range of 0.05–1). Look at its panel: for which true values does recovery break down, and why? (Hint: if decay is very fast, most of the decline happens before the second test interval, so what do the data look like?) Here the recovered λ and s are *not* correlated ($r = 0.03$), so this is imprecision rather than a trade-off. Contrast this with the clear A–d trade-off of the power model found in Week 5. 3. What design change would improve recovery of λ and s? Test one idea in Exercise 3.

Part D: model recovery (40 min)

Code · کد: `code/week07.py`, cell 5 · سلول 5

Expected output (about 1–2 minutes):

```
AIC confusion matrix (rows = generating model, columns = selected model):
          exp   pow  expasym
exp      0.91  0.00     0.09
pow      0.03  0.82     0.15
expasym  0.06  0.25     0.69
AIC inversion matrix (rows = generating model, columns = selected model):
          exp   pow  expasym
exp      0.91  0.00     0.10
pow      0.03  0.77     0.16
expasym  0.06  0.23     0.74

BIC confusion matrix (rows = generating model, columns = selected model):
          exp   pow  expasym
exp      0.99  0.01     0.00
pow      0.05  0.94     0.01
expasym  0.09  0.58     0.33
BIC inversion matrix (rows = generating model, columns = selected model):
          exp   pow  expasym
exp      0.88  0.01     0.00
pow      0.04  0.61     0.03
expasym  0.08  0.38     0.97
```

Interpret: 1. Which model is most often confused with which? (With AIC, 25% of expasym datasets are labeled power; with BIC, 58%.) Explain the mimicry in terms of the curves' shapes. 2. Compare AIC and BIC. Which recovers the simpler models better, and which the more complex one? 3. In Part B, BIC selected the power model for the real data. Using the BIC inversion matrix, how confident should we be that the data were generated by a power model? (Only 61% of BIC's "power" verdicts came from power-generated data; 38% came from expasym.)

Part E: conclusions about the Week 2 data (15 min)

Write a paragraph (6–10 sentences) that a reviewer would accept, summarizing what the Week 2 data say about the form of forgetting. Use Parts A–D and your Week 6 CV results, and state what *cannot* be concluded at this design.

Exercises (increasing difficulty)

Exercise 1 $\star$. Compute AIC and BIC weights for the worked example by hand (calculator) and check them against Part B.

Exercise 2 $\star$. Repeat Part B with AICc. For which N would AICc and AIC differ noticeably for the two-trace model?

Exercise 3 $\star\star$ (design for recovery). Rerun the expasym recovery of Part C with intervals extended to $\{0.25, 1, 2, 4, 7, 14, 30, 60, 90\}$ days, and separately with $n = 200$ items. Which change helps λ and s recovery more?

Exercise 4 ⋆⋆ (CV in model recovery). Replace AIC/BIC in Part D by 5-fold CV (from Week 6) for 30 simulations per model. Is CV better or worse than AIC at recovering the models? Is it worth the computation time?

Exercise 5 ⋆⋆⋆ (Advanced: SDT recovery). For the payoff experiment of Week 4 (three conditions, 80 + 80 trials per condition), run a model recovery for "shared d'" vs. "separate d'" using the likelihood-ratio test at $\alpha = .05$ and BIC. How often does each method wrongly claim that payoffs change sensitivity? How many trials per condition would you need to detect a true sensitivity difference of 0.3 in 80% of simulations?

Lab check-off

Show: (1) your IC table with weights (Part B), (2) the recovery scatter plots with one sentence on which parameter is least well recovered and why, (3) the AIC and BIC confusion matrices, and (4) your Part E paragraph.

3.2.7 Connection to AI & LLMs

Comparing learning curves of an AI model. LLMs can learn a new task from a few examples in the prompt ("in-context learning"). How the error falls with the number of examples k is a learning curve, and the question "exponential or power?" is the same question as in this week's lab (and Week 2's forgetting curves). Information criteria, model recovery and identifiability apply unchanged. The design matters as much as before: with few values of k and few test items, the two curves may be impossible to tell apart.

Code · كد: `code/week07.py`, cell 6 · سلول 6

Expected output:

```
{'exponential': 41.6, 'power': 40.4} Akaike weights: [0.362 0.638]
```

The data were generated by a power law, but the evidence is weak (weights 0.64 vs. 0.36).

Try it (no API needed). - ⋆ Run a model recovery (Week 7, Part D) at this design: 50 datasets from each curve. Build the confusion matrix. - ⋆⋆ Change the design: more values of k (up to 128) or 200 test items per k. Which change improves model recovery more? - ⋆⋆ Discussion: a paper concludes from one fitted curve that "in-context learning follows a power law, like human practice". What would you need to see before accepting the comparison with humans?

3.2.8 Common mistakes & misconceptions

1. Interpreting an AIC value on its own. Only differences between models fitted to the same data mean anything.
2. Using LRTs for non-nested models, or reading a non-significant LRT as evidence *for* the simpler model.
3. Thinking BIC is "Bayesian AIC" with the same goal. BIC aims at identifying the data-generating model; AIC aims at prediction. They can disagree for good reasons.
4. Unclear N. Always report what counted as an observation; BIC and AICc depend on it.
5. Skipping recovery. A parameter that cannot be recovered at your design must not be interpreted, no matter how good the fit.
6. Running recovery at a different design or with a different fitting procedure than the real analysis. Recovery results are design-specific.
7. Reading the confusion matrix as if it were the inversion matrix. "Data from model A are recovered as A 90% of the time" is not the same as "when A wins, it is the true model 90% of the time".
8. Choosing parameter ranges for recovery that are wider or narrower than plausible values. Too wide inflates correlations; too narrow hides problems.

3.2.9 Check your understanding

1. Write the formulas for AIC and BIC. For which N does BIC penalize each parameter more than AIC? Answer AIC $= -2\hat{\ell} + 2k$; BIC $= -2\hat{\ell} + k \ln N$. BIC's penalty exceeds AIC's when $\ln N > 2$, i.e., $N > e^2 \approx 7.4$, so for $N \geq 8$.

2. Model A has AIC = 210.4, model B has AIC = 212.9. Compute the Akaike weights and interpret them. Answer $\Delta = (0, 2.5)$, so $e^{-\Delta/2} = (1, 0.287)$ and $w = (0.777, 0.223)$. A is favored about 3.5:1 as the better predictor, but B retains non-negligible support.

3. Why can the likelihood-ratio test compare exponential and exponential-with-asymptote but not exponential and power? Answer The exponential is a special case of the asymptote model (fix $s = 0$), so the models are nested and $2\Delta\ell$ has a known χ^2 distribution under the null. Exponential and power are not nested: neither is a special case of the other.

4. Describe the steps of a parameter-recovery study. Answer Choose plausible true parameter values; simulate data at the real design; fit with the real fitting procedure; compare recovered with true values (scatter plots, correlation, bias, RMSE); check correlations among the recovered parameters for trade-offs.

5. In a recovery study, the recovered λ and s correlate at $r = -0.8$, although the true values were drawn independently. What does that mean? Answer A trade-off: at this design, different combinations of decay rate and asymptote produce nearly the same predictions, so errors in one are compensated by the other. Individual values of λ and s should not be interpreted separately. A better design or a reparameterization is needed.

6. Explain the difference between a confusion matrix and an inversion matrix in model recovery. Answer Confusion: $P(\text{selected} \mid \text{generated})$, with rows summing to 1, which describes the method's behavior for each true model. Inversion: $P(\text{generated} \mid \text{selected})$, with columns summing to 1 (assuming equal priors for the generating models), which tells us how much to trust a given winner when analyzing real data.

7. A model recovery shows that AIC selects the three-parameter model 40% of the time when the two-parameter model generated the data. What would you do? Answer Treat AIC wins for the complex model with caution at this design: check BIC or CV recovery, which may be more conservative, improve the design (more trials, more informative conditions), and report the recovery results alongside the real-data comparison.

3.2.10 Assignment

A3 continues (due at the start of the Week 9 lecture)

Tasks 4 (information criteria) and 5 (parameter and model recovery) of A3 use this week's material. The full brief is in Week 6. Reuse today's `parameter_recovery` code and the model-recovery loop: they are designed to be adapted directly.

Final project: teams and proposal brief released

- Teams: 1–2 students. Register your team with the TA by the end of this week.
- Task: choose a cognitive model and apply the full pipeline to a dataset (simulated, open, or your own): fit, evaluate (at least one model comparison with an alternative model), recover (at least one parameter-recovery analysis), interpret. Pairs also include a Bayesian/MCMC or hierarchical analysis and an individual contribution statement.
- Proposal (due at the start of the Week 9 lecture, max. 2 pages): research question, model(s) with equations, data source and design, planned comparison and recovery analyses, and a timeline. The full project brief and rubric are distributed separately (`project_brief.md`).
- Not sure what to model? The topics of Weeks 14–15 (exemplar categorization, drift-diffusion, reinforcement learning) all make good projects, as do extensions of the forgetting and SDT models from this course.

3.2.11 Further reading

- Farrell & Lewandowsky (2018), chapter 10. Model comparison: likelihood-ratio tests, AIC, BIC and their weights; their assumptions and limitations. *(Main reading.)*
- Lee & Wagenmakers (2014), chapter 7 (introduction). How the Bayesian approach to model comparison relates to the methods of this week, as preparation for Week 10.

- *Optional, outside the course list:* Wilson, R. C., & Collins, A. G. E. (2019). Ten simple rules for the computational modeling of behavioral data. *eLife, 8*, e49547. A practical guide that treats parameter and model recovery as standard practice.

Preview for Week 8: the midterm. The lecture is an integrated review in which we run the whole pipeline (simulate, fit, evaluate, recover, interpret) on a new dataset; the lab slot is the midterm exam.

3.3 Week 8: Midterm Review: The Whole Pipeline in One Case Study

Chapters 1–3 review · Lecture 1 h (integrated case study) · Lab slot: midterm exam (90 min) + project consultation (30 min)

3.3.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
formal model / formalize	فرمال کردن / مدل فرمال	نوشتن نظریه به شکل معادله یا الگوریتم
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
parameter recovery	بازیابی پارامتر	آیا برازش، پارامترهای معلوم شبیه‌سازی را برمی‌گرداند
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
criterion (c)	ملاک	آستانه‌ی تصمیم؛ نشان‌دهنده‌ی سوگیری پاسخ
least squares	کمترین مربعات	روش برازش با کمینه کردن مجموع مربع خطاها
cross-validation (CV)	اعتبارسنجی متقابل (cross-validation)	برازش روی بخشی از داده و آزمون روی بخش کنارگذاشته
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند

English	فارسی	توضیح کوتاه
SSE / RMSD	ریشه‌ی میانگین / مجموع مربعات خطا مربع انحراف	معیارهای فاصله‌ی داده و پیش‌بینی
log-likelihood / NLL	منفی آن / لگاریتم درست‌نمایی	مقداری که بهینه‌سازها کمینه یا بیشینه می‌کنند
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
noise	نویز	تغییرات تصادفی در شواهد یا در داده
deviance (G^2)	انحراف (deviance)	معیار عدم برازش نسبت به مدل اشباع‌شده
AIC / BIC	بیزی / معیار اطلاعاتی آکائیکه	برازش به‌علاوه‌ی جریمه برای تعداد پارامتر
research design	طرح آزمایش	شرایط، تعداد کوشش‌ها و نمونه‌ها
confusion matrix	ماتریس درهم‌ریختگی (confusion matrix)	P(مدل انتخاب‌شده)
hit	اصابت (hit)	پاسخ «بله» وقتی علامت وجود دارد
response time (RT)	زمان واکنش	فاصله‌ی ارائه‌ی محرک تا پاسخ
signal detection theory (SDT)	نظریه‌ی تشخیص علامت	مدلی که حساسیت را از سوگیری پاسخ جدا می‌کند
evidence	شواهد	اطلاعاتی که برای تصمیم جمع می‌شود
exponential / power function	توانی / تابع نمایی	دو شکل رایج برای منحنی فراموشی و یادگیری
training / test data	آزمون / داده‌ی آموزش	داده‌ی کنارگذاشته برای / داده‌ی برازش سنجش

3.3.2 Learning outcomes

By the end of this week, students will be able to:

1. Execute the complete modeling pipeline (formalize → simulate → fit → evaluate → recover → interpret) on a new problem without step-by-step guidance.
2. Select an appropriate likelihood, fitting method and evaluation method for a given dataset, and justify each choice.
3. Interpret a set of competing model fits, information criteria and recovery results, and write a defensible conclusion that states its limits.
4. Recall and apply the key formulas of Weeks 1–7 (the midterm formula sheet).

3.3.3 Motivation & big picture

Over seven weeks we built a toolkit piece by piece: formal models (Weeks 1–2), least squares and optimization (Week 3), maximum likelihood (Week 4), uncertainty and multiple participants (Week 5), cross-validation (Week 6), information criteria and recovery (Week 7). Each lab practiced one or two pieces. Real research needs all of them at once, in the right order, with judgment about what matters.

Today's lecture runs the entire pipeline on a new problem: does studying a word more often make it more *memorable* (higher sensitivity), and if so, how does sensitivity grow with repetitions? The case study is also the best preparation for the midterm, which asks you to make and justify the same kinds of decisions.

3.3.4 Lecture notes

In plain words This week puts all the pieces of the first half together on one new question: *does studying a word more often make it easier to recognize later?* We write the theory as an SDT model in which sensitivity d' depends on the number of repetitions. We propose three versions of that dependence (no effect, a smooth increase, any pattern), fit all three by maximum likelihood, check whether each misfits (G^2), compare them (LRT, AIC, BIC), check that the method works at this design (recovery), and finally interpret the winner in plain psychological language, including what the data *cannot* tell us. If you can follow this one example from start to end, you understand the first half of the course.

Suggested timing (60 min): concept map 10 · case study 40 · exam briefing 10.

1. Concept map of Weeks 1–7

Pipeline step	Question	Tools	Week
Formalize	What exactly does the theory claim?	Five components of a model; parameters as psychological constructs	1–2
Simulate	What can (and can't) the model do?	Forward simulation, analytic checks, prediction plots	1–2
Fit	Which parameter values best account for the data?	SSE / least squares; NLL / MLE; grid, Nelder–Mead, L-BFGS-B; multi-start; bounds and reparameterization	3–4

Pipeline step	Question	Tools	Week
Quantify uncertainty	How precise are the estimates?	Hessian (Wald) SEs, profile likelihood, bootstrap	5
Evaluate (absolute)	Does the model misfit beyond noise?	Residuals, G^2	2–4
Evaluate (relative)	Which model generalizes best?	CV, LRT, AIC/AICc/BIC, weights	6–7
Recover	Can the method work at this design?	Parameter recovery, model recovery (confusion and inversion matrices)	7
Interpret	What does it mean psychologically, and what can't we conclude?	Parameter meaning, design limits, aggregation artifacts	all

Decision guide for the likelihood:

Data	Natural noise model	Week
Counts out of n (recall, hits, false alarms)	Binomial	2, 4
Single binary choices	Bernoulli	4
Continuous, roughly symmetric, equal variance (e.g., block-mean RTs)	Gaussian (= least squares)	4–5

2. Case study: does repetition increase memory sensitivity?

Design. In a recognition-memory experiment, words are studied 1, 2 or 4 times. At test, 100 old words per repetition condition and 100 new words (lures) are presented, and the participant says "old" or "new". Because lures were never studied, there is one false-alarm rate, shared by all conditions, and three hit rates.

Formalization (SDT with an absolute criterion). New words produce evidence $X \sim \mathcal{N}(0, 1)$; old words studied r times produce $X \sim \mathcal{N}(d'_r, 1)$; the participant says "old" if $X > \kappa$, with one criterion κ for all test items (they are intermixed, so the participant cannot know an item's condition). Then

$$H_r = \Phi(d'_r - \kappa), \qquad F = \Phi(-\kappa).$$

Three candidate models of how d' depends on repetitions:

Model	d'_r	k	Claim
$\mathcal{M}_0$ (no effect)	δ_0	2	Repetition does not change sensitivity
$\mathcal{M}_1$ (log-linear)	$\delta_0 + \delta_1 \log_2 r$	3	Each *doubling* of study adds the same sensitivity
$\mathcal{M}_2$ (free)	d'_1, d'_2, d'_4	4	Any pattern (saturated: 4 parameters for 4 rates)

$\mathcal{M}_0 \subset \mathcal{M}_1 \subset \mathcal{M}_2$, so all pairs are nested.

Likelihood. Binomial for each of the four counts (three hit counts, one false-alarm count).

The lab code below runs every step. Students run it at home as exam preparation; the lecture walks through the outputs.

Reading the math

- $H_r = \Phi(d'_r - \kappa)$ and $F = \Phi(-\kappa)$: the same SDT as Week 2, but the criterion is written as an absolute position κ on the evidence axis (measured from the center of the "new word" curve) instead of relative to the midpoint. New words: curve at 0, so F is the area right of κ, i.e. $\Phi(-\kappa)$. Old words studied r times: curve at d'_r, so $H_r = \Phi(d'_r - \kappa)$.
- One κ for all conditions: all test words are mixed together, so the participant cannot use a different threshold for words studied once and words studied four times.
- $\log_2 r$: the base-2 logarithm, so $\log_2 1 = 0$, $\log_2 2 = 1$, $\log_2 4 = 2$. In model $\mathcal{M}_1$, $d'_r = \delta_0 + \delta_1 \log_2 r$ means: δ_0 is the sensitivity after one study, and every *doubling* of study adds δ_1.
- Numbers from the worked example: $\hat{\kappa} = 1.04$ gives $F = \Phi(-1.04) = 0.15$ ✓. With $\hat{d}'_1 = 0.82$: $H_1 = \Phi(0.82 - 1.04) = \Phi(-0.22) = 0.41$, close to the observed 0.40 ✓.

For an engineer The pipeline of the concept map is the system identification workflow you know: choose candidate model structures, estimate parameters (ML), validate (residual tests $\approx G^2$), select the model order (LRT/AIC/BIC $\approx$ GLRT/AIC/MDL), and verify the procedure on simulated data with known truth (recovery). Two differences deserve emphasis in class: the "system" is a person whose parameters are themselves the scientific result, and the amount of data is tiny by engineering standards (400 binary responses), so uncertainty and recovery matter much more.

For the instructor

- Use the concept-map table as the backbone of the session. For each row, point to the corresponding output of the case study (e.g. "Evaluate (absolute)" $\rightarrow$ the G^2 column). Students see that every tool has a place.

- Ask the class to interpret before you do. Show the results table without the interpretation paragraph and let small groups write two sentences: "what do we conclude, and what can't we conclude?" Then compare with the text. This is exactly the skill the midterm tests.
- Exam briefing: tell students that the exam rewards *justified decisions* (which likelihood, which criterion, why recovery matters) more than computation. Show one past-style question from the "Check your understanding" section.
- Project consultation (lab slot): ask each team three questions: What is your cognitive question? Which parameter or model comparison answers it? Can your design recover it? Most weak proposals fail on the third.

3.3.5 Worked example

The case-study results for one simulated participant (true model $\mathcal{M}_1$ with $\delta_0 = 0.8$, $\delta_1 = 0.6$, $\kappa = 1.0$), from the code in the lab section. Numbers are rounded.

Step 1: data.

Condition	Old words	"Old" responses	Rate
$r = 1$	100	40	0.40
$r = 2$	100	66	0.66
$r = 4$	100	80	0.80
New (lures)	100	15	0.15

Step 2: fit and evaluate (MLE, $N = 400$ responses):

Model	k	NLL	G^2 (df, p)	AIC	BIC	w_{AIC}	w_{BIC}	$\hat{d}'_{1,2,4}$	$\hat{\kappa}$
$\mathcal{M}_0$ none	2	27.27	35.55 (2, < .001)	58.54	66.52	0.00	0.00	1.34, 1.34, 1.34	1.04
$\mathcal{M}_1$ log-linear	3	9.77	0.54 (1, .46)	25.53	37.51	0.67	0.94	0.82, 1.37, 1.93	1.04
$\mathcal{M}_2$ free	4	9.50	0 (saturated)	26.99	42.96	0.33	0.06	0.78, 1.45, 1.88	1.04

Likelihood-ratio tests: $\mathcal{M}_0$ vs. $\mathcal{M}_1$: $\Lambda = 35.01$, df = 1, $p < .001$; $\mathcal{M}_1$ vs. $\mathcal{M}_2$: $\Lambda = 0.54$, df = 1, $p = .46$.

Recovery at this design (lab code, second block): $\mathcal{M}_1$'s parameters are well recovered ($r = 0.89$–0.93), and BIC identifies the generating model in 87–98% of simulations.

Step 3: interpret. Repetition clearly increases sensitivity: the no-effect model misfits badly ($G^2 = 35.5$, $p < .001$) and is rejected by every criterion. The log-linear model fits as well as the saturated model (LRT $p = .46$) with one parameter fewer, and both AIC and BIC prefer it. Its estimate $\hat{\delta}_1 \approx 0.55$ means each doubling of study adds roughly half a d' unit. Recovery results show that, at this design, both the parameters and the model choice can be trusted. Caveats: this is one simulated participant; with only three repetition levels, a log-linear trend cannot be distinguished from many other increasing curves (e.g., a power function of r); and a single criterion κ assumes the participant cannot tell how often an item was studied before responding.

3.3.6 Lab (2 hours): midterm exam + project consultation

Midterm exam (90 minutes, in the lab slot)

- Format: written, closed book. A formula sheet (below) is provided. No laptops: questions involve reading short code fragments and interpreting outputs, not writing code from scratch.
- Coverage: Weeks 1–7 (Chapters 1–3), with the emphasis on *understanding and interpretation* (the course's specific goal 2).
- Question types: (1) short conceptual questions; (2) one calculation by hand (e.g., d' and c, a likelihood or NLL, AIC/BIC and weights); (3) reading code and predicting or explaining its output; (4) interpreting a fit table, a residual plot or a recovery/confusion matrix; (5) one short design question ("what would you change, and why?").
- Weight: 3 of 20 points.
- The midterm exam itself, with model answers, is in the course assessments (Stage 3 materials).

Formula sheet provided at the exam:

$$d' = \Phi^{-1}(H) - \Phi^{-1}(F), \quad c = -\tfrac{1}{2}[\Phi^{-1}(H) + \Phi^{-1}(F)], \quad H = \Phi(d'/2 - c), \quad F = \Phi(-d'/2 - c)$$

$$f_{\text{exp}}(t) = Ae^{-\lambda t}, \quad f_{\text{pow}}(t) = A(1+t)^{-d}, \quad P_c^{\text{RW}} = \frac{1-r^z}{1-r^a},\ r = \frac{1-v}{1+v}, \quad \mathbb{E}[T] = \frac{aP_c - z}{v}$$

$$\text{SSE} = \textstyle\sum_i (y_i - \hat{y}_i)^2, \quad \text{RMSD} = \sqrt{\text{SSE}/N}$$

$$\ell(\boldsymbol{\theta}) = \textstyle\sum_i \left[\ln \binom{n_i}{y_i} + y_i \ln f_i + (n_i - y_i)\ln(1 - f_i)\right], \quad \text{NLL} = -\ell$$

$$G^2 = 2(\ell_{\text{sat}} - \hat{\ell}), \quad \Lambda = 2(\hat{\ell}_1 - \hat{\ell}_0) \sim \chi^2_{\Delta k}$$

$$\text{SE} \approx \sqrt{\text{diag}(\text{H}^{-1})}, \quad \text{Wald CI: } \hat{\theta} \pm 1.96\,\text{SE}, \quad \text{profile CI: } 2(\text{NLL}_{\text{prof}} - \text{NLL}_{\text{min}}) \le 3.84$$

$$\text{AIC} = 2\,\text{NLL} + 2k, \quad \text{AICc} = \text{AIC} + \tfrac{2k(k+1)}{N-k-1}, \quad \text{BIC} = 2\,\text{NLL} + k\ln N, \quad w_i = \frac{e^{-\Delta_i/2}}{\sum_j e^{-\Delta_j/2}}$$

Critical values: $\chi^2_{1,.95} = 3.84$, $\chi^2_{2,.95} = 5.99$, $\chi^2_{3,.95} = 7.81$. Quantiles: $\Phi^{-1}(0.84) \approx 1.0$, $\Phi^{-1}(0.975) = 1.96$. Logs: $\ln 2 = 0.693$, $\ln 10 = 2.303$, $\ln 100 = 4.605$, $\ln 400 = 5.991$.

Project consultation (30 minutes, after the exam)

Teams meet the instructor or TA for 5–10 minutes each to discuss their project idea before the proposal deadline (start of Week 9). Bring a one-paragraph description of your question, model and data.

Self-study: the case study in code (run before the exam)

Code · کد: `code/week08.py`, cell 1 · سلول 1

Code · کد: `code/week08.py`, cell 2 · سلول 2

Expected output (first block):

```
hits per condition: [40 66 80]  false alarms: 15
           k     NLL      G2  df  p(G2)     AIC     BIC
           ↪  d'_1,2,4  kappa  w_AIC  w_BIC
model
M0_none    2  27.269  35.549   2  0.000  58.538  66.521  [1.34, 1.34,
 ↪  1.34]   1.04  0.000  0.000
M1_loglin  3   9.766   0.543   1  0.461  25.533  37.507  [0.82, 1.37,
 ↪  1.93]   1.04  0.674  0.938
M2_free    4   9.495   0.000   0    NaN  26.990  42.956  [0.78, 1.45,
 ↪  1.88]   1.04  0.326  0.062

LRT M0 vs M1: 35.01 (df 1, p = 0.0000);  M1 vs M2: 0.54 (df 1, p = 0.461)
```

Expected output (second block, about 1 minute):

```
M1 recovery delta0 : r = 0.895, RMSE = 0.181
M1 recovery delta1 : r = 0.930, RMSE = 0.113
M1 recovery kappa  : r = 0.887, RMSE = 0.142

BIC model recovery, P(selected | generated):
           M0_none  M1_loglin  M2_free
M0_none       0.98       0.02     0.00
M1_loglin     0.05       0.93     0.02
M2_free       0.10       0.03     0.87
```

Note on the $\mathcal{M}_2$ generator in model recovery: its three d' values are drawn so that d'_4 lies *between* d'_1 and d'_2, a non-monotone pattern that $\mathcal{M}_1$ cannot produce. Otherwise $\mathcal{M}_2$ data would often be perfectly consistent with $\mathcal{M}_1$, and "confusions" would be meaningless.

Self-check questions on the case study (discussed in the lecture): 1. Why is there only one false-alarm rate, and how does that change the SDT model compared with Week 2? 2. Which model does each criterion favor, and do the LRTs agree? 3. Is δ_1 recoverable at this design? Is $\mathcal{M}_1$ distinguishable from $\mathcal{M}_0$ and $\mathcal{M}_2$? 4. Write the one-sentence psychological conclusion, with its main caveat.

3.3.7 Connection to AI & LLMs

Chatbots as study partners: trust, but verify. Many students use LLMs to explain concepts and check exam answers. That can help, but LLMs can produce fluent, confident and wrong calculations. Treat a chatbot's answer as you would a model's prediction: check it against something you can compute. The code below checks a pre-made chatbot answer to midterm question B3 (AIC for three models). The chatbot used $2\,\mathrm{NLL} + k$ instead of $2\,\mathrm{NLL} + 2k$.

Code · کد: `code/week08.py`, cell 3 · سلول 3

Expected output:

```
M1: chatbot 222.0, correct 224.0  WRONG
M2: chatbot 218.0, correct 221.0  WRONG
M3: chatbot 218.8, correct 223.8  WRONG
Akaike weights, chatbot: [0.07 0.55 0.37]  correct: [0.15 0.68 0.17]
```

The chatbot happens to pick the right winner ($\mathcal{M}_2$), but every number is wrong, and it doubles the support for the most complex model. A grader would give little credit, and in a real analysis the error would favor overfitting.

Try it (no API needed). - ⋆ Which error in the formula produces exactly the chatbot's numbers? What would BIC look like with the same error? - ⋆⋆ Write three small "test cases" (inputs with known answers) for the SDT formulas of Week 2 that you could use to check any answer, from a chatbot or a classmate. - Reminder: no AI tools are allowed in the exam itself, and the course policy on AI use for assignments is in `assessments/README.md`.

3.3.8 Common mistakes & misconceptions (top 10 from Weeks 1–7)

1. Treating a good fit as evidence that a model is true (Weeks 3, 6).
2. Forgetting the minus sign: optimizers minimize the negative log-likelihood (Week 4).
3. Not clipping probabilities, so $\ln 0$ crashes the fit (Week 4).
4. Trusting `success=True`, a single start, or a bounded Nelder–Mead that has collapsed against a bound (Weeks 3–4).
5. Least squares for counts or choices without thinking about the noise model (Weeks 3–4).
6. Reporting estimates without uncertainty, or using Wald intervals near bounds (Week 5).
7. Drawing conclusions about individuals from averaged curves (Week 5).
8. Leaking test data into training in cross-validation (Week 6).
9. Comparing non-nested models with an LRT, or reading AIC values on their own instead of as differences (Week 7).
10. Interpreting parameters or model comparisons without a recovery study at the actual design (Week 7).

3.3.9 Check your understanding (practice questions in exam style)

1. (Concept) Explain in three sentences why the exponential-with-asymptote model fitted the Week 2 data better than the power model, although the power model generated the data. Answer The asymptote model has an extra parameter, so it can adapt to the particular noise of the sample, here the flat-looking long-delay points. Better training fit is guaranteed for more flexible models and is not evidence of truth. Generalization measures (CV, AIC/BIC) and recovery studies are needed to judge it.

2. (Calculation) An observer has $H = 0.84$ and $F = 0.16$. Compute d' and c (use $\Phi^{-1}(0.84) \approx 1.0$). Answer $d' = 1.0 - (-1.0) = 2.0$; $c = -\frac{1}{2}(1.0 + (-1.0)) = 0$, so the observer is unbiased.

3. (Calculation) Two models are fitted to $N = 100$ binary responses. Model A: $k = 2$, NLL = 60.0. Model B: $k = 4$, NLL = 57.5. Compute AIC and BIC for both. Which does each criterion prefer? ($\ln 100 = 4.605$) Answer AIC: A = 124.0, B = 123.0, so AIC prefers B (slightly). BIC: A = 120 + 9.21 = 129.21; B = 115 + 18.42 = 133.42, so BIC prefers A. The criteria disagree because BIC penalizes each parameter by 4.6 instead of 2.

4. (Code reading) What is wrong with this NLL function, and what will the optimizer do?

> Code · کد `code/week08.py`, cell 4 · سلول 4

Answer (i) It returns the log-likelihood, not its negative, so `minimize` will drive the parameters toward the *worst* fit (log-likelihood toward $-\infty$). (ii) `p` is not clipped, so predicted values of 0 or above 1 give −inf or nan. Fix: `return -np.sum(binom.logpmf(y, n, np.clip(p, EPS, 1-EPS)))`.

5. (Interpretation) A recovery study gives $r(\text{true}, \text{recovered}) = 0.95$ for parameter A and 0.35 for parameter B. What do you conclude about a paper reporting group differences in B at this design? Answer B is poorly recoverable at this design: estimates are dominated by noise or trade-offs, so group differences in estimated B are not interpretable. The paper should improve the design or not interpret B.

6. (Interpretation) A BIC confusion matrix has P(selected = power | generated = asymptote) = 0.58. Your real data's BIC winner is the power model. What should you write? Answer That the power model is preferred by BIC but, at this design, BIC often selects the power model for data generated by the asymptote model, so the evidence does not discriminate between them well. Report the inversion probabilities, AIC/CV results, and recommend a design with longer delays.

7. (Design) You want to know whether a drug changes caution (boundary a) or evidence quality (drift v) in the random walk model. What data must you collect, and why is accuracy alone not enough? Answer Choices and RTs (ideally whole RT distributions) under drug and placebo. Both a larger a and a larger v raise accuracy, but a slows responses while v speeds them, so only the joint pattern separates them (Week 1).

8. (Concept) Why must cross-validation folds for the forgetting data be formed within intervals if we want to test prediction of new trials? Answer So that every training set contains all retention intervals (conditions), and the test asks about new items in known conditions. Leaving out whole intervals would test extrapolation to new conditions, which is a different generalization claim.

3.3.10 Assignment

- A3 (model comparison and recovery) is due at the start of the Week 9 lecture.
- Project proposal is due at the start of the Week 9 lecture (max. 2 pages; see Week 7).
- No new assignment this week. Prepare for the midterm by running the case study above and answering the practice questions without looking at the answers.

3.3.11 Further reading

- Farrell & Lewandowsky (2018), chapters 2–5 and 10. Reread the chapter summaries as a review of the whole first half of the course.
- Lee & Wagenmakers (2014), chapter 11 (SDT). Read the model description as preparation for Week 9, where we start treating parameters as random variables with priors.

Preview for Week 9: the second half of the course turns Bayesian. Instead of a single best estimate plus a confidence interval, we will compute the full probability distribution of the parameters given the data.

Chapter 4

Bayesian Modeling

4.1 Week 9: Bayesian Modeling I: Priors, Posteriors and Grid Approximation

Chapter 4: Bayesian modeling · Lecture 1 h · Lab 2 h Running data: a recall proportion (30 of 50) and the Week 2 SDT participant (68/100 hits, 11/100 false alarms).

4.1.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
prior	توزیع پیشین (prior)	باور درباره‌ی پارامتر پیش از دیدن داده
posterior	توزیع پسین (posterior)	باور درباره‌ی پارامتر بعد از دیدن داده
grid approximation	تقریب شبکه‌ای (grid)	محاسبه‌ی پسین روی شبکه‌ای از مقادیر
signal detection theory (SDT)	نظریه‌ی تشخیص علامت	مدلی که حساسیت را از سوگیری پاسخ جدا می‌کند
hit	اصابت (hit)	پاسخ «بله» وقتی علامت وجود دارد
Bayes' rule	قاعده‌ی بیز	پیشین × درست‌نمایی ∝ پسین
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
evidence	شواهد	اطلاعاتی که برای تصمیم جمع می‌شود
beta-binomial model	مدل beta-binomial	پیشین Beta با داده‌ی دوجمله‌ای

English	فارسی	توضیح کوتاه
binomial distribution	توزیع دوجمله‌ای	توزیع تعداد موفقیت در n کوشش
MAP estimate	برآورد MAP	مُد توزیع پسین
credible interval / HDI	بازه‌ی باورپذیر (credible interval)	بازه‌ای که با احتمال مشخص پارامتر را دارد
confidence interval (CI)	بازه‌ی اطمینان	بازه‌ای که با روش مشخص پارامتر واقعی را در بر می‌گیرد
prior predictive check	بررسی پیش‌بینی پیشین	شبیه‌سازی داده از پیشین برای سنجش معقول بودن آن
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
maximum likelihood estimate (MLE)	برآورد بیشینه‌ی درست‌نمایی	پارامترهایی که داده را محتمل‌ترین می‌کنند
cognitive model	مدل شناختی	توصیف دقیق و ریاضی یا محاسباتی از یک فرایند ذهنی
trial	کوشش (trial)	یک بار انجام تکلیف توسط شرکت‌کننده
conjugate prior	پیشین مزدوج (conjugate)	پیشینی که پسین هم‌خانواده می‌دهد
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان
accuracy	دقت	نسبت پاسخ‌های درست

4.1.2 Learning outcomes

By the end of this week, students will be able to:

1. State Bayes' rule for parameters and identify the roles of prior, likelihood, evidence and posterior.
2. Compute posteriors analytically for the beta-binomial model and numerically by grid approximation, for one and two parameters.
3. Summarize a posterior (mean, median, MAP, SD, equal-tailed and highest-density intervals) and distinguish a credible interval from a confidence interval.
4. Explain how the prior and the amount of data jointly determine the posterior, and use prior predictive simulation to check whether a prior is sensible.
5. Interpret posterior probabilities of psychologically meaningful hypotheses (e.g., "$P(d' > 1 \mid \text{data})$").

4.1.3 Motivation & big picture

Since Week 3 we have asked: "which single parameter value best accounts for the data?" and then, in Week 5, "how much would that estimate vary across imaginary replications?" Researchers often want to say something more direct:

> *"Given these data, there is a 95% probability that this participant's sensitivity lies between 1.2 and 2.1." "The probability that the payoff manipulation made participants more liberal is 0.98."*

Frequentist confidence intervals do not license such statements (Week 5). Bayesian inference does, at a price: we must state what we believed about the parameters before seeing the data, in the form of a prior distribution. Bayes' rule then combines the prior with the likelihood (which we already know how to write) into a posterior distribution: everything the model and the data together say about the parameters.

Bayesian methods also make several of our earlier headaches easier: uncertainty comes for free (no Hessians or bootstraps), hierarchical models of many participants become natural (Week 13), and model comparison gets a principled answer (Week 10). The cost is computation, which is why Weeks 11–13 are about sampling.

> Reminder (Week 1): this week is about Bayesian data analysis, a method we as scientists use to estimate the parameters of *any* model. It makes no claim that the mind is Bayesian. Bayesian *models of cognition* come in Week 15.

Where is this used?

- Small samples and individual cases. Clinical neuropsychology often has one patient and a few dozen trials. A Bayesian analysis with a sensible prior gives usable, honest uncertainty where MLE is unstable.
- Direct probability statements that clinicians and policy makers understand: "there is a 97% probability that the new training improved memory".
- Adaptive testing: computerized tests (e.g. vision and hearing tests, adaptive exams) update a posterior after every response and choose the next item where it is most informative.
- Combining evidence: today's posterior is tomorrow's prior, so results of earlier studies can be built in formally.
- Modern cognitive modeling software (PyMC, Stan, JAGS) is Bayesian; most hierarchical models in recent papers are fitted this way (Week 13).

4.1.4 Lecture notes

In plain words So far we found *one* best parameter value and then asked how much it might vary. The Bayesian approach instead describes our knowledge about the parameter as a whole probability distribution. Before the data, the prior says which values we consider plausible (e.g. "recall probability is probably moderate, not 0 or 1"). The data then re-weight every possible value by its likelihood (Week 4). The result, the posterior, says how plausible each value is *after* seeing the data. In words: *posterior* $\propto$ *likelihood* $\times$ *prior*. From the posterior we can read off a best guess (its mean), an uncertainty (its spread), and direct statements such as "the probability that $d' > 1$ is 0.93". With little data the prior matters a lot; with much data the posterior is dominated by the likelihood and ends up close to the MLE. Computing the posterior is easy for one or two parameters (on a grid) and needs sampling methods beyond that (Weeks 11–13).

Suggested timing (60 min): Bayes' rule 10 · priors 10 · beta-binomial 12 · grid approximation 10 · summaries and credible intervals 12 · prior predictive checks 6.

1. Bayes' rule for parameters

$$\underbrace{p(\boldsymbol{\theta} \mid \mathrm{y})}_{\text{posterior}} = \frac{\overbrace{p(\mathrm{y} \mid \boldsymbol{\theta})}^{\text{likelihood}}\ \overbrace{p(\boldsymbol{\theta})}^{\text{prior}}}{\underbrace{p(\mathrm{y})}_{\text{evidence (marginal likelihood)}}}, \qquad p(\mathrm{y}) = \int p(\mathrm{y} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})\, d\boldsymbol{\theta}.$$

(We suppress "$\mathcal{M}$" in the notation this week: everything is conditional on the model.)

- The likelihood is exactly the function from Week 4.
- The prior expresses what is known or plausible before the data.
- The evidence $p(\mathrm{y})$ does not depend on $\boldsymbol{\theta}$. For estimation it is just the normalizing constant that makes the posterior integrate to 1:

$$p(\boldsymbol{\theta} \mid \mathrm{y}) \propto p(\mathrm{y} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}).$$

Posterior $\propto$ likelihood $\times$ prior. In Week 10 the evidence becomes the star of Bayesian model comparison.

Intuition. The posterior is a compromise: it is high where the parameter values are both plausible *a priori* and make the data probable. With little data the prior matters; with a lot of data the likelihood dominates and the posterior concentrates around the MLE.

Reading the math

- $p(\boldsymbol{\theta} \mid \mathrm{y})$: "probability (density) of the parameter value $\boldsymbol{\theta}$ *given* the data". This is what we want.
- $p(\mathrm{y} \mid \boldsymbol{\theta})$: the likelihood, read as a function of $\boldsymbol{\theta}$ (Week 4).
- $p(\boldsymbol{\theta})$: the prior, chosen before the data.

- $p(\mathrm{y}) = \int p(\mathrm{y} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta})\, d\boldsymbol{\theta}$: the integral adds up "likelihood × prior" over *all* possible parameter values. It is one number, the same for every $\boldsymbol{\theta}$, so for estimation it only rescales the curve to total area 1.
- $\propto$ means "proportional to": equal up to a constant factor. In practice we compute likelihood × prior at many values and then divide by their sum, exactly as in the 4-point table of the worked example (0.0004479 / 0.0009574 = 0.468).

2. Choosing priors

Kind	Example for a recall probability θ	When
Flat / uniform	Beta$(1, 1)$	When every value in the range is genuinely equally plausible (rarely true)
Weakly informative	Beta$(2, 2)$: mild preference for moderate values	Default for most cognitive models: rules out absurd values, otherwise lets the data speak
Informative	Beta$(20, 5)$: "recall after one day is around 0.8"	When previous studies or theory provide real knowledge

Principles: - Priors must be fixed before seeing the data. - They should respect the parameter's range (e.g., no prior mass on negative decay rates). - They should be checked by prior predictive simulation (§6): which *data* does the prior imply? - A "vague" prior on one scale can be very informative on another scale (lab Part D). - Report your priors and check the sensitivity of conclusions to reasonable alternatives (Week 10).

3. A closed-form case: the beta-binomial model

For y successes in n binomial trials with a Beta(a_0, b_0) prior on θ:

$$p(\theta \mid y) \propto \underbrace{\theta^{y}(1-\theta)^{n-y}}_{\text{likelihood}} \cdot \underbrace{\theta^{a_0-1}(1-\theta)^{b_0-1}}_{\text{prior}} = \theta^{a_0+y-1}(1-\theta)^{b_0+n-y-1},$$

so the posterior is again a Beta distribution:

$$\theta \mid y \sim \text{Beta}(a_0 + y,\ b_0 + n - y).$$

When prior and posterior come from the same family, the prior is called conjugate.

- a_0 and b_0 act like pseudo-counts of prior successes and failures.

- Posterior mean: $\dfrac{a_0 + y}{a_0 + b_0 + n}$, a weighted average of the prior mean $\frac{a_0}{a_0+b_0}$ and the observed proportion y/n, with weights $a_0 + b_0$ and n.
- Connection to Weeks 2 and 4: the log-linear correction $(y+0.5)/(n+1)$ is exactly the posterior mean under a Beta$(0.5, 0.5)$ prior, the so-called *Jeffreys prior*. The "correction" was a Bayesian estimate in disguise.

Reading the math

- The Beta distribution Beta(a, b) is a flexible bell-like curve on the interval 0–1, ideal for describing beliefs about a probability. Its mean is $a/(a + b)$. Beta$(1, 1)$ is flat; Beta$(20, 5)$ is a narrow bump around 0.8.
- The update rule Beta(a_0, b_0) $\rightarrow$ Beta$(a_0 + y, b_0 + n - y)$ is just counting: add the observed successes to a and the failures to b. The prior behaves as if we had already seen a_0 successes and b_0 failures (pseudo-counts).
- Example: prior Beta$(2, 2)$ ("as if" 2 recalls and 2 failures), data 7 of 10 $\rightarrow$ Beta$(9, 5)$, mean $9/14 = 0.643$. The prior pulls the observed 0.70 slightly toward 0.5, because it adds 4 pseudo-observations to 10 real ones.

4. Grid approximation

Most cognitive models have no conjugate prior. If there are only one or two parameters, we can approximate the posterior on a grid:

1. Define a grid of parameter values $\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_G$ covering all plausible values.
2. At each grid point compute the log prior plus the log likelihood (logs avoid underflow, as in Week 4).
3. Subtract the maximum, exponentiate, and normalize so that the values sum to 1 (or integrate to 1).

The result is a discrete approximation of the posterior from which we can compute any summary. Grid approximation is exact up to the grid resolution and is ideal for building intuition. It fails beyond 2–3 parameters (the curse of dimensionality, Week 3), which motivates MCMC (Weeks 11–13).

5. Summarizing a posterior

Summary	Definition	Remark
Posterior mean	$\mathbb{E}[\theta \mid \mathrm{y}]$	Minimizes expected squared error
Posterior median	50th percentile	Robust to skew
MAP	$\arg\max_\theta p(\theta \mid \mathrm{y})$	Equals the MLE under a flat prior

Summary	Definition	Remark
Posterior SD	$\sqrt{\text{Var}[\theta \mid \text{y}]}$	The Bayesian counterpart of a SE
95% equal-tailed interval	2.5th to 97.5th percentiles	Easy to compute
95% HDI (highest-density interval)	The narrowest interval containing 95% of the posterior; every value inside is more probable than every value outside	Preferred for skewed posteriors

Credible vs. confidence interval. A 95% credible interval *does* mean "given the model, the prior and the data, the parameter lies in this interval with probability 0.95". A confidence interval is a statement about the long-run behavior of the procedure (Week 5). With weak priors and plenty of data, the two often coincide numerically, but their interpretations differ.

Posterior probabilities of hypotheses come for free: $P(d' > 1 \mid \text{y})$ is just the posterior mass above 1.

6. Prior predictive checks

The prior predictive distribution describes the data the model expects *before* seeing any data:

$$p(\text{y}^{\text{rep}}) = \int p(\text{y}^{\text{rep}} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})\, d\boldsymbol{\theta}.$$

To simulate from it: draw $\boldsymbol{\theta}$ from the prior, then simulate data from the model. If the simulated data look absurd (e.g., nearly all hit rates at exactly 0 or 1, or recall that *increases* over a month), the prior is unreasonable. Prior predictive checks are the Bayesian version of the "simulate before you fit" habit from Week 2.

> Advanced: the posterior as updated prior. Because $p(\boldsymbol{\theta} \mid \text{y}_1, \text{y}_2) \propto p(\text{y}_2 \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta} \mid \text{y}_1)$ for independent data, today's posterior is tomorrow's prior. Bayesian updating is therefore naturally sequential, which is also why Bayesian inference is a popular normative model of *learning* (Week 15).

For an engineer

- This is Bayesian estimation as in detection and estimation theory: the posterior is what an MAP or MMSE estimator works with. The posterior mean is the MMSE estimate; the posterior mode is the MAP estimate; with a flat prior MAP = ML.
- The conjugate Beta–binomial update is the same idea as conjugate Gaussian updating, the scalar core of the Kalman filter (prior mean and variance + measurement $\rightarrow$ posterior mean and variance), here for a probability instead of a Gaussian state.
- Sequential updating ("today's posterior is tomorrow's prior") is exactly recursive Bayesian filtering.

- What is different in psychology: the prior is usually weak and must be *justified to readers*, and the main output is not a decision but a description of uncertainty about a psychological parameter. Prior predictive checks are the habit that makes priors defensible.

For the instructor

- Do the 4-point grid on the board (worked example, Part 1) with a column for prior, likelihood, product and posterior. It contains the whole of Bayesian inference in one table. Then change the prior to (0.1, 0.1, 0.4, 0.4) and recompute live: students see the prior at work.
- Use the Beta pseudo-count story. "A Beta(2, 2) prior is like having already seen 2 recalls and 2 failures." Students who find priors arbitrary relax once they see them as imaginary data.
- Address the "subjectivity" worry directly. Every analysis has assumptions (the likelihood is one too); Bayesian analysis makes one more of them explicit, and sensitivity analysis (Week 10) checks how much it matters.
- Typical questions.
  - *"Which prior is correct?"* None is uniquely correct. Use weakly informative priors, check them with prior predictive simulations, and report sensitivity.
  - *"Is a credible interval the same as a confidence interval?"* Often similar numerically with weak priors and enough data, but only the credible interval means "95% probability that the parameter is in here".
  - *"Is the brain Bayesian, then?"* That is a different question (Week 15). This week Bayes is a tool for the scientist.

4.1.5 Worked example

Part 1: a four-point grid by hand

A participant recalls $y = 7$ of $n = 10$ items. To keep the arithmetic simple, suppose θ can only take the values 0.2, 0.4, 0.6 or 0.8, each with prior probability 1/4.

θ	Prior	Likelihood $\theta^7(1-\theta)^3$ (constant dropped)	Prior × likelihood	Posterior
0.2	0.25	0.0000066	0.0000016	0.002
0.4	0.25	0.0003539	0.0000885	0.092
0.6	0.25	0.0017916	0.0004479	0.468
0.8	0.25	0.0016777	0.0004194	0.438
Sum	1		0.0009574	1.000

Each posterior value = (prior × likelihood) / sum. The binomial coefficient was dropped because it cancels in the normalization. Interpretation: after the data, 0.6 and 0.8 are about equally plausible, and 0.2 is practically excluded.

Part 2: the same data with a continuous Beta prior

Prior Beta$(2, 2)$ (mean 0.5, mildly favoring moderate values). Posterior: Beta$(2 + 7, 2 + 3) =$ Beta$(9, 5)$.

- Posterior mean $= 9/14 = 0.643$: between the prior mean (0.5) and the observed proportion (0.7), closer to the data because $n = 10 > a_0 + b_0 = 4$.
- MAP (mode) $= (9 - 1)/(9 + 5 - 2) = 0.667$.
- 95% equal-tailed credible interval $= [0.386, 0.861]$.

Interpretation. Given a mild prior and 10 items, the recall probability is probably around 0.64, but anything from about 0.39 to 0.86 remains credible. Ten items tell us little.

Part 3: the prior matters less with more data

With $y = 30$ of $n = 50$:

Prior	Posterior	Mean	95% credible interval
Beta$(1, 1)$ (flat)	Beta$(31, 21)$	0.596	[0.461, 0.724]
Beta$(0.5, 0.5)$ (Jeffreys)	Beta$(30.5, 20.5)$	0.598	[0.462, 0.727]
Beta$(20, 5)$ (informative, "$\approx$ 0.8")	Beta$(50, 25)$	0.667	[0.557, 0.768]

The flat and Jeffreys priors give practically the same answer, also equal to the Week 5 profile-likelihood interval [0.462, 0.728]. The informative prior, which is worth 25 pseudo-observations, still pulls the estimate noticeably toward 0.8: 50 observations are not enough to overwhelm a strong prior. Whether that is good depends on whether the prior knowledge was right.

4.1.6 Lab (2 hours)

Goals

1. Compute beta-binomial posteriors analytically and by grid approximation, and see the prior–data compromise.
2. Watch the posterior sharpen with data (sequential updating).

3. Compute a 2-D grid posterior for SDT (d', c); summarize it with means, HDIs and posterior probabilities.
4. Use prior predictive simulation to expose a bad "vague" prior.

Timing: Part A 25 min · Part B 15 min · Part C 40 min · Part D 25 min · Part E 15 min.

Setup

Code · کد: `code/week09.py`, cell 1 · سلول 1

Part A: beta-binomial, analytically and on a grid (25 min)

Code · کد: `code/week09.py`, cell 2 · سلول 2

Expected output:

```
flat Beta(1,1)           grid mean = 0.5962, exact mean = 0.5962, 95% CI
↪  = [0.461 0.724]
Jeffreys Beta(.5,.5)     grid mean = 0.5980, exact mean = 0.5980, 95% CI
↪  = [0.462 0.727]
informative Beta(20,5)   grid mean = 0.6667, exact mean = 0.6667, 95% CI
↪  = [0.557 0.768]
```

The grid and exact posteriors coincide. Look at where each posterior sits relative to its prior and the likelihood.

Part B: sequential updating (15 min)

Code · کد: `code/week09.py`, cell 3 · سلول 3

What to notice: the posterior starts as the prior, narrows as items accumulate, and concentrates around the true value. Its width shrinks roughly like $1/\sqrt{n}$, as the SE did in Week 5.

Part C: a two-parameter grid posterior for SDT (40 min)

The Week 2 participant: 68 hits out of 100 signal trials and 11 false alarms out of 100 noise trials. Priors: $d' \sim \mathcal{N}(1, 1.5)$ (sensitivity is usually positive, rarely above 4) and $c \sim \mathcal{N}(0, 1)$ (bias is usually modest).

Code · کد: `code/week09.py`, cell 4 · سلول 4

Expected output:

```
    post. mean  post. SD    MAP  95% HDI low  95% HDI high
d'       1.688     0.209  1.675        1.250         2.062
c        0.376     0.106  0.375        0.163         0.575
MLE (uncorrected formulas): d' = 1.694, c = 0.379
P(d' > 1 | data) = 1.000;  P(c > 0 | data) = 1.000
Posterior correlation of d' and c: 0.245
```

("1.000" means that none of the 20 000 posterior samples fell below the threshold, so the probability is > 0.9999. Report it as such, not as certainty.)

Interpret (write in your notebook): 1. How do the posterior mean and MAP compare with the MLE? Why are they so close here? 2. Translate the HDI for d' and $P(c > 0 \mid \text{data})$ into one plain-language sentence each about this participant. 3. Increase the prior SD on d' to 10, then decrease it to 0.3 (centered at 1). Which summaries change, and by how much?

Part D: prior predictive checks (25 min)

Before trusting a prior, look at the data it implies. Compare our prior with a "vague" one, $d' \sim \mathcal{N}(0, 10)$ and $c \sim \mathcal{N}(0, 10)$, which is sometimes used because it is "uninformative".

Code · کد: `code/week09.py`, cell 5 · سلول 5

Expected output:

```
our prior: d'~N(1,1.5), c~N(0,1): 14% of simulated datasets have a rate
↪  of exactly 0 or 1; 24% have H < F
'vague' prior: d'~N(0,10), c~N(0,10): 96% of simulated datasets have a
↪  rate of exactly 0 or 1; 23% have H < F
```

What to notice: the "vague" prior is not neutral at all. On the probability scale it puts 96% of simulated datasets at hit or false-alarm rates of exactly 0 or 1, which practically never happens in real detection experiments. It expresses a strong and absurd belief about the data. Vagueness on the parameter scale is not vagueness on the data scale.

Our own prior is not perfect either: about a quarter of its simulated datasets have $H < F$, because $\mathcal{N}(1, 1.5)$ puts about 25% of its mass on negative d'. For a task where participants are known to be above chance, a prior restricted to $d' > 0$ (e.g., a half-normal or a Gamma) would be more defensible. Try it, and check whether it changes the posterior for this participant. (Hint: with 200 informative trials, hardly at all.)

Part E: interpret (15 min)

Write a short "results paragraph" (4–6 sentences) for this participant as it would appear in a paper. Report the priors, the posterior means and 95% HDIs for d' and c, $P(d' > 1 \mid \text{data})$, and one sentence on prior sensitivity (from Part C, question 3).

Exercises (increasing difficulty)

Exercise 1 $\star$. Show algebraically that the posterior mean of the beta-binomial model is a weighted average of the prior mean and y/n. What are the weights?

Exercise 2 $\star$. Using the Part A grid, compute the 95% HDI and the equal-tailed interval for $\text{Beta}(1 + 2, 1 + 18)$ (2 successes out of 20). Why do they differ?

Exercise 3 $\star\star$ (forgetting by grid). Compute the grid posterior of (A, d) for the power forgetting model with the Week 2 data, using priors $A \sim \text{Beta}(8, 2)$ and $d \sim \text{Gamma}(\text{shape } 2, \text{scale } 0.3)$. Compare the posterior means and 95% HDIs with the MLE and the Week 5 profile and bootstrap intervals.

Exercise 4 $\star\star$ (equivalence of MAP and MLE). In Part C, replace both priors by flat priors over the grid. Show that the MAP equals the MLE. Why is the posterior *mean* still slightly different?

Exercise 5 $\star\star\star$ (Advanced: sequential vs. batch). For the SDT model, update the grid posterior trial by trial (each trial's posterior becomes the next trial's prior) through 200 simulated trials, and show that the final posterior is identical (up to numerical error) to the batch posterior from all trials at once. Plot the posterior SD of d' against the number of trials.

Lab check-off

Show: (1) the three-panel prior/likelihood/posterior plot from Part A, (2) the SDT summary table and joint posterior from Part C, and (3) the two prior predictive plots from Part D with one sentence on what is wrong with the vague prior.

4.1.7 Connection to AI & LLMs

How good is the model, really? A Bayesian answer for small evaluations. Researchers often test an LLM on a small set of items, for example 25 hand-written problems. The beta-binomial model of this week gives what such studies usually lack: a full posterior for the accuracy, direct probability statements ("the probability that accuracy exceeds 60%"), and a direct comparison of two models.

> Code · كد: `code/week09.py`, cell 6 · سلول 6

Expected output:

```
Model A: posterior mean = 0.704, 95% interval = [0.522, 0.857],
 ↪  P(accuracy > 0.6) = 0.878
Model B: posterior mean = 0.593, 95% interval = [0.406, 0.766],
 ↪  P(accuracy > 0.6) = 0.479
P(A more accurate than B | data) = 0.807
```

With 25 items, the posterior intervals are about 0.35 wide, and there is a 19% posterior probability that B is actually the better model.

Try it (no API needed). - ⋆ Use an informative prior from an earlier, larger evaluation (e.g., Beta(14, 6) for model A). How much do the conclusions change? - ⋆⋆ The comparison above treats the two models as independent, although they answered the same items. Why is this a problem (see the Week 5 box)? Sketch a model that respects the pairing. - ⋆⋆ Discussion: what prior would you use for the accuracy of a brand-new model on a four-choice test, and why should it not put much mass below 0.25?

4.1.8 Common mistakes & misconceptions

1. "The prior makes Bayesian inference subjective, so the results are arbitrary." Priors are explicit assumptions, just like the choice of likelihood. Report them, justify them, and check sensitivity. With adequate data, reasonable priors lead to similar conclusions.
2. "Flat or very wide priors are uninformative." They can be strongly informative on the data scale (Part D). Use weakly informative priors and check them with prior predictive simulation.
3. Choosing the prior after looking at the data. That uses the data twice.
4. Multiplying probabilities on a grid without logs, which causes underflow (Week 4). Work with log-prior + log-likelihood, then subtract the maximum before exponentiating.
5. Too coarse or too narrow a grid. If the posterior is cut off at the grid edge, all summaries are wrong. Check that the posterior is near zero at the edges.
6. Reading a credible interval as a confidence interval, or vice versa. Only the credible interval supports "95% probability the parameter is in here" (given the model and prior).
7. Reporting only the MAP. The point of the posterior is the whole distribution. Report a central value *and* an interval, and probabilities for the hypotheses you care about.

4.1.9 Check your understanding

1. Write Bayes' rule for a parameter θ and name each term. Answer $p(\theta \mid y) = p(y \mid \theta)p(\theta)/p(y)$: posterior = likelihood × prior / evidence (marginal likelihood). For estimation, $p(\theta \mid y) \propto p(y \mid \theta)p(\theta)$.

2. Prior Beta$(3, 3)$, data: 12 recalled out of 20. What is the posterior, and its mean? Answer Beta$(3 + 12, 3 + 8) =$ Beta$(15, 11)$, with mean $15/26 = 0.577$.

3. Why does the posterior get closer to the MLE as n increases? Answer The log-likelihood is a sum over observations and grows with n, while the log-prior stays fixed. With much data the likelihood dominates the product, and the posterior concentrates around the likelihood's maximum.

4. What is the difference between a 95% HDI and a 95% equal-tailed interval? When do they coincide? Answer The HDI is the narrowest interval containing 95% of the posterior mass, and every value inside has higher density than any value outside. The equal-tailed interval cuts 2.5% from each tail. They coincide for symmetric unimodal posteriors and differ for skewed ones.

5. Why does grid approximation become impractical for models with many parameters? Answer The number of grid points grows exponentially with the number of parameters (e.g., 100 points per parameter means 100^k evaluations). That is the curse of dimensionality. MCMC sampling (Weeks 11–13) avoids it.

6. A colleague uses $d' \sim \mathcal{N}(0, 100)$ "to be safe". What would you do to convince them it is a bad idea? Answer A prior predictive simulation: show that this prior implies hit and false-alarm rates that are almost always exactly 0 or 1, an absurd belief about real detection data. Suggest a weakly informative prior instead.

7. Explain the statement "the log-linear correction for extreme rates is a Bayesian estimate". Answer The corrected rate $(y + 0.5)/(n + 1)$ equals the posterior mean of θ under a Beta$(0.5, 0.5)$ prior: $(0.5 + y)/(0.5 + 0.5 + n)$. The correction implicitly adds half a pseudo-observation of each type.

4.1.10 Assignment

- A3 (model comparison and recovery) and the project proposal are due at the start of this week's lecture.
- No new graded assignment this week. A4 (Bayesian analysis by grid) is released in Week 10.
- Optional practice (ungraded): lab Exercise 3 (forgetting by grid) is the core of A4. Starting it now will save you time.

4.1.11 Further reading

- Kruschke (2014), chapters 5–6. Bayes' rule, and inferring a binomial probability via exact mathematical analysis (the beta-binomial model). Very gentle and visual. *(Main reading.)*
- Lee & Wagenmakers (2014), chapters 1 and 3. The basics of Bayesian analysis, and inferences with binomials.
- Martin, Kumar & Lao (2021), chapter 1. Bayesian inference: the components of a Bayesian model and prior/posterior predictive distributions, with Python code.

- Farrell & Lewandowsky (2018), chapter 6. Bayesian parameter estimation: basic concepts, linked to the likelihood framework of Week 4.
- Azimifar & Ahmadi (1396), vol. 1 (in Persian): the section on Bayesian parameter estimation and its relation to ML estimation [verify chapter].

Preview for Week 10: the posterior tells us about parameters *within* a model. Next week we ask how the data change our belief *between* models, using the evidence $p(\mathrm{y} \mid \mathcal{M})$ that we have been ignoring as a normalizing constant, and how to check whether a model's posterior predictions look like the real data.

4.2 Week 10: Bayesian Modeling II: Posterior Predictive Checks and Bayes Factors

> Chapter 4: Bayesian modeling (with Chapter 3: evaluation) · Lecture 1 h · Lab 2 h Running data: the Week 2 forgetting data (exponential vs. power), and a weak-evidence SDT participant (is $d' = 0$?).

4.2.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
posterior	توزیع پسین (posterior)	باور درباره‌ی پارامتر بعد از دیدن داده
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان
exponential / power function	توانی / تابع نمایی	دو شکل رایج برای منحنی فراموشی و یادگیری
evidence	شواهد	اطلاعاتی که برای تصمیم جمع می‌شود
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز
posterior predictive check (PPC)	بررسی پیش‌بینی پسین	مقایسه‌ی داده‌ی شبیه‌سازی‌شده از پسین با داده‌ی واقعی
marginal likelihood / evidence	درست‌نمایی حاشیه‌ای	احتمال داده تحت مدل، میانگین روی پیشین
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
Bayes factor (BF)	عامل بیز (Bayes factor)	نسبت درست‌نمایی‌های حاشیه‌ای دو مدل

English	فارسی	توضیح کوتاه
model flexibility / complexity	پیچیدگی مدل / انعطاف‌پذیری	دامنه‌ی الگوهایی که مدل می‌تواند تولید کند
grid approximation	تقریب شبکه‌ای (grid)	محاسبه‌ی پسین روی شبکه‌ای از مقادیر
Savage–Dickey ratio	نسبت Savage–Dickey	از نسبت چگالی پسین به پیشین در BF مقدار صفر
AIC / BIC	بیزی / معیار اطلاعاتی آکائیکه	برازش به‌علاوه‌ی جریمه برای تعداد پارامتر
prior	توزیع پیشین (prior)	باور درباره‌ی پارامتر پیش از دیدن داده
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
posterior odds	بخت پسین (odds)	نسبت احتمال پسین دو مدل
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
noise	نویز	تغییرات تصادفی در شواهد یا در داده
adequacy check	بررسی کفایت مدل	آیا مدل ویژگی‌های اصلی داده را بازتولید می‌کند
nested models	مدل‌های تودرتو (nested)	یک مدل حالت خاص مدل دیگر است
trial	کوشش (trial)	یک بار انجام تکلیف توسط شرکت‌کننده
accuracy	دقت	نسبت پاسخ‌های درست

4.2.2 Learning outcomes

By the end of this week, students will be able to:

1. Generate posterior predictive distributions and conduct graphical and numerical posterior predictive checks (PPCs).
2. Define the marginal likelihood and the Bayes factor, and explain why the marginal likelihood penalizes model flexibility automatically.
3. Compute Bayes factors by grid integration and with the Savage–Dickey density ratio, and compare them with the BIC approximation.
4. Evaluate the sensitivity of Bayes factors to the prior, and contrast it with the robustness of posterior estimates.
5. Interpret Bayes factors on a graded evidence scale and combine them with prior odds into posterior model probabilities.

4.2.3 Motivation & big picture

Last week gave us posteriors *within* a model. Two questions remain.

1. Is the model any good at all? A posterior can be narrow and confident even when the model is badly wrong: the exponential model will happily give a precise posterior for λ for the Week 2 data. The Bayesian answer is to ask the fitted model to generate new data and check whether they look like the real data. This is a *posterior predictive check*, the Bayesian counterpart of residual plots and G^2.

2. Which model do the data support more? In Week 7 we compared models with AIC and BIC, which penalize parameter counts. Bayesian inference has a built-in answer: compute how well each model predicted the data *before* it was fitted, averaging over all parameter values its prior allows. That is the marginal likelihood, and the ratio of two marginal likelihoods is the Bayes factor. A flexible model spreads its predictions over many possible datasets, so it assigns less probability to any one of them. The Bayes factor thus implements Occam's razor automatically, and it responds to functional-form flexibility, not just to k.

Bayes factors are also used to test null hypotheses ("is sensitivity zero?"). Unlike p-values, they can quantify evidence for the null.

Where is this used?

- Evidence for "no effect". Replication studies often ask whether an effect is absent. A p-value cannot support the null; a Bayes factor can ("the data are 8 times more likely under no effect").
- Psychology journals and software: Bayes factors are reported routinely with tools such as JASP and the R package BayesFactor, for t-tests, ANOVAs and correlations as well as for cognitive models.
- Model checking in every Bayesian analysis: posterior predictive checks are a standard step before any result is trusted (e.g. in clinical trials analyzed with Bayesian methods and in cognitive modeling papers).
- Comparing theories with different flexibility: when two models have the same number of parameters but one can produce many more data patterns, the Bayes factor is one of the few tools that notices.

4.2.4 Lecture notes

In plain words Two separate questions this week. (1) Is my model good enough? Let the fitted model *imagine* new data: take plausible parameter values from the posterior, simulate a new experiment, repeat many times. If the real data look like a typical imagined dataset, the model is adequate; if the real data sit outside the cloud of imagined data, the model is missing something. This is a posterior predictive check. (2) Which of two models do the data support more? Ask each model, *before fitting*, how probable it found the data we actually got, averaging over all the parameter values its prior allows. That number is the marginal likelihood, and the ratio for two models is the Bayes

factor. A model that bets on everything (very flexible) must spread its bets thinly, so it wins less when the data fall where a simpler model bet. That is Occam's razor, built into probability theory. The catch: the Bayes factor depends on the prior, so you must report how it changes with reasonable priors.

Suggested timing (60 min): posterior predictive checks 15 · marginal likelihood and Bayes factors 20 · computing BFs (grid, BIC, Savage–Dickey) 15 · prior sensitivity and comparison of methods 10.

1. Posterior predictive checks

The posterior predictive distribution describes new data $\mathrm{y}^{\mathrm{rep}}$ that the fitted model expects:

$$p(\mathrm{y}^{\mathrm{rep}} \mid \mathrm{y}) = \int p(\mathrm{y}^{\mathrm{rep}} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta} \mid \mathrm{y})\, d\boldsymbol{\theta}.$$

Simulation recipe: for $s = 1, \ldots, S$, draw $\boldsymbol{\theta}^{(s)}$ from the posterior and simulate a dataset $\mathrm{y}^{\mathrm{rep}(s)}$ from the model with the real design. This propagates *both* parameter uncertainty and sampling noise.

Checks: - Graphical: plot the observed data on top of the predictive intervals (e.g., the central 95% of y^{rep} per condition). Systematic departures (observed points repeatedly outside the bands, or on one side of them) show where the model fails. - Numerical: choose a test statistic $T(\mathrm{y})$ that captures a feature you care about (e.g., recall at the longest delays, the steepness of the early drop). The posterior predictive p-value is $p_B = P\big(T(\mathrm{y}^{\mathrm{rep}}) \geq T(\mathrm{y}) \mid \mathrm{y}\big)$. Values near 0 or 1 indicate that the model cannot reproduce this feature.

PPCs assess absolute adequacy, i.e., whether the model is good enough, not which of two models is better. They are a routine step of every Bayesian analysis (Week 13).

Reading the math

- $\mathrm{y}^{\mathrm{rep}}$: "replicated" data, an imagined repetition of the experiment.
- $\int p(\mathrm{y}^{\mathrm{rep}} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta} \mid \mathrm{y})\, d\boldsymbol{\theta}$: average the model's predictions over all parameter values, each weighted by its posterior plausibility. In practice this integral is replaced by the two-step simulation recipe: draw $\boldsymbol{\theta}$ from the posterior, then simulate data with it.
- $p_B = P(T(\mathrm{y}^{\mathrm{rep}}) \geq T(\mathrm{y}) \mid \mathrm{y})$: the fraction of imagined datasets whose summary T is at least as large as the real one. If only 1% of imagined datasets show recall as high as observed at 30 days ($p_B = 0.01$), the model cannot produce that feature.

2. The marginal likelihood and the Bayes factor

The marginal likelihood (evidence) of model $\mathcal{M}$ is the likelihood averaged over the prior:

$$p(\mathrm{y} \mid \mathcal{M}) = \int p(\mathrm{y} \mid \boldsymbol{\theta}, \mathcal{M})\, p(\boldsymbol{\theta} \mid \mathcal{M})\, d\boldsymbol{\theta}.$$

It is the probability the model assigned to the observed data *before seeing them*. Bayes' rule at the level of models gives:

$$\underbrace{\frac{p(\mathcal{M}_1 \mid \mathrm{y})}{p(\mathcal{M}_2 \mid \mathrm{y})}}_{\text{posterior odds}} = \underbrace{\frac{p(\mathrm{y} \mid \mathcal{M}_1)}{p(\mathrm{y} \mid \mathcal{M}_2)}}_{\mathrm{BF}_{12}} \times \underbrace{\frac{p(\mathcal{M}_1)}{p(\mathcal{M}_2)}}_{\text{prior odds}}.$$

BF_{12} is the factor by which the data change the odds in favor of $\mathcal{M}_1$ over $\mathcal{M}_2$, and $\mathrm{BF}_{21} = 1/\mathrm{BF}_{12}$.

The automatic Occam's razor. Both models must spread a total probability of 1 over all possible datasets. A simple model concentrates its predictions on a few datasets; a flexible model spreads them thin. If the data fall where the simple model predicted, the simple model gets the higher marginal likelihood, even though the flexible model can also fit them well after its parameters are tuned. Unlike AIC/BIC, this penalty depends on *how* flexible a model is (the prior predictive spread), not merely on k.

A common evidence scale (an adaptation of Jeffreys' classic categories, as used by Lee & Wagenmakers):

BF_{12}	Evidence for $\mathcal{M}_1$
1–3	Anecdotal
3–10	Moderate
10–30	Strong
30–100	Very strong
> 100	Extreme

(Values below 1 are read the same way for $\mathcal{M}_2$.) These labels are aids, not thresholds. Report the number.

Reading the math

- $p(\mathrm{y} \mid \mathcal{M}) = \int p(\mathrm{y} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})\, d\boldsymbol{\theta}$: the likelihood averaged over the prior, not maximized. Compare: MLE takes the *best* parameter value; the marginal likelihood takes the *prior-weighted average* over all of them. A model with many bad parameter values in its prior gets a low average.
- Posterior odds = Bayes factor × prior odds. Odds are probability ratios: odds of 3 mean 3:1, i.e. probability 0.75. If both models were equally plausible before (prior odds = 1) and $\mathrm{BF}_{12} = 3$, then afterwards $P(\mathcal{M}_1 \mid \mathrm{y}) = 3/(1+3) = 0.75$.
- Worked example in one line: with 7 of 10 correct, chance predicted the data with probability 0.117 and the "any accuracy" model with 0.091, so $\mathrm{BF}_{01} = 1.29$: the data hardly change the odds.

3. Computing Bayes factors

(a) Grid integration (1–2 parameters): $p(\mathrm{y} \mid \mathcal{M}) \approx \sum_g p(\mathrm{y} \mid \boldsymbol{\theta}_g)\, p(\boldsymbol{\theta}_g)\, \Delta_g$, where Δ_g is the cell area. Do it in log space (log-sum-exp), and check that the prior mass inside the grid is ≈ 1.

(b) The BIC approximation. For large N, $\ln p(\mathrm{y} \mid \mathcal{M}) \approx -\frac{1}{2}\mathrm{BIC}$, so

$$\mathrm{BF}_{12} \approx \exp\big(-\tfrac{1}{2}(\mathrm{BIC}_1 - \mathrm{BIC}_2)\big).$$

It is quick but crude: it ignores the actual prior (it implicitly assumes a "unit-information" prior) and functional-form flexibility.

(c) The Savage–Dickey density ratio (nested models). If $\mathcal{M}_0$ fixes $\phi = \phi_0$ in $\mathcal{M}_1$, and the priors on the remaining parameters match, then

$$\mathrm{BF}_{01} = \frac{p(\phi = \phi_0 \mid \mathrm{y}, \mathcal{M}_1)}{p(\phi = \phi_0 \mid \mathcal{M}_1)} :$$

the posterior density at the null value divided by the prior density at the null value, both under the larger model. If the data make ϕ_0 more plausible than it was a priori, the evidence favors the null. One posterior from $\mathcal{M}_1$ is all we need.

(Further methods, such as bridge sampling and thermodynamic integration, compute marginal likelihoods from MCMC output for complex models. They are beyond this course.)

Reading the math Savage–Dickey $\mathrm{BF}_{01} = p(\phi_0 \mid \mathrm{y})/p(\phi_0)$: look at the height of the posterior and of the prior at the null value ϕ_0 (e.g. $\theta = 0.5$). If the posterior is *higher* there than the prior was, the data made the null value more plausible, so the evidence favors the null. In the worked example the prior (flat) has height 1 at 0.5 and the posterior $\mathrm{Beta}(8, 4)$ has height 1.29, so $\mathrm{BF}_{01} = 1.29$. A picture of two curves and one vertical line at ϕ_0 is all you need to explain this.

4. Prior sensitivity: estimation vs. testing

For estimation, a reasonable prior matters less and less as data accumulate. For Bayes factors, the prior on a tested parameter *never* stops mattering. Making the prior under $\mathcal{M}_1$ wider spreads its predictions more thinly, which lowers $p(\mathrm{y} \mid \mathcal{M}_1)$, so an arbitrarily vague prior makes the BF favor the null no matter what the data are (the Jeffreys–Lindley paradox). Consequences: - Priors for model comparison must be chosen thoughtfully, ideally reflecting realistic effect sizes. - Always report a sensitivity analysis: the BF over a range of reasonable prior widths.

5. Comparison of model-evaluation methods

Method	Question	Penalizes	Prior needed?	Week
G^2, residuals, PPC	Is the model adequate?	—	PPC: yes	4, 10
LRT	Does the extra parameter improve fit beyond chance? (nested only)	Δk via χ^2	No	7

Method	Question	Penalizes	Prior needed?	Week
AIC / CV	Which model predicts new data best?	Estimated over-fitting	No	6–7
BIC	Approximately: which model has the higher evidence?	$k \ln N$	Implicit	7
Bayes factor	How much do the data change the odds between models?	Prior predictive spread (true flexibility)	Yes, and the result depends on it	10

> Advanced: Bayes factors vs. predictive criteria. The marginal likelihood can be written as a product of sequential predictions, $p(\mathrm{y}) = p(y_1)\, p(y_2 \mid y_1) \cdots p(y_N \mid y_{1:N-1})$. It therefore scores how well the model predicts each data point from the earlier ones, *starting from the prior*. LOO-CV scores predictions from (almost) all the other data. This is why BFs depend on the prior and LOO hardly does, and why they can disagree on the same data.

For an engineer

- Posterior odds = BF × prior odds is the MAP (Bayes) detector between two hypotheses, where the Bayes factor plays the role of the likelihood ratio.
- The difference from the GLRT (Week 4 and 7): with unknown parameters, the GLRT *maximizes* the likelihood over them; the Bayes factor *integrates* (averages) over them with a prior. Maximizing favors flexible hypotheses; averaging penalizes them automatically.
- The Jeffreys–Lindley paradox has an engineering flavor: if you let the alternative hypothesis allow absurdly large signal amplitudes (a very wide prior), almost any observation is better explained by "noise only".
- Posterior predictive checks are like validating a channel model by simulating received signals from the fitted model and comparing their statistics (histograms, autocorrelations, eye diagrams) with measured ones.

For the instructor

- Explain Occam's razor with a betting story. Two forecasters each have 100 chips to place on possible outcomes. One spreads them over everything; the other puts most chips on a narrow range. If the outcome lands in that narrow range, the focused forecaster wins more. The marginal likelihood is the payout.

- Show prior sensitivity live. In the lab code, change the width of the prior under $\mathcal{M}_1$ and watch the BF move. This is more convincing than any warning.
- Keep the two questions apart. Write "Is the model good enough? $\rightarrow$ PPC" and "Which model is better? $\rightarrow$ BF / CV / AIC" on the board. Students often use a BF to claim a model is *good*; it only says it is *better than the other*.
- Typical questions.
  - *"BF or p-value?"* They answer different questions. The BF compares two specific hypotheses and can support the null; the p-value only measures surprise under the null.
  - *"BF = 2.5, is that significant?"* There is no threshold; it is anecdotal evidence. Report the number.
  - *"Why do BF and AIC disagree?"* They aim at different targets (evidence vs. prediction) and the BF depends on the prior.

4.2.5 Worked example

Part 1: is recall at chance? A Bayes factor by hand

A participant chooses between two alternatives on $n = 10$ trials and is correct on $y = 7$. Is performance at chance?

- $\mathcal{M}_0$: $\theta = 0.5$ (no free parameters).
- $\mathcal{M}_1$: $\theta \sim \text{Beta}(1, 1)$ (any accuracy equally plausible a priori).

Step 1. $p(y \mid \mathcal{M}_0) = \binom{10}{7} 0.5^{10} = 120/1024 = 0.1172$.

Step 2. Under a uniform prior, every count $0, \ldots, n$ is equally likely a priori, so $p(y \mid \mathcal{M}_1) = \int_0^1 \binom{10}{7} \theta^7 (1-\theta)^3 \, d\theta = \frac{1}{n+1} = \frac{1}{11} = 0.0909$.

Step 3. $\text{BF}_{01} = 0.1172/0.0909 = 1.29$. The data slightly *favor chance*, which is anecdotal evidence either way.

Step 4 (Savage–Dickey check). The posterior under $\mathcal{M}_1$ is $\text{Beta}(8, 4)$, whose density at 0.5 is 1.289. The prior density at 0.5 is 1. The ratio is 1.29, identical to Step 3.

More data:

Data	p (two-sided binomial test)	BF_{01}	Reading
7/10	0.34	1.29	Anecdotal, either way
60/100	≈ 0.06	1.10	Still anecdotal: the data barely distinguish chance from "some accuracy"

Data	p (two-sided binomial test)	BF_{01}	Reading
70/100	< 0.001	0.0023 ($\mathrm{BF}_{10} \approx 430$)	Extreme evidence against chance

Interpretation. With 60/100, a p-value near .05 tempts researchers to claim an effect, yet the Bayes factor says the data are about equally likely under both hypotheses. The reason: the alternative with a uniform prior "predicted" that accuracy could be anywhere from 0 to 1. An observed accuracy of 0.6 is only modestly better predicted by it than by exactly 0.5. A narrower, more realistic prior under $\mathcal{M}_1$ (e.g., accuracies between 0.5 and 0.8) would give a different BF, which is why prior sensitivity must be reported.

Part 2: posterior model probabilities

For 7/10 with prior odds of 1: posterior odds $= 1.29$, so $P(\mathcal{M}_0 \mid y) = 1.29/2.29 = 0.56$. If a reviewer believes a priori that chance performance is 4 times less likely than above-chance performance (prior odds 1:4), then posterior odds $= 1.29 \times 0.25 = 0.32$, so $P(\mathcal{M}_0 \mid y) = 0.24$. The BF is the same for everyone; the posterior model probabilities depend on the prior odds.

4.2.6 Lab (2 hours)

Goals

1. Run graphical and numerical posterior predictive checks for the exponential and power forgetting models.
2. Compute marginal likelihoods and the Bayes factor for power vs. exponential by grid integration; compare with BIC.
3. Test $d' = 0$ for a weak-evidence SDT participant with the Savage–Dickey ratio, and verify it by direct computation.
4. Map the sensitivity of the Bayes factor to the prior, and contrast it with the posterior estimate.

Timing: Part A 35 min · Part B 25 min · Part C 30 min · Part D 20 min · Part E 10 min.

Setup

Code · کد: `code/week10.py`, cell 1 · سلول 1

Expected output:

```
exp: posterior mean A = 0.655, rate = 0.049
pow: posterior mean A = 0.867, rate = 0.417
```

Part A: posterior predictive checks (35 min)

Code · کد: `code/week10.py`, cell 2 · سلول 2

Expected output:

```
exp: T(observed) = 0.267, mean T(rep) = 0.240, posterior predictive p =
↪  0.302
pow: T(observed) = 0.267, mean T(rep) = 0.243, posterior predictive p =
↪  0.320
```

Interpret: look at the plots first. For the exponential model, the observed recall at the shortest delay (42/50) lies *above* the 95% band and recall at 7 days (13/50) lies *below* it. The exponential cannot drop steeply early and then level off. The power model's band contains every observed point.

Yet the numerical check with T = mean recall at the three longest delays gives unremarkable p_B values for both models (about 0.3). The exponential's misfit is in the *shape* (early vs. middle delays), which this statistic does not look at. A PPC can only detect the failures its test statistic is sensitive to. Choose statistics that target the features that distinguish the theories; in Exercise 2 a "first drop" statistic exposes the exponential model ($p_B \approx 0.02$).

Part B: Bayes factor by grid integration (25 min)

Code · کد: `code/week10.py`, cell 3 · سلول 3

Expected output:

```
exp: log p(y|M) = -34.938   (prior mass inside grid = 0.9996)
pow: log p(y|M) = -22.872   (prior mass inside grid = 0.9991)

BF(pow vs exp) = exp(12.07) = 1.74e+05
BIC approximation: BF(pow vs exp) = exp(9.98) = 2.16e+04
```

Interpret: BF $\approx 1.7 \times 10^5$ is extreme evidence for the power model over the exponential. BIC gives the same verdict but a different number (2×10^4). Why do the grid BF and the BIC approximation differ? (Hint: which prior does each use, and are the two models equally flexible under our priors?)

Part C: Savage–Dickey: is this participant's sensitivity zero? (30 min)

A participant in a hard detection task: 12 hits out of 20 signal trials, 8 false alarms out of 20 noise trials. $\mathcal{M}_1$: $d' \sim \mathcal{N}(0,1)$, $c \sim \mathcal{N}(0,1)$. $\mathcal{M}_0$: $d' = 0$, $c \sim \mathcal{N}(0,1)$.

> Code · کد: `code/week10.py`, cell 4 · سلول 4

Expected output:

```
BF01 Savage-Dickey = 1.328;  BF01 direct = 1.328;  posterior mean d' =
 ↪  0.444
MLE d' = 0.507
```

The two computations agree, which is a useful check that both are implemented correctly.

Part D: prior sensitivity (20 min)

> Code · کد: `code/week10.py`, cell 5 · سلول 5

Expected output:

```
 prior SD of d'   BF01  BF10  posterior mean d'
           0.25  0.938 1.066              0.144
           0.50  0.972 1.028              0.313
           1.00  1.328 0.753              0.444
           2.00  2.314 0.432              0.496
           5.00  5.541 0.180              0.513
          10.00 11.011 0.091              0.515
```

What to notice: as the prior on d' widens, the posterior mean of d' quickly stabilizes, but BF_{01} keeps growing in favor of the null. A very wide prior "predicts" huge sensitivities that did not occur, so the alternative is penalized. For estimation the prior washes out; for testing it does not.

Part E: interpret (10 min)

Write 4–5 sentences: does this participant detect the signal? Report the BF with your chosen prior and justify the prior, mention the sensitivity analysis, and state what additional data would make the answer clearer.

Exercises (increasing difficulty)

Exercise 1 $\star$. Verify by numerical integration (`scipy.integrate.quad`) that $\int_0^1 \binom{10}{7}\theta^7(1-\theta)^3 d\theta = 1/11$.

Exercise 2 ⋆. In Part A, use a different test statistic: the drop in recall from the first to the second interval. Which model, if any, fails this check? (Expected: the exponential, $p_B \approx 0.02$; the power model passes.)

Exercise 3 ⋆⋆ (prior sensitivity of the forgetting BF). Repeat Part B with the rate priors widened (scale $\times$ 3) and narrowed (scale $\div$ 3). Does the conclusion "power $\gg$ exponential" survive? Why is it so robust here, unlike in Part D?

Exercise 4 ⋆⋆ (an informed alternative). In Part C, replace the prior on d' by a half-normal $d' \sim \mathcal{N}^+(0, 1)$ (only positive values; remember to renormalize). Compute the Savage–Dickey BF. Why does a directional prior change the result?

Exercise 5 ⋆⋆⋆ (Advanced: BF for the repetition study). For the Week 8 case study ($\mathcal{M}_0$ vs. $\mathcal{M}_1$), choose priors for δ_0, δ_1 and κ, compute BF_{10} with the Savage–Dickey ratio at $\delta_1 = 0$ using a 3-D grid (keep it coarse), and compare it with the BIC approximation.

Lab check-off

Show: (1) the two PPC plots with p_B values, (2) the grid BF for power vs. exponential and its BIC approximation, and (3) the prior-sensitivity table from Part D with one sentence of interpretation.

4.2.7 Connection to AI & LLMs

Do LLMs have a "theory of mind"? Some studies report that LLMs pass false-belief tasks, classic tests of whether one understands that others can hold false beliefs. Others show that small changes to the stories make performance collapse [verify: Kosinski, 2023/2024; Ullman, 2023]. Many such tests have two answer options, so chance is 50%, and the number of items is small. A Bayes factor answers the right first question: *do the data support above-chance performance at all, compared with chance?*

> Code · كد: `code/week10.py`, cell 6 · سلول 6

Expected output:

```
26/40 correct (65%): BF10 = 1.16
130/200 correct (65%): BF10 = 785.26
```

The same 65% is no evidence at all with 40 items, and overwhelming evidence with 200 items. Above-chance performance is also only the first step: the posterior predictive check of this week asks *how* the model succeeds and fails. For example, does it fail on exactly the altered stories, where a human would not?

Try it (no API needed). - ⋆ Repeat with a prior concentrated on above-chance values, e.g. Beta(5, 2) restricted to $\theta > 0.5$ (renormalize the density). How does BF_{10} for 26/40 change? - ⋆⋆ Simulate a

"shortcut" model that answers correctly on 90% of standard stories and 30% of altered stories (half of each). Its overall accuracy is 60%. Design a posterior predictive check that reveals the shortcut. - ⋆⋆ Discussion: is "above chance on false-belief tasks" a claim at Marr's computational or algorithmic level?

4.2.8 Common mistakes & misconceptions

1. "A narrow posterior means the model is right." Precision is not adequacy. Always run a PPC.
2. Using the posterior predictive check to choose between models. PPCs check adequacy. Use BFs, CV or information criteria to compare.
3. "The Bayes factor is the posterior probability of the model." It is the *change* in odds. Posterior probabilities also need prior odds.
4. Using very vague priors in Bayes-factor tests. They bias the BF toward the null without limit (Part D). Use realistic priors and report the sensitivity.
5. Savage–Dickey with mismatched priors. The nuisance-parameter priors must be the same in both models, and the prior density must be evaluated *at the null value* in the larger model.
6. Computing marginal likelihoods on a grid that cuts off prior mass. Check that the prior mass inside the grid is close to 1.
7. Treating "BF = 2.9" vs. "BF = 3.1" as qualitatively different. The evidence scale is continuous; labels are aids, not decision thresholds.

4.2.9 Check your understanding

1. How is a posterior predictive distribution simulated, and what does it combine? Answer Draw parameters from the posterior, then simulate data from the model with those parameters (and the real design). It combines uncertainty about the parameters with the sampling noise of new data.

2. Define the marginal likelihood in words. Answer The probability (density) of the observed data under the model, averaged over all parameter values weighted by the prior: how well the model predicted these data before it was fitted.

3. Why does the marginal likelihood penalize flexible models even without counting parameters? Answer Probability over possible datasets must sum to 1. A flexible model spreads its prior predictions over many datasets and so assigns each one, including the observed one, less probability, unless the data fall where only the flexible model predicts them.

4. $\text{BF}_{10} = 6$ and the prior odds are 1:3 (against $\mathcal{M}_1$). What are the posterior odds and $P(\mathcal{M}_1 \mid \text{y})$? Answer Posterior odds $= 6 \times 1/3 = 2$, so $P(\mathcal{M}_1 \mid \text{y}) = 2/3 \approx 0.67$.

5. State the Savage–Dickey ratio and the conditions for using it. Answer $\text{BF}_{01} = p(\phi_0 \mid \text{y}, \mathcal{M}_1)/p(\phi_0 \mid \mathcal{M}_1)$: the posterior over the prior density at the null value under the larger model. It requires nested models (the null fixes $\phi = \phi_0$) and identical priors on the other parameters.

6. Why does widening the prior on the tested parameter increase BF_{01}, even with fixed data? Answer A wider prior assigns more probability to large effects and less to the small effect sizes compatible with the data, so the alternative's marginal likelihood decreases while the null's does not change.

7. BIC gives $BF_{12} \approx 20$, while the grid computation with your priors gives 3. Which do you report and why? Answer The grid BF, because it uses the actual priors and accounts for functional-form flexibility; BIC is a large-sample approximation with an implicit prior. Mention the discrepancy and, ideally, a prior sensitivity analysis.

4.2.10 Assignment

A4: A Bayesian analysis by grid (1.0 point of 20; released Week 10, due at the start of the Week 12 lecture)

Data: the A2 dataset (`rng=np.random.default_rng(2024)`, 9 intervals, $n = 20$ items; see Week 4).

Tasks.

1. Priors (1.5 pts). Choose and justify priors for (A, λ) of the exponential model and (A, d) of the power model. Show prior predictive simulations of forgetting curves for both models and revise the priors if needed.
2. Posterior (2 pts). Compute the grid posteriors for both models. Report posterior means, SDs and 95% HDIs, and the posterior correlation of the two parameters. Plot the joint posteriors.
3. Posterior predictive checks (2 pts). Graphical PPC for both models, plus a numerical PPC with a test statistic you choose and justify.
4. Bayes factor (2 pts). Compute $BF_{pow,exp}$ by grid integration (check the prior mass), compare it with the BIC approximation, and repeat with two alternative prior settings (sensitivity analysis).
5. Comparison with A2 (1 pt). Compare the posterior summaries with your A2 MLEs and their uncertainty.
6. Interpretation (1.5 pts, max. 300 words). What do you conclude about forgetting in these data, and how robust is the conclusion to the prior?

Rubric: points as above; within each task: correctness 60%, figures and tables 20%, interpretation 20%. Late submissions lose 10% per day (max. 3 days).

4.2.11 Further reading

- Lee & Wagenmakers (2014), chapter 7. Bayesian model comparison: marginal likelihoods, Bayes factors and the Savage–Dickey method, with worked examples. *(Main reading.)*
- Farrell & Lewandowsky (2018), chapter 11. Bayesian model comparison using Bayes factors, in the context of cognitive models.
- Kruschke (2014), chapter 10. Model comparison and hierarchical modeling, a gentle introduction to Bayes factors and their prior sensitivity.
- Martin, Kumar & Lao (2021), chapter 2. Exploratory analysis of Bayesian models, including posterior predictive checks.

Preview for Week 11: our grids worked for two parameters. For five, ten or a hundred parameters we need a smarter way to explore the posterior: Markov chain Monte Carlo. You will build your first sampler from scratch.

Chapter 5

Parameter Estimation with MCMC

5.1 Week 11: Monte Carlo Methods and the Metropolis–Hastings Algorithm

Chapter 5: Parameter estimation with data-driven methods (MCMC) · Lecture 1 h · Lab 2 h Running data: the Week 2 forgetting data with the Week 10 priors, so that we can check the sampler against the grid posterior we already have.

5.1.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
Monte Carlo	Monte Carlo (مونت‌کارلو)	تقریب با میانگین نمونه‌های تصادفی
Metropolis algorithm	الگوریتم Metropolis	پیشنهاد یک حرکت و پذیرش یا رد آن
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
estimate / estimator	برآورد / برآوردگر	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
Markov chain Monte Carlo (MCMC)	Monte Carlo زنجیره‌ی مارکوف	نمونه‌گیری از پسین با یک زنجیره‌ی تصادفی
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان
prior	توزیع پیشین (prior)	باور درباره‌ی پارامتر پیش از دیدن داده
sampler	نمونه‌گیر (sampler)	الگوریتمی که از توزیع نمونه می‌گیرد

English	فارسی	توضیح کوتاه
grid approximation	تقریب شبکه‌ای (grid)	محاسبه‌ی پسین روی شبکه‌ای از مقادیر
posterior	توزیع پسین (posterior)	باور درباره‌ی پارامتر بعد از دیدن داده
chain	زنجیره	دنباله‌ی نمونه‌های MCMC
proposal	پیشنهاد (proposal)	مقدار جدیدی که الگوریتم امتحان می‌کند
acceptance rate	نرخ پذیرش	نسبت پیشنهادهای پذیرفته‌شده
trace plot	نمودار trace	مقدار نمونه‌ها در برابر شماره‌ی تکرار
autocorrelation	خودهمبستگی	وابستگی نمونه‌های پشت سر هم
burn-in / warm-up	دوره‌ی گرم‌شدن (burn-in)	نمونه‌های اولیه که دور ریخته می‌شوند
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
starting point	نقطه‌ی شروع	سوگیری اولیه به سمت یکی از پاسخ‌ها
response bias	سوگیری پاسخ	تمایل به یک پاسخ، مستقل از محرک
large language model (LLM)	مدل بزرگ زبانی	شبکه‌ی عصبی‌ای که کلمه‌ی بعدی متن را پیش‌بینی می‌کند
self-consistency	self-consistency (رأی اکثریت)	چند بار نمونه‌گیری جواب و انتخاب رأی اکثریت
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
MCSE	خطای استاندارد Monte Carlo	خطای شبیه‌سازی در میانگین پسین

5.1.2 Learning outcomes

By the end of this week, students will be able to:

1. Explain why posterior quantities are integrals, and how Monte Carlo sampling turns integrals into averages, with an error that shrinks like $1/\sqrt{S}$.
2. Describe the logic of Markov chain Monte Carlo: a chain whose long-run distribution is the posterior, using only the *unnormalized* posterior.
3. Implement the Metropolis (and Metropolis–Hastings) algorithm from scratch for a one- and a two-parameter cognitive model.
4. Tune the proposal distribution using the acceptance rate, trace plots and autocorrelation, and identify burn-in.
5. Compute posterior summaries and derived quantities (e.g., a half-life) from samples, and validate a sampler against a known answer.

5.1.3 Motivation & big picture

Everything we want from a posterior is an integral: the posterior mean $\int \theta\, p(\theta \mid \mathrm{y})\, d\theta$, a marginal posterior (integrating out other parameters), $P(d' > 1 \mid \mathrm{y})$, the posterior predictive distribution, the marginal likelihood. In Weeks 9–10 we did these integrals on grids. That worked for two parameters. A hierarchical SDT model for 20 participants (Week 13) has more than 40 parameters; a grid with only 10 points per parameter would need 10^{40} evaluations.

Monte Carlo methods replace integration by sampling: if we could draw many values from the posterior, any integral becomes an average over the draws, and the accuracy depends on the number of draws, not on the number of parameters. The problem is that we cannot sample from a posterior directly; we usually cannot even compute its normalizing constant.

Markov chain Monte Carlo (MCMC) solves this with a beautifully simple trick: a random walk through parameter space that moves toward more probable values more often than toward less probable ones, in exactly the right proportions. After a while, the places the walker visits are draws from the posterior. The best-known version, the Metropolis algorithm, fits in 15 lines of Python, and you will write it today before using any library.

Where is this used?

- All modern Bayesian software (PyMC, Stan, JAGS, BUGS) is built on MCMC. Every hierarchical model in recent psychology and neuroscience papers was fitted with some version of it.
- Physics and chemistry (where Metropolis was invented in 1953, to simulate particles), epidemiology (estimating infection parameters), phylogenetics (reconstructing evolutionary trees) and finance all rely on MCMC.
- Cognitive science also uses sampling as a *theory of the mind*: some researchers propose that people approximate Bayesian inference by mentally sampling a few possibilities, which explains certain biases.
- AI: sampling answers from a language model is Monte Carlo (see the AI & LLMs section).

5.1.4 Lecture notes

In plain words Every question we ask a posterior ("what is the average value?", "what is the probability that $d' > 1$?") is an integral, and grids cannot compute integrals over many parameters. Monte Carlo replaces the integral with an average: if we had many random values *drawn from* the posterior, the average of those values would be the posterior mean, the fraction above 1 would be $P(d' > 1)$, and so on. The difficulty is getting those draws. The Metropolis algorithm gets them with a clever random walk. Stand at some parameter value, propose a small random step, and compare how plausible (likelihood × prior) the new spot is. If it is more plausible, move there. If it is less plausible, move there only *sometimes*, with probability equal to the ratio of plausibilities; otherwise stay where you are and record the current value again. Run this long enough and the places you have visited are

a sample from the posterior. Remarkably, you never need the hard normalizing constant, because only ratios are used.

Suggested timing (60 min): Monte Carlo integration 12 · Markov chains 8 · the Metropolis algorithm 18 · Metropolis–Hastings 5 · tuning, burn-in and autocorrelation 12 · preview 5.

1. Monte Carlo integration

If $\theta^{(1)}, \ldots, \theta^{(S)}$ are independent draws from $p(\theta \mid \mathrm{y})$, then for any function g:

$$\mathbb{E}[g(\theta) \mid \mathrm{y}] = \int g(\theta)\, p(\theta \mid \mathrm{y})\, d\theta \;\approx\; \frac{1}{S} \sum_{s=1}^{S} g(\theta^{(s)}).$$

Want	Choose $g(\theta)$	Monte Carlo estimate
Posterior mean	θ	Average of the draws
$P(\theta > 1 \mid \mathrm{y})$	indicator $\mathbb{1}[\theta > 1]$	Proportion of draws above 1
Posterior of a derived quantity, e.g., half-life $2^{1/d} - 1$	$g(d)$	Apply g to each draw, then summarize
Credible interval	—	Percentiles (or HDI) of the draws
Marginal posterior of θ_1	—	Simply ignore the other columns

The Monte Carlo standard error of the mean is $\mathrm{SD}(g)/\sqrt{S}$ for independent draws, independent of the number of parameters. That is why sampling beats grids in high dimensions.

Reading the math

- $\mathbb{E}[g(\theta) \mid \mathrm{y}] = \int g(\theta)\, p(\theta \mid \mathrm{y})\, d\theta$: the posterior average of some quantity $g(\theta)$. Choosing g chooses the question (see the table).
- $\approx \frac{1}{S} \sum_s g(\theta^{(s)})$: replace the integral by the plain average over S draws $\theta^{(1)}, \ldots, \theta^{(S)}$. The superscript (s) is the draw number.
- Error: $\mathrm{SD}(g)/\sqrt{S}$. To halve the error you need four times as many draws, whatever the number of parameters.
- Example: 10,000 draws from $\mathrm{Beta}(8, 4)$; 9,000 of them are above 0.5, so $P(\theta > 0.5 \mid \mathrm{y}) \approx 0.90$, with Monte Carlo error $\sqrt{0.9 \times 0.1/10000} = 0.003$.

2. Markov chains in one paragraph

A Markov chain is a sequence $\theta^{(1)}, \theta^{(2)}, \ldots$ in which each value is generated from the previous one only, by a fixed transition rule. Under mild conditions, a chain "forgets" its starting point and settles into a stationary distribution: the long-run proportion of time it spends in each region. MCMC

constructs a transition rule whose stationary distribution is the posterior. Two consequences: - The early part of the chain still reflects the starting point. It is discarded as burn-in (warm-up). - Successive draws are correlated, so S correlated draws carry less information than S independent ones (the effective sample size, Week 12).

3. The Metropolis algorithm

Let $\tilde{p}(\theta) = p(\mathrm{y} \mid \theta)\, p(\theta)$ be the unnormalized posterior (likelihood × prior). Choose a symmetric proposal distribution, usually a normal random walk $\theta' \sim \mathcal{N}(\theta, \sigma_{\text{prop}})$.

1. Start at some $\theta^{(0)}$.
2. For $s = 1, 2, \ldots, S$:
   - Propose $\theta' \sim \mathcal{N}(\theta^{(s-1)}, \sigma_{\text{prop}})$.
   - Compute the acceptance probability

$$\alpha_{\text{acc}} = \min\left(1,\ \frac{\tilde{p}(\theta')}{\tilde{p}(\theta^{(s-1)})}\right).$$

   - Draw $u \sim \text{Uniform}(0, 1)$. If $u < \alpha_{\text{acc}}$, accept: $\theta^{(s)} = \theta'$. Otherwise reject: $\theta^{(s)} = \theta^{(s-1)}$ (the current value is *repeated* in the chain).

Why it works intuitively: uphill moves are always accepted; downhill moves are accepted with probability equal to the density ratio. So the chain spends time in each region in proportion to its posterior probability.

Why only the unnormalized posterior is needed: the evidence $p(\mathrm{y})$ appears in both numerator and denominator of the ratio and cancels. This is the key to MCMC's usefulness.

Practical details: - Work in logs: accept if $\ln u < \ln \tilde{p}(\theta') - \ln \tilde{p}(\theta)$. - Bounded parameters: if a proposal falls outside the support (e.g., $A > 1$), its prior is 0, so $\ln \tilde{p} = -\infty$ and the proposal is automatically rejected. (Alternatively, sample on a transformed, unbounded scale, as in Week 3; then the Jacobian must be added to the log posterior.) - Multiple parameters: propose all parameters at once with a vector of proposal SDs, or update one parameter at a time (which leads to Gibbs sampling, Week 12).

> Advanced: detailed balance. A transition rule $T(\theta \to \theta')$ leaves π stationary if $\pi(\theta)\, T(\theta \to \theta') = \pi(\theta')\, T(\theta' \to \theta)$ for all pairs. Check it for Metropolis with $\tilde{p}(\theta') < \tilde{p}(\theta)$: the left side is $\pi(\theta)\, q(\theta' \mid \theta)\, \tilde{p}(\theta')/\tilde{p}(\theta)$ and the right side is $\pi(\theta')\, q(\theta \mid \theta') \cdot 1$. With symmetric q and $\pi \propto \tilde{p}$, the two are equal.

Reading the math

- $\tilde{p}(\theta) = p(\mathrm{y} \mid \theta)\, p(\theta)$: likelihood × prior, *without* dividing by the evidence. The tilde (~) marks "unnormalized".
- $\alpha_{\text{acc}} = \min(1, \tilde{p}(\theta')/\tilde{p}(\theta))$: if the proposal is more plausible, the ratio is > 1 and min caps it at 1 (always accept). If it is half as plausible, accept with probability 0.5.

- "Draw $u \sim \text{Uniform}(0, 1)$; accept if $u < \alpha$" is simply a way to make an event happen with probability α (a biased coin).
- Worked example, step 3: the proposal 0.95 is only 0.052 times as plausible as 0.80; $u = 0.30 > 0.052$, so it is rejected and 0.80 is written into the chain a second time.

4. Metropolis–Hastings: asymmetric proposals

If the proposal is not symmetric, $q(\theta' \mid \theta) \neq q(\theta \mid \theta')$ (e.g., a log-normal proposal for a positive rate), the acceptance probability needs a correction:

$$\alpha_{\text{acc}} = \min\left(1,\ \frac{\tilde{p}(\theta')\, q(\theta \mid \theta')}{\tilde{p}(\theta)\, q(\theta' \mid \theta)}\right).$$

Metropolis is the special case where the q terms cancel. Gibbs sampling (Week 12) is another special case, in which every proposal is accepted.

5. Tuning and diagnosing a sampler

The proposal SD σ_{prop} controls efficiency:

σ_{prop}	Acceptance rate	Behavior	Trace plot looks like
Too small	Very high (> 0.9)	Tiny steps; explores slowly; strong autocorrelation	A slowly wandering line
Too large	Very low (< 0.1)	Most proposals land in low-probability regions and are rejected; the chain sticks	Flat steps with long plateaus
About right	$\approx$ 0.2–0.5	Good mixing	A "fuzzy caterpillar"

Classic theoretical results suggest an optimal acceptance rate of about 0.44 for one parameter, falling toward about 0.23 for many parameters. Treat these as targets, not laws. Tune σ_{prop} during a preliminary run (not during the run you keep). Good proposal SDs are of the order of the posterior SDs, and correlated posteriors need correlated (or reparameterized) proposals.

Burn-in. Discard the initial part of the chain before it reaches the high-probability region; the trace plot shows where. Thinning (keeping every m-th draw) reduces storage but throws away information. It is not needed for correct inference.

Autocorrelation. The lag-h autocorrelation is the correlation between $\theta^{(s)}$ and $\theta^{(s+h)}$. High autocorrelation means slow exploration and fewer effectively independent draws.

Always validate a new sampler on a problem with a known answer (here, the conjugate beta-binomial and the Week 10 grid).

For an engineer

- Monte Carlo integration is what you do when you estimate a BER or an outage probability by simulation: average an indicator over many random realizations. The $1/\sqrt{S}$ error law is the same, and it is why low error rates need very long simulations.
- An MCMC chain is a Markov process whose stationary distribution is designed to be the posterior; burn-in is the transient, the stationary part is the steady state.
- Autocorrelation of the chain is exactly the autocorrelation of a random process. Correlated samples carry less information, just as correlated noise samples average out more slowly; Week 12 turns this into the "effective sample size".
- Proposal tuning is a step-size problem like choosing the step size of an adaptive filter: too small converges slowly, too large becomes unstable (here: most steps are rejected).
- The difference from ordinary simulation: we cannot generate independent samples from the target directly, so we build a process that converges to it.

For the instructor

- Walk the chain on the board. Draw the curve $\theta^7(1-\theta)^3$ and run the four steps of the worked example with a student rolling the "u" values (a random number app works). The rejected step that *repeats* 0.80 is the key moment; ask why it must be recorded.
- An analogy that works: a tourist exploring a city at night who prefers lit streets. Brighter street: always go. Darker street: go only sometimes. After a long walk, the time spent on each street is proportional to its brightness.
- Show the three trace plots (too small, good, too large) from lab Part D side by side. Students learn to read trace plots in five minutes this way.
- Typical questions.
  - *"Why not just use a fine grid?"* With 40 parameters, a grid is impossible; sampling cost does not grow that way.
  - *"How do we know the chain has converged?"* That is Week 12 (multiple chains, $\hat{R}$).
  - *"Why throw away burn-in?"* Early draws still reflect the arbitrary starting point, not the posterior.

5.1.5 Worked example

Model: recall $y = 7$ of $n = 10$ with a flat prior, so $\tilde{p}(\theta) = \theta^7(1-\theta)^3$ (constants dropped). Exact posterior: Beta$(8, 4)$, mean 0.667. Proposal: $\theta' = \theta + \mathcal{N}(0, 0.1)$.

Step	Current θ	Proposal θ'	$\tilde{p}(\theta)$	$\tilde{p}(\theta')$	Ratio	u	Decision	New θ
1	0.50	0.58	0.000977	0.001636	1.675	0.42	accept (uphill)	0.58
2	0.58	0.80	0.001636	0.001678	1.026	0.71	accept (uphill)	0.80
3	0.80	0.95	0.001678	0.000087	0.052	0.30	reject: $u > 0.052$	0.80
4	0.80	0.66	0.001678	0.002144	1.278	0.88	accept (uphill)	0.66

Chain so far: 0.50, 0.58, 0.80, 0.80, 0.66. The rejected step *repeats* 0.80 in the chain. That is essential: repeated values are how the chain records that 0.80 is more probable than 0.95. Dropping them would bias the sample.

Interpretation. In four steps the chain moved from the start into the region of high posterior density (0.6–0.8) and refused a jump into the tail. Over thousands of steps, the proportion of time spent near each value converges to Beta$(8, 4)$.

5.1.6 Lab (2 hours)

Goals

1. Monte Carlo integration with independent draws: see the $1/\sqrt{S}$ error.
2. Write a Metropolis sampler and validate it on the beta-binomial model.
3. Sample the posterior of the power forgetting model, compare with the Week 10 grid, and compute a derived quantity.
4. Tune the proposal: compare too-small, good and too-large proposal SDs with trace plots, acceptance rates and autocorrelation.

Timing: Part A 15 min · Part B 25 min · Part C 35 min · Part D 30 min · Part E 15 min.

Setup

Code · کد: `code/week11.py`, cell 1 · سلول 1

Part A: Monte Carlo integration (15 min)

Code · کد: `code/week11.py`, cell 2 · سلول 2

Expected output:

```
Exact: mean = 0.5926, P(theta > 0.65) = 0.1967
S =    100: mean = 0.5865 (MC s.e. 0.0065), P(theta > 0.65) = 0.1800
S =   1000: mean = 0.5935 (MC s.e. 0.0021), P(theta > 0.65) = 0.2160
S =  10000: mean = 0.5937 (MC s.e. 0.0007), P(theta > 0.65) = 0.2022
S = 100000: mean = 0.5927 (MC s.e. 0.0002), P(theta > 0.65) = 0.1971
```

Ten times more draws gives about $\sqrt{10} \approx 3.2$ times less Monte Carlo error.

Part B: a Metropolis sampler, validated (25 min)

Code · کد: `code/week11.py`, cell 3 · سلول 3

Expected output:

```
Acceptance rate: 0.59
Metropolis: mean = 0.5930, SD = 0.0660, 95% interval = [0.458 0.718]
Exact:      mean = 0.5926, SD = 0.0663, 95% interval = [0.46  0.719]
```

Part C: Metropolis for the power forgetting model (35 min)

Same priors as in Week 10: $A \sim \text{Beta}(8, 2)$, $d \sim \text{Gamma}(2, \text{scale } 0.3)$.

Code · کد: `code/week11.py`, cell 4 · سلول 4

Expected output:

```
Acceptance rate: 0.49
    mean     SD   2.5%  97.5%
A  0.868  0.052  0.762   0.96
d  0.418  0.048  0.325   0.51
Week 10 grid posterior means for comparison: A = 0.867, d = 0.417
```

Derived quantities come for free. Compute the posterior of the half-life $t_{1/2} = 2^{1/d} - 1$ (Week 2) and the predicted recall after 60 days:

Code · کد: `code/week11.py`, cell 5 · سلول 5

Expected output:

```
half-life (days)       mean = 4.493, 95% interval = [2.889 7.443]
P(recall) at 60 days   mean = 0.158, 95% interval = [0.113 0.211]
P(half-life < 3 days | data) = 0.039
```

Part D: tuning the proposal (30 min)

Code · کد: `code/week11.py`, cell 6 · سلول 6

Expected output:

```
too small    proposal SD = [0.002, 0.002]: acceptance = 0.97, lag-10
↪  autocorrelation of d = 0.98, mean of d = 0.473
about right  proposal SD = [0.04, 0.05]: acceptance = 0.50, lag-10
↪  autocorrelation of d = 0.26, mean of d = 0.416
too large    proposal SD = [0.5, 1.5]: acceptance = 0.00, lag-10
↪  autocorrelation of d = 0.94, mean of d = 0.420
```

What to notice: the "too small" chain accepts almost everything but crawls (a slowly wandering trace with autocorrelation near 1 even at lag 60); the "too large" chain rejects almost everything and sits still for long stretches (flat steps). Both give unreliable estimates from 10 000 iterations: the "too small" chain's mean of d (0.473) is far from the true posterior mean (0.417), and the "too large" chain's mean happens to be close only because it barely moved from its starting value (0.42). The well-tuned chain looks like a fuzzy caterpillar and its autocorrelation decays quickly.

Part E: interpret (15 min)

Answer in your notebook: 1. Do the Metropolis posterior summaries for (A, d) agree with the Week 10 grid? What does that agreement establish? 2. How would you report the half-life of this participant's memory, with uncertainty, in one sentence? 3. Why must rejected proposals be recorded as repeated values in the chain?

Exercises (increasing difficulty)

Exercise 1 $\star$. Using the Part B sampler, estimate $P(\theta > 0.65 \mid \text{y})$ for 30/50 and compare with the exact value from Part A. How many iterations do you need to get within ± 0.01?

Exercise 2 $\star$. Run the Part C sampler from four very different starting points, e.g., $(0.3, 0.05)$, $(0.99, 2.5)$, $(0.5, 1.0)$ and $(0.9, 0.1)$. Plot all traces together. After how many iterations do they become indistinguishable? (This is the idea behind R-hat, Week 12.)

Exercise 3 $\star\star$ (exponential model). Write `log_post_exp` with the Week 10 priors, sample it, and compare the posterior with the grid. Then compute the posterior predictive distribution from your samples and reproduce the Week 10 PPC plot.

Exercise 4 $\star\star$ (Metropolis–Hastings with an asymmetric proposal). Sample d with a log-normal random-walk proposal, $\ln d' \sim \mathcal{N}(\ln d, 0.1)$, which never proposes negative values. Derive the Hastings correction $q(d \mid d')/q(d' \mid d) = d'/d$ and include it. Show that omitting the correction biases the posterior.

Exercise 5 $\star\star\star$ (Advanced: adaptive tuning). Write a warm-up phase that adjusts the proposal SDs every 500 iterations to push the acceptance rate toward 0.3 (e.g., multiply the SD by 1.2 if the rate is too high, by 0.8 if too low), then freezes them for the main run. Why must the adaptation stop before the samples you keep?

Lab check-off

Show: (1) the validation of your sampler against the exact Beta posterior (Part B), (2) the trace plots and summary for the power model with the comparison to the grid (Part C), and (3) the tuning figure with acceptance rates (Part D).

5.1.7 Connection to AI & LLMs

Sampling answers from an LLM is Monte Carlo. Because an LLM samples its output (Week 1 box), asking the same question several times gives different answers. A popular trick called self-consistency samples k answers and takes the majority vote [Wang et al., 2023, ICLR]. Does it help, and how much? This is a Monte Carlo question: we cannot compute the majority-vote accuracy easily in closed form, but we can simulate it and report the Monte Carlo standard error, as in this week's lab.

Code · کد: `code/week11.py`, cell 7 · سلول 7

Expected output:

```
k =  1 samples: majority-vote accuracy = 0.406 (MCSE 0.003)
k =  5 samples: majority-vote accuracy = 0.506 (MCSE 0.004)
k = 15 samples: majority-vote accuracy = 0.671 (MCSE 0.003)
k = 45 samples: majority-vote accuracy = 0.873 (MCSE 0.002)
```

Try it (no API needed). - ⋆ Change `p_answer` so that one *wrong* answer is the most likely (e.g., `[0.30,0.35,0.20,0.15]`). What does majority voting do now? Why? - ⋆⋆ How many simulated questions (`n_sim`) do you need for an MCSE of 0.001? Check your answer with the formula $\sqrt{p(1-p)/n}$. - ⋆⋆ Connection to cognition: the "wisdom of the crowd within" idea says that averaging a person's repeated guesses improves accuracy. Which assumption of this simulation (independent samples) is unlikely to hold for repeated human guesses, or for an LLM at low temperature?

5.1.8 Common mistakes & misconceptions

1. Dropping rejected steps instead of repeating the current value. This biases the sample toward the tails.
2. Using the normalized posterior. Not needed and usually impossible; the evidence cancels in the ratio.
3. Computing the ratio without logs. Likelihoods underflow. Compare log densities.

4. Forgetting the prior support. Proposals outside the support must get $\ln \tilde{p} = -\infty$, otherwise the likelihood may be evaluated at impossible values (e.g., negative probabilities produce `nan`).
5. Judging the sampler only by acceptance rate. A 30% rate does not guarantee convergence. Look at traces and autocorrelation, and run several chains (Week 12).
6. Keeping the burn-in or tuning the proposal during the samples you keep. Both bias the results.
7. "More iterations always fix it." If the chain is stuck in one mode, or the proposal is badly scaled, more iterations of the same chain may not help in practical time. Diagnose first.
8. Forgetting the Hastings correction with asymmetric proposals.

5.1.9 Check your understanding

1. Why is the Monte Carlo error independent of the number of parameters, while grid approximation is not? Answer The MC estimate is an average of S draws, and its standard error is $\text{SD}/\sqrt{S}$ regardless of the dimension of θ. A grid must cover the whole parameter space, so its size grows exponentially with the number of parameters.

2. In the Metropolis algorithm, why do we not need the evidence $p(\text{y})$? Answer Acceptance depends only on the *ratio* of posterior densities at the proposed and current values; $p(\text{y})$ appears in both and cancels.

3. Current $\tilde{p}(\theta) = 0.004$, proposed $\tilde{p}(\theta') = 0.001$, and $u = 0.18$. Accept or reject? What if $u = 0.31$? Answer The ratio is 0.25. With $u = 0.18 < 0.25$, accept; with $u = 0.31 > 0.25$, reject and repeat the current value.

4. A chain has acceptance rate 0.97. What is probably wrong, and how does its trace look? Answer The proposal SD is too small: almost every tiny step is accepted, the chain explores very slowly, and the trace looks like a slowly wandering line with high autocorrelation. Increase the proposal SD.

5. What is burn-in, and how do you decide its length? Answer The initial iterations, before the chain has reached the high-probability region, which still reflect the arbitrary starting value. Choose the length from trace plots (and from several chains started at different points), and discard those iterations.

6. When is the Hastings correction needed? Give an example. Answer When the proposal is asymmetric, $q(\theta' \mid \theta) \neq q(\theta \mid \theta')$, e.g., a log-normal proposal for a positive rate, or a proposal that depends on the current value's position in a bounded interval.

7. How do you obtain the posterior of the half-life $t_{1/2} = 2^{1/d} - 1$ from MCMC samples of d? Answer Transform every sample of d into a half-life and summarize the transformed samples (mean, percentiles, HDI). No extra derivation is needed; this is one of the big practical advantages of sampling.

5.1.10 Assignment

No new graded assignment this week. A4 (Bayesian analysis by grid) is due at the start of the Week 12 lecture. A5 (MCMC from scratch) is released in Week 12 and builds directly on today's `metropolis` function, so keep your lab notebook tidy.

Optional practice (ungraded): Exercise 2 (multiple starting points) is the best preparation for Week 12.

5.1.11 Further reading

- Kruschke (2014), chapter 7. Markov chain Monte Carlo: an intuitive introduction to the Metropolis algorithm (the "politician" example), its logic and representativeness of samples. *(Main reading.)*
- Farrell & Lewandowsky (2018), chapter 7. Bayesian parameter estimation with MCMC, applied to cognitive models.
- Martin, Kumar & Lao (2021): the sections on inference engines and MCMC [verify chapter].

Preview for Week 12: Gibbs sampling (no rejections at all), and the question every MCMC user must answer: has my chain converged? We will compute R-hat and the effective sample size ourselves, then check them against ArviZ.

5.2 Week 12: Gibbs Sampling and Convergence Diagnostics

> Chapter 5: Parameter estimation with data-driven methods (MCMC) · Lecture 1 h · Lab 2 h Running data: log response times of one participant (a normal model with unknown mean and precision), and the power forgetting model sampled with last week's Metropolis sampler.

5.2.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
Gibbs sampling	نمونه‌گیری Gibbs	به‌روزرسانی هر پارامتر از توزیع شرطی کاملش
convergence	همگرایی	رسیدن زنجیره‌ها به توزیع هدف
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
Markov chain Monte Carlo (MCMC)	زنجیره‌ی مارکوف Monte Carlo	نمونه‌گیری از پسین با یک زنجیره‌ی تصادفی
response time (RT)	زمان واکنش	فاصله‌ی ارائه‌ی محرک تا پاسخ
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان
Metropolis algorithm	الگوریتم Metropolis	پیشنهاد یک حرکت و پذیرش یا رد آن
sampler	نمونه‌گیر (sampler)	الگوریتمی که از توزیع نمونه می‌گیرد
full conditional	توزیع شرطی کامل (full conditional)	توزیع یک پارامتر به شرط بقیه و داده
conjugate prior	پیشین مزدوج (conjugate)	پیشینی که پسین هم‌خانواده می‌دهد
R-hat	R-hat	مقایسه‌ی واریانس بین و درون زنجیره‌ها؛ باید ≈ 1 باشد
effective sample size (ESS)	اندازه‌ی مؤثر نمونه (ESS)	تعداد نمونه‌ی مستقل هم‌ارز
chain	زنجیره	دنباله‌ی نمونه‌های MCMC
trace plot	نمودار trace	مقدار نمونه‌ها در برابر شماره‌ی تکرار
Monte Carlo	Monte Carlo (مونت‌کارلو)	تقریب با میانگین نمونه‌های تصادفی
proposal	پیشنهاد (proposal)	مقدار جدیدی که الگوریتم امتحان می‌کند
posterior	توزیع پسین (posterior)	باور درباره‌ی پارامتر بعد از دیدن داده
MCSE	خطای استاندارد Monte Carlo	خطای شبیه‌سازی در میانگین پسین
prior	توزیع پیشین (prior)	باور درباره‌ی پارامتر پیش از دیدن داده
HMC / NUTS	Hamiltonian Monte Carlo / NUTS	MCMC مبتنی بر گرادیان
burn-in / warm-up	دوره‌ی گرم‌شدن (burn-in)	نمونه‌های اولیه که دور ریخته می‌شوند
autocorrelation	خودهمبستگی	وابستگی نمونه‌های پشت سر هم
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز

5.2.2 Learning outcomes

By the end of this week, students will be able to:

1. Derive the full conditional distributions of a semi-conjugate normal model and implement a Gibbs sampler from scratch.
2. Explain when Gibbs sampling is efficient and when it mixes slowly (correlated parameters), and how Metropolis-within-Gibbs handles non-conjugate conditionals.
3. Implement split-$\hat{R}$ and the effective sample size (ESS), and verify them against ArviZ.
4. Diagnose non-convergence from multiple chains using trace plots, $\hat{R}$, ESS and the Monte Carlo standard error.
5. Apply a reporting checklist for MCMC results.

5.2.3 Motivation & big picture

Last week's Metropolis sampler had one tuning knob, the proposal SD, and a nagging question: *how do we know the chain has converged?* This week answers both concerns.

Gibbs sampling removes the tuning problem for a large and important class of models. Instead of proposing a jump for all parameters and sometimes rejecting it, it updates one parameter at a time, drawing it *exactly* from its distribution given the current values of all the others. Every draw is accepted. Many classic Bayesian programs (BUGS, JAGS, used throughout Lee & Wagenmakers and Kruschke) are built on this idea.

Convergence diagnostics are the honest part of MCMC. A chain is only useful if it has (a) forgotten its starting point and (b) explored the whole posterior, and nothing in a single chain proves either. The standard remedy is to run several chains from different starting points and check that they agree. $\hat{R}$ quantifies "do the chains agree?", and the ESS quantifies "how much independent information do the correlated draws contain?". No MCMC result should be reported without them.

Where is this used?

- Every published MCMC analysis must report convergence diagnostics; reviewers in psychology and neuroscience now routinely ask for $\hat{R}$ and ESS.
- Gibbs sampling powers the classic Bayesian programs BUGS and JAGS, which are used in many cognitive modeling textbooks (Lee & Wagenmakers, Kruschke) and in ecology, epidemiology and education research.
- Image processing and communications: Gibbs samplers have been used for image restoration and for detection in multi-antenna (MIMO) receivers.
- Topic models (finding themes in large text collections) were classically fitted with Gibbs sampling.

5.2.4 Lecture notes

In plain words Metropolis needs a well-chosen step size. Gibbs sampling avoids that for many models by updating one parameter at a time: fix all the other parameters at their current values, and draw the chosen parameter directly from its exact distribution under those conditions (its full conditional). Then move to the next parameter, and so on. Nothing is ever rejected. The second half of the week answers the uncomfortable question "has my sampler actually worked?". We run several chains from very different starting points. If they all end up describing the same distribution, that is good evidence of convergence; $\hat{R}$ ("R-hat") is a single number that measures whether the chains agree (it should be below 1.01). Because neighboring draws are correlated, 4,000 draws may contain only the information of, say, 1,000 independent ones: that number is the effective sample size (ESS), and it tells us how precise our averages are.

Suggested timing (60 min): Gibbs sampling 20 · when Gibbs struggles 5 · convergence and multiple chains 8 · $\hat{R}$ 10 · ESS and MCSE 10 · reporting checklist 7.

1. Gibbs sampling

For parameters $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_k)$, the full conditional of θ_m is its posterior given the data and *all other* parameters, $p(\theta_m \mid \boldsymbol{\theta}_{-m}, \mathrm{y})$. Gibbs sampling cycles through them:

1. Start at $\boldsymbol{\theta}^{(0)}$.
2. For $s = 1, \ldots, S$:
   - draw $\theta_1^{(s)} \sim p(\theta_1 \mid \theta_2^{(s-1)}, \ldots, \theta_k^{(s-1)}, \mathrm{y})$
   - draw $\theta_2^{(s)} \sim p(\theta_2 \mid \theta_1^{(s)}, \theta_3^{(s-1)}, \ldots, \mathrm{y})$
   - ... (always using the newest values)
   - draw $\theta_k^{(s)} \sim p(\theta_k \mid \theta_1^{(s)}, \ldots, \theta_{k-1}^{(s)}, \mathrm{y})$

It is a special case of Metropolis–Hastings in which the proposal is the full conditional itself, so the acceptance probability is always 1.

Finding full conditionals. Write the joint density $p(\mathrm{y} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})$, keep only the factors that contain θ_m, and recognize a known distribution. This works when priors are (conditionally) conjugate.

Example: a normal model for log RTs. Let y_i be log response times, $y_i \sim \mathcal{N}(\mu, 1/\sqrt{\tau})$, where $\tau = 1/\sigma^2$ is the precision. Priors: $\mu \sim \mathcal{N}(m_0, s_0)$ and $\tau \sim \mathrm{Gamma}(a_0, \text{rate } b_0)$, independent. Then:

$$\mu \mid \tau, \mathrm{y} \sim \mathcal{N}\left(\frac{m_0/s_0^2 + \tau \sum_i y_i}{1/s_0^2 + N\tau},\ \frac{1}{\sqrt{1/s_0^2 + N\tau}} \right),$$

$$\tau \mid \mu, \mathrm{y} \sim \mathrm{Gamma}\left(a_0 + \frac{N}{2},\ \text{rate} = b_0 + \frac{1}{2} \sum_i (y_i - \mu)^2 \right).$$

Reading the formulas. The conditional mean of μ is a precision-weighted average of the prior mean and the data mean (as in the beta-binomial of Week 9). The conditional distribution of τ adds half an observation to the shape and half a squared residual to the rate for each data point.

> Note on parameterizations. NumPy's `rng.gamma(shape,scale)` uses scale = 1/rate. Mixing up rate and scale is the most common bug in Gibbs samplers.

Reading the math

- $\boldsymbol{\theta}_{-m}$: "all parameters except θ_m".
- Precision $\tau = 1/\sigma^2$: the inverse of the variance. Large precision = small noise. Working with precisions makes combining information a simple addition.
- The μ update: the conditional mean $\dfrac{m_0/s_0^2 + \tau \sum y_i}{1/s_0^2 + N\tau}$ is a weighted average of the prior mean m_0 (weight $1/s_0^2$, the prior's precision) and the data mean (total weight $N\tau$, the data's precision). The conditional precision is the *sum* of the two precisions. Worked example: $1+4\times10 = 41$, so the data dominate.
- The τ update: a Gamma distribution whose shape grows by $1/2$ per observation and whose rate grows by half of each squared residual. Its mean, shape/rate, is roughly "1 / (average squared residual)", i.e. the precision suggested by the data.

2. When Gibbs struggles, and Metropolis-within-Gibbs

- Strong posterior correlations. Gibbs moves parallel to the coordinate axes. If the posterior is a narrow diagonal ridge (like the A–d trade-off of the power model), each step can move only a tiny distance, and the chain zig-zags slowly along the ridge. Remedies: reparameterize, update correlated parameters jointly ("blocking"), or use gradient-based samplers (Week 13).
- Non-conjugate conditionals. Most cognitive models (forgetting curves, SDT, the GCM) have no closed-form conditionals. Metropolis-within-Gibbs updates each parameter with a one-dimensional Metropolis step targeting its full conditional, and uses exact Gibbs draws where available.

> Advanced: JAGS and BUGS automatically pick a sampler for each parameter (conjugate Gibbs, slice sampling, adaptive Metropolis), which is why they run on almost any model. Modern Hamiltonian Monte Carlo (Stan, PyMC's NUTS) instead uses the gradient of the log posterior to move all parameters jointly along the posterior's geometry, and handles correlated posteriors far better (Week 13).

3. Convergence: the logic of multiple chains

An MCMC chain has converged when its draws are representative of the posterior. Two failure modes: 1. Not yet stationary: the chain still reflects the starting value. 2. Stuck or slow: the chain explores only part of the posterior (e.g., one of two modes, or one end of a long ridge).

Neither can be detected reliably from one chain, because a chain stuck in one region can look perfectly stable. Run $C \geq 4$ chains from over-dispersed starting points: if they all end up describing the same distribution, that is good (though never conclusive) evidence of convergence.

Trace plots of all chains overlaid should look like one well-mixed "hairy caterpillar", with no chain visibly separated or trending.

4. The potential scale reduction factor $\hat{R}$

Compare the variance between chains with the variance within chains. With C chains of length S (after burn-in), chain means $\bar{\theta}_c$ and within-chain variances s_c^2:

$$W = \frac{1}{C}\sum_c s_c^2, \qquad B = \frac{S}{C-1}\sum_c (\bar{\theta}_c - \bar{\theta})^2, \qquad \widehat{\text{var}}^+ = \frac{S-1}{S}W + \frac{1}{S}B, \qquad \hat{R} = \sqrt{\frac{\widehat{\text{var}}^+}{W}}.$$

If the chains have not mixed, B is large relative to W and $\hat{R} > 1$. As chains converge, $\hat{R} \to 1$.

- Split-$\hat{R}$: split each chain into a first and a second half and treat them as $2C$ chains. This also detects a single chain that is still drifting.
- Threshold: the classic rule was $\hat{R} < 1.1$; current recommendations (Vehtari et al., 2021) use $\hat{R} < 1.01$, computed after rank-normalization (as ArviZ does).

5. Effective sample size and Monte Carlo standard error

Autocorrelated draws carry less information than independent ones. The effective sample size is

$$\text{ESS} = \frac{C \cdot S}{1 + 2\sum_{h=1}^{\infty} \rho_h},$$

where ρ_h is the lag-h autocorrelation, estimated from all chains and truncated when the estimates become noise. ESS is the number of *independent* draws that would give the same precision.

- The Monte Carlo standard error of a posterior mean is MCSE $=$ posterior SD$/\sqrt{\text{ESS}}$. It must be small relative to the precision you report (e.g., don't report 3 decimals if the MCSE is 0.01).
- Rule of thumb: bulk-ESS $\geq$ 400 (with 4 chains) for reliable posterior means and intervals; tail-ESS $\geq$ 400 for reliable extreme quantiles.

Reading the math

- $\hat{R}$: W is the average variance *inside* each chain; B measures how far the chain *means* are from each other. $\widehat{\text{var}}^+$ is an estimate of the total posterior variance that includes the disagreement between chains. If the chains agree, B adds almost nothing and $\hat{R} = \sqrt{\widehat{\text{var}}^+/W} \approx 1$. In the "not converged" worked example, the two chains sit around 1.0 and 2.0, B is huge and $\hat{R} = 4.4$.
- ESS: $C \cdot S$ is the total number of draws; the denominator $1 + 2\sum \rho_h$ inflates for correlated draws. With $\rho_h = 0.5^h$, the sum is $0.5 + 0.25 + \cdots = 1$, so the draws count only one third: 4,000 draws are worth 1,333 independent ones.

- MCSE $=$ posterior SD$/\sqrt{\text{ESS}}$ is the simulation error of the reported mean; decide the number of decimals from it.

6. MCMC reporting checklist

1. Sampler, number of chains, iterations, burn-in (warm-up), and any thinning.
2. Starting values (dispersed?).
3. $\hat{R}$ (which version) and ESS for all reported parameters, plus the maximum $\hat{R}$ and minimum ESS.
4. Trace plots (in a supplement) and a statement that they were inspected.
5. Priors, and the posterior summaries with MCSE-appropriate precision.
6. A posterior predictive check (Week 10).

For an engineer

- Precision-weighted averaging in the μ update is the same rule as maximal-ratio combining or the Kalman measurement update: combine estimates in proportion to their inverse noise variances.
- Gibbs sampling updates one variable given all the others, like coordinate-wise iterative algorithms (alternating optimization) and the iterative exchange of conditional information in turbo-style receivers; Gibbs samplers have been proposed for MIMO detection.
- $\hat{R}$ is a one-way ANOVA (between-group vs. within-group variance) across independent simulation runs: the same check you would do by running a Monte Carlo simulation with several seeds and verifying that the runs agree.
- ESS $= N/(1+2\sum \rho_h)$ is the standard formula for the effective number of independent samples of a correlated random process: averaging N correlated samples reduces the variance only as much as averaging ESS independent ones (related to the correlation time).

For the instructor

- Draw Gibbs on a contour plot. Two parameters, an elliptical posterior. Gibbs moves in horizontal and vertical steps only (a staircase). Then draw a narrow diagonal ellipse: the staircase steps become tiny. That one picture explains both how Gibbs works and when it struggles.
- Make $\hat{R}$ visual first: overlay four trace plots, once with chains that agree and once with one chain stuck elsewhere. Ask "do these chains describe the same distribution?" before showing any formula.
- Teach the reporting checklist as a habit. Ask students to apply it to their project and to every MCMC result in the lab.
- Typical questions.
  - *"$\hat{R}$ is 1.00, so my model is correct?"* No. $\hat{R}$ only checks that the chains agree; the model can still be wrong (check with a PPC) or the code can still be buggy.
  - *"Should I thin my chains?"* Not for correctness; it only saves memory.

- *"Rate or scale?"* NumPy's Gamma uses scale = 1/rate; the most common bug in student Gibbs samplers.

5.2.5 Worked example

Part 1: one Gibbs iteration by hand

Data: $N = 4$ log RTs, y $= (6.0, 6.3, 6.1, 6.4)$, so $\sum y_i = 24.8$. Priors: $m_0 = 6$, $s_0 = 1$, $a_0 = 2$, $b_0 = 0.1$. Current values: $\mu = 6.0$, $\tau = 10$.

Update μ. Precision $= 1/1^2 + 4 \times 10 = 41$; mean $= (6/1 + 10 \times 24.8)/41 = (6 + 248)/41 = 6.195$; SD $= 1/\sqrt{41} = 0.156$. Draw, say, $\mu = 6.22$.

Update τ given $\mu = 6.22$. Squared residuals: $0.0484 + 0.0064 + 0.0144 + 0.0324 = 0.1016$. Shape $= 2 + 4/2 = 4$; rate $= 0.1 + 0.1016/2 = 0.1508$. The conditional mean of τ is $4/0.1508 = 26.5$ (i.e., $\sigma \approx 0.19$). Draw, say, $\tau = 24.1$.

New state: $(\mu, \tau) = (6.22, 24.1)$. No accept/reject step was needed.

Part 2: $\hat{R}$ by hand

Two chains of $S = 4$ draws each.

Not converged. Chain 1: (1.0, 1.2, 0.8, 1.0); chain 2: (2.0, 2.2, 1.8, 2.0). - Chain means: 1.0 and 2.0. Within-chain variances (ddof = 1): both $0.08/3 = 0.0267$, so $W = 0.0267$. - $B = S \times \text{var}(\text{means}) = 4 \times 0.5 = 2.0$. - $\widehat{\text{var}}^+ = \frac{3}{4} \times 0.0267 + \frac{1}{4} \times 2.0 = 0.520$. - $\hat{R} = \sqrt{0.520/0.0267} = 4.4$. The chains clearly disagree.

Converged (realistic numbers). $S = 1000$ draws per chain, $W = 0.05$, chain means 1.00 and 1.02: $B = 1000 \times \text{var}(1.00, 1.02) = 1000 \times 0.0002 = 0.2$; $\widehat{\text{var}}^+ = 0.999 \times 0.05 + 0.2/1000 = 0.05015$; $\hat{R} = \sqrt{0.05015/0.05} = 1.0015$. Below 1.01: fine.

Part 3: ESS by hand

$C \cdot S = 4000$ draws with autocorrelations $\rho_h = 0.5^h$. Then $\sum_{h \geq 1} \rho_h = 1$, so ESS $= 4000/(1 + 2) = 1333$. With a posterior SD of 0.2, MCSE $= 0.2/\sqrt{1333} = 0.0055$: report the posterior mean to two decimals.

5.2.6 Lab (2 hours)

Goals

1. Implement a Gibbs sampler for the normal model of log RTs and validate it against a grid posterior.
2. Run four dispersed chains and inspect them.
3. Implement split-$\hat{R}$ and ESS; compare with ArviZ.
4. Break convergence on purpose (the power model with a bad Metropolis sampler) and watch the diagnostics catch it.

Timing: Part A 30 min · Part B 15 min · Part C 30 min · Part D 30 min · Part E 15 min.

Setup

Code · کد: `code/week12.py`, cell 1 · سلول 1

Part A: a Gibbs sampler (30 min)

Code · کد: `code/week12.py`, cell 2 · سلول 2

Expected output:

```
N = 40, sample mean = 6.200, sample SD = 0.225
Gibbs: mu = 6.1994 (SD 0.0358), sigma = 0.2283 (SD 0.0255)
Grid:  mu = 6.1996, sigma = 0.2287
```

Part B: four dispersed chains (15 min)

Code · کد: `code/week12.py`, cell 3 · سلول 3

Expected output:

```
Chain means of mu: [6.2006 6.1988 6.1996 6.1992]
```

Gibbs forgets even absurd starting values within a few iterations here, because μ and τ are nearly uncorrelated in this posterior.

Part C: $\hat{R}$ and ESS from scratch, checked against ArviZ (30 min)

Code · کد: `code/week12.py`, cell 4 · سلول 4

Expected output:

```
       R-hat (ours)  R-hat (ArviZ)  ESS (ours)  bulk-ESS (ArviZ)  MCSE
        ↪  of mean
param
mu           1.0002         1.0004   9980.4260        10032.6243
↪  0.0004
sigma        0.9997         0.9998   8772.9605         8861.7940
↪  0.0003
      ess_bulk ess_tail r_hat mcse_mean  mcse_sd
mu       10032     9427  1.00   0.00036  0.00027
sigma     8861     9714  1.00   0.00027  0.00021
```

(The exact column layout of `az.summary` differs slightly between ArviZ versions; the output above is from ArviZ 1.x.) Our simple estimators agree closely with ArviZ. With $4 \times 2500 = 10\,000$ draws, ESS $\approx$ 9000–10 000 means the Gibbs draws are almost independent. ArviZ additionally rank-normalizes the draws (Vehtari et al., 2021), which makes its $\hat{R}$ and bulk-ESS robust to heavy tails, so small differences are expected. In practice use ArviZ; today's point is to know what it computes.

Part D: catching non-convergence (30 min)

We reuse last week's Metropolis sampler on the power forgetting model, once badly tuned and short, once well tuned.

Code · کد: `code/week12.py`, cell 5 · سلول 5

Expected output:

```
bad (tiny steps, short)   split-R-hat(d) = 3.583, ArviZ R-hat(d) =
↪  3.424, ESS(d) = 4, chain means of d = [0.248 0.785 0.451 0.328]
good (tuned, longer)      split-R-hat(d) = 1.002, ArviZ R-hat(d) =
↪  1.002, ESS(d) = 1025, chain means of d = [0.414 0.414 0.415 0.413]
```

What to notice: with tiny steps and short chains, each chain is still close to where it started. The traces are separate, $\hat{R}$ is far above 1.01 and the ESS is tiny. *Each chain on its own looks smooth and stable*, which is exactly why one chain is never enough. The tuned sampler gives overlapping traces, $\hat{R} = 1.002$ and an ESS of about 1000 from 20 000 draws. The Gibbs sampler of Part C, with almost uncorrelated draws, reached an ESS of about 9000–10 000 from 10 000 draws.

Part E: interpret and report (15 min)

Write an MCMC methods-and-results paragraph for the log-RT model, following the checklist in Lecture §6: sampler, chains, iterations, burn-in, starting values, $\hat{R}$, ESS, posterior means and 95% intervals of μ and σ (back-transform μ to milliseconds with e^{μ} and say what it represents).

Exercises (increasing difficulty)

Exercise 1 $\star$. Show by algebra that the full conditional of τ is a Gamma distribution: write $p(\tau \mid \mu, \mathrm{y}) \propto \tau^{N/2} \exp(-\frac{\tau}{2}\sum(y_i-\mu)^2) \cdot \tau^{a_0-1}e^{-b_0\tau}$ and read off the shape and rate.

Exercise 2 $\star$. Introduce the rate/scale bug (`scale=B0+...` instead of `1/(...)`) into `gibbs_normal`. Do the diagnostics catch it? Does the validation against the grid? What does this tell you about the difference between *convergence* and *correctness*?

Exercise 3 $\star\star$ (correlated Gibbs). Write a Gibbs sampler for a bivariate normal "posterior" with correlation ρ (full conditionals: $\theta_1 \mid \theta_2 \sim \mathcal{N}(\rho\theta_2, \sqrt{1-\rho^2})$ and vice versa). Compare the ESS for $\rho = 0, 0.9, 0.99$ with 5000 iterations, and plot the zig-zag path for $\rho = 0.99$.

Exercise 4 $\star\star$ (Metropolis-within-Gibbs). Sample the power forgetting model by updating A and d one at a time, each with a 1-D Metropolis step. Compare the ESS per second with the joint Metropolis sampler. Why might it be worse here? (Hint: the A–d correlation.)

Exercise 5 $\star\star\star$ (Advanced: two conditions). Extend the Gibbs sampler to log RTs from two conditions with separate means μ_1, μ_2 and a shared precision τ. Simulate data with a true difference of 0.1, sample, and report the posterior of $\mu_2 - \mu_1$ and $P(\mu_2 > \mu_1 \mid \mathrm{y})$ with diagnostics.

Lab check-off

Show: (1) the Gibbs-vs-grid validation (Part A), (2) your $\hat{R}$/ESS table next to ArviZ's (Part C), and (3) the bad-vs-good trace plots with their diagnostics (Part D).

5.2.7 Connection to AI & LLMs

Gibbs sampling from a masked language model. Models like BERT are trained to fill in a masked word given all other words: they learn full conditionals, $P(w_i \mid \text{all other words})$. That is exactly what a Gibbs sampler needs. Repeatedly masking one position and resampling it generates whole sentences (Wang & Cho, 2019). The toy model below knows only the two full conditionals of a two-word "language", and Gibbs sampling recovers the joint distribution of sentences. Because the language is "sticky" (dog goes with barks, cat with meows), the chain mixes slowly, and $\hat{R}$ catches this in short runs.

Code · کد `code/week12.py`, cell 6 · سلول 6

Expected output:

```
  50 iterations: P(subject = dog) = 0.285 (exact 0.35), R-hat = 1.151
3000 iterations: P(subject = dog) = 0.354 (exact 0.35), R-hat = 1.000
```

Try it (no API needed). - ⋆ Make the language even stickier (e.g., 0.33 on the diagonal and 0.001 elsewhere, renormalized). How many iterations do you now need for $\hat{R} < 1.01$? Compute the ESS (Week 12, Part C) of the `is_dog` chains. - ⋆⋆ Real masked models are trained separately for each position, so their conditionals need not come from any single joint distribution. Perturb `p_v_given_s` slightly (without changing `p_s_given_v`) and check whether the Gibbs sampler still recovers `joint`. What does this tell you about trusting a sampler whose conditionals are learned?

Optional reading: Wang, A., & Cho, K. (2019). BERT has a mouth, and it must speak: BERT as a Markov random field language model. *arXiv:1902.04094*.

5.2.8 Common mistakes & misconceptions

1. Rate vs. scale in the Gamma distribution. NumPy and SciPy use scale = 1/rate. Write down which one your formula uses.
2. Using old values within a Gibbs sweep. Each conditional must use the *newest* values of the other parameters.
3. "$\hat{R} \approx 1$, so the results are correct." Diagnostics test agreement between chains, not correctness of the model or the code (Exercise 2). Validate new samplers on known cases.
4. One long chain instead of several. A stuck chain can look perfectly stable. Use ≥ 4 dispersed chains.
5. Reporting results with low ESS. With ESS = 50, a posterior mean has an MCSE of about 14% of the posterior SD, and tail quantiles are unreliable.
6. Thinning to "fix" autocorrelation. Thinning lowers storage but does not increase the information. ESS already accounts for autocorrelation.
7. Using the 1.1 threshold uncritically. Modern guidance is $\hat{R} < 1.01$ with rank-normalized split-$\hat{R}$.
8. Diagnosing only some parameters. Check *all* reported parameters, including derived ones.

5.2.9 Check your understanding

1. Why does Gibbs sampling never reject a draw? Answer Each draw comes exactly from the full conditional distribution. Viewed as Metropolis–Hastings with the full conditional as proposal, the acceptance ratio is always 1.

2. What do you need to be able to run a pure Gibbs sampler? Answer Full conditional distributions for every parameter (or block) that you can sample from directly, typically because the priors are conditionally conjugate.

3. A Gibbs sampler for two strongly correlated parameters has a very low ESS. Why, and what could you do? Answer Gibbs updates one coordinate at a time, so on a narrow diagonal ridge every step is tiny and the chain zig-zags slowly. Remedies: reparameterize to reduce the correlation, update the correlated parameters jointly (blocking), or use HMC/NUTS.

4. Explain $\hat{R}$ in one sentence. Which value indicates convergence? Answer It compares the variance between chains with the variance within chains (it estimates how much the posterior spread could shrink if the chains ran forever); values near 1 (< 1.01 by current guidance) indicate that the chains agree.

5. You have 4000 draws and an ESS of 200 for a parameter with posterior SD 0.5. What is the MCSE of its posterior mean? How many decimals should you report? Answer MCSE $= 0.5/\sqrt{200} = 0.035$. Report the mean to one decimal (or two at most), because the third decimal is Monte Carlo noise.

6. Why can't one chain demonstrate convergence? Answer A chain stuck in one region (one mode, or one part of a ridge) can look stationary and well mixed, and nothing in the chain itself reveals the unexplored regions. Only chains started elsewhere, which end up somewhere different, expose the problem.

7. Name four things a methods section should report about an MCMC analysis. Answer Any four of: sampler/software; number of chains; iterations and warm-up; starting values; $\hat{R}$ and ESS for all parameters; that trace plots were inspected; priors; a posterior predictive check.

5.2.10 Assignment

A5: MCMC from scratch, with diagnostics (1.0 point of 20; released Week 12, due at the start of the Week 14 lecture)

Data: the payoff SDT experiment from Week 4, Part D (regenerate it with the same seed: `np.random.default_rng(11)`), and the log-RT data from today's lab (`np.random.default_rng(12)`).

Tasks.

1. Metropolis for SDT (3 pts). Implement a random-walk Metropolis sampler for the shared-d' payoff model (d', c_1, c_2, c_3) with priors $d' \sim \mathcal{N}(1, 1.5)$ and $c_m \sim \mathcal{N}(0, 1)$. Tune the proposal (report the acceptance rate), and run 4 dispersed chains.
2. Diagnostics (2 pts). Report split-$\hat{R}$ and ESS for all parameters with your own functions *and* with ArviZ; show trace plots. State whether you would trust the results and why.
3. Inference (1.5 pts). Report posterior means and 95% intervals; compute the posterior of $c_{\text{conservative}} - c_{\text{liberal}}$ and the probability that it is positive. Compare d' with the Week 4 MLE.
4. Gibbs (2 pts). Implement the two-condition Gibbs sampler of Exercise 5 and report the posterior of the mean difference with diagnostics.
5. Interpretation (1.5 pts, max. 300 words). What do the payoffs do to sensitivity and bias, according to the posterior? Compare with the likelihood-ratio conclusion of Week 4.

Rubric: points as above; within each task: correctness 60%, figures/tables 20%, interpretation 20%. Late submissions lose 10% per day (max. 3 days).

5.2.11 Further reading

- Kruschke (2014), chapter 7 (sections on Gibbs sampling and on MCMC representativeness, accuracy and efficiency): diagnostics, ESS and MCSE explained visually. *(Main reading.)*
- Martin, Kumar & Lao (2021), chapter 2. Exploratory analysis of Bayesian models: trace plots, $\hat{R}$, ESS and MCSE with ArviZ.
- Lee & Wagenmakers (2014), chapter 4. Inferences with Gaussians: the normal model whose Gibbs sampler you built today, in its Bayesian-modeling context.
- *Optional, outside the course list:* Vehtari, A., Gelman, A., Simpson, D., Carpenter, B., & Bürkner, P.-C. (2021). Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC. *Bayesian Analysis, 16*(2), 667–718.

Preview for Week 13: writing samplers by hand teaches how they work, but it is slow and error-prone. Next week we describe models in PyMC, let its NUTS sampler do the work, and use the time we save for the most useful model class in cognitive science: hierarchical models of many participants.

5.3 Week 13: Probabilistic Programming and Hierarchical Models

> Chapter 5: Parameter estimation with data-driven methods (MCMC), bridging to Chapter 6 · Lecture 1 h · Lab 2 h Running data: a recognition/detection experiment with 12 participants and only 50 + 50 trials each: hierarchical SDT in PyMC.

5.3.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
hierarchical model	مدل سلسله‌مراتبی (چندسطحی)	پارامترهای فردی از یک توزیع گروهی می‌آیند

English	فارسی	توضیح کوتاه
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
trial	کوشش (trial)	یک بار انجام تکلیف توسط شرکت‌کننده
probabilistic programming	برنامه‌نویسی احتمالاتی	تعریف مدل با کد و نمونه‌گیری خودکار (PyMC)
prior	توزیع پیشین (prior)	باور درباره‌ی پارامتر پیش از دیدن داده
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
prior predictive check	بررسی پیش‌بینی پیشین	شبیه‌سازی داده از پیشین برای سنجش معقول بودن آن
posterior	توزیع پسین (posterior)	باور درباره‌ی پارامتر بعد از دیدن داده
posterior predictive check (PPC)	بررسی پیش‌بینی پسین	مقایسه‌ی داده‌ی شبیه‌سازی‌شده از پسین با داده‌ی واقعی
HMC / NUTS	Hamiltonian Monte Carlo / NUTS	MCMC مبتنی بر گرادیان
divergent transition	divergence (گذار واگرا)	هشدار NUTS برای ناحیه‌ای که درست کاوش نشده
complete / no / partial pooling	ادغام جزئی / بدون ادغام / ادغام کامل (pooling)	/ جدا برای هر فرد / یک پارامتر برای همه سلسله‌مراتبی
shrinkage	انقباض به سمت میانگین (shrinkage)	کشیده شدن برآوردهای پرنویز به سمت میانگین گروه
centered / non-centered parameterization	پارامتری‌سازی centered / non-centered	دو شکل نوشتن مدل سلسله‌مراتبی
group-level parameter	پارامتر سطح گروه	میانگین و پراکندگی پارامترها در جمعیت
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
hit	اصابت (hit)	پاسخ «بله» وقتی علامت وجود دارد
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز
gradient	گرادیان	جهت و شیب تغییر تابع
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
reparameterization	بازپارامتری‌سازی	نوشتن پارامتر در مقیاس دیگر، مثلاً $\lambda = e^{\eta}$
standard error (SE)	خطای استاندارد	پراکندگی برآورد در تکرارهای فرضی آزمایش
benchmark	benchmark (مجموعه‌آزمون استاندارد)	مجموعه سؤال برای مقایسه‌ی مدل‌های هوش مصنوعی

5.3.2 Learning outcomes

By the end of this week, students will be able to:

1. Specify a cognitive model in PyMC (priors, deterministic transformations, likelihood) and run a complete Bayesian workflow: prior predictive check, sampling, diagnostics, posterior predictive check.
2. Explain conceptually how Hamiltonian Monte Carlo / NUTS differs from random-walk Metropolis, and what a divergence signals.
3. Formulate a hierarchical (multilevel) model and explain partial pooling and shrinkage, contrasting it with the complete- and no-pooling approaches of Week 5.
4. Implement a non-centered parameterization and justify its use.
5. Interpret group-level and individual-level posteriors and report them according to the Week 12 checklist.

5.3.3 Motivation & big picture

In Week 5 we faced a dilemma. Fitting the average of all participants is stable but can distort the model (aggregation artifacts) and ignores individual differences. Fitting each participant separately respects differences, but with few trials per person the estimates are noisy. Some become extreme or infinite (a participant with 50/50 hits has $d' = \infty$ without a correction). The second-stage statistics on those estimates also ignore that some are much less certain than others.

Hierarchical models dissolve the dilemma. They assume that each participant has their own parameter, *and* that these individual parameters come from a group-level distribution whose mean and spread are also estimated. Each individual estimate then borrows strength from the others: noisy, extreme estimates are pulled ("shrunk") toward the group mean, by exactly as much as their uncertainty warrants. The group-level parameters answer the population question ("what is the typical sensitivity, and how much do people differ?") with all sources of uncertainty included.

Writing samplers for such models by hand would be a research project. Probabilistic programming languages like PyMC let us *describe* the model in a few lines and use a state-of-the-art sampler, NUTS, automatically. The hand-built samplers of Weeks 11–12 are what make this black box transparent to you.

Where is this used?

- Most modern cognitive modeling studies with many participants fit hierarchical models: hierarchical drift-diffusion models (the HDDM package), hierarchical reinforcement-learning models in computational psychiatry, hierarchical SDT in memory research.
- Clinical and developmental studies with short sessions: children and patients cannot do hundreds of trials, so individual estimates are noisy; partial pooling makes them usable.
- Education and social science: students within classes within schools (multilevel models) are the textbook example of hierarchical structure.

- Sports analytics and medicine: estimating a player's skill or a hospital's success rate from few observations by borrowing strength from the group is the same idea.

5.3.4 Lecture notes

In plain words When each person gives us only a little data, their individual estimates are noisy, and some come out absurd (a perfect score gives $d' = \infty$). A hierarchical model says: each person has their own parameter, *but* people are similar to each other, so their parameters come from a common group distribution, whose mean and spread we also estimate. The effect is sensible compromise: a person with little or extreme data is pulled toward the group average (shrinkage), and a person with plenty of data keeps an estimate close to their own. How strong the pull is depends on how different people turn out to be, which is learned from the data. Writing a sampler for such a model by hand is hard, so we use PyMC: we write the model almost like the math, and PyMC runs an advanced sampler (NUTS) for us. The last part of the week is a step-by-step workflow for doing this safely.

Suggested timing (60 min): probabilistic programming 10 · HMC/NUTS concepts 10 · hierarchical models 20 · parameterization and divergences 10 · the Bayesian workflow 10.

1. Probabilistic programming with PyMC

A PyMC model is a direct transcription of the mathematical model:

Math	PyMC
$\mu_d \sim \mathcal{N}(1, 1)$	`mu_d=pm.Normal("mu_d",1,1)`
$\sigma_d \sim \mathcal{N}^+(0, 1)$	`sigma_d=pm.HalfNormal("sigma_d", 1)`
$d'_j = \mu_d + \sigma_d z_j$	`d=pm.Deterministic("d",mu_d+sigma_d*z_d)`
$H_j = \Phi(d'_j/2 - c_j)$	`H=Phi(d/2-c)` (an expression, built with `pm.math`)
$\text{hits}_j \sim \text{Binomial}(n_s, H_j)$	`pm.Binomial("hits",n=n_s,p=H,observed=hits)`

Then `pm.sample()` compiles the model, computes gradients automatically, and runs NUTS. The result is an ArviZ `InferenceData`/`DataTree` object with groups such as `posterior`, `sample_stats`, `prior_predictive` and `posterior_predictive`, which ArviZ summarizes and diagnoses (Week 12).

Practical notes: use `cores=1` when running a plain `.py` script on Windows (or protect the code with `if__name__=="__main__":`); give every variable a unique name; use `dims`/`coords` to label participants.

2. Hamiltonian Monte Carlo and NUTS (conceptually)

Random-walk Metropolis proposes blind steps and wastes most of them in high dimensions or along correlated ridges (Weeks 11–12). Hamiltonian Monte Carlo (HMC) treats the negative log posterior as a landscape and the current parameter vector as a frictionless ball. Give the ball a random push (momentum), let it roll for a while following the landscape's gradient, and propose where it ends up. Proposals can be far away yet still in high-probability regions, so acceptance is high and autocorrelation low. NUTS (the No-U-Turn Sampler) tunes the rolling time automatically. HMC needs gradients, which PyMC computes for you.

Divergences. The rolling is simulated numerically in small steps. Where the posterior's curvature changes abruptly (typically in "funnels", below), the simulation becomes inaccurate and NUTS flags a divergent transition. Even a few divergences mean the posterior is not being explored reliably there. Do not ignore them; reparameterize.

> Advanced: HMC simulates Hamiltonian dynamics with energy $H(\boldsymbol{\theta}, \mathrm{p}) = -\ln \tilde{p}(\boldsymbol{\theta}) + \frac{1}{2}\mathrm{p}^\top M^{-1}\mathrm{p}$ using the leapfrog integrator. Exact dynamics conserve H, so proposals would always be accepted. The leapfrog error makes a Metropolis correction necessary, and a divergence is a trajectory whose energy error explodes.

3. Hierarchical models

Structure (for SDT with J participants):

$$
\begin{aligned}
&\text{Group level:} && \mu_d \sim \mathcal{N}(1,1),\ \ \sigma_d \sim \mathcal{N}^+(0,1),\ \ \mu_c \sim \mathcal{N}(0,1),\ \ \sigma_c \sim \mathcal{N}^+(0,1)\\
&\text{Individual level:} && d'_j \sim \mathcal{N}(\mu_d, \sigma_d),\ \ c_j \sim \mathcal{N}(\mu_c, \sigma_c),\quad j = 1, \ldots, J\\
&\text{Data:} && \text{hits}_j \sim \text{Binomial}\big(n_s, \Phi(d'_j/2 - c_j)\big),\ \ \text{FAs}_j \sim \text{Binomial}\big(n_n, \Phi(-d'_j/2 - c_j)\big)
\end{aligned}
$$

Three ways to pool information (cf. Week 5):

Approach	Model	What it assumes
Complete pooling	One d' and one c for everyone	$\sigma_d = \sigma_c = 0$: no individual differences
No pooling	Separate d'_j, c_j with fixed, independent priors	$\sigma \to \infty$: participants tell us nothing about each other
Partial pooling	Hierarchical; σ_d, σ_c *estimated*	The data decide how similar participants are

Shrinkage. For a normal approximation, the hierarchical estimate of participant j is a precision-weighted average of their own estimate $\hat{d}_j$ (with standard error SE_j) and the group mean:

$$\mathbb{E}[d'_j \mid \text{data}] \approx \frac{\hat{d}_j/\text{SE}_j^2 + \mu_d/\sigma_d^2}{1/\text{SE}_j^2 + 1/\sigma_d^2}.$$

Participants with noisier data (larger SE_j) are shrunk more. If people really differ a lot (large σ_d), shrinkage is weak; if they are similar, it is strong. This is learned from the data.

Why shrinkage is good, not a distortion: across participants, shrunken estimates are on average *closer to the true values* than the raw individual estimates. This is a famous result in statistics (Stein's paradox). Extreme raw estimates are partly extreme by chance, and shrinkage corrects for that regression to the mean.

Group-level questions are answered directly by the posteriors of μ_d (typical sensitivity) and σ_d (individual differences), and group comparisons are made by adding group or condition effects to the group level.

Reading the math

- Two levels. Group level: μ_d (typical sensitivity) and σ_d (how much people differ). Individual level: $d'_j \sim \mathcal{N}(\mu_d, \sigma_d)$, person j's sensitivity is drawn from the group distribution. Data level: the hits and false alarms of person j depend only on that person's d'_j and c_j.
- $\mathcal{N}^+(0, 1)$: a "half-normal" distribution, a normal curve folded at 0 so only positive values remain (a standard deviation cannot be negative).
- The shrinkage formula is a precision-weighted average (as in Week 12): the person's own estimate $\hat{d}_j$ has precision $1/\text{SE}_j^2$, the group distribution has precision $1/\sigma_d^2$. Example from the worked example: $\text{SE} = 0.40$ gives precision 6.25, the group gives 4, so the estimate is $(6.25 \times 2.4 + 4 \times 1.3)/10.25 = 1.97$: the raw 2.4 moves 40% of the way toward the group mean 1.3.
- If people really differ a lot (large σ_d), the group precision is small and shrinkage is weak. If they are similar, shrinkage is strong.

4. Centered vs. non-centered parameterization

Writing $d'_j \sim \mathcal{N}(\mu_d, \sigma_d)$ directly (the centered form) creates a "funnel" in the posterior: when σ_d is small, all d'_j must be squeezed close together, and when it is large they can spread out. NUTS struggles with this changing geometry and produces divergences, especially with few data per participant. The non-centered form samples standardized deviations and builds d'_j deterministically:

$$z_j \sim \mathcal{N}(0, 1), \qquad d'_j = \mu_d + \sigma_d z_j.$$

It describes the same model but has a much friendlier geometry. Use it by default when data per participant are sparse.

Reading the math $z_j \sim \mathcal{N}(0, 1)$, $d'_j = \mu_d + \sigma_d z_j$: instead of sampling d'_j directly, sample a standardized "how unusual is this person" score z_j (in units of the group SD) and rebuild d'_j from it. Both versions describe exactly the same model. The difference is only in what the sampler has to explore:

in the centered form, the allowed range of every d'_j shrinks to almost nothing when σ_d is small (a funnel); in the non-centered form, the z_j always live on the same scale, so the sampler's step size fits everywhere.

5. The Bayesian workflow

1. Specify the model and priors (with justification).
2. Prior predictive check (Week 9): does the prior imply plausible data?
3. Fit with several chains.
4. Diagnose: $\hat{R}$, ESS, divergences, trace plots (Week 12).
5. Posterior predictive check (Week 10): can the fitted model reproduce the data?
6. Compare models if needed (LOO via ArviZ, Bayes factors; Weeks 6, 10).
7. Interpret and report.

For an engineer

- Hierarchical model = estimation with a population prior. It is like estimating each user's channel gain from a few pilots while also learning the distribution of gains in the cell, and using it as the prior: an empirical-Bayes / MMSE estimator. Shrinkage toward the group mean is the same phenomenon as the James–Stein estimator, which beats separate ML estimates in total squared error when there are three or more units.
- HMC/NUTS simulates Hamiltonian dynamics (a frictionless particle on the surface −log posterior) with a numerical integrator (leapfrog), as in physics simulations. A divergence is the integrator becoming unstable because the step size is too large for the local curvature, like a stiff ODE integrated with a too-large step.
- Non-centered parameterization is a change of variables that normalizes the variables to unit scale, i.e. it improves the conditioning of the problem, much like whitening or normalizing inputs before an adaptive algorithm.
- PyMC computes gradients automatically (automatic differentiation), the same technology that trains neural networks.

For the instructor

- Introduce shrinkage with a sports or exam example. A student who answered 3 of 3 questions correctly is probably not a "100% student"; a student with 95 of 100 probably is near 95%. Ask the class how they would combine individual and class information: they invent partial pooling themselves.
- Show the shrinkage plot from lab Part D: raw estimates, hierarchical estimates and the group mean on one figure, with arrows. It makes the idea stick.
- Do not teach HMC mathematics in depth. One picture (a ball rolling on the negative log posterior, given a random push) is enough; the important practical skill is reading the diagnostics (divergences, $\hat{R}$, ESS).
- Running PyMC in class: on Windows use `cores=1` in scripts, and start sampling early (Part C), since it can take a few minutes.

- Typical questions.
  - *"Isn't shrinkage biasing the results?"* Slightly, but it reduces the total error, so estimates are closer to the truth on average.
  - *"Is a hierarchical model always better?"* When individuals have enough data, results are similar to separate fits; the gain is largest with little data per person.
  - *"What does a divergence mean for my results?"* That part of the posterior was not explored reliably. Reparameterize and rerun until there are none.

5.3.5 Worked example

Shrinkage by hand. The group posterior has $\mu_d \approx 1.3$ and $\sigma_d \approx 0.5$ (group precision $1/0.5^2 = 4$). Two participants both have a raw estimate $\hat{d}_j = 2.4$.

Participant	SE_j	Individual precision $1/\text{SE}_j^2$	Hierarchical estimate	Posterior SD
P1 (few trials)	0.40	6.25	$(6.25 \times 2.4 + 4 \times 1.3)/(6.25 + 4) = 20.2/10.25 =$ 1.97	$1/\sqrt{10.25} = 0.31$
P2 (many trials)	0.20	25	$(25 \times 2.4 + 4 \times 1.3)/(25 + 4) = 65.2/29 = 2.25$	$1/\sqrt{29} = 0.19$

Interpretation. Both participants *look* equally sensitive, but P1's estimate rests on less data, so the model trusts it less and moves it 0.43 toward the group mean. P2 moves only 0.15. Both posterior SDs are smaller than the raw SEs, because the group distribution adds information. (In the full model, μ_d and σ_d are themselves uncertain and estimated jointly; this calculation fixes them for illustration.)

5.3.6 Lab (2 hours)

Goals

1. Simulate a 12-participant experiment and compute individual (no-pooling) estimates.
2. Write the hierarchical SDT model in PyMC with a non-centered parameterization; run a prior predictive check.
3. Sample with NUTS; check $\hat{R}$, ESS and divergences.
4. Visualize shrinkage and compare the estimates with the true values.
5. Run a posterior predictive check and interpret the group-level parameters.

Timing: Part A 10 min · Part B 20 min · Part C 25 min · Part D 30 min · Part E 20 min · Part F 15 min.

> Run time: sampling takes from under a minute (conda-forge install with a compiler) up to about 2–3 minutes (no compiler). Start Part C early.

Part A: data and no-pooling estimates (10 min)

> Code · کد: `code/week13.py`, cell 1 · سلول 1

Expected output:

```
    hits  fas  d_true  d_indiv  c_true  c_indiv
0     46    2    2.21     3.01    0.13     0.15
1     16   25   -0.24    -0.46    0.42     0.23
2     28    6    1.78     1.29    0.41     0.50
3     40   11    1.33     1.57    0.05    -0.03
4     43    9    1.96     1.94    0.09    -0.08
5     43   17    1.49     1.45   -0.34    -0.32
6     31    0    2.21     2.63    0.70     1.02
7     35   11    1.32     1.27    0.13     0.12
8     24    8    1.04     0.92    0.60     0.51
9     39    8    1.59     1.72    0.33     0.11
10    27    3    1.52     1.58    0.78     0.69
11    21    9    1.12     0.69    0.66     0.54
```

Part B: the model in PyMC, and a prior predictive check (20 min)

> Code · کد: `code/week13.py`, cell 2 · سلول 2

Expected output:

```
Share of prior-predictive rates exactly 0 or 1: 0.30
```

The prior implies a broad range of hit and false-alarm rates, mostly above the diagonal. About 30% of simulated participants have a hit or false-alarm count of exactly 0 or 50. These are mostly participants simulated with very high sensitivity and a strong bias, which produce zero false alarms. This is far better than the 96% of the "vague" prior in Week 9, but still on the generous side: tightening the priors on `sigma_d` and `sigma_c` (e.g., `HalfNormal(0.5)`) would remove most of them. Try it and check whether the posterior changes.

Part C: sampling and diagnostics (25 min)

Code · کد: `code/week13.py`, cell 3 · سلول 3

Expected output:

```
        ess_bulk ess_tail r_hat mcse_mean mcse_sd
mu_d        1303     1651  1.00    0.0075  0.0054
sigma_d     1639     2088  1.00    0.0055  0.0045
mu_c        1312     1474  1.00    0.0036   0.003
sigma_c     1203     1748  1.00    0.0034  0.0047
Number of divergent transitions: 0
Max R-hat over all individual d and c: 1.0023234826538692
```

Part D: shrinkage and group-level inference (30 min)

Code · کد: `code/week13.py`, cell 4 · سلول 4

Expected output:

```
RMSE vs. true d':  no pooling = 0.339,  hierarchical = 0.279
mu_d     posterior mean = 1.45, 95% interval = [0.90, 1.95]
sigma_d  posterior mean = 0.90, 95% interval = [0.56, 1.42]
mu_c     posterior mean = 0.29, 95% interval = [0.04, 0.55]
sigma_c  posterior mean = 0.38, 95% interval = [0.22, 0.65]
P(mu_c > 0 | data) = 0.987
```

Interpret (write in your notebook): 1. Which participants are shrunk most, and why? Find one whose raw estimate is extreme and describe what the hierarchical model does with it. 2. Is the hierarchical estimate closer to the truth on average? Is it closer for *every* participant? Why is that not required? 3. What do μ_d and σ_d tell you about this population? Compare σ_d with the SD used to simulate the data (0.5) and explain the difference (hint: 12 participants).

Part E: posterior predictive check (20 min)

Code · کد: `code/week13.py`, cell 5 · سلول 5

Expected output:

```
Share of observed counts inside their 95% predictive interval: 1.00
```

Part F: report (15 min)

Write a methods-and-results paragraph following the Week 12 checklist: model and priors (in words and symbols), sampler and settings, diagnostics (max $\hat{R}$, min ESS, divergences), the PPC result, and the group-level conclusions (μ_d, σ_d, μ_c with 95% intervals; $P(\mu_c > 0 \mid \text{data})$).

Exercises (increasing difficulty)

Exercise 1 $\star$. Fit the complete-pooling model (one d' and one c for everyone) in PyMC. Compare its estimate of d' with μ_d from the hierarchical model, and its posterior predictive check with Part E. Which participants does it fail to predict?

Exercise 2 $\star$. Rerun Part C with the centered parameterization (`d=pm.Normal("d",mu_d,sigma_d,dims="participant")`). Compare divergences, ESS and run time. Then reduce the data to 10 + 10 trials per participant and compare again.

Exercise 3 $\star\star$ (unequal trial numbers). Give half of the participants 20 + 20 trials and the other half 200 + 200 (simulate new data). Show that shrinkage depends on the number of trials, as in the worked example.

Exercise 4 $\star\star$ (model comparison with LOO). Compute the log-likelihood with `pm.compute_log_likelihood(idata)`. ArviZ's `loo` needs one pointwise log-likelihood per observation: add the `hits` and `fas` log-likelihood arrays into a single per-participant array (e.g., `idata.log_likelihood["obs"]=idata.log_likelihood["hits"]+idata.log_likelihood["fas"]`) and call `az.loo(idata,var_name="obs")`. Compare the hierarchical and complete-pooling models with `az.compare`. Check the Pareto-k diagnostics and say what they mean for leave-one-participant-out prediction.

Exercise 5 $\star\star\star$ (Advanced: a group effect). Simulate two groups of 12 participants that differ in mean sensitivity by 0.4. Extend the model with a group effect β on μ_d ($d'_j = \mu_d + \beta \cdot \text{group}_j + \sigma_d z_j$), estimate β, and report $P(\beta > 0 \mid \text{data})$. Compare with a t-test on the individual d' estimates.

Lab check-off

Show: (1) the diagnostics output (no divergences, $\hat{R}$, ESS), (2) the shrinkage plot with the RMSE comparison, and (3) the PPC plot with your Part F paragraph.

Project progress check (this week): submit a one-page status report (what is done, a first fit or simulation figure, open problems). It is not graded; written feedback is returned within a week.

5.3.7 Connection to AI & LLMs

Partial pooling for AI benchmarks. Benchmarks are split into task categories, and some categories have only a handful of items. Reporting raw accuracy per category gives noisy, sometimes extreme numbers, just like individual SDT fits with few trials. A hierarchical model treats category accuracies as draws from a common distribution and shrinks small categories toward the overall mean. The code below uses a fast empirical-Bayes version, a beta-binomial model whose group parameters are estimated by maximizing the marginal likelihood on a grid. In the exercises you fit the full Bayesian version in PyMC.

Code · کد: `code/week13.py`, cell 6 · سلول 6

Expected output:

```
items   true    raw  shrunk
    5  0.72  0.80   0.71
    8  0.63  0.88   0.74
   10  0.64  0.50   0.63
   15  0.63  0.67   0.68
   25  0.76  0.88   0.80
   40  0.69  0.55   0.60
   60  0.80  0.78   0.76
   80  0.51  0.57   0.60
group mean = 0.69, concentration = 19.9;  RMSE raw = 0.125, shrunk =
↪  0.066
```

The 8-item category (raw 0.88, true 0.63) is pulled down strongly, and the shrunken estimates are about twice as accurate overall (RMSE 0.066 vs. 0.125).

Try it (no API needed). - ⋆ Which categories move most, and why? Relate your answer to the shrinkage formula in this week's lecture notes. - ⋆⋆ Fit the same model in PyMC: `mu~Beta(2,2)`, `kappa~Gamma(2,0.1)` (shape and rate), `theta_j~Beta(mu*kappa,(1-mu)*kappa)`, `y_j~Binomial(n_j,theta_j)`. Compare the posterior means with the empirical-Bayes estimates, and check the diagnostics. - ⋆⋆ Discussion: a leaderboard highlights "the category where Model X is weakest", a category with 5 items. What would you tell the authors?

5.3.8 Common mistakes & misconceptions

1. "Shrinkage biases the results." It trades a little bias for a large reduction in variance and gives more accurate estimates on average. It is principled, not an ad-hoc correction.
2. Ignoring divergences. Even a handful indicate unreliable exploration. Reparameterize (non-centered), tighten priors on scale parameters, or increase `target_accept`.

3. Centered parameterization with sparse data. It invites funnels and divergences. Default to non-centered for hierarchical models with few trials per participant.
4. Vague priors on group SDs. Very wide priors on σ can produce implausible prior predictions and sampling problems. Half-normal priors on a sensible scale are usually better.
5. Treating the black box as magic. PyMC samples whatever model you wrote, including wrong ones. The workflow (prior predictive check, diagnostics, PPC) still applies in full.
6. Running multi-process sampling from a script on Windows without a main guard, which causes cryptic multiprocessing errors. Use `cores=1` or `if__name__=="__main__":`.
7. Reporting only individual estimates. The group-level parameters often answer the actual research question.

5.3.9 Check your understanding

1. What are the three ways to combine data across participants, and what does each assume about individual differences? Answer Complete pooling (everyone identical, $\sigma = 0$), no pooling (participants unrelated, so no information is shared), and partial pooling/hierarchical (participants come from a common distribution whose spread is estimated from the data).

2. Two participants have the same raw d' estimate. Why may the hierarchical model shrink one of them much more? Answer Shrinkage depends on the precision of each individual estimate. The participant with fewer trials (larger SE) carries less information and is pulled more strongly toward the group mean.

3. Why is the non-centered parameterization often preferred? Answer The centered form creates a funnel-shaped posterior (individual parameters must collapse together when the group SD is small) that NUTS explores poorly, causing divergences. Sampling standardized z_j and computing $d'_j = \mu + \sigma z_j$ gives a geometry that is much easier to explore.

4. What is a divergent transition, and what should you do about it? Answer A NUTS trajectory whose numerical simulation broke down, usually in a region of high curvature; it signals that part of the posterior is not explored reliably. Reparameterize, use more informative priors for scale parameters, or increase `target_accept`, then check that the divergences disappear.

5. Why does HMC usually mix much better than random-walk Metropolis in high dimensions? Answer It uses gradient information to make long, directed moves that follow the shape of the posterior, so distant proposals still have a high acceptance probability. Random-walk proposals become almost always rejected unless they are tiny.

6. In a hierarchical SDT model, which parameter answers "how much do people differ in sensitivity?", and which answers "what is the typical sensitivity?" Answer σ_d (the group-level SD) and μ_d (the group-level mean), respectively.

7. List the steps of the Bayesian workflow. Answer Specify the model and priors $\rightarrow$ prior predictive check $\rightarrow$ fit with several chains $\rightarrow$ diagnose ($\hat{R}$, ESS, divergences, traces) $\rightarrow$ posterior predictive check $\rightarrow$ compare models if needed $\rightarrow$ interpret and report.

5.3.10 Assignment

- A5 (MCMC from scratch) is due at the start of the Week 14 lecture.
- Project progress check this week (see above).
- No new graded assignment: the remaining weeks are reserved for the final project.

5.3.11 Further reading

- Lee & Wagenmakers (2014), chapter 11. Signal detection theory, including the hierarchical SDT model you implemented today. *(Main reading.)*
- Kruschke (2014), chapter 9. Hierarchical models: partial pooling and shrinkage explained step by step.
- Farrell & Lewandowsky (2018), chapter 9. Multilevel or hierarchical modeling of cognitive data.
- Martin, Kumar & Lao (2021), chapters 2 and 10. Diagnostics with ArviZ, and probabilistic programming languages (how PyMC works under the hood).

Preview for Week 14: with the full toolkit in hand, we tour models from different areas of cognitive science, starting with categorization (the exemplar model GCM) and the speed–accuracy machinery of the drift-diffusion model.

Chapter 6

Applications in Cognitive Science

6.1 Week 14: Modeling in Cognitive Science I: Categorization (GCM) and Decision Making (Drift-Diffusion)

Chapter 6: Examples of modeling in different areas of cognitive science · Lecture 1 h · Lab 2 h Running data: simulated classification data for the classic 5-4 category structure, and simulated choice-RT data from a drift-diffusion process in a speed/accuracy × easy/hard design.

6.1.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
categorization	دسته‌بندی	قرار دادن محرک در یک دسته
Generalized Context Model (GCM)	مدل بافت تعمیم‌یافته	مدل exemplar دسته‌بندی بر پایه‌ی شباهت
drift rate	نرخ رانش	سرعت میانگین جمع شدن شواهد؛ نشانه‌ی کیفیت شواهد
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
response time (RT)	زمان واکنش	فاصله‌ی ارائه‌ی محرک تا پاسخ
drift-diffusion model (DDM)	مدل drift-diffusion (رانش–انتشار)	انباشت پیوسته‌ی شواهد بین دو مرز
speed–accuracy trade-off	بده‌بستان سرعت و دقت	سریع‌تر پاسخ دادن به قیمت خطای بیشتر
accuracy	دقت	نسبت پاسخ‌های درست

English	فارسی	توضیح کوتاه
exemplar	نمونه (exemplar)	یک مورد ذخیره‌شده در حافظه از یک دسته
prototype	پیش‌نمونه (prototype)	خلاصه یا میانگین یک دسته
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
attention weight	وزن توجه	سهم توجه به هر بُعد محرک
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
EZ-diffusion	EZ-diffusion	برآورد ساده‌ی پارامترهای از سه DDM آماره
parameter recovery	بازیابی پارامتر	آیا برازش، پارامترهای معلوم شبیه‌سازی را برمی‌گرداند
evidence	شواهد	اطلاعاتی که برای تصمیم جمع می‌شود
stimulus	محرک	چیزی که به شرکت‌کننده نشان داده می‌شود
boundary separation	فاصله‌ی مرزها	مقدار شواهد لازم برای پاسخ؛ نشانه‌ی احتیاط
non-decision time	زمان غیرتصمیمی	زمان دیدن محرک و اجرای حرکت پاسخ
similarity	شباهت	نزدیکی دو محرک در فضای روان‌شناختی
trial	کوشش (trial)	یک بار انجام تکلیف توسط شرکت‌کننده
embedding	embedding (بردار معنایی)	برداری که مدل برای یک کلمه یا متن می‌سازد

6.1.2 Learning outcomes

By the end of this week, students will be able to:

1. Describe exemplar and prototype theories of categorization and formalize them as the GCM and a prototype model.
2. Fit the GCM and the prototype model to classification data by MLE, compare them, and interpret attention weights and sensitivity.
3. Describe the drift-diffusion model (DDM) as the continuous limit of the Week 1 random walk, and map its parameters onto cognitive constructs.
4. Estimate DDM parameters with the closed-form EZ-diffusion method and validate it by parameter recovery.

5. Interpret experimental effects in terms of model parameters ("this manipulation changed caution, not evidence quality").

6.1.3 Motivation & big picture

The first 13 weeks built a toolkit, mostly on two small running examples (forgetting and SDT). Chapter 6 shows the toolkit at work on models that have shaped whole research areas. This week covers two of the most influential.

Categorization. How do we decide that a new animal is a dog? *Prototype theory* says that we compare it with a summary of the category, the average dog. *Exemplar theory* says that we compare it with stored memories of every dog (and every cat) we have seen. Both sound plausible, and verbal arguments went back and forth for years. Formal models made the difference: Medin and Schaffer's 5-4 category structure was designed so that the two theories make opposite predictions for specific stimuli. Nosofsky's Generalized Context Model (GCM) turned exemplar theory into a precise, fit-able model with interpretable attention weights.

Decision making. In Week 1 we built a random walk that accumulates evidence to a boundary. Its continuous version, the drift-diffusion model, is arguably the most successful model in cognitive psychology: it explains accuracy *and* the full RT distributions of correct and error responses, with parameters for evidence quality, caution and non-decision time. It is used to study ageing, ADHD, depression, learning and the effect of drugs. Fitting the full DDM is computationally heavy, but the EZ-diffusion method gives good estimates from three summary statistics with closed-form equations.

Where is this used?

- Categorization models explain how people learn concepts (medical students learning to recognize skin lesions, children learning words), and they are used to design better training: which examples to show, and in what order.
- Exemplar ideas in machine learning: nearest-neighbor classifiers and kernel methods classify a new case by its similarity to stored examples, the same principle as the GCM.
- The drift-diffusion model is one of the most used models in cognitive neuroscience and clinical research: ageing, ADHD, depression, anxiety, effects of drugs and sleep deprivation. It is also used in consumer research (how fast people choose between products) and in neuroeconomics.
- EZ-diffusion is popular because it needs only three summary statistics per person and no optimization, so it can be applied to existing datasets quickly.

6.1.4 Lecture notes

In plain words Two famous models this week. Categorization: how do we decide that a new animal is a dog? One theory says we compare it with an average "ideal dog" (prototype). Another says we compare it with all the dogs and cats we remember and see which group it resembles more overall (exemplars). The GCM turns the exemplar idea into math: measure the distance between the new item and each stored example (paying more attention to some features than others), turn distance into similarity (closer = much more similar), and choose category A with probability "total similarity to A / total similarity to all". A clever set of stimuli (the 5-4 structure) makes the two theories predict *opposite* things for some items. Decisions: the drift-diffusion model is the continuous version of Week 1's random walk. Evidence drifts toward the correct boundary at an average speed v (quality of evidence) while being jiggled by noise; a response is made when a boundary a (caution) is reached; a fixed time T_{er} is added for seeing and pressing. EZ-diffusion is a shortcut that computes v, a and T_{er} from just three numbers: accuracy, and the mean and variance of correct RTs.

Suggested timing (60 min): categorization theories and the 5-4 structure 10 · GCM 15 · prototype model and the critical predictions 5 · the drift-diffusion model 15 · EZ-diffusion 10 · interpretation 5.

1. The 5-4 category structure

Stimuli vary on four binary dimensions (e.g., the shape, color, size and position of a geometric figure in Medin & Schaffer's design; facial features in later versions). Participants learn to classify 9 training stimuli into categories A and B, then classify 7 new transfer stimuli without feedback. The commonly used stimulus set is [verify against Medin & Schaffer, 1978, or Smith & Minda, 2000]:

Category A	Category B	Transfer
A1 1110	B1 1100	T1 1001
A2 1010	B2 0110	T2 1000
A3 1011	B3 0001	T3 1111
A4 1101	B4 0000	T4 0010
A5 0111		T5 0101
		T6 0011
		T7 0100

The category A prototype is 1111 and the B prototype is 0000. The critical comparison: A1 (1110) shares three features with the A prototype, while A2 (1010) shares only two. A prototype model therefore predicts that A1 is classified *better* than A2. But A2 is highly similar to two other A exemplars (A1 and A3) and to no B exemplar, whereas A1 is highly similar to two B exemplars (B1 and B2). An exemplar model can therefore predict that A2 is classified *at least as well* as A1, and in the empirical data A2 is typically classified better.

2. The Generalized Context Model (GCM)

Distance between stimulus i and stored exemplar e, with attention weights $w_m \geq 0$, $\sum_m w_m = 1$ (city-block metric for separable dimensions):

$$\text{dist}_{ie} = \sum_{m=1}^{4} w_m \, |x_{im} - x_{em}|.$$

Similarity decays exponentially with distance, with sensitivity c:

$$s_{ie} = \exp(-c \cdot \text{dist}_{ie}).$$

Choice rule (summed similarity, Luce ratio):

$$P(A \mid i) = \frac{\sum_{e \in A} s_{ie}}{\sum_{e \in A} s_{ie} + \sum_{e \in B} s_{ie}}.$$

Parameters: c (how steeply similarity falls with distance; large c means only very close exemplars matter) and three free attention weights (the fourth is \$1 - \$ the others), so $k = 4$. (This is the overloaded c from the Week 1 notation table; this week it means sensitivity, not an SDT criterion.)

Interpretation of attention weights: w_m is the share of attention given to dimension m. Optimal attention emphasizes dimensions that are diagnostic of the category. In the 5-4 structure, dimensions 1 and 3 are the most diagnostic.

Prototype model: the same distance, similarity and choice rule, but each category is represented by its prototype only: $P(A \mid i) = s_{i,P_A}/(s_{i,P_A} + s_{i,P_B})$. Also $k = 4$, so the comparison is not affected by parameter counting.

Likelihood: if stimulus i is classified n_i times and called "A" y_i times, then $y_i \sim \text{Binomial}(n_i, P(A \mid i))$, fitted by MLE with multi-start (Weeks 3–4). Weights are best estimated through a softmax reparameterization (Week 3), $w = \text{softmax}(\eta_1, \eta_2, \eta_3, 0)$, and $c = e^{\gamma}$.

> Advanced: the full GCM adds a response-bias parameter b_A, a similarity exponent (Gaussian vs. exponential similarity), a metric parameter r (city-block $r = 1$ vs. Euclidean $r = 2$) and sometimes a response-determinism parameter γ (raising summed similarities to a power). Lee & Wagenmakers (chapter 17) develop a Bayesian, hierarchical GCM.

Reading the math

- Distance $\sum_m w_m |x_{im} - x_{em}|$: count the features on which the new item i and the stored example e differ, but weight each feature by its attention w_m. With 4 binary features and equal weights 0.25, one mismatching feature gives distance 0.25.
- Similarity $e^{-c \cdot \text{dist}}$: distance 0 gives similarity 1; with $c = 2$, distance 0.25 gives $e^{-0.5} = 0.61$, distance 1 gives $e^{-2} = 0.14$. A larger c makes similarity drop faster, so only very close examples matter.

- Choice rule: add the similarities to all A examples and to all B examples, and divide. In the worked example, T3 has total similarity 2.79 to A and 1.09 to B, so $P(A) = 2.79/3.89 = 0.72$.

3. The drift-diffusion model (DDM)

Take the Week 1 random walk and let the step size and step duration shrink together. Evidence $X(t)$ then evolves continuously:

$$dX = v\,dt + s\,dW(t), \qquad X(0) = z,$$

where dW is Gaussian white noise (a Wiener process). A response is made when X reaches a (correct) or 0 (error); RT $= T_{\text{decision}} + T_{er}$.

Parameter	Meaning	Typical manipulation that changes it
v (drift rate)	Quality / rate of evidence accumulation	Stimulus difficulty; perceptual sensitivity; attention
a (boundary separation)	Response caution (speed–accuracy setting)	Speed vs. accuracy instructions
z (starting point)	A priori bias toward one response	Base rates, payoffs
T_{er} (non-decision time; also written t_0)	Encoding + motor time	Response modality, stimulus degradation
s (within-trial noise)	Scaling constant	Fixed (by convention $s = 0.1$ in EZ, $s = 1$ elsewhere)

The full DDM also includes across-trial variability in drift, starting point and non-decision time. Week 1, Exercise 5 showed that drift variability produces slow errors.

Signature predictions: right-skewed RT distributions; a speed–accuracy trade-off when a changes; easier stimuli giving responses that are faster *and* more accurate (higher v); a change in T_{er} shifting the whole RT distribution without changing its shape.

Reading the math $dX = v\,dt + s\,dW(t)$ reads: "in a tiny time step dt, the evidence changes by a deterministic push $v\,dt$ plus a random kick $s\,dW$". dW is Gaussian noise whose variance is proportional to dt (so over a step of length dt the kick has SD $s\sqrt{dt}$). In the lab simulation this becomes exactly Week 1's random walk with small steps: `x+=v*dt+s*sqrt(dt)*randn()`. The table of parameters is the important part to memorize: v = evidence quality, a = caution, z = bias, T_{er} = non-decision time.

4. EZ-diffusion

Wagenmakers, van der Maas and Grasman (2007) derived closed-form estimates of v, a and T_{er} (assuming $z = a/2$, no across-trial variability, $s = 0.1$) from three statistics: the proportion correct P_c, the variance of correct RTs V_{RT} (in s²) and the mean of correct RTs M_{RT} (in s):

$$L = \text{logit}(P_c) = \ln \frac{P_c}{1 - P_c}, \qquad x = \frac{L\,(LP_c^2 - LP_c + P_c - 0.5)}{V_{RT}},$$

$$v = \text{sign}(P_c - 0.5)\; s\; x^{1/4}, \qquad a = \frac{s^2 L}{v},$$

$$y = -\frac{va}{s^2}, \qquad \text{MDT} = \frac{a}{2v} \cdot \frac{1 - e^y}{1 + e^y}, \qquad T_{er} = M_{RT} - \text{MDT}.$$

Edge cases: the formulas fail at $P_c = 0.5$ (then $v = 0$) and $P_c = 1$ (then $L = \infty$). For $P_c = 1$, a standard fix is to use $P_c = 1 - 1/(2N)$, analogous to the extreme-rate corrections of Week 2.

Strengths and limits. EZ is fast, needs no optimization and recovers parameters well when its assumptions hold (lab Part C). It ignores error RTs and the shape of the RT distribution beyond mean and variance, and it cannot estimate bias or across-trial variabilities. For those, fit the full DDM by MLE or with hierarchical Bayesian methods (e.g., the HDDM package or the Wiener likelihood in PyMC/Stan).

Reading the math The EZ equations look intimidating but are a recipe; nobody needs to memorize them.

1. $L = \ln(P_c/(1 - P_c))$: the logit of accuracy ("log odds of being correct"). $P_c = 0.8$ gives $L = \ln 4 = 1.39$.
2. x combines accuracy with the variance of correct RTs: the more variable the RTs for a given accuracy, the lower the drift.
3. v and a then follow from L and x; MDT is the predicted *mean decision time*.
4. $T_{er} = M_{RT} - \text{MDT}$: whatever part of the mean RT is not decision time is non-decision time.

Use the worked example (Part 2) to show each step with numbers; then let the computer do it (the function `ez_diffusion` in the lab code).

For an engineer

- GCM is a similarity-based classifier with an exponential kernel and a weighted L1 (city-block) metric, close to a kernel / Parzen-window classifier. The attention weights act like learned feature weights; c is the inverse kernel width.
- The DDM is a Wiener process with drift between two absorbing boundaries: the classic first-passage-time problem. The continuous-time SPRT (Week 1 box) is exactly this process, with $v \propto$ signal amplitude and s the noise level.
- RT distributions are first-passage-time distributions, which are right-skewed for the same reason detection delays of a sequential detector are.

- EZ-diffusion is a method-of-moments estimator: it matches three sample moments (accuracy, mean and variance of correct RTs) to their closed-form model expressions, instead of maximizing a likelihood.

For the instructor

- Let students be the participants. Before the GCM, show the 9 training stimuli of the 5-4 structure as simple drawings (e.g. shapes with 4 binary features), teach the categories with feedback, then test A1 and A2. The class's own data often show the exemplar pattern.
- Draw the DDM as Week 1's picture with a smooth, noisy path. Then change one parameter at a time on the board (wider boundaries, steeper drift, later start) and ask what happens to accuracy and RT.
- Do not derive EZ. Present it as a "calculator" and spend the time on interpretation (lab Part E): which manipulation changed which parameter.
- Typical questions.
  - *"Prototype or exemplar, who won?"* For small, poorly structured categories exemplar models usually fit better; for large, well-structured ones prototypes do well. Both ideas survive in modern models.
  - *"Why not just compare mean RTs?"* A slower group could be more cautious, have worse evidence, or be slower motorically; only the DDM separates them.
  - *"Are the parameters of EZ and the full DDM the same?"* They estimate the same quantities, but EZ assumes no bias and no trial-to-trial variability.

6.1.5 Worked example

Part 1: one GCM prediction by hand

Stimulus T3 (1111), equal attention $w_m = 0.25$, sensitivity $c = 2$.

Exemplar	Mismatching dimensions	Distance	Similarity $e^{-2\,\text{dist}}$
A1 1110	1	0.25	0.607
A2 1010	2	0.50	0.368
A3 1011	1	0.25	0.607
A4 1101	1	0.25	0.607
A5 0111	1	0.25	0.607
Sum A			2.794
B1 1100	2	0.50	0.368
B2 0110	2	0.50	0.368

Exemplar	Mismatching dimensions	Distance	Similarity $e^{-2\,\text{dist}}$
B3 0001	3	0.75	0.223
B4 0000	4	1.00	0.135
Sum B			1.094

$$P(A \mid \text{T3}) = \frac{2.794}{2.794 + 1.094} = 0.719.$$

Interpretation. T3 is the A prototype, yet the exemplar model predicts only about 72% "A" responses with these parameters: two B exemplars (B1 and B2) are only two features away. A larger c (sharper similarity) would raise this probability, because the nearest neighbors (four A exemplars at distance 0.25) would dominate.

Part 2: EZ-diffusion by hand

A participant has $P_c = 0.802$, $V_{RT} = 0.112$ s², $M_{RT} = 0.723$ s.

1. $L = \ln(0.802/0.198) = 1.399$.
2. $x = 1.399 \times (1.399 \times 0.643 - 1.399 \times 0.802 + 0.802 - 0.5)/0.112 = 0.998$.
3. $v = 0.1 \times 0.998^{1/4} = 0.100$.
4. $a = 0.01 \times 1.399/0.100 = 0.140$.
5. $y = -0.100 \times 0.140/0.01 = -1.399$, so MDT $= (0.140/0.200) \times (1 - e^{-1.399})/(1 + e^{-1.399}) = 0.423$ s.
6. $T_{er} = 0.723 - 0.423 = 0.300$ s.

Interpretation. Of the mean correct RT of 723 ms, about 423 ms is spent accumulating evidence and 300 ms on encoding and responding. Drift 0.10 and boundary 0.14 (in EZ units with $s = 0.1$) are typical values for a moderately difficult task.

6.1.6 Lab (2 hours)

Goals

1. Implement the GCM and the prototype model for the 5-4 structure; simulate data from the GCM.
2. Fit both models by MLE, compare them, and examine the critical A1-vs-A2 prediction.
3. Check parameter recovery for the GCM.
4. Simulate a drift-diffusion process, recover its parameters with EZ-diffusion, and interpret a 2 × 2 experiment.

Timing: Part A 20 min · Part B 25 min · Part C 15 min · Part D 40 min · Part E 20 min.

Part A: the 5-4 structure and two models (20 min)

Code · کد: `code/week14.py`, cell 1 · سلول 1

Expected output:

```
stimulus features  P(A) true  A responses / 20
      A1     1110       0.84                15
      A2     1010       0.88                18
      A3     1011       0.91                18
      A4     1101       0.60                10
      A5     0111       0.60                11
      B1     1100       0.47                10
      B2     0110       0.47                11
      B3     0001       0.16                 4
      B4     0000       0.12                 2
      T1     1001       0.58                11
      T2     1000       0.48                11
      T3     1111       0.87                20
      T4     0010       0.48                10
      T5     0101       0.26                 6
      T6     0011       0.58                12
      T7     0100       0.19                 4
```

Part B: fit and compare (25 min)

Code · کد: `code/week14.py`, cell 2 · سلول 2

Expected output:

```
GCM       NLL =  28.36  AIC =  64.72  BIC =  79.79  c = 3.82  w = [0.397
↪  0.111 0.426 0.066]
prototype NLL =  30.02  AIC =  68.04  BIC =  83.11  c = 1.90  w = [0.435
↪  0.009 0.461 0.095]
stimulus  observed   GCM  prototype
      A1      0.75 0.842      0.822
      A2      0.90 0.884      0.818
```

Interpret: 1. Which model is preferred, and how strong is the evidence? (ΔAIC $= 3.3$, so Akaike weights of about 0.84 vs. 0.16: a moderate preference, not a decisive one.) 2. Look at A1 and A2. The data show A2 (0.90) classified better than A1 (0.75). The GCM predicts this ordering (0.88 vs. 0.84); the prototype model cannot (0.82 vs. 0.82). Why does this comparison discriminate between the theories more sharply than the overall fit statistic? 3. Interpret the GCM attention weights. Which dimensions did the participant attend to, and does that make sense given the category structure?

Part C: is the GCM recoverable at this design? (15 min)

Code · کد: `code/week14.py`, cell 3 · سلول 3

Expected output (about 1 minute):

```
c: r(true, recovered) = 0.08
w1: r(true, recovered) = 0.91
w2: r(true, recovered) = 0.76
w3: r(true, recovered) = 0.89
w4: r(true, recovered) = 0.85
```

What to notice: the attention weights are recovered well (r = 0.76–0.91), but the sensitivity c is essentially not recoverable at this design (r = 0.08). With only 16 stimuli and 20 trials each, a large c and a moderate c can produce very similar response proportions (once c is large, predictions saturate), and c trades off with how concentrated the weights are. So we should interpret the attention weights from Part B but not the value $\hat{c} = 3.82$. More trials, a wider range of stimuli, or a Bayesian prior on c would help. This is exactly the kind of result a recovery study is for (Week 7).

Part D: drift-diffusion and EZ (40 min)

Code · کد: `code/week14.py`, cell 4 · سلول 4

Expected output (about 30 s):

```
instruction difficulty    Pc   MRT  v_true  v_EZ  a_true  a_EZ  Ter_EZ
      speed       easy 0.870 0.423    0.25 0.243    0.08 0.078   0.304
      speed       hard 0.732 0.466    0.12 0.115    0.08 0.088   0.288
   accuracy       easy 0.975 0.572    0.25 0.261    0.14 0.140   0.317
   accuracy       hard 0.818 0.697    0.12 0.111    0.14 0.135   0.313
```

Code · کد: `code/week14.py`, cell 5 · سلول 5

Part E: interpret the 2 × 2 experiment (20 min)

Answer in your notebook: 1. Which EZ parameter changes with the instruction manipulation, and which with difficulty? Did either manipulation affect the parameter it should *not* affect? 2. Describe how the RT distribution changes from speed to accuracy instructions (Part D figure). Which part of the distribution changes most: the leading edge or the tail? 3. A clinical study finds that patients are slower than controls but equally accurate. Sketch two different DDM accounts of this result, and say which data would distinguish them.

Exercises (increasing difficulty)

Exercise 1 $\star$. Compute $P(A \mid \text{A1})$ and $P(A \mid \text{A2})$ by hand for equal weights and $c = 2$ under both models. Do they reproduce the critical prediction?

Exercise 2 $\star$. Apply EZ to the worked example's statistics and to $P_c = 1.0$ with $N = 100$ trials (use the edge correction). What happens without the correction?

Exercise 3 $\star\star$ (prototype data). Generate data from the prototype model and fit both models. Does the model comparison identify the generating model? Run a small model recovery (20 datasets per generating model) and build the 2×2 confusion matrix.

Exercise 4 $\star\star$ (EZ recovery and bias). For 200 simulated participants with random (v, a, T_{er}) and only 100 trials each, compare EZ estimates with the true values. Which parameter is recovered worst? Then add across-trial drift variability to the simulator (Week 1, Exercise 5) and check which EZ estimate becomes biased.

Exercise 5 $\star\star\star$ (Advanced: Bayesian GCM). Fit the GCM in PyMC with priors $c \sim \text{Gamma}(2, \text{scale } 2)$ and $w \sim \text{Dirichlet}(1, 1, 1, 1)$. Compare the posterior of the attention weights with the MLE, and run a posterior predictive check on the A1-vs-A2 difference. (See Lee & Wagenmakers, chapter 17, for a hierarchical version.)

Lab check-off

Show: (1) the fit table and prediction plot for the GCM vs. the prototype model with your answer to question 2, (2) the EZ table for the 2×2 design, and (3) your answer to Part E, question 1.

6.1.7 Connection to AI & LLMs

Exemplar models in a learned "semantic space". The GCM needs a psychological space in which to measure distances. Traditionally, that space comes from similarity ratings and multidimensional scaling. Language models offer an alternative: every word or sentence is mapped to an embedding vector, and the distances between vectors are often used as a stand-in for psychological similarity. Combining embeddings with the GCM gives a model that can classify *any* new word with no extra similarity ratings. Whether embedding distances match human similarity is an empirical question, and you can test it with the tools of this course.

Code · كد: `code/week14.py`, cell 6 · سلول 6

Expected output:

```
P(fruit | tomato) = 0.902
P(fruit | knife) = 0.094
P(fruit | nutcracker) = 0.370
```

Try it (no API needed). - ⋆ Change c to 1 and to 10. How do the three predictions change, and why does "nutcracker" change the most? - ⋆⋆ Suppose 20 participants classified the three probes, with 19, 1 and 12 "fruit" responses. Fit c by MLE (binomial likelihood, Week 4) and run a G^2 test. Does the embedding space with a single parameter fit? - ⋆⋆ Discussion: the embeddings were learned from text, not from perception. For which categories might text-based similarity differ systematically from human similarity (think of color or shape)?

6.1.8 Common mistakes & misconceptions

1. Unconstrained attention weights. Without a sum-to-one constraint, c and w trade off perfectly (non-identifiable). Use a softmax reparameterization.
2. Judging exemplar vs. prototype models by overall fit only. The theoretically diagnostic predictions (A1 vs. A2) are often more informative than small differences in AIC.
3. Averaging classification data over participants who attend to different dimensions. This can make an exemplar-like individual look prototype-like (and vice versa); recall Week 5's aggregation artifacts.
4. Using EZ with incorrect units. RTs must be in seconds and the variance in s², with $s = 0.1$. Mixing ms and s gives nonsense.
5. Applying EZ when its assumptions are violated (strong bias, slow or fast errors). Check error RTs; use the full DDM if needed.
6. "A slower group has a lower drift rate." Slowness can come from a higher boundary, a lower drift, or a longer non-decision time. Only a model fitted to accuracy *and* RT distributions can tell which.
7. Forgetting the edge cases $P_c = 1$ or $P_c \leq 0.5$ in EZ.

6.1.9 Check your understanding

1. State the core difference between exemplar and prototype theories in one sentence each. Answer Exemplar: a category is represented by all stored instances, and classification depends on summed similarity to them. Prototype: a category is represented by a single summary (central tendency), and classification depends on similarity to that summary.

2. In the 5-4 structure, why is the A1-vs-A2 comparison diagnostic? Answer A1 is closer to the A prototype (3 of 4 features), so a prototype model predicts it is classified better than A2. But A2 is

very similar to other A exemplars and to no B exemplar, while A1 is similar to two B exemplars, so the exemplar model can predict A2 $\geq$ A1. The theories make opposite ordinal predictions.

3. What do the GCM parameters c and w_m represent? Answer c is sensitivity: how quickly similarity decreases with psychological distance (large c means only very close exemplars matter). w_m is the proportion of attention allocated to dimension m, which stretches distances along attended dimensions.

4. Which DDM parameter would you expect to change under (a) speed vs. accuracy instructions, (b) harder stimuli, (c) a slower motor response (e.g., foot instead of finger)? Answer (a) Boundary separation a. (b) Drift rate v. (c) Non-decision time T_{er}.

5. What three statistics does EZ-diffusion need, and what are its main assumptions? Answer Proportion correct, the variance of correct RTs, and the mean of correct RTs. It assumes an unbiased starting point ($z = a/2$), no across-trial variability in parameters, and the scaling $s = 0.1$.

6. How is the DDM related to the random walk model of Week 1? Answer It is the continuous-time, continuous-evidence limit of the random walk: as step size and step duration go to zero together, the random walk converges to a Wiener diffusion process with drift v and noise s.

6.1.10 Assignment

- A5 (MCMC from scratch) is due at the start of this week's lecture.
- No new assignment: work on the final project. If your project uses the GCM or DDM, today's code is a good starting point; cite the course materials and say what you changed.

6.1.11 Further reading

- Lee & Wagenmakers (2014), chapter 17. The Generalized Context Model, fitted in a Bayesian (and hierarchical) way to category-learning data. *(Main reading for categorization.)*
- Farrell & Lewandowsky (2018): the chapter on models of choice response time [verify chapter number] for the drift-diffusion model and its relatives.
- Verguts (2022): the chapters on categorization and on decision making [verify chapters].
- *Optional, outside the course list:* Wagenmakers, E.-J., van der Maas, H. L. J., & Grasman, R. P. P. P. (2007). An EZ-diffusion model for response time and accuracy. *Psychonomic Bulletin & Review, 14*, 3–22. Medin, D. L., & Schaffer, M. M. (1978). Context theory of classification learning. *Psychological Review, 85*, 207–238.

Preview for Week 15: learning from rewards (reinforcement learning), the mind as a Bayesian observer, and how cognitive models and AI systems inform each other.

6.2 Week 15: Modeling in Cognitive Science II: Reinforcement Learning, Bayesian Cognition and AI

Chapter 6: Examples of modeling in different areas of cognitive science · Lecture 1 h · Lab 2 h Running data: simulated choices in a two-armed bandit task with a reversal; a simulated visual–haptic cue-combination experiment.

6.2.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
bandit task	تکلیف bandit	انتخاب مکرر بین گزینه‌هایی با پاداش احتمالی
cue combination	ترکیب نشانه‌ها (cues)	ترکیب اطلاعات چند حس برای یک برآورد
Q-learning	Q-learning	قاعده‌ی یادگیری ارزش گزینه‌ها
softmax / inverse temperature (β)	softmax / دمای معکوس	تبدیل ارزش‌ها به احتمال انتخاب
learning rate (α)	نرخ یادگیری	اندازه‌ی به‌روزرسانی بعد از هر خطا
temperature	دما (temperature)	پارامتر نویز در انتخاب؛ $\beta = 1/T$
trial	کوشش (trial)	یک بار انجام تکلیف توسط شرکت‌کننده
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
parameter recovery	بازیابی پارامتر	آیا برازش، پارامترهای معلوم شبیه‌سازی را برمی‌گرداند
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
ideal observer	مشاهده‌گر ایده‌آل	مدل استنباط بهینه با اطلاعات موجود
cognitive model	مدل شناختی	توصیف دقیق و ریاضی یا محاسباتی از یک فرایند ذهنی
exploration / exploitation	بهره‌برداری / کاوش	انتخاب بهترین / امتحان گزینه‌های دیگر
prediction error	خطای پیش‌بینی	تفاوت پاداش گرفته‌شده و پاداش مورد انتظار

English	فارسی	توضیح کوتاه
reinforcement learning (RL)	یادگیری تقویتی	یادگیری از پاداش و پیامد
noise	نویز	تغییرات تصادفی در شواهد یا در داده
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
reliability	پایایی (قابلیت اطمینان)	دقت یک نشانه؛ عکس واریانس آن
large language model (LLM)	مدل بزرگ زبانی	شبکه‌ی عصبی‌ای که کلمه‌ی بعدی متن را پیش‌بینی می‌کند
parameter trade-off	بده‌بستان پارامترها	تغییر یک پارامتر که با پارامتر دیگر جبران می‌شود
RLHF / reward model	RLHF / مدل پاداش	آموزش chatbot از روی ترجیحات انسانی
Bradley–Terry model	مدل Bradley–Terry	مدل انتخاب برای مقایسه‌های جفتی

6.2.2 Learning outcomes

By the end of this week, students will be able to:

1. Formalize reward learning with the Rescorla–Wagner / Q-learning rule and a softmax choice rule, and interpret the learning rate and inverse temperature.
2. Write the trial-by-trial likelihood of a learning model, fit it by MLE, and evaluate parameter recovery and model recovery against simpler strategies.
3. Derive the predictions of a Bayesian ideal observer for cue combination, and test whether simulated behavior is optimal.
4. Distinguish Bayesian models of cognition (a claim about the mind) from Bayesian data analysis (a method for scientists).
5. Discuss how cognitive models and AI systems inform each other, and how the evaluation tools of this course apply to AI systems proposed as models of cognition.

6.2.3 Motivation & big picture

Learning from consequences. A participant repeatedly chooses between two slot machines, one paying off more often than the other. Halfway through, the machines secretly swap. How quickly does the participant notice and adapt? How much do they explore versus exploit? One of the most successful models in all of psychology and neuroscience answers with a single equation: update your expectation by a fraction of the prediction error, the difference between what you got and what you expected. The same error-driven rule appears in classical conditioning (Rescorla–Wagner), in

connectionist learning (the delta rule), in the firing of dopamine neurons, and in the reinforcement-learning algorithms behind modern AI systems. Fitting it to choice data is one of the most common uses of cognitive modeling today, in computational psychiatry for instance.

The mind as a statistician. When you see and touch an object at the same time, vision and touch both give noisy estimates of its size. A *Bayesian ideal observer* combines them by weighting each cue by its reliability, which gives an estimate more precise than either alone. Do people do this? This is a different use of the word "Bayesian" from Weeks 9–13: here Bayes' rule is a hypothesis about the mind, at Marr's computational level (Week 1).

Cognitive models $\leftrightarrow$ AI. Many ideas travel in both directions between cognitive modeling and AI: error-driven learning, neural networks, reinforcement learning, Bayesian inference. Today AI systems, including large language models, are themselves proposed as models of human cognition. The tools of this course (likelihoods, model comparison, generalization tests, recovery) are exactly what is needed to evaluate such claims.

Where is this used?

- Computational psychiatry: reinforcement-learning models are fitted to choices of patients with depression, addiction, schizophrenia or OCD, to test whether they learn differently from rewards and punishments (e.g. a lower learning rate after losses).
- Neuroscience: the prediction error of the delta rule matches the activity of dopamine neurons, one of the best-known links between a cognitive model and the brain.
- Education and games: adaptive learning systems and game design use RL ideas to model how players and learners adjust to feedback.
- Perception and robotics: optimal cue combination explains how people combine vision, touch and hearing, and the same weighting rule is used in sensor fusion.
- AI: modern reinforcement learning and the training of chatbots from human preferences (RLHF) use the same choice models (see the AI & LLMs section).

6.2.4 Lecture notes

In plain words Learning from rewards. Each option (e.g. each of two slot machines) has a "value" in the participant's head: how much reward they expect from it. After choosing an option and seeing the outcome, the value is nudged toward what actually happened: *new value* = *old value* + *learning rate* $\times$ *(reward* $-$ *old value)*. The term in brackets is the prediction error, the surprise. The learning rate α says how big the nudge is. Choices are not perfectly greedy: the softmax rule picks the better-valued option more often, and the inverse temperature β sets how strongly (low β = more random exploring, high β = almost always the best). Fitting this model means finding the α and β that make the participant's actual sequence of choices most likely. The Bayesian mind: when two senses give noisy estimates of the same thing (seeing and touching an object's size), the best guess is a weighted average that trusts the more reliable sense more. People often come close to this optimal weighting; the model predicts it with no free parameters, which makes it a strong test.

Suggested timing (60 min): reward learning 20 · fitting learning models 10 · Bayesian models of cognition 15 · cognitive models and AI 10 · course synthesis 5.

1. Reinforcement learning: the delta rule and Q-learning

Rescorla–Wagner / delta rule. After choosing option o on trial t and receiving reward $r_t \in \{0, 1\}$:

$$\delta_t = r_t - Q_t(o) \quad \text{(prediction error)}, \qquad Q_{t+1}(o) = Q_t(o) + \alpha\, \delta_t,$$

and the values of unchosen options stay the same. The learning rate $\alpha \in [0, 1]$ sets how strongly new outcomes overwrite old expectations. A high α adapts quickly but is noisy; a low α is stable but slow to notice a reversal.

Softmax choice rule. Values are turned into choice probabilities with inverse temperature $\beta \geq 0$:

$$P(\text{choose } o \text{ on trial } t) = \frac{\exp(\beta\, Q_t(o))}{\sum_{o'} \exp(\beta\, Q_t(o'))}.$$

$\beta = 0$ means random choice (pure exploration); a large β means always picking the currently best option (exploitation). For two options this is a logistic function of the value difference: $P(o_1) = 1/(1 + e^{-\beta(Q_1 - Q_2)})$.

Connections: - The update is the delta rule of connectionist networks (Verguts, 2022): a single-layer network trained by error correction. - Prediction errors δ_t resemble the phasic responses of midbrain dopamine neurons, which made RL models central to computational neuroscience. - Q-learning with temporal-difference errors is the basis of modern reinforcement learning in AI.

Reading the math

- $Q_t(o)$: the value of option o on trial t (the subscript t is the trial number).
- $\delta_t = r_t - Q_t(o)$: reward received minus reward expected. Positive = pleasant surprise, negative = disappointment.
- $Q_{t+1}(o) = Q_t(o) + \alpha\, \delta_t$. Example: $Q = 0.5$, $\alpha = 0.3$, reward 1: $\delta = 0.5$, new $Q = 0.5 + 0.15 = 0.65$. With no reward: $\delta = -0.5$, new $Q = 0.35$.
- Softmax for two options: $P(\text{option 1}) = 1/(1 + e^{-\beta(Q_1 - Q_2)})$. With $Q_1 - Q_2 = 0.25$: $\beta = 0$ gives 0.50 (random), $\beta = 3$ gives 0.68, $\beta = 20$ gives 0.99 (almost always option 1).

2. Fitting a learning model

The likelihood is sequential. Values depend on the participant's own past choices and rewards, so we run the model through the participant's actual trial sequence and multiply the probabilities it assigned to the choices actually made:

$$\ell(\alpha, \beta) = \sum_{t=1}^{T} \ln P\big(\text{choice}_t \mid Q_t(\cdot;\ \alpha, \text{history}_{<t}),\ \beta\big).$$

Practical points: - Reparameterize: $\alpha = 1/(1 + e^{-\eta_\alpha})$ and $\beta = e^{\eta_\beta}$ (Week 3), and use multi-start. - α–β trade-off: noisy choice (low β) and slow or erratic learning (extreme α) can produce similar-looking behavior, so the estimates are often correlated across fits, and the sign of the correlation depends on the design and the parameter range. A further problem is that very large β values are hard to estimate at all (lab Part C). Recovery studies at the actual design are essential (Week 7). - The design matters: reversals and enough trials are needed to estimate α; stable reward probabilities give little information about it. - Alternative strategies must be compared: e.g., *win-stay/lose-shift* (WSLS: repeat a rewarded choice, switch after no reward, with lapse rate ε), or a biased-random model. A good RL fit means little if WSLS fits just as well.

Reading the math $\ell(\alpha, \beta) = \sum_t \ln P(\text{choice}_t \mid Q_t, \beta)$: run the learning model through the participant's *own* sequence of choices and rewards, trial by trial. On each trial, compute the probability the model gave to the choice the participant actually made, take its log, and add up. The worked example (Part 1) does this for four trials: $\ln 0.5 + \ln 0.679 + \ln 0.593 + \ln 0.755 = -1.884$. The best (α, β) maximize this sum. It is Week 4's likelihood, except that each trial's probability depends on the history before it.

3. Bayesian models of cognition

Cue combination. A visual cue $x_V \sim \mathcal{N}(S, \sigma_V)$ and a haptic cue $x_H \sim \mathcal{N}(S, \sigma_H)$ both measure the true size S. With a flat prior, the posterior over S is normal with mean

$$\hat{S} = w_V x_V + w_H x_H, \qquad w_V = \frac{1/\sigma_V^2}{1/\sigma_V^2 + 1/\sigma_H^2}, \quad w_H = 1 - w_V,$$

and SD

$$\sigma_{VH} = \sqrt{\frac{\sigma_V^2\,\sigma_H^2}{\sigma_V^2 + \sigma_H^2}} \;\le\; \min(\sigma_V, \sigma_H).$$

Testable predictions (tested, e.g., by Ernst & Banks, 2002): (1) the weights should track the cues' reliabilities, which the experimenter can manipulate by degrading one cue; (2) bimodal precision should be *better* than the best single cue, by exactly the amount above. The single-cue SDs are measured separately, so the ideal-observer predictions have no free parameters. That makes them strong, falsifiable tests.

What a Bayesian model of cognition claims. That behavior approximates optimal inference given the agent's noise, prior and goals: a computational-level account (Marr). It is silent about the algorithm: the brain might approximate Bayes with sampling, heuristics or neural population codes.

Criticisms to keep in mind: with free choices of priors, likelihoods and loss functions, almost any behavior can be called "optimal" after the fact (the flexibility problem of Week 6). Strong tests fix these components independently, as in cue combination.

Two meanings, one more time:

	Bayesian model of cognition	Bayesian data analysis
Who does the inference?	The participant's mind (hypothetically)	The scientist
Prior represents	The participant's beliefs about the world	The scientist's knowledge about parameters
Can be wrong if…	People are not optimal	The model or priors are poorly chosen
Examples in this course	Cue combination (today)	Weeks 9–13

You can analyze a Bayesian model of cognition with maximum likelihood, and a non-Bayesian model (e.g., Q-learning) with Bayesian data analysis.

Reading the math

- $w_V = \dfrac{1/\sigma_V^2}{1/\sigma_V^2 + 1/\sigma_H^2}$: the weight of vision is its precision (1/variance) divided by the total precision. A sense twice as precise (half the SD, a quarter of the variance) gets four times the weight.
- $\sigma_{VH} = \sqrt{\sigma_V^2 \sigma_H^2 / (\sigma_V^2 + \sigma_H^2)}$: the combined SD, always smaller than the better single sense. Example: $\sigma_V = 1$, $\sigma_H = 2$: $w_V = 1/(1 + 1/4) = 0.8$ and $\sigma_{VH} = \sqrt{4/5} = 0.894$, about 11% better than vision alone.
- The test: measure σ_V and σ_H separately (one sense at a time), predict w_V and σ_{VH} with no free parameters, then check the two-sense data.

For an engineer

- The delta rule is the LMS update. $Q \leftarrow Q + \alpha(r - Q)$ is a one-tap LMS (or an exponentially weighted moving average with forgetting factor $1 - \alpha$): large α tracks changes quickly but is noisy, small α is smooth but slow, the familiar tracking–misadjustment trade-off.
- Softmax is the Boltzmann/Gibbs distribution; β is an inverse temperature in the physical sense. Sampling with a temperature is also how language models generate text (Week 1 AI box).
- Fitting α and β from choices is system identification of an adaptive system from its binary outputs only: you never observe Q, only the decisions it produces. That is why α and β trade off and why recovery checks are essential.
- Optimal cue combination uses exactly the inverse-variance weights of maximal-ratio combining and of sensor fusion in a Kalman filter: the combined estimate has variance $(1/\sigma_V^2 + 1/\sigma_H^2)^{-1}$. The scientific question is whether the *brain* implements this fusion.

4. Cognitive models and artificial intelligence

Direction	Examples
Cognitive science → AI	Neural networks (inspired by neurons and learning rules); reinforcement learning (from animal learning theory); exemplar/prototype ideas in machine-learning classifiers (nearest-neighbor vs. centroid methods)
AI → cognitive science	Deep networks as models of visual cortex; RL agents as models of decision making; large language models proposed as models of language processing and of human-like reasoning
Shared mathematics	Likelihoods and loss functions; gradient-based optimization (Week 3); Bayesian inference and MCMC; cross-validation and generalization (Week 6)

Evaluating an AI system as a cognitive model uses exactly the course toolkit: - Does it predict human data *beyond* fitting it, e.g., new conditions and individual differences (generalization, Week 6)? - Does it beat simpler, interpretable cognitive models on a fair comparison (Weeks 7, 10)? - Can we tell from data whether humans and the model use the same mechanism, or only produce similar outputs (model mimicry and recovery, Week 7)? - Are its parameters or internal states interpretable as psychological constructs (Week 1)?

A model with billions of parameters that fits behavior well is not automatically an *explanation*: flexibility, identifiability and interpretability matter as much as fit.

5. Course synthesis

Across very different domains (forgetting, detection, categorization, decisions, learning, perception), the workflow has been the same: formalize → simulate → fit → evaluate → recover → interpret. The domain changes the model; the logic of modeling stays.

For the instructor

- Play the bandit game with the class. Two "machines" (two students with hidden coins or a random number app, one paying 80% of the time and one 20%), and halfway through secretly swap them. The class experiences exploration, exploitation and reversal before seeing any equation.
- Do the four Q-learning trials of the worked example on the board with a table: value before, choice, reward, prediction error, value after. It takes ten minutes and demystifies the code.
- For cue combination, use your engineering intuition openly: "combining two noisy sensors" is a natural way to present it, and students from psychology find it illuminating.
- Keep the AI discussion grounded. Section 4 can trigger long debates; use the four evaluation questions (generalization, fair comparison, mimicry, interpretability) to keep them focused.

- Typical questions.
  - *"Why is β hard to estimate?"* Once choices are almost deterministic, increasing β further hardly changes the likelihood.
  - *"Is the brain really doing Q-learning?"* It is a good description of many choice data and matches dopamine signals, but more complex models (model-based learning) also have support.
  - *"Do people always combine cues optimally?"* Often close, but not always; the lab participant over-weights touch.

6.2.5 Worked example

Part 1: four trials of Q-learning by hand

Two options; $\alpha = 0.5$, $\beta = 3$; initial values $Q(1) = Q(2) = 0.5$.

Trial	$Q(1)$	$Q(2)$	$P(\text{choose } 1)$	Choice	Reward	δ	Update
1	0.50	0.50	0.500	1	1	$1 - 0.5 = +0.5$	$Q(1) = 0.5 + 0.5 \times 0.5 = 0.75$
2	0.75	0.50	$1/(1+e^{-3\times 0.25}) = 0.679$	1	0	$0 - 0.75 = -0.75$	$Q(1) = 0.75 - 0.375 = 0.375$
3	0.375	0.50	$1/(1 + e^{-3\times(-0.125)}) = 0.407$	2	1	$1 - 0.5 = +0.5$	$Q(2) = 0.75$
4	0.375	0.75	$1/(1 + e^{-3\times(-0.375)}) = 0.245$	2	1	$+0.25$	$Q(2) = 0.875$

Log-likelihood of these four choices: $\ln 0.5 + \ln 0.679 + \ln(1-0.407) + \ln(1-0.245) = -0.693 - 0.387 - 0.523 - 0.281 = -1.884$.

Interpretation. With $\alpha = 0.5$ a single non-reward halves the value of option 1, and the model predicts a switch, which the participant made. The likelihood rewards parameter values that predict the participant's actual choice sequence.

Part 2: an ideal observer

Visual SD $\sigma_V = 1$ mm, haptic SD $\sigma_H = 2$ mm. Weights: $w_V = 1/(1+1/4) = 0.8$, $w_H = 0.2$. Predicted combined SD: $\sqrt{(1\times 4)/(1+4)} = 0.894$ mm, about 11% better than vision alone. If $x_V = 50$ and $x_H = 54$ mm, the optimal estimate is $0.8 \times 50 + 0.2 \times 54 = 50.8$ mm.

Interpretation. An observer who simply *averaged* the cues ($w = 0.5$) would have SD $\sqrt{0.25\times 1 + 0.25 \times 4} = 1.118$ mm, *worse* than vision alone. Optimal weighting is what makes combining cues worthwhile.

6.2.6 Lab (2 hours)

Goals

1. Simulate a Q-learning agent in a reversal bandit task.
2. Fit (α, β) by MLE; run parameter recovery and diagnose the α–β trade-off.
3. Compare Q-learning with win-stay/lose-shift and biased-random models.
4. Test whether a simulated observer combines cues optimally.

Timing: Part A 20 min · Part B 20 min · Part C 25 min · Part D 30 min · Part E 25 min.

Part A: a Q-learning agent in a reversal task (20 min)

Code · کد: `code/week15.py`, cell 1 · سلول 1

Expected output:

```
Total reward: 151 / 200;  chose the better option on 91% of trials
```

Part B: the likelihood and an MLE fit (20 min)

Code · کد: `code/week15.py`, cell 2 · سلول 2

Expected output:

```
MLE: alpha = 0.491 (true 0.3), beta = 3.80 (true 5.0), NLL = 51.77
Chance-level NLL (always p = 0.5): 138.63
```

Part C: parameter recovery and the α–β trade-off (25 min)

Code · کد: `code/week15.py`, cell 3 · سلول 3

Expected output (about 30–60 s):

```
alpha: r(true, recovered) = 0.90
beta: r(true, recovered) = 0.17
corr(recovered alpha, recovered beta) = 0.31 (true values were drawn
↪  independently: 0.03)
```

Interpret: with 200 trials and one reversal, α is recovered well (r = 0.90) but β hardly at all (r = 0.17). This also explains the Part B fit ($\hat{\beta} = 3.8$ for a true 5.0). Look at the large-β region of the β panel. Why are high inverse temperatures hard to pin down? (Hint: once choices are almost deterministic, increasing β further barely changes the likelihood.) The recovered α and β are also correlated (0.31) although the true values were independent: a trade-off. What does this imply for a study that correlates β with, say, a clinical questionnaire?

Part D: is it really reinforcement learning? Model comparison (30 min)

Code · کد: `code/week15.py`, cell 4 · سلول 4

Expected output (about 1 minute):

```
AIC for the Part A participant: {'Q-learning': 107.5, 'WSLS': 200.3,
 ↪  'bias': 278.9}

Model recovery (rows: generating, columns: selected by AIC):
            Q-learning  WSLS  bias
Q-learning          20     0     0
WSLS                 0    20     0
```

Part E: an ideal observer for cue combination (25 min)

A simulated participant judges the size of objects seen and felt simultaneously. We know the single-cue noise levels from unimodal blocks. Does the participant combine the cues optimally?

> Code · کد: `code/week15.py`, cell 5 · سلول 5

Expected output:

```
Ideal observer: w_V = 0.80, predicted bimodal SD = 0.894 mm
Participant: empirical w_V = 0.56, bimodal SD = 1.006 mm
Best single cue SD = 1.000 mm;  simple average would give 1.118 mm
```

Interpret: is this participant optimal? The empirical visual weight (0.56) is well below the optimal 0.80: the participant over-weights the less reliable haptic cue. As a result, their bimodal SD (1.006 mm) is no better than vision alone (1.000 mm), instead of the 0.894 mm an ideal observer would achieve. Note that the ideal-observer prediction had no free parameters: that is what makes this a strong test.

(We simulated this participant with a true weight of 0.65. The estimate 0.56 is off by about 3 standard errors, an unusually unlucky sample but a useful reminder that empirical weights also have sampling error. Exercise 4 puts bootstrap intervals around them.)

Exercises (increasing difficulty)

Exercise 1 $\star$. Reproduce the worked example's four Q-learning trials with `nll_q` (hint: compute the NLL with `theta=(logit0.5,log3)` on the four-trial sequence). Do you get -1.884 for the log-likelihood?

Exercise 2 $\star$. Rerun Part C with 50 trials (no reversal) and with 400 trials (three reversals). How does recovery of α change? What does this imply for experimental design?

Exercise 3 $\star\star$ (two learning rates). Many studies find different learning rates for positive and negative prediction errors. Implement a model with α^+ and α^-, simulate an agent with $\alpha^+ = 0.4$, $\alpha^- = 0.1$, and test with AIC/BIC and a recovery study whether the extra parameter is identifiable at the Part A design.

Exercise 4 ⋆⋆ (optimal weights). In Part E, simulate an observer with $w_V = 0.8$ (optimal) and one with $w_V = 0.5$ (averaging), 100 bimodal trials each. Use a bootstrap (Week 5) to put confidence intervals around the empirical weights and SDs. With 100 trials, can you distinguish the two observers?

Exercise 5 ⋆⋆⋆ (Advanced: hierarchical RL). Simulate 20 agents with individual (α_j, β_j) and only 80 trials each. Fit them individually by MLE, and then with a hierarchical Bayesian model in PyMC. (Hint: the trial loop can be written with `pytensor.scan`; alternatively, implement a hand-written Metropolis-within-Gibbs sampler using `nll_q` for the individual parameters and conjugate updates for the group means.) Compare recovery of α_j between the two approaches.

Lab check-off

Show: (1) the MLE fit and the recovery plots with your interpretation of the β panel, (2) the model-recovery table, and (3) your answer on the optimality of the cue-combination observer.

6.2.7 Connection to AI & LLMs

The choice rule behind chatbot training. Modern chatbots are fine-tuned with *reinforcement learning from human feedback* (RLHF). People see two responses and say which they prefer, a reward model is fitted to these choices, and the chatbot is then trained by reinforcement learning to maximize the learned reward (Christiano et al., 2017; Ouyang et al., 2022). The reward model is usually the Bradley–Terry model, $P(i \succ j) = 1/(1 + e^{-(r_i - r_j)})$. This is the softmax choice rule of this week with $\beta = 1$, and the Luce choice rule of the GCM. Fitting it is an MLE problem from Week 4. So human choice models sit at the heart of how today's AI systems are aligned with human preferences, and so do their problems: noisy choices, identifiability (only *differences* in r matter, so one value is fixed), and the gap between what people choose and what is actually good.

Code · کد: `code/week15.py`, cell 6 · سلول 6

Expected output:

```
304 comparisons; estimated rewards = [0.   0.56 0.5  1.61] (true [0.
↪  0.5 1.  2. ])
```

With about 300 noisy comparisons, the estimated order of responses 1 and 2 is even reversed. A reward model is a fitted cognitive model, and it needs uncertainty estimates and recovery checks like any other.

Try it (no API needed). - ⋆ Compute Hessian-based standard errors for the three estimates (Week 5). Is the reversal of responses 1 and 2 within the uncertainty? - ⋆⋆ Run a parameter recovery: how many comparisons are needed before the order of all four responses is recovered correctly in 90%

of simulated datasets? - ⋆⋆ Simulate raters who prefer *longer* answers regardless of quality (add a length bonus to the true reward). What does the fitted reward model learn, and what will the chatbot then be trained to do?

Optional reading: Christiano, P. F., et al. (2017). Deep reinforcement learning from human preferences. *NeurIPS*. Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. *NeurIPS*.

6.2.8 Common mistakes & misconceptions

1. Fitting a learning model to aggregated choice proportions. The likelihood must follow each participant's own trial sequence, because values depend on their own history.
2. Interpreting α without a recovery check. α and β trade off, and α is poorly identified without reversals or enough trials.
3. Not comparing with simple strategies. Win-stay/lose-shift or perseveration can mimic learning. Show that the RL model wins a fair comparison.
4. Forgetting that the softmax uses value *differences*. Adding a constant to all Q-values changes nothing, while the scale of the values and β trade off.
5. "People are Bayesian" from a post-hoc fit. If the prior and likelihood were chosen to fit the behavior, optimality is assumed, not tested. Strong tests fix them independently (cue combination).
6. Confusing the two Bayesianisms (model of the mind vs. method of analysis).
7. "A large AI model that fits human data is a good cognitive model." Fit is not explanation. Ask about generalization, mimicry, identifiability and interpretability, as for any model.

6.2.9 Check your understanding

1. Write the delta-rule update and explain the role of the prediction error. Answer $Q \leftarrow Q+\alpha(r-Q)$. The prediction error $r - Q$ is the difference between outcome and expectation: positive errors raise the value, negative errors lower it, and the learning rate scales the change.

2. What does the inverse temperature β control? What behavior do $\beta = 0$ and $\beta \to \infty$ produce? Answer The sensitivity of choices to value differences (exploration vs. exploitation). $\beta = 0$: random choice regardless of values. $\beta \to \infty$: always choose the option with the highest current value.

3. Why must a learning model's likelihood be computed trial by trial on each participant's actual sequence? Answer The model's predictions on trial t depend on that participant's own previous choices and rewards (through the Q-values), so the probability of each observed choice can only be computed by running the model through the observed history.

4. Explain the α–β trade-off. Answer Different combinations of learning rate and choice noise can produce similar choice sequences. For example, sluggish adaptation can come from slow learning

(low α) or from noisy, value-insensitive choice (low β). The estimates are therefore correlated and individually uncertain; the direction of the correlation depends on the design. In addition, high β values are weakly identified because near-deterministic choices look the same for many large β.

5. Vision has SD 2 and touch SD 2. What are the optimal weights and the predicted combined SD? Answer Equal reliabilities give $w_V = w_H = 0.5$, and the combined SD is $\sqrt{4 \times 4/8} = \sqrt{2} = 1.41$, better than either cue alone (2).

6. Give one example each of an idea that moved from cognitive science to AI and from AI to cognitive science. Answer Cognitive science $\rightarrow$ AI: error-driven learning (Rescorla–Wagner / delta rule) leading to neural-network training and reinforcement learning. AI $\rightarrow$ cognitive science: deep convolutional networks used as models of the ventral visual stream, or RL agents and language models proposed as models of human behavior.

7. A researcher shows that a large language model's responses correlate highly with human responses in a reasoning task. What further evidence would you ask for before accepting it as a cognitive model? Answer Prediction of new conditions and of individual differences (generalization); a fair comparison against simpler cognitive models; tests that distinguish the model's mechanism from alternatives that produce the same outputs (mimicry); and some correspondence between its internal states and psychological constructs.

6.2.10 Assignment

No new assignment: final project presentations are next week, and the written report and code are due before the final exam. Use this week's lab time for project work and the course wrap-up.

6.2.11 Further reading

- Verguts (2022): the chapters on the delta rule and on reinforcement learning (error-driven learning and its neural basis) [verify chapters]. *(Main reading for Part 1.)*
- Farrell & Lewandowsky (2018): the chapters on neural network models and on models in neuroscience [verify chapter numbers].
- Anderson (2014): the chapters on neural networks and learning, which connect cognitive and computational neuroscience models [verify chapters].
- *Optional, outside the course list:* Ernst, M. O., & Banks, M. S. (2002). Humans integrate visual and haptic information in a statistically optimal fashion. *Nature, 415*, 429–433. Wilson, R. C., & Collins, A. G. E. (2019). Ten simple rules for the computational modeling of behavioral data. *eLife, 8*, e49547 (its running example is a bandit task like today's).

Preview for Week 16: final project presentations, and a closing session on good modeling practice.

6.3 Week 16: Final Project Presentations and Good Modeling Practice

Course wrap-up · Lecture slot: synthesis + first presentations · Lab slot: presentations and peer feedback

---

6.3.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
peer feedback	بازخورد همتا	نقد سازنده‌ی کار هم‌کلاسی‌ها
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
parameter recovery	بازیابی پارامتر	آیا برازش، پارامترهای معلوم شبیه‌سازی را برمی‌گرداند
reproducibility	بازتولیدپذیری	امکان به دست آوردن دوباره‌ی همان نتایج
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
Bayes factor (BF)	عامل بیز (Bayes factor)	نسبت درست‌نمایی‌های حاشیه‌ای دو مدل
bounds / constraints	قیدها / کران‌ها	محدودیت‌های مجاز برای مقدار پارامترها
posterior	توزیع پسین (posterior)	باور درباره‌ی پارامتر بعد از دیدن داده
hierarchical model	مدل سلسله‌مراتبی (چندسطحی)	پارامترهای فردی از یک توزیع گروهی می‌آیند
adequacy check	بررسی کفایت مدل	آیا مدل ویژگی‌های اصلی داده را بازتولید می‌کند
posterior predictive check (PPC)	بررسی پیش‌بینی پسین	مقایسه‌ی داده‌ی شبیه‌سازی‌شده از پسین با داده‌ی واقعی
cross-validation (CV)	اعتبارسنجی متقابل (cross-validation)	برازش روی بخشی از داده و آزمون روی بخش کنارگذاشته
AIC / BIC	بیزی / معیار اطلاعاتی آکائیکه	برازش به‌علاوه‌ی جریمه برای تعداد پارامتر
research design	طرح آزمایش	شرایط، تعداد کوشش‌ها و نمونه‌ها

English	فارسی	توضیح کوتاه
model recovery	بازیابی مدل	آیا مقایسه‌ی مدل، مدل مولد را درست انتخاب می‌کند
drift-diffusion model (DDM)	مدل drift-diffusion (رانش–انتشار)	انباشت پیوسته‌ی شواهد بین دو مرز
preregistration	پیش‌ثبت (preregistration)	ثبت برنامه‌ی تحلیل پیش از دیدن داده
prior	توزیع پیشین (prior)	باور درباره‌ی پارامتر پیش از دیدن داده
random seed	seed تصادفی	عدد شروع مولد اعداد تصادفی برای تکرارپذیری
large language model (LLM)	مدل بزرگ زبانی	شبکه‌ی عصبی‌ای که کلمه‌ی بعدی متن را پیش‌بینی می‌کند
accuracy	دقت	نسبت پاسخ‌های درست

6.3.2 Learning outcomes

By the end of this week, students will be able to:

1. Communicate a complete modeling study (question, model, fitting, evaluation, recovery, interpretation) clearly and within a time limit.
2. Critique other modeling studies constructively, using the course's evaluation criteria.
3. Apply a checklist of good modeling practice (reproducibility, transparency, robustness) to their own project report.
4. Integrate the methods of the course into one coherent workflow and explain when to use which.

6.3.3 Motivation & big picture

A model is only as useful as others' ability to understand, check and build on it. Over 15 weeks, every lab has followed the same arc: formalize → simulate → fit → evaluate → recover → interpret. This week you show that arc in your own project, and you practice the other half of scientific modeling: reading someone else's model critically but fairly. The session ends with the practices that make modeling work trustworthy and reusable: reproducible code, transparent reporting, honest statements of limitations.

6.3.4 Lecture notes

In plain words The last week has no new method. Students present their projects, and the course is summarized as one workflow: formalize a theory as a model, simulate it to see what it predicts, fit it to data, evaluate it (does it fit, and does it beat alternatives?), check that the method can recover the truth at this design, and interpret the result in psychological terms, including what cannot be concluded. The checklist of good practice turns these steps into concrete things to verify in every report: reproducible code, reported uncertainty, adequacy checks, recovery and honest limitations.

Suggested timing (lecture slot, 60 min): course synthesis 15 · good-practice checklist 15 · first presentations 30.

1. The course in one table

Question you face	Tools	Week
How do I turn a theory into a model?	Five components; simulation; analytic checks	1–2
How do I estimate parameters?	Least squares; maximum likelihood; optimization (grid, Nelder–Mead, L-BFGS-B), multi-start, bounds	3–4
How certain are the estimates?	Hessian SEs, profile likelihood, bootstrap; posterior intervals	5, 9
How do I handle many participants?	Individual fits; aggregation artifacts; hierarchical models	5, 13
Is the model adequate?	Residuals, G^2, posterior predictive checks	2–4, 10
Which model is better?	CV, LRT, AIC/BIC, Bayes factors	6–7, 10
Can my design answer the question?	Parameter recovery, model recovery	7
How do I get a posterior for complex models?	Grid $\rightarrow$ Metropolis–Hastings $\rightarrow$ Gibbs $\rightarrow$ NUTS (PyMC); $\hat{R}$, ESS, divergences	9–13
What do models look like in different fields?	SDT, forgetting, practice, GCM, DDM/EZ, RL, ideal observers	2, 5, 14–15

For an engineer Looking back, most of the course maps onto tools you already know, and the table below can serve as your own summary when answering students' questions:

Course topic	Engineering counterpart
Random walk / DDM (Weeks 1, 14)	Sequential detection (SPRT), first-passage times
SDT, ROC (Week 2)	Binary detection, P_D / P_{FA}, ROC
Least squares, optimizers (Week 3)	LS estimation, gradient methods, Gauss–Newton
Maximum likelihood, G^2, LRT (Weeks 4, 7)	ML estimation, goodness-of-fit tests, GLRT
Hessian SEs, bootstrap (Week 5)	Fisher information / CRLB, Monte Carlo performance evaluation
CV, AIC, BIC (Weeks 6–7)	Model-order selection (AIC, MDL), validation data
Parameter/model recovery (Week 7)	Testing estimators on synthetic data with known ground truth
Bayesian inference, BF (Weeks 9–10)	MAP/MMSE estimation, Bayesian detection
MCMC, $\hat{R}$, ESS (Weeks 11–12)	Monte Carlo simulation, Markov processes, correlated-sample statistics
Hierarchical models (Week 13)	Empirical Bayes, James–Stein shrinkage
Delta rule, cue combination (Week 15)	LMS adaptation, MRC / Kalman sensor fusion

What is genuinely different in cognitive modeling: the "system" is a human being, the data are small and noisy, and the parameters themselves are the scientific findings, so interpretation, uncertainty and recovery matter more than raw performance.

2. A checklist of good modeling practice

Before fitting - [ ] The research question is stated in terms of model parameters or model comparisons. - [ ] Candidate models (including at least one simpler alternative) are specified *before* looking at the data, ideally preregistered. - [ ] Priors (if Bayesian) are justified and checked with prior predictive simulation. - [ ] Parameter and model recovery have been checked at the actual design.

Fitting and evaluation - [ ] The likelihood matches the data type (binomial for counts, sequential for learning, …). - [ ] Optimization uses multi-start and sensible bounds, and fits are checked visually; for MCMC, $\hat{R}$, ESS and divergences are reported. - [ ] Absolute fit (residuals, G^2 or PPC) *and* relative fit (CV/IC/BF) are reported. - [ ] Uncertainty is reported for every interpreted parameter.

Reporting and reproducibility - [ ] All code and (where allowed) data are shared; random seeds are fixed. - [ ] Software versions are recorded (e.g., `condaenvexport>environment.yml`). - [ ] One command or notebook regenerates every number and figure in the report. - [ ] Limitations are stated: what the design cannot distinguish, which parameters are not recoverable, which assumptions were not tested.

Recording the computing environment (include in every project notebook):

Code · کد: `code/week16.py`, cell 1 · سلول 1

3. Presentation format

- Length: 8 minutes (individual) or 10 minutes (pairs), plus 4 minutes of questions. Strict time-keeping.
- Suggested structure (one slide each, roughly):
  1. The question and why it matters.
  2. The model(s): equations and the psychological meaning of every parameter.
  3. Data and design.
  4. Fitting method and a fit figure (data + predictions).
  5. Evaluation: model comparison *and* an adequacy check.
  6. Recovery: can the design answer the question?
  7. Interpretation and limitations.
- Pairs: both members present, and the Bayesian/MCMC or hierarchical component must be shown.
- Schedule: posted one week in advance; the order is drawn at random. Sessions run across the lecture and lab slots (about 15–18 presentations in 3 hours, depending on enrolment).

4. Giving and receiving feedback

Each student completes a peer-feedback form for three assigned presentations (below). Feedback is anonymous to the presenter, visible to the instructor, and counts toward participation (lab check-off). Good feedback is specific ("the recovery plot shows β is not identifiable, so the group difference in β should not be interpreted") rather than general ("nice talk"), and constructive (it suggests a concrete improvement).

Peer-feedback form

Item	Rating (1–4)	Comment
The question was clear and the model matched it		
The model's parameters were explained in psychological terms		
The fit was shown and evaluated (adequacy + comparison)		
Recovery or identifiability was addressed		
Conclusions followed from the results, and limitations were stated		

Item	Rating (1–4)	Comment
One thing that was particularly well done		
One concrete suggestion for the written report		

For the instructor

- Grade with the rubric, comment with the checklist. During presentations, keep the good-practice checklist next to you and note one strength and one concrete improvement per team; this makes the written feedback fast and fair.
- Questions to ask presenters (they map onto the course): Which parameter answers your question? How do you know it is recoverable? Does the model fit in absolute terms, not only better than the alternative? What can your design *not* tell us?
- Use the worked critique (fictional Q-learning study) as a model for how students should give feedback: specific, tied to a week of the course, and constructive.
- Close the course with the one-table summary. Ask students which row they found hardest, and point them to the corresponding week for exam revision.

6.3.5 Worked example: critiquing a modeling study

Summary of a (fictional) study. *"We fitted a Q-learning model to 30 patients and 30 controls who performed 60 trials of a two-armed bandit task with stable reward probabilities. Each participant was fitted by MLE. Patients had significantly lower learning rates ($t(58) = 2.4$, $p = .02$). We conclude that the disorder impairs reward learning."*

Critique using the checklist.

1. Recovery at this design (Week 7, Week 15 lab). With 60 trials and *stable* reward probabilities, the learning rate is weakly identifiable, and α and β trade off. Without a recovery study, a difference in $\hat{\alpha}$ could reflect a difference in choice noise (β) or just estimation noise. *Suggestion:* report parameter recovery at this design, and include reversals.
2. Model comparison (Week 15, Part D). Did Q-learning beat simpler strategies (WSLS, perseveration) in both groups? If patients are better described by a different strategy, comparing their α with controls' is not meaningful. *Suggestion:* report AIC/BIC or CV per group, with model recovery.
3. Second-stage t-test on point estimates (Weeks 5, 13). It ignores the uncertainty of individual estimates, which is large with 60 trials. *Suggestion:* a hierarchical model with a group effect on α, reporting its posterior.
4. Adequacy (Weeks 4, 10). No posterior predictive check or learning-curve plot is shown. Does the model reproduce each group's learning curve?

5. Interpretation. “Impairs reward learning” is a strong process claim. A lower $\hat{\alpha}$ is also compatible with more exploration, lower motivation (lower β), or different priors on initial values. *Suggestion:* temper the conclusion, or add a design that separates these accounts.

Overall: a reasonable first study whose conclusion is not yet supported. Three additions (recovery, model comparison, hierarchical estimation) would make it convincing. Notice that each point maps directly onto one week of this course.

6.3.6 Lab (2 hours): presentations and peer feedback

Goals: present your project; give written feedback on three presentations; note the feedback you receive for your final report.

Logistics: - Upload slides (PDF) to the course page before the session starts. - Bring your laptop with your project notebook in case of questions about the code. - Fill in the peer-feedback forms during the session (paper or online form) and submit them by the end of the day.

After the presentations: the instructor gives brief oral feedback to each team. Written feedback (instructor + peers) is returned within 3 days so it can be used in the final report.

6.3.7 Connection to AI & LLMs

Good practice when AI is part of your modeling work. AI tools now enter cognitive modeling in three roles, and each needs the course’s habits of checking: 1. As a coding assistant. Generated code can look right and be wrong. Test it the modeling way: simulate data with known parameters and check that the code recovers them (parameter recovery as a unit test). 2. As a “simulated participant”. Some studies give LLMs psychology experiments and compare the answers with human data, or even propose LLM-based models of human behavior (e.g., Binz & Schulz, 2023; Dillion et al., 2023 [verify]). Evaluate such claims like any model: generalization to new tasks, comparison with simpler cognitive models, mimicry, and interpretability (Week 15). 3. As an object of study. If you test an LLM, record the model name and version, the date, the exact prompts, the temperature and the number of samples. Without these, the study cannot be reproduced.

The code below tests a pre-made SDT function “written by an assistant”. It looks plausible, and it even passes the case $c = 0$, but a recovery test shows that it gets the sign of c wrong.

Code · کد: `code/week16.py`, cell 2 · سلول 2

Expected output:

```
true (d', c) = (1.5, 0.3) -> recovered (1.50, -0.30)  FAIL
true (d', c) = (1.0, -0.4) -> recovered (1.00, 0.40)  FAIL
true (d', c) = (2.0, 0.0) -> recovered (2.00, 0.00)  OK
```

A liberal observer would be reported as conservative, which would reverse the conclusion of a bias study. The test with $c = 0$ alone would not have caught it.

Try it (no API needed). - ⋆ Fix the function and rerun the tests. - ⋆⋆ Write a similar recovery test for the EZ-diffusion function of Week 14, using the simulator from the same lab. - ⋆⋆ For your own project: add one paragraph to the reproducibility statement that describes how AI tools were used and how their output was verified (see `assessments/project_brief.md`, section 6).

Optional reading: Binz, M., & Schulz, E. (2023). Using cognitive psychology to understand GPT-3. *PNAS, 120*(6), e2218523120. Dillion, D., Tandon, N., Gu, Y., & Gray, K. (2023). Can AI language models replace human participants? *Trends in Cognitive Sciences, 27*, 597–600 [verify].

6.3.8 Common mistakes & misconceptions (in project presentations and reports)

1. Too much on the method, too little on the meaning. Every parameter and every result needs a psychological interpretation.
2. Showing a fit statistic without a fit figure. Always plot data against predictions.
3. Model comparison without an adequacy check, or vice versa.
4. Interpreting parameters that were not shown to be recoverable.
5. Overclaiming: "the model is correct", "the mind is Bayesian". State what the evidence supports and what it does not.
6. Irreproducible results: missing seeds, unrecorded package versions, notebook cells run out of order.
7. Running over time. Practice the talk; one idea per slide.

6.3.9 Check your understanding (integrative review questions for the final exam)

1. A colleague has accuracy and RT data from a speed/accuracy manipulation and wants to know whether the manipulation affects caution or evidence quality. Outline a full modeling plan, naming the model, the fitting method, the evaluation, and the recovery check. Answer Model: drift-diffusion (EZ-diffusion for a first pass, or the full DDM by MLE / hierarchical Bayes). Fit per participant and condition. Compare models in which the manipulation affects only a, only v, or both (AIC/BIC or a hierarchical model with condition effects; PPC on RT quantiles). Recovery: simulate data at the actual design with each model, and check that parameters and the "which parameter changed" conclusion are recovered. Interpret the condition effects on a and v with uncertainty.

2. When would you choose a Bayes factor over cross-validation for comparing two models, and vice versa? Answer Bayes factor: when the question is which model is more probable given the data (evidence, including support for a null), when meaningful priors are available, and with a prior sensitivity analysis. CV: when the goal is predictive accuracy for new data of a specific kind, priors are hard to justify, or the BF is too sensitive to the priors.

3. Explain how a parameter can be estimated very precisely (narrow CI or posterior) and yet be meaningless. Answer Precision only reflects the curvature of the likelihood under the assumed model. If the model is misspecified (fails PPC/G^2), if the parameter trades off with another so that its meaning depends on an arbitrary constraint, or if the optimizer or sampler failed (local minimum, bound, non-convergence), the precise estimate does not measure the intended construct.

4. Why do hierarchical models often give better individual estimates than separate individual fits? Answer They share information across participants through the group distribution, shrinking noisy individual estimates toward the group mean in proportion to their uncertainty. This reduces estimation variance more than it adds bias, so estimates are on average closer to the truth.

5. Name three different ways in which this course used simulation, and why each matters. Answer (i) Exploring a model's predictions before fitting (what can and can't it produce). (ii) Parameter and model recovery (can the design and method answer the question). (iii) Bootstrap and predictive checks (uncertainty and adequacy). Further: Monte Carlo and MCMC for computing posteriors.

6. A model has the best AIC among four candidates but a posterior predictive check shows a systematic misfit at long delays. What do you report and conclude? Answer Report both. The model is the best of the candidates for prediction, but it is not adequate in absolute terms: none of the candidates captures the long-delay behavior. Conclusions should be restricted accordingly, and a model revision targeting that feature is the next step.

6.3.10 Assignment

Final project: written report and code

- Due: one week after this session, and in any case before the final exam (exact date on the course page).
- Submission: report (PDF, max. 10 pages for individuals, 14 for pairs, excluding references and appendix) plus a code archive or repository link that reproduces all results.
- Grading: see the project brief and rubric (`assessments/project_brief.md`). The report and code are worth 3 of the 6 project points; the proposal (1) and presentation (2) are already graded.
- Use the feedback from today: the rubric rewards visible responses to presentation feedback.

Final exam

Written, cumulative (emphasis on Chapters 4–6), formula sheet provided, no laptops; worth 3 of 20 points. See `assessments/final_exam.md` for the format and a sample paper.

6.3.11 Further reading

- Farrell & Lewandowsky (2018): the chapter on using models in psychology (the role of models in theory development and common pitfalls) [verify chapter number].
- Lee & Wagenmakers (2014): browse the case studies in Part IV for ideas on how to extend your project.
- Martin, Kumar & Lao (2021), chapter 9. End-to-end Bayesian workflows, a good model for structuring a report.
- *Optional, outside the course list:* Wilson, R. C., & Collins, A. G. E. (2019). Ten simple rules for the computational modeling of behavioral data. *eLife, 8*, e49547.

Thank you for a great course, and keep modeling!

Glossary

Terms are listed alphabetically. The number in brackets is the week where the term is introduced or treated most fully. Notation follows the course-wide table in Week 1.

Term	Definition
Acceptance rate [11]	Proportion of Metropolis proposals that are accepted. Very high: steps too small; very low: steps too large. A rough target is 0.2–0.5.
Aggregation artifact [5]	A pattern in averaged data that no individual shows, e.g., a power-law curve from averaging exponential learners.
AIC / AICc [7]	Akaike information criterion, $2\,\mathrm{NLL} + 2k$; estimates out-of-sample predictive accuracy. AICc adds a small-sample correction.
Akaike weight [7]	$w_i \propto \exp(-\Delta_i/2)$: relative support for model i as the best predictor among the candidates.
Attention weight (w_m) [14]	GCM parameter: the share of attention given to stimulus dimension m (weights sum to 1).
Bayes factor (BF_{12}) [10]	Ratio of marginal likelihoods $p(\mathrm{y} \mid \mathcal{M}_1)/p(\mathrm{y} \mid \mathcal{M}_2)$: how much the data change the odds between two models.
Bayesian data analysis [9]	Using Bayes' rule to infer parameters and models from data; a method of the scientist, applicable to any model.
Bayesian model of cognition [15]	A hypothesis that the mind performs (approximately) Bayesian inference; a computational-level theory.
Beta-binomial model [9]	A binomial likelihood with a Beta prior; the posterior is again a Beta distribution (conjugacy).
BIC [7]	Bayesian information criterion, $2\,\mathrm{NLL} + k \ln N$; approximates $-2 \ln p(\mathrm{y} \mid \mathcal{M})$ and favors simpler models than AIC.

Term	Definition
Bias–variance trade-off [6]	Expected prediction error = noise + bias^2 + variance. Flexible models lower bias but raise variance.
Bootstrap (parametric) [5]	Simulate many datasets from the fitted model, refit each one, and use the spread of the estimates as their uncertainty.
Boundary separation (a) [1, 14]	Random walk / DDM parameter: the distance between the response boundaries. Represents caution.
Burn-in (warm-up) [11]	Initial MCMC iterations discarded because they still reflect the starting values.
Centered / non-centered parameterization [13]	Two equivalent ways to write a hierarchical model; the non-centered form ($d_j = \mu + \sigma z_j$) avoids funnel geometry and divergences.
Complete / no / partial pooling [5, 13]	One parameter set for everyone / independent parameters per person / individual parameters drawn from an estimated group distribution.
Confusion matrix (model recovery) [7]	$P(\text{selected model} \mid \text{generating model})$, estimated by simulation.
Conjugate prior [9, 12]	A prior that yields a posterior in the same family (e.g., Beta for binomial data); enables Gibbs sampling.
Credible interval / HDI [9]	An interval containing a stated posterior probability (e.g., 95%); the HDI is the narrowest such interval.
Criterion (c) [2]	SDT response-bias parameter: the criterion location relative to the midpoint between the noise and signal distributions. Positive values are conservative.
Cross-validation (K-fold, LOO) [6]	Fit on part of the data and score predictions on the held-out part, rotating through K folds; LOO holds out one observation at a time.
d' (sensitivity) [2]	SDT parameter: the distance between the signal and noise distributions in SD units.
Delta rule / Rescorla–Wagner rule [15]	Error-driven update $Q \leftarrow Q + \alpha(r - Q)$.
Deterministic core [2]	The part of a model that gives its average prediction, $f(x; \boldsymbol{\theta})$.
Deviance (G^2) [4]	$2(\ell_{\text{sat}} - \hat{\ell})$: misfit relative to the saturated model; approximately χ^2 with df = cells $- k$.

Term	Definition
Divergent transition [13]	A NUTS trajectory whose numerical integration broke down; it signals unreliable exploration of part of the posterior.
Drift rate (v) [1, 14]	Random walk / DDM parameter: the average rate of evidence accumulation. Represents evidence quality.
Drift-diffusion model (DDM) [14]	Continuous evidence accumulation with noise between two boundaries; predicts choices and RT distributions.
Effective sample size (ESS) [12]	The number of independent draws that would give the same precision as the autocorrelated MCMC draws.
Error surface [3]	The discrepancy (SSE or NLL) as a function of the parameters; its shape reveals minima, valleys (trade-offs) and plateaus.
Evidence / marginal likelihood [9, 10]	$p(\mathrm{y} \mid \mathcal{M}) = \int p(\mathrm{y} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta})\, d\boldsymbol{\theta}$: how well the model predicted the data before fitting.
EZ-diffusion [14]	Closed-form estimates of drift, boundary and non-decision time from accuracy and the mean and variance of correct RTs.
Flexibility (model complexity) [6]	The range of data patterns a model can produce; it depends on the number of parameters *and* the functional form.
Forward / inverse problem [1]	Simulation (parameters $\rightarrow$ data) vs. estimation (data $\rightarrow$ parameters).
Full conditional [12]	The distribution of one parameter given all others and the data; Gibbs sampling draws from it.
GCM (Generalized Context Model) [14]	Exemplar model of categorization: classification by summed similarity to stored exemplars.
Generalization criterion [6]	Evaluating models by predicting new conditions (extrapolation) rather than new trials.
Gibbs sampling [12]	MCMC that updates each parameter by drawing from its full conditional; every draw is accepted.
Grid approximation [9]	Computing the posterior on a grid of parameter values; feasible for 1–2 parameters.
Hamiltonian Monte Carlo (HMC) / NUTS [13]	Gradient-based MCMC that simulates physical dynamics to make distant, high-acceptance proposals; NUTS tunes the trajectory length automatically.

Term	Definition
Hessian / observed Fisher information [5]	Matrix of second derivatives of the NLL at the MLE; its inverse approximates the covariance of the estimates.
Hierarchical (multilevel) model [13]	A model with individual-level parameters drawn from group-level distributions.
Identifiability (structural / practical) [7]	Whether parameters can be uniquely determined, in principle (structural) or from the available data (practical).
Ideal observer [15]	A model of optimal inference given the available information, e.g., reliability-weighted cue combination.
Inverse temperature (β) [15]	Softmax parameter: the sensitivity of choices to value differences (exploration vs. exploitation).
Inversion matrix (model recovery) [7]	$P(\text{generating model} \mid \text{selected model})$: how much to trust a model-selection outcome.
Learning rate (α) [15]	How strongly prediction errors update values.
Likelihood (L), log-likelihood (ℓ), NLL [4]	$p(\text{y} \mid \boldsymbol{\theta})$ viewed as a function of $\boldsymbol{\theta}$; its log; and the negative log, which optimizers minimize.
Likelihood-ratio test (LRT) [7]	For nested models, $2\Delta\ell \sim \chi^2_{\Delta k}$ under the simpler model.
Linking assumption [2]	How model quantities map onto observable behavior.
Local minimum [3]	A point lower than its surroundings but not the global minimum; found by single-start optimization. Use multi-start.
MAP [9]	Maximum a posteriori estimate (posterior mode); equals the MLE under a flat prior.
Markov chain Monte Carlo (MCMC) [11]	Sampling from a posterior by simulating a Markov chain whose stationary distribution is the posterior.
Maximum likelihood estimate (MLE) [4]	The parameter values that maximize the likelihood.
MCSE [12]	Monte Carlo standard error, posterior SD/$\sqrt{\text{ESS}}$: the simulation error of a posterior mean.
Metropolis / Metropolis–Hastings [11]	MCMC that proposes a move and accepts it with probability $\min(1, \text{ratio})$; Hastings adds a correction for asymmetric proposals.
Model mimicry [7]	One model producing data that look as if generated by another.

Term	Definition
Model recovery [7]	Simulating from each candidate model and checking whether the comparison method selects the generating model.
Monte Carlo integration [11]	Approximating an expectation by the average over random draws; error $\propto 1/\sqrt{S}$.
Multi-start [3]	Running an optimizer from many starting values and keeping the best result.
Nelder–Mead simplex [3]	Derivative-free optimizer that moves a simplex by reflection, expansion, contraction and shrinkage.
Nested models [7]	One model is a special case of the other (obtained by fixing parameters).
Noise model [2]	The distribution of observations around the deterministic prediction (e.g., binomial, Gaussian).
Non-decision time (t_0, T_{er}) [1, 14]	The part of RT not spent accumulating evidence (encoding and motor execution).
Overfitting [6]	Fitting the noise of a sample, which improves fit but worsens prediction of new data.
Parameter recovery [7]	Simulating data from known parameters at the actual design and checking that fitting returns them.
Parameter trade-off [3, 5]	Changes in one parameter can be compensated by another, which shows up as a tilted valley in the error surface and correlated estimates.
Posterior [9]	$p(\boldsymbol{\theta} \mid \mathrm{y}) \propto p(\mathrm{y} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta})$.
Posterior predictive check (PPC) [10]	Comparing data simulated from the fitted model with the observed data, to check adequacy.
Power / exponential forgetting [2]	$A(1+t)^{-d}$ (decreasing relative loss rate) vs. $Ae^{-\lambda t}$ (constant relative loss rate).
Prior / prior predictive check [9]	The distribution of parameters before the data / simulating data from the prior to check that the prior is sensible.
Profile likelihood [5]	The NLL minimized over the other parameters for each value of the parameter of interest; gives likelihood-ratio intervals.
Prototype model [14]	Categorization by similarity to a single summary representation per category.

Term	Definition
R-hat ($\hat{R}$) [12]	Ratio comparing between-chain and within-chain variance; ≈ 1 (< 1.01) when chains agree.
Random walk model [1]	Discrete-step evidence accumulation to a boundary; the precursor of the DDM.
Relative loss rate (hazard) [2]	$-f'(t)/f(t)$: the proportional rate of forgetting at time t.
Reparameterization [3, 13]	Optimizing or sampling on a transformed scale (e.g., $\lambda = e^{\eta}$) to enforce constraints or improve geometry.
RMSD [3]	Root mean squared deviation, $\sqrt{\text{SSE}/N}$, in data units.
ROC / zROC [2]	Hit rate vs. false-alarm rate as the criterion varies / the same on z (probit) axes.
Saturated model [4]	A model with one free parameter per data cell, which reproduces the data exactly.
Savage–Dickey density ratio [10]	For nested models, BF_{01} = posterior density / prior density at the null value.
Shrinkage [13]	The pull of individual estimates toward the group mean in hierarchical models, stronger for noisier data.
Signal detection theory (SDT) [2]	A model of detection decisions with noisy evidence and a criterion; separates sensitivity from bias.
Softmax [15]	$P(o) \propto \exp(\beta Q(o))$: turns values into choice probabilities.
SSE / MSE [3]	Sum / mean of squared errors between data and predictions.
Standard error [5]	The SD of an estimator's sampling distribution.
Wald interval [5]	$\hat{\theta} \pm 1.96\,\text{SE}$, with the SE from the Hessian; assumes approximate normality.
Weighted least squares [3]	Squared errors weighted by the inverse of their variance.

6.3.12 AI & LLM terms (from the "Connection to AI & LLMs" sections)

Term	Definition
Benchmark contamination [6]	Test items (or their answers) that appeared in a model's training data; a form of data leakage that inflates scores.
Bradley–Terry model [15]	$P(i \succ j) = 1/(1 + e^{-(r_i - r_j)})$: choice model for paired comparisons; used for RLHF reward models. Equivalent to a two-option softmax with $\beta = 1$.
Cross-entropy loss [4]	The training loss of language models: the negative log-likelihood of the next token, averaged over tokens.
Embedding [14]	A vector that a model assigns to a word or text; distances between embeddings are often used as a proxy for similarity.
In-context learning [7]	An LLM's improvement on a task from examples given in the prompt, without changing its parameters.
Large language model (LLM) [1]	A neural network (usually a transformer) trained to predict the next token in very large text corpora.
Perplexity [4]	$\exp(\text{NLL per token})$: the effective number of equally likely next tokens; lower is better.
RLHF [15]	Reinforcement learning from human feedback: a reward model is fitted to human preference choices, and the language model is then trained to maximize that reward.
Scaling law [3]	An empirical (usually power-law) relation between model loss and model size, data size or compute.
Self-consistency [11]	Sampling several answers from a model and taking the majority vote.
Temperature [1]	Divides the logits before the softmax in an LLM; low temperature gives near-deterministic output, high temperature more random output ($\beta = 1/T$).
Token [1]	The unit a language model reads and predicts: a word or part of a word.

Reading list

6.3.13 Course references (from the course specification)

Reference	Role in the course	Weeks
Farrell, S., & Lewandowsky, S. (2018). *Computational modeling of cognition and behavior.* Cambridge University Press.	Primary text for the first half: from verbal to formal models, parameter estimation, model comparison; also Bayesian estimation, hierarchical models and model families	1–10, 12–16
Lee, M. D., & Wagenmakers, E.-J. (2014). *Bayesian cognitive modeling: A practical course.* Cambridge University Press.	Primary text for the second half: Bayesian inference, model comparison and case studies (SDT, memory retention, GCM)	2, 4, 7–10, 12–14, 16
Kruschke, J. K. (2014). *Doing Bayesian data analysis: A tutorial with R, JAGS, and Stan* (2nd ed.). Academic Press.	Gentle, visual introduction to Bayes' rule, MCMC, diagnostics, hierarchical models and Bayes factors	9–13
Martin, O. A., Kumar, R., & Lao, J. (2021). *Bayesian modeling and computation in Python.* CRC Press.	Python/PyMC/ArviZ practice: Bayesian workflow, diagnostics, PPCs	9–13, 16
Anderson, B. (2014). *Computational neuroscience and cognitive modelling.* SAGE.	Context on computational neuroscience; neural network and learning models	1, 15
Verguts, T. (2022). *Introduction to modeling cognitive processes.* MIT Press.	Accessible modeling introduction; optimization and gradient descent; delta rule and reinforcement learning; categorization and decisions	1, 3, 14–15

Reference	Role in the course	Weeks
Ghodousian, A. (1401). *Mabani-ye ravesh-haye mostaghim dar behine-sazi gheyr-khatti* (Persian).	Direct-search methods in nonlinear optimization (theory behind Nelder–Mead)	3
Azimifar, Z., & Ahmadi, E. (1396). *Shenasayi-ye amari-ye olgu: yadgiri-ye nazari-ye moghaddamati*, vol. 1 (Persian).	Statistical learning view: ML and Bayesian estimation; generalization and the bias–variance trade-off	4, 6, 9

6.3.14 Chapter pointers by week

> Please verify. The chapter numbers below were assigned from memory of the books' tables of contents. Those for Farrell & Lewandowsky, Lee & Wagenmakers and Kruschke are believed to be correct but should be checked against the editions in the library. For Anderson, Verguts, Ghodousian and Azimifar & Ahmadi only topics are given; chapter numbers are marked [verify].

Week	Readings
1	F&L ch. 1–2; Verguts, introductory chapter [verify]; Anderson, introductory chapter [verify]
2	F&L ch. 2; L&W ch. 11 (SDT, preview), ch. 10 (memory retention)
3	F&L ch. 3; Ghodousian (direct search methods) [verify]; Verguts (optimization/gradient descent) [verify]
4	F&L ch. 4; L&W ch. 3; Azimifar & Ahmadi (ML estimation) [verify]
5	F&L ch. 5 (and ch. 4 on parameter uncertainty)
6	F&L ch. 10; Azimifar & Ahmadi (generalization, bias–variance) [verify]
7	F&L ch. 10; L&W ch. 7 (introduction)
8	Review: F&L ch. 2–5, 10; L&W ch. 11
9	Kruschke ch. 5–6; L&W ch. 1, 3; MKL ch. 1; F&L ch. 6; Azimifar & Ahmadi (Bayesian estimation) [verify]

Week	Readings
10	L&W ch. 7; F&L ch. 11; Kruschke ch. 10; MKL ch. 2
11	Kruschke ch. 7; F&L ch. 7; MKL (MCMC/inference engines) [verify]
12	Kruschke ch. 7; MKL ch. 2; L&W ch. 4
13	L&W ch. 11; Kruschke ch. 9; F&L ch. 9; MKL ch. 2, 10
14	L&W ch. 17; F&L (choice response times) [verify chapter]; Verguts (categorization, decisions) [verify]
15	Verguts (delta rule, reinforcement learning) [verify]; F&L (neural networks, neuroscience) [verify]; Anderson (networks, learning) [verify]
16	F&L (using models in psychology) [verify]; L&W Part IV; MKL ch. 9

Abbreviations: F&L = Farrell & Lewandowsky; L&W = Lee & Wagenmakers; MKL = Martin, Kumar & Lao.

6.3.15 Optional readings outside the course list

These are cited in individual weeks for students who want the original sources. All are well-known publications; please check the details against the publisher's record before distributing.

- Busemeyer, J. R., & Wang, Y.-M. (2000). Model comparisons and model selections based on generalization criterion methodology. *Journal of Mathematical Psychology, 44*, 171–189. (Week 6)
- Ernst, M. O., & Banks, M. S. (2002). Humans integrate visual and haptic information in a statistically optimal fashion. *Nature, 415*, 429–433. (Week 15)
- Heathcote, A., Brown, S., & Mewhort, D. J. K. (2000). The power law repealed: The case for an exponential law of practice. *Psychonomic Bulletin & Review, 7*, 185–207. (Week 5)
- Marr, D. (1982). *Vision.* W. H. Freeman. (Week 1)
- Medin, D. L., & Schaffer, M. M. (1978). Context theory of classification learning. *Psychological Review, 85*, 207–238. (Week 14)
- Roberts, S., & Pashler, H. (2000). How persuasive is a good fit? A comment on theory testing. *Psychological Review, 107*, 358–367. (Week 6)
- Vehtari, A., Gelman, A., Simpson, D., Carpenter, B., & Bürkner, P.-C. (2021). Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC. *Bayesian Analysis, 16*, 667–718. (Week 12)
- Wagenmakers, E.-J., van der Maas, H. L. J., & Grasman, R. P. P. P. (2007). An EZ-diffusion model for response time and accuracy. *Psychonomic Bulletin & Review, 14*, 3–22. (Week 14)

- Wilson, R. C., & Collins, A. G. E. (2019). Ten simple rules for the computational modeling of behavioral data. *eLife, 8*, e49547. (Weeks 7, 15, 16)